The Backend Engineer's Library

Computer Architecture

Volume 1

Contents

Why Architecture Matters: Mechanical Sympathy

The Modern CPU: Pipelines, Out-of-Order, Speculation

The Memory Hierarchy and Caches

Cache Coherence and Hardware Memory Consistency

Storage Hardware: HDD, SSD, NVMe, and Persistent Memory

Multi-Socket Systems and NUMA

Data Parallelism: SIMD, Vectorization, and GPUs for Backend

Performance Anti-Patterns: False Sharing, Branch Misprediction, Cache Thrashing

Number Representation and Floating Point

Hardware Support for Virtualization and Isolation

Chapter 1 — Why Architecture Matters: Mechanical Sympathy

What this chapter covers. This is the opening chapter of a volume about the machine underneath your services. Its job is to convince you that a senior backend engineer — one who spends their days thinking in terms of services, queues, RPCs, and SLOs — cannot reason about performance, cost, or tail latency without a working model of the hardware those abstractions run on. We introduce *mechanical sympathy*: the discipline of writing software that works *with* the grain of the hardware rather than against it. We anchor it in the numbers every engineer should carry in their head — the latency hierarchy that spans roughly nine orders of magnitude from a register access to a cross-continent round trip. We trace the abstraction stack from your source code down to circuits and show where performance *leaks* through those abstractions in ways your source cannot express. We argue that all of this matters *more*, not less, at fleet scale, where a constant-factor inefficiency multiplies across thousands of nodes into real money and blown latency budgets. Finally, we lay out the mental model and the roadmap for the rest of the volume.

This chapter is deliberately a motivating opener. The concrete effects we tease here — cache misses, branch mispredictions, NUMA, false sharing, sequential vs. random I/O — each get a full chapter later. Here we draw the map.

Learning goals — after this chapter you should be able to:

- Define mechanical sympathy and explain its origin and its backend-engineering meaning: that latency, throughput, and cost are ultimately set by hardware behavior your high-level code hides.
- Recite the latency hierarchy to the correct *order of magnitude* — register, L1/L2/L3 cache, DRAM, SSD, intra-datacenter network, disk seek, cross-continent — and explain why spanning nine orders of magnitude means *which tier you hit* dominates performance.
- Name the layers of the abstraction stack and identify, for a given performance symptom, which layer is leaking (a cache miss, a page fault, a syscall, a GC pause).

- Explain why data *movement*, not compute, usually dominates modern backend performance, and reason about a workload as compute-bound, memory-bound, or I/O-bound.
- Articulate why per-node hardware efficiency is a fleet-level cost and tail-latency concern, and why the network is just another (slow) tier of the same latency hierarchy.
- Apply the practitioner's stance: design for locality and sequential access, assume you cannot intuit modern hardware, and measure.

The term, and why a racing driver owns it

The phrase *mechanical sympathy* comes from motorsport. Sir Jackie Stewart, three-time Formula One world champion, is widely quoted for the idea that you don't have to be an engineer to be a racing driver, but you do need mechanical sympathy: an understanding of how the car works so you can get the best out of it and not destroy it. A driver who understands what the gearbox, the tyres, and the engine are actually doing brakes, shifts, and corners in a way that cooperates with the machine. A driver without that understanding fights the car and loses — either lap time or the whole engine.

Martin Thompson, one of the architects of the LMAX Disruptor, borrowed the phrase for software and made it a term of art among performance engineers. The Disruptor is the canonical demonstration. LMAX ran a retail financial-exchange matching engine that needed to process on the order of millions of orders per second on a single thread with predictable, low latency. The team found that conventional concurrency primitives — bounded queues guarded by locks — were fundamentally at odds with how the hardware works. Their replacement, the Disruptor, is a pre-allocated ring buffer coordinated by a small set of sequence counters. It has almost nothing to do with clever algorithms in the big-O sense and almost everything to do with respecting the machine: keep data in contiguous memory so the prefetchers win, avoid the cache-line ping-pong that locks and shared counters cause, avoid allocation so the garbage collector stays quiet, and let a single writer own a cache line so no coherence traffic is needed to update it. The result outperformed the queue-based design by a large margin not because it did less algorithmic work, but because it did far less *mechanical* work — fewer cache misses, less coherence traffic, less garbage.

That is the whole idea. **Your high-level code describes what to compute. The hardware decides how fast that computation actually runs, and it decides based on properties your source code never mentions:** where the data lives in the memory hierarchy, whether it is laid out contiguously, whether the branch predictor can guess your control flow, whether two threads are quietly fighting over the same cache line. Mechanical sympathy is the practice of holding a model of those properties in your head while you design, so that the machine and your code pull in the same direction.

For a backend engineer the stakes are not lap times. They are the three numbers you are actually judged on:

- **Latency** — how long a single request takes, and specifically the tail (p99, p999), where hardware effects dominate.
- **Throughput** — how many requests a node sustains, which sets how many nodes you need.
- **Cost** — nodes times price, plus power, plus the human cost of chasing performance regressions you don't understand.

All three are downstream of hardware behavior. A service that touches memory in a cache-hostile pattern doesn't return a wrong answer; it returns the right answer using two to ten times more machine, which shows up as a bigger fleet, a fatter cloud bill, and a p99 that violates your SLO under load. None of that is visible in the source diff. It is all mechanical sympathy — or the lack of it.

The numbers every engineer should know

In 2009 Jeff Dean began circulating, and Peter Norvig popularized, a table titled "Latency Numbers Every Programmer Should Know." It is the single most useful thing a performance-minded engineer can memorize, because it collapses an intimidating machine into a short list of magnitudes. The exact figures are era-dependent and hardware-dependent — they have shifted as CPUs, DRAM, SSDs, and networks have improved — so treat every number below as *approximate and order-of-magnitude*. The point is never the third significant digit; the point is the *ratio between tiers*, and those ratios are remarkably stable.

Here is a representative version, normalized to nanoseconds, for a modern server-class machine circa the mid-2020s:

Operation	Approx. latency	In nanoseconds	Relative to L1 (~1 ns)
CPU register access / one clock cycle	~0.3 ns	~0.3	~0.3×
L1 cache reference	~1 ns	1	1×
Branch mispredict	~3 ns	3	~3×
L2 cache reference	~4 ns	4	~4×
L3 cache reference	~10–20 ns	~15	~15×
Main memory (DRAM) reference	~100 ns	100	~100×
Read 1 MB sequentially from DRAM	~10–20 µs	~15,000	~15,000×
NVMe SSD random 4 KB read	~10–100 µs	~50,000	~50,000×
Read 1 MB sequentially from SSD	~50–200 µs	~100,000	~100,000×
Round trip within a datacenter	~0.5 ms	~500,000	~500,000×
Disk (spinning HDD) seek	~5–10 ms	~7,000,000	~7,000,000×
Read 1 MB sequentially from HDD	~5–20 ms	~10,000,000	~10,000,000×
Round trip, same continent	~10–50 ms	~30,000,000	~30,000,000×
Round trip, cross-continent (e.g., CA↔Europe)	~150 ms	~150,000,000	~150,000,000×

Read the last column again. From an L1 hit to a cross-continent round trip is a factor of roughly one hundred million — and if you include a sub-nanosecond register access at the top, the full span is close to **nine orders of magnitude**. That span is the most important fact in this book.

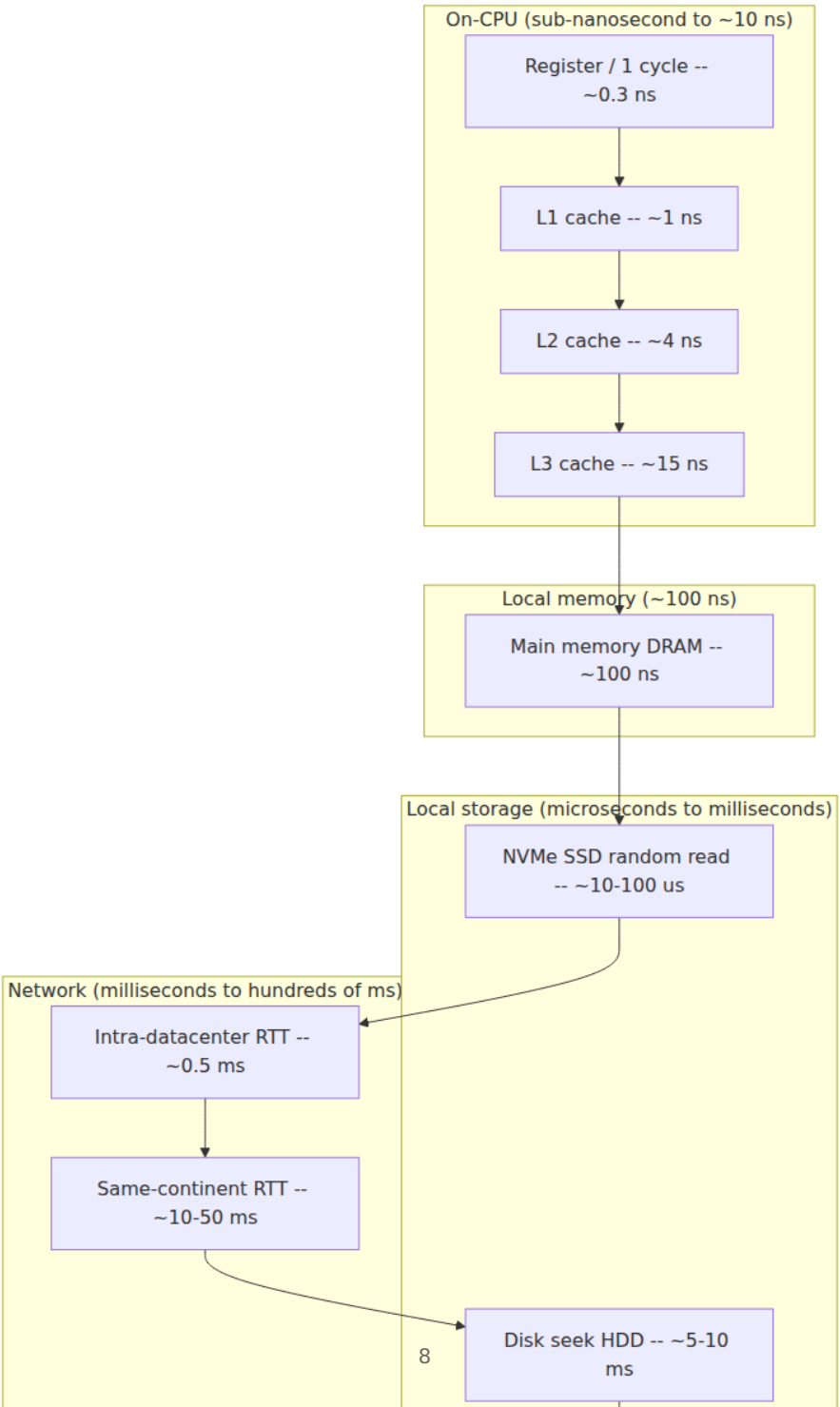
It means that the dominant term in any latency budget is almost always *which tier of the hierarchy you touched*, not how much work you did once you got there. Sorting a thousand elements that are already in L1 is free compared to the single DRAM miss you took to find the pointer to them. A function that is algorithmically optimal but chases pointers across DRAM will lose to a "worse" algorithm that stays in cache.

A useful trick for building intuition is to scale everything up by a billion, turning nanoseconds into seconds — human time:

- **L1 cache (1 ns → 1 second):** grabbing something off your desk.
- **DRAM (100 ns → ~2 minutes):** walking to another room to fetch it.
- **NVMe SSD random read (~50 µs → ~14 hours):** an overnight errand.
- **Intra-DC round trip (0.5 ms → ~6 days):** shipping something across the country.
- **Disk seek (~10 ms → ~4 months):** a long, slow logistics operation.
- **Cross-continent round trip (150 ms → ~5 years):** a multi-year expedition.

When you internalize that a network hop is *days* and a cross-region call is *years* on the scale where a cache hit is *one second*, you stop being surprised that a service which "just adds one more downstream call" doubled its p99. The call didn't add compute; it teleported the request several tiers down the latency hierarchy and back.

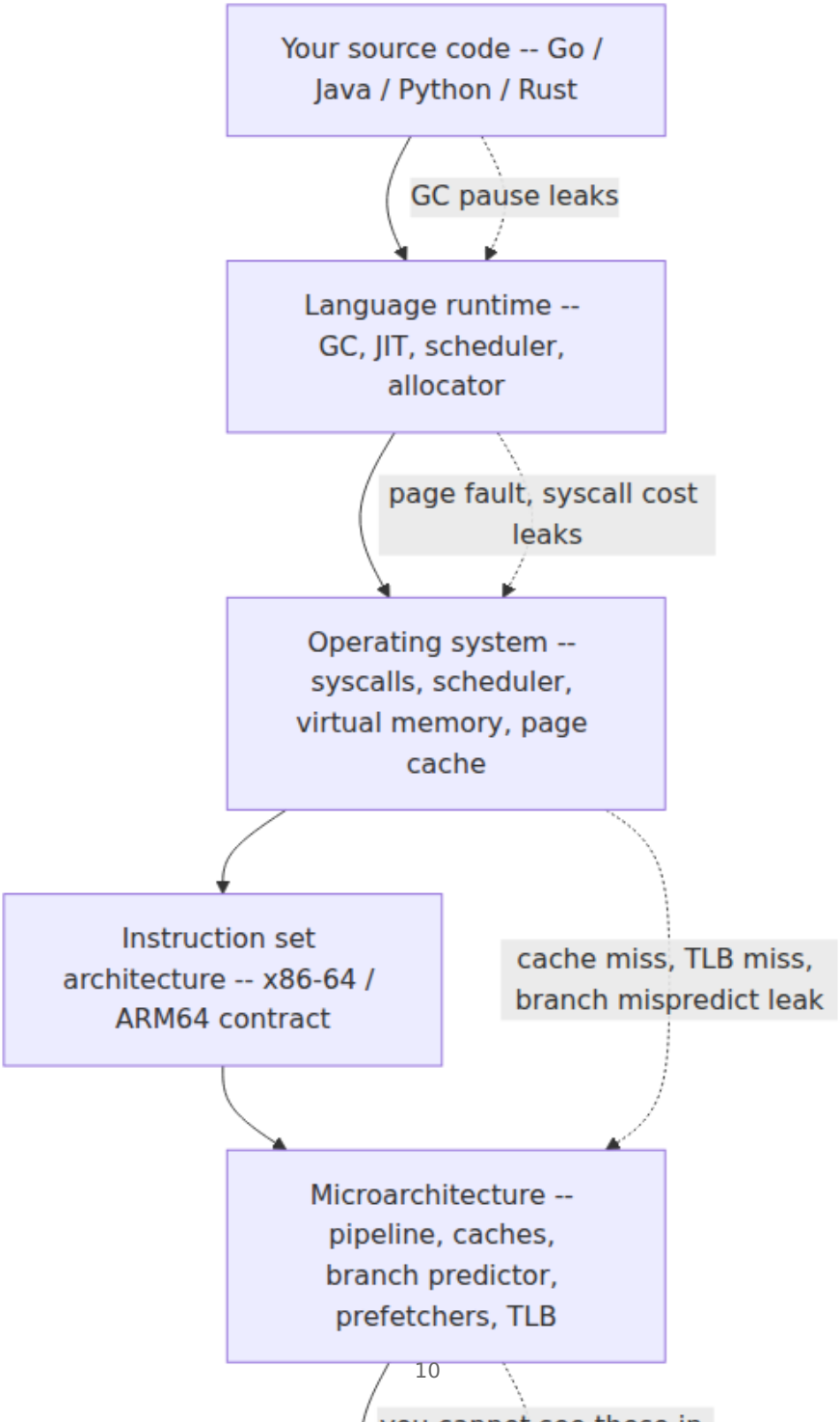
The following diagram renders the same hierarchy as a log-scale ladder. Each rung is roughly an order of magnitude below the one above it; the vertical distance is the whole game.


On-CPU (sub-nanosecond to ~10 ns)
Register / 1 cycle -- ~0.3 ns
L1 cache -- ~1 ns
L2 cache -- ~4 ns
L3 cache -- ~15 ns
Local memory (~100 ns)
Main memory DRAM -- ~100 ns
Local storage (microseconds to milliseconds)
NVMe SSD random read -- ~10-100 us
Network (milliseconds to hundreds of ms)
Intra-datacenter RTT -- ~0.5 ms
Same-continent RTT -- ~10-50 ms
Disk seek HDD -- ~5-10 ms

Two caveats keep this honest. First, the numbers *move*: L3 sizes and latencies vary across microarchitectures, NVMe has compressed the storage tier by orders of magnitude relative to the HDDs the original table assumed, and datacenter networks have gotten faster. Always re-measure on your actual hardware. Second, the ladder is not perfectly monotonic in every dimension — a large sequential SSD read can beat a scattered set of DRAM-missing random accesses in *throughput* even though DRAM wins on *per-access latency*. That tension between latency and throughput is a theme we return to below.

The abstraction stack, and where it leaks

You write in Go, Java, Python, Rust, or C++. Beneath that source is a tower of abstractions, each of which exists to let you *not* think about the layer below it:


Your source code -- Go / Java / Python / Rust
GC pause leaks
Language runtime -- GC, JIT, scheduler, allocator
page fault, syscall cost leaks
Operating system -- syscalls, scheduler, virtual memory, page cache
Instruction set architecture -- x86-64 / ARM64 contract
cache miss, TLB miss, branch mispredict leak
Microarchitecture -- pipeline, caches, branch predictor, prefetchers, TLB

Each arrow downward is an abstraction doing its job: your source doesn't name registers, the runtime hides the allocator, the OS hides physical memory behind virtual addresses, the ISA hides the pipeline behind a clean sequential-execution contract, the microarchitecture hides the wild reality of out-of-order speculation behind that contract. This is a triumph. It is why you can be productive at all.

But abstractions leak, and they leak specifically in the dimension they were *not* designed to hide: **performance**. The ISA promises that instructions execute one after another with well-defined results. It says nothing about *how long* each takes — and the answer varies by orders of magnitude depending on state that the abstraction deliberately conceals. Consider what is invisible in your source code yet decisive in production:

- **A cache miss.** `user.balance` is a field access. If `user` is in L1, it costs ~1 ns. If it was evicted and lives in DRAM, the *same source line* costs ~100 ns — a 100× difference the syntax cannot express.
- **A branch misprediction.** An `if` is one keyword. If the CPU's branch predictor guesses right, the branch is nearly free because the pipeline stays full; if it guesses wrong on a data-dependent branch, the pipeline flushes and you eat ~10–20 cycles. The source looks identical in both cases.
- **A page fault.** A memory read is a memory read — until the page isn't resident, and the OS takes over to fetch it (from the page cache, or worse, from disk), turning a nanosecond-scale access into a microsecond or millisecond stall.
- **A syscall.** `read()` and `write()` look like function calls. They are privilege-level transitions that cost far more than an ordinary call, flush microarchitectural state, and may block. The cost of crossing the user/kernel boundary is a recurring theme in high-performance I/O (it is why `io_uring`, batching, and kernel-bypass networking exist).
- **A garbage-collection pause.** In a managed runtime, an innocuous allocation can trigger a collection. Your code did not "call the GC"; the runtime did, on its own schedule, possibly pausing your thread. GC pauses are one of the most common causes of backend tail latency, and — crucially — they interact with the memory hierarchy: a collector that walks the heap evicts your working set from cache, so even a short pause leaves a cold-cache crater behind it.

None of these appear in a code review. All of them appear in a flame graph, a `perf` profile, or a latency histogram. This is the core reason the rest of this volume exists: the abstractions are excellent at hiding *correctness* details and terrible at hiding *performance* details, and performance is your job. Mechanical sympathy is, operationally, the skill of reasoning about the leaks — knowing which layer a symptom is coming from, and knowing which of your design choices controls it.

Why compute is cheap and moving data is expensive

There is a structural reason the latency table looks the way it does, and it drives almost every design decision in this volume: **arithmetic has gotten enormously faster than memory access, so modern machines are overwhelmingly bottlenecked on data movement, not computation.**

A single core can retire several instructions per cycle, and with SIMD it can perform dozens of floating-point operations per cycle. At a few GHz, that is tens to hundreds of billions of operations per second per core. Meanwhile a single DRAM access costs ~100 ns — during which that same core could have executed hundreds of instructions. This gap, sometimes called the *memory wall*, has widened for decades: compute throughput grew far faster than memory latency shrank. The entire cache hierarchy — L1, L2, L3, prefetchers, out-of-order execution — is elaborate machinery built for one purpose: to hide memory latency so the arithmetic units don't starve.

The practical consequence is a rule of thumb worth tattooing somewhere: **for most backend workloads, the question is not "how many operations does this do?" but "how far does the data have to travel, and how predictably?"** Two algorithms with identical big-O complexity can differ by an order of magnitude in wall-clock time purely because one respects the memory hierarchy and the other thrashes it. Big-O counts operations; it says nothing about the *constant factor* that data movement imposes, and at these ratios the constant factor is often the whole story.

The classic demonstration: array scan vs. linked-list traversal

Consider summing a million integers. In an array (or a Go slice, or a `std::vector`), the elements are contiguous in memory. The hardware

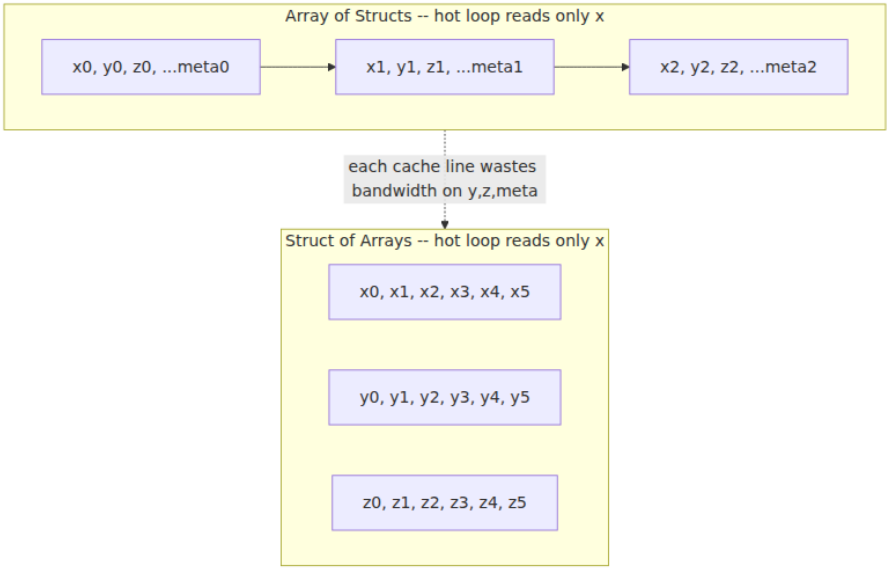
prefetcher recognizes the sequential access pattern and streams the next cache lines into L1 *before* you ask for them, so almost every access is an L1 hit. You are essentially bandwidth-limited, and DRAM bandwidth is large.

In a linked list, each node holds a value and a pointer to the next node, and those nodes may be scattered anywhere in memory (especially after a program has run for a while and the allocator has interleaved them with other objects). Each `node = node->next` is a pointer chase: you cannot compute the next address until the current node has arrived, so the prefetcher can't help and the accesses can't overlap. Every hop risks a cache miss that serializes behind ~100 ns of DRAM latency. Same operation count, same O(n), but the array scan can run *an order of magnitude faster* because it respects locality and the linked list defeats it. This is why performance-sensitive code overwhelmingly favors flat, contiguous structures over pointer-linked ones, and why "cache-friendly data structures" is a design discipline unto itself.

Array-of-structs vs. struct-of-arrays

The same principle governs how you lay out records. Suppose you have ten million particles, each with a position and a large bag of other fields, and a hot loop that reads only the position. The natural object-oriented layout is an *array of structs* (AoS): each particle's fields sit together, so the array in memory alternates position, other-stuff, position, other-stuff. Memory moves in cache-line units (typically 64 bytes). When your loop reads one particle's position, the CPU pulls in a whole cache line — most of which is the *other* fields you don't need. You pay full memory bandwidth to move data you immediately discard, and your effective bandwidth for the field you care about collapses.

The *struct of arrays* (SoA) layout stores all positions contiguously in one array and the other fields in separate arrays. Now the position loop touches only position data; every byte in every cache line is useful; the prefetcher streams perfectly; and if you later want to vectorize the loop with SIMD, the data is already in the shape the vector units want.



Neither layout is universally correct — if your access pattern reads *all* fields of one record at a time, AoS is better because it keeps that record's fields on one line. The point is that the *right* layout is the one that matches your *access pattern* to the cache line, and you cannot make that call without a mechanical model of how memory moves. Chapter 3 (memory hierarchy) and Chapter 7 (SIMD) develop this in depth.

Sequential vs. random I/O

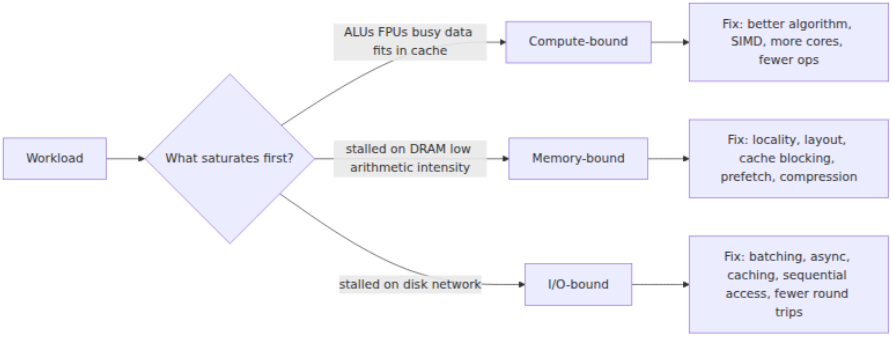
The same locality logic extends all the way down the hierarchy to storage. On a spinning disk, sequential reads avoid the mechanical seek that dominates random-access latency, so sequential throughput can exceed random throughput by two to three orders of magnitude. NVMe SSDs have no heads to move and are far more forgiving of random access, but even they strongly prefer sequential, aligned, batched I/O because of how flash pages, the flash translation layer, and internal parallelism work (Chapter 5). This single fact — sequential beats random — shapes the design of nearly every high-performance storage system you use. Log-structured storage engines (LSM-trees in RocksDB, Cassandra, and their kin) exist largely to turn random writes into sequential ones. Write-ahead logs are sequential by design. Kafka's throughput comes in significant part from treating the disk as an append-

only sequential log and letting the OS page cache do the rest. Locality is not a CPU-only concern; it is the same idea repeated at every tier.

Compute-bound, memory-bound, I/O-bound: the roofline way of thinking

Given that data movement usually dominates, how do you reason about what is actually limiting a specific workload? The most useful framework is the *roofline model*, introduced by Williams, Waterman, and Patterson. Its full form plots attainable performance against *arithmetic intensity* — the ratio of compute operations to bytes moved from memory — and shows that any kernel is bounded by one of two ceilings: a horizontal *compute roof* (the machine's peak FLOP/s) or a slanted *memory-bandwidth roof* (peak bytes/s times arithmetic intensity). Where your workload's intensity falls determines which roof you hit.

You don't need the formal plot to use the idea. The practical version is a triage question: **for this workload, which resource saturates first — the arithmetic units, the memory system, or I/O?** Every workload is dominated by one of three regimes:

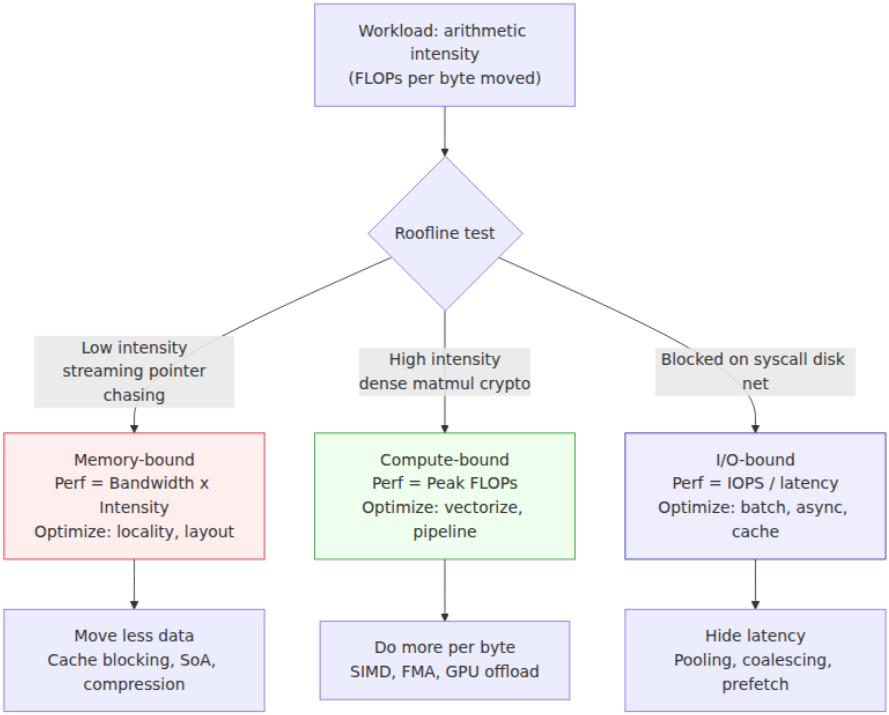


- **Compute-bound.** The arithmetic units are the bottleneck; the working set fits in cache so memory isn't the limit. Cryptographic hashing, compression, media transcoding, and dense numeric kernels often live here. Remedies are algorithmic (do fewer operations), parallel (more cores), or vectorized (SIMD, Chapter 7).
- **Memory-bound.** The cores spend most of their time *stalled* waiting for DRAM; the arithmetic intensity is low, so you move a lot of bytes per operation. A great deal of ordinary backend code — traversing large in-memory structures, hash-map lookups over big tables, JSON

parsing, pointer-chasing object graphs — is memory-bound. Adding cores barely helps because they all queue on the same memory system; the fixes are the locality and layout techniques above, plus compression to move fewer bytes.

- **I/O-bound.** The bottleneck is storage or, most often for services, the *network*. The CPU is mostly idle, waiting on a database, a cache tier, or a downstream RPC. Remedies are batching, asynchrony/pipelining, caching to avoid the trip entirely, sequential access patterns, and — the highest-leverage of all — *eliminating round trips*.

The reason this triage matters is that **the three regimes have disjoint fixes, and applying the wrong fix wastes effort or makes things worse.** Adding CPU cores to a memory-bound service buys almost nothing because the new cores just deepen the queue at the memory controller. Micro-optimizing a hot function in an I/O-bound service is pure motion — the CPU was idle anyway. Before you optimize, you must know which roof you are under, and the only reliable way to know is to measure: a profiler and the CPU's performance counters (cache-miss rate, stalled cycles, IPC) will tell you whether you are starved for compute, for memory bandwidth, or for data from another machine.



Why it matters more at scale, not less

A reasonable objection at this point: modern hardware is fast and cheap, most services are I/O-bound on the network anyway, and premature micro-optimization is a genuine sin. All true. At one request per second, nobody should care about a cache miss, and reaching for `perf` to shave nanoseconds off a rarely-called handler is malpractice. So why does a backend engineer need any of this?

Because **the distributed-systems setting inverts the usual economics of micro-optimization.** The reasons small inefficiencies are usually ignorable — they're rare, they're per-process, the machine is idle anyway — all evaporate when the same code runs on thousands of nodes, millions of times per second, under a latency SLO. Three multipliers turn hardware behavior from a curiosity into a first-order business concern.

Constant factors multiply across the fleet

Suppose a poor memory-access pattern makes your service's hot path 2× slower than it needs to be — a very ordinary amount of "wrong data layout." At low traffic this is invisible. But if that service runs a fleet of, say, 2,000 nodes to handle its load, then fixing the 2× lets the same traffic run on ~1,000. That is a thousand machines of compute, RAM, power, cooling, rack space, and network — recurring, every month, forever. The constant factor that big-O analysis throws away is precisely the quantity that, multiplied by fleet size, shows up on the invoice. At hyperscale, *backend performance is hardware efficiency*, and hardware efficiency is money and energy. A 10% CPU reduction on a large service is a widely-shared, career-making result at big companies precisely because 10% of a giant fleet is enormous.

Utilization amplifies latency non-linearly

Constant factors don't just cost machines; they cost *tail latency*, and non-linearly. Queueing theory tells us that as a server's utilization ρ approaches 1, its queueing delay grows roughly as $1/(1-\rho)$ — it heads to infinity as the system saturates. A workload that runs your nodes at 80% utilization has a very different, and far worse, latency tail than one that runs them at 40%, even though average service time is unchanged. So an inefficiency that raises per-request work doesn't just cost more nodes; if you *don't* add those nodes, it pushes every existing node up its utilization curve into the region where the tail explodes. Hardware efficiency and latency SLOs are the same problem viewed from two angles.

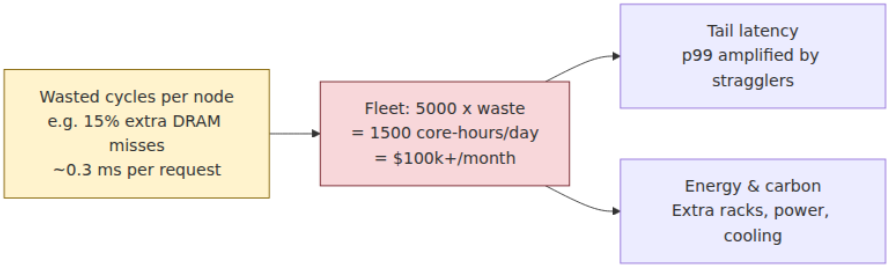
Tail latency is hardware-effect-dominated

This is the deepest reason the volume exists. At scale, you are not judged on average latency; you are judged on the tail — p99, p999 — because a request to a service that fans out to many backends is only as fast as its *slowest* dependency. Jeff Dean and Luiz Barroso made this precise in "The Tail at Scale": if each backend has a 1-in-100 chance of a slow response, a request that touches 100 backends will *almost always* hit at least one slow one, so the system-level p99 is governed by the per-node p999 and worse. Rare per-node slowness becomes common system-level slowness.

And what causes those rare per-node slow responses? Overwhelmingly, *hardware and runtime effects*:

- A **GC pause** that stalls the thread — and, as noted, evicts the working set from cache so the post-pause requests run cold and slow too.
- A **NUMA** effect (Chapter 6) where a thread migrated to a socket whose local memory doesn't hold its data, so every access pays the remote-memory penalty.
- **Cache and TLB** pressure from a noisy neighbor on the same host evicting your lines.
- A **page fault** or a cold **page cache** turning an expected memory read into a disk read.
- **False sharing** (Chapter 8) where two threads' unrelated variables land on the same cache line and ping-pong it between cores under the coherence protocol.

These are exactly the leaks from the abstraction stack, and they are exactly what dominates the tail. You cannot debug a p999 problem with a model of the machine that stops at the source code. The tail lives in the microarchitecture, and reasoning about it *requires* the mechanical model this volume builds.



The distributed-systems lens: the network is just another tier

The unifying insight for a distributed-systems engineer is that **the latency hierarchy does not stop at the edge of the machine — the network is simply its slowest tiers, and the same locality thinking spans the entire ladder from register to region.**

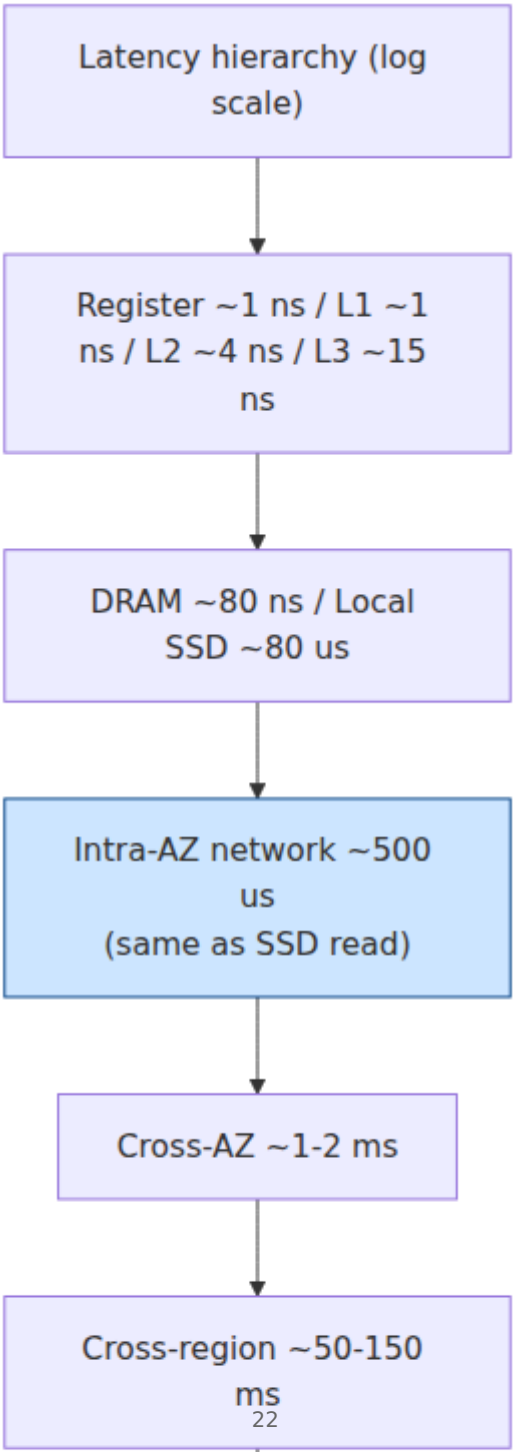
Look again at the latency table. Cache → DRAM → SSD → intra-DC network → cross-region network is one continuous ladder of increasing cost. A cross-region round trip (~150 ms) sits below a disk seek (~10 ms) sits below an SSD read (~50 µs) sits below a DRAM access (~100 ns) sits below a cache hit (~1 ns) on the same logarithmic scale. The mental discipline you apply to cache locality — *keep the data you need close, touch it sequentially, avoid unnecessary trips to slower tiers* — is the identical discipline behind every distributed-systems performance pattern you already know:

- **Caching tiers** (a local cache, then Redis/Memcached, then the database) are literally a memory hierarchy built out of network hops instead of silicon: each layer trades capacity for latency exactly as L1/L2/L3/DRAM do.
- **Data locality / colocation** — putting compute next to data, keeping a request's data in one region, sharding so a request hits one shard — is cache locality at datacenter scale.
- **Batching and pipelining** RPCs to amortize round trips is the network version of amortizing DRAM latency by streaming cache lines; both are attacks on the same latency-per-access problem.
- **Avoiding chatty protocols** — collapsing N round trips into one — is the network's version of avoiding pointer chasing. Every avoidable round trip is a jump several tiers down the ladder, and, as the human-scaled numbers showed, a cross-region trip is *years* where a cache hit is a *second*.

Two further points make the lens concrete for a fleet operator:

Per-node efficiency multiplies globally. Every constant-factor improvement on one node — better layout, fewer cache misses, less GC — is multiplied by the node count and by the request rate. At fleet scale, per-node mechanical sympathy is a *global* cost, energy, and carbon lever, not a local one. This is why the largest operators invest heavily in profiling entire fleets continuously (Google's fleet-wide profiler and the "Datacenter Tax" analysis found that a double-digit percentage of all cycles across the fleet went to a handful of low-level primitives — memory allocation, memory movement, hashing, compression, serialization, RPC — the "datacenter tax." Shaving those primitives is worth thousands of machines.)

Capacity planning is hardware-behavior modeling. When you answer "how many nodes do we need for peak?" you are, whether you say so or not, modeling hardware: how many requests a node sustains before its memory system saturates or its tail latency crosses the SLO. Get the mechanical model wrong and you over-provision (burning money) or under-provision (missing the SLO under load). Good capacity planning is applied mechanical sympathy plus queueing theory.


Latency hierarchy (log scale)
Register ~1 ns / L1 ~1 ns / L2 ~4 ns / L3 ~15 ns
DRAM ~80 ns / Local SSD ~80 us
Intra-AZ network ~500 us
(same as SSD read)
Cross-AZ ~1-2 ms
Cross-region ~50-150 ms

The engineer's mental model

Pulling the chapter together, here is the working model to carry into the rest of the volume and into your day job:

1. **Know the hierarchy and its magnitudes.** Keep the latency ladder in your head to the nearest order of magnitude. When you add a step to a request, ask which tier it lands on — because *which tier* dominates, not how clever the step is.
2. **Assume data movement dominates, not compute.** Modern cores are starved for data. Reason about a workload in terms of bytes moved and how far, and classify it as compute-, memory-, or I/O-bound before you touch it. The fixes for the three are disjoint.
3. **Design for locality and sequential access — at every tier.** Prefer contiguous, flat data structures over pointer-linked ones. Match your data layout (AoS vs. SoA) to your access pattern and the cache line. Turn random access into sequential where you can. Then apply the same thinking to storage and to the network: colocate, batch, cache, and eliminate round trips.
4. **Respect the abstractions — but know where they leak.** The stack from source to circuits is what makes you productive; don't fight it needlessly. But when performance is the requirement, remember that the abstractions hide correctness, not cost. A cache miss, a branch mispredict, a page fault, a syscall, and a GC pause are all invisible in the source and decisive in the profile.
5. **You cannot intuit modern hardware — measure.** Out-of-order execution, speculation, prefetching, and multi-level caching make performance genuinely counterintuitive; expert guesses about what is slow are wrong a large fraction of the time. Profile with real tools (`perf`, flame graphs, hardware performance counters for cache-miss rate, IPC, and stalled cycles) on representative workloads. Mechanical sympathy tells you *what to look for* and *what your options are*; it does not replace measurement, it makes measurement legible.

The two halves of that last point matter equally. Without a mechanical model, a profile is a wall of numbers you can't act on — you see a high cache-miss rate but don't know it points at your data layout. Without measurement, a mechanical model is a way to be confidently wrong at

higher resolution. The engineer who is dangerous, in the good sense, has both.

Two hands-on measurements make this concrete on any Linux server. Run them now; they take seconds and they anchor the abstractions above in numbers you produced yourself.

False sharing, seen in `perf stat`. The canonical hardware leak that source cannot express is two threads hammering *different* variables that happen to share a cache line. The coherence protocol ping-pongs the line between cores, and throughput collapses.

```
// false_sharing.c — compile: gcc -O2 -pthread false_sharing.c -o
false_sharing
#include <pthread.h>
#include <stdint.h>

// Case A: two counters on the SAME cache line (false sharing)
struct { uint64_t a; uint64_t b; } shared;          // a and b likely
same 64-B line

// Case B: padded so each counter owns its line — swap to this to see
the fix:
// struct { uint64_t a; char pad[56]; uint64_t b; } shared;
// (56 = 64 - sizeof(uint64_t); or use alignas(64) per field)

void *inc_a(void *_) { for (long i = 0; i < 100000000L; i++) shared.a+
+; return NULL; }
void *inc_b(void *_) { for (long i = 0; i < 100000000L; i++) shared.b+
+; return NULL; }
int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, inc_a, NULL);
    pthread_create(&t2, NULL, inc_b, NULL);
    pthread_join(t1, NULL); pthread_join(t2, NULL);
    return 0;
}
```

```
# Same line (false sharing) — high coherence traffic
$ perf stat -e cache-misses,cache-references,cycles,instructions,LLC-
load-misses ./false_sharing 2>&1 | tail -n 12
# ── trimmed output (Intel Xeon, 2 threads, 100M increments each)
──────────

     1,842,103,411      cache-misses              #   38.2% of all
cache refs
     4,821,905,203      cache-references
    12,403,812,009      cycles
     3,102,445,871      instructions              #    0.25  insn per
cycle
       891,204,118      LLC-load-misses
       4.82 seconds time elapsed

# Padded to separate lines — same work, coherence traffic gone
$ perf stat -e cache-misses,cache-references,cycles,instructions,LLC-
load-misses ./false_sharing_padded 2>&1 | tail -n 12
# ── trimmed output (same machine, same work)
──────────────────────────

        42,103,812      cache-misses              #    2.1% of all
cache refs
     2,011,482,093      cache-references
     2,901,445,201      cycles
     3,098,102,441      instructions              #    1.07  insn per
cycle
        11,204,118      LLC-load-misses
       0.92 seconds time elapsed
# IPC 0.25→1.07 and ~4× fewer cycles: the only change was layout.
Chapter 8
# dissects why and how to detect it with perf c2c / toplev.
```

What to read: `cache-misses` / `LLC-load-misses` collapse by ~40×, cycles by ~4×, and instructions-per-cycle recovers from 0.25 (stalled on coherence) to >1. The fix is pure layout — padding or `alignas(64)` — not fewer operations. This is the constant factor that fleet-wide profiling (Kanev et al., \"Profiling a Warehouse-Scale Computer\") repeatedly finds dominating real services.

Memory latency, seen in one line. `lat_mem_rd` from lmbench (or a single `fio` rand-read) measures the hierarchy you just memorized. No setup beyond the package:

```
$ lat_mem_rd 32 512  # stride 32 B, up to 512 MB working set — prints
latency vs size
# ── trimmed output (lmbench 3, EPYC Milan, DDR4, single socket)
─────────────
"stride=32"
0.00049 1.02    # 0.5 KB working set — L1 hit
0.00391 1.15    # 4 KB — still L1
0.01562 3.8     # 16 KB — L2
0.06250 12.4    # 64 KB — L3 hit region
0.25000 18.1    # 256 KB — still L3
1.00000 78.5    # 1 MB — spilling to DRAM
4.00000 92.3    # 4 MB — DRAM
32.0000 104.1   # 32 MB — DRAM (TLB effects visible)
128.000 112.7   # 128 MB — DRAM
# First column: working-set MB. Second: load latency (ns). Steps at ~32
KB,
# ~512 KB, ~16 MB are L1→L2→L3→DRAM transitions. Compare your machine's
steps
# to the table above; the absolute numbers move, the ladder shape does
not.

# Equivalent quick check with fio (if lmbench not installed):
$ fio --name=randread --rw=randread --bs=4k --size=1G --runtime=5 --
time_based \
      --iodepth=1 --direct=1 --filename=/dev/nvme0n1 --output-format=js
on 2>&1 \
  | python3 -c "import json,sys; d=json.load(sys.stdin); j=d['jobs'][0]
['read']; print(f\"IOPS={j['iops']:.0f}  p50 lat={j['clat_ns']
['percentile']['50.000000']/1000:.0f}µs  p99={j['clat_ns']
['percentile']['99.000000']/1000:.0f}µs\")"
IOPS=184203  p50 lat=38µs  p99 lat=71µs
# Random 4 KB over NVMe: ~38 µs median — place it on the hierarchy and
note the
# gap to DRAM (~100 ns) and to a cross-region RPC (~150 ms).
```

Both snippets are runnable as written. The `perf stat` counters exist on any modern x86-64 or ARM64 Linux with `perf` installed (`apt install linux-tools-generic` / `yum install perf`); on VMs without PMU access, `cache-misses` may read zero — use `perf stat -e task-clock,cycles` as a fallback. `lat_mem_rd` is `apt install lmbench`; `fio` is `apt install fio`.

Roadmap to the volume

The rest of Volume 1 turns each teaser in this chapter into a working understanding. Read it in order for a build-up from silicon to systems, or jump to the chapter that matches the roof you're currently under.

- **Chapter 2 — The Modern CPU: Pipelines, Out-of-Order, Speculation.** Why a CPU is not the sequential machine the ISA pretends it is, how pipelining and out-of-order execution hide latency, and why branch prediction and speculation both give you your performance and, occasionally, take it (and your security) away.
- **Chapter 3 — The Memory Hierarchy and Caches.** The heart of the volume: cache lines, associativity, the levels, prefetchers, and the locality principles that make the AoS/SoA and array/linked-list differences from this chapter concrete and quantifiable.
- **Chapter 4 — Cache Coherence and Hardware Memory Consistency.** What actually happens when multiple cores share data: the coherence protocol, memory ordering, and the hardware reality underneath the memory models your concurrent code relies on.
- **Chapter 5 — Storage Hardware: HDD, SSD, NVMe, and Persistent Memory.** Why sequential beats random all the way down, how flash and the FTL really behave, and what NVMe and persistent memory change about storage-engine design.
- **Chapter 6 — Multi-Socket Systems and NUMA.** When "main memory" stops being uniform, why a thread's socket affinity moves its latency by a large factor, and how NUMA shapes the tail.
- **Chapter 7 — Data Parallelism: SIMD, Vectorization, and GPUs for Backend.** Getting many operations per instruction, why data layout (hello again, SoA) determines whether vectorization is even possible, and when a GPU belongs in a backend.
- **Chapter 8 — Performance Anti-Patterns: False Sharing, Branch Misprediction, Cache Thrashing.** A field guide to the specific ways well-intentioned code fights the hardware, with how to spot each in a profile.
- **Chapter 9 — Number Representation and Floating Point.** How integers and IEEE-754 floats really behave, the correctness and performance traps in each, and why "just use a float" is sometimes a bug.

- **Chapter 10 — Hardware Support for Virtualization and Isolation.** The hardware underneath VMs and containers, what isolation actually costs in mechanical terms, and how virtualization reshapes every layer above it.

Key takeaways

- **Mechanical sympathy** — the term from Jackie Stewart, brought to software by Martin Thompson and demonstrated by the LMAX Disruptor — is the practice of writing software that works with the hardware's grain. For a backend engineer, latency, throughput, and cost are all ultimately set by hardware behavior your high-level code hides.
- The **latency hierarchy** spans roughly nine orders of magnitude from a register/L1 access (~1 ns) to a cross-continent round trip (~150 ms). Treat the numbers as approximate and era-dependent, but know the *ratios* cold: which tier you touch dominates performance far more than how much work you do once you're there.
- Abstractions from source down to circuits hide **correctness details, not performance details.** Cache misses, branch mispredictions, page faults, syscalls, and GC pauses are all invisible in the source and decisive in the profile — the "leaks" are where performance actually lives.
- **Data movement, not compute, dominates.** Because arithmetic outran memory (the memory wall), most backend code is memory- or I/O-bound. Classify a workload as compute-, memory-, or I/O-bound (the roofline idea) before optimizing — the three regimes have disjoint fixes.
- **Design for locality and sequential access at every tier.** Contiguous over pointer-linked; layout matched to access pattern (AoS vs. SoA) and cache line; sequential over random I/O; and on the network, colocate, batch, cache, and cut round trips.
- Hardware behavior matters **more at scale, not less.** Constant factors multiply across the fleet into real money and energy; utilization amplifies latency non-linearly; and **tail latency (p99/p999) is dominated by hardware and runtime effects** — GC, NUMA, false sharing, cache pressure — per "The Tail at Scale."

- **The network is just the slowest tier** of the same latency ladder. Distributed caching, data locality, batching, and avoiding chatty protocols are the datacenter-scale versions of cache locality, prefetching, and avoiding pointer chasing.
- You **cannot intuit modern hardware — measure.** Mechanical sympathy tells you what to look for and what your options are; profilers and performance counters tell you where you actually are.

Further reading

- Jeff Dean and Peter Norvig, "Latency Numbers Every Programmer Should Know." Widely circulated slide/table; see Peter Norvig's "Teach Yourself Programming in Ten Years" (norvig.com/21-days.html) for the canonical figures, and the interactive "Latency Numbers Every Programmer Should Know" visualizations maintained by the community (e.g., colin-scott.github.io/personal_website/research/interactive_latency.html).
- Martin Thompson, "Mechanical Sympathy" blog (mechanical-sympathy.blogspot.com) and the LMAX Disruptor technical paper: Thompson, Farley, Barker, Gee, Stewart, "Disruptor: High Performance Alternative to Bounded Queues for Exchanging Data Between Concurrent Threads" (2011).
- Jeffrey Dean and Luiz André Barroso, "The Tail at Scale," *Communications of the ACM*, 56(2),
- The definitive treatment of why per-node hardware effects dominate system-level tail latency.
- Samuel Williams, Andrew Waterman, David Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, 52(4), 2009.
- Ulrich Drepper, "What Every Programmer Should Known About Memory" (2007). A long, deep, and still-relevant treatment of the memory hierarchy and its performance implications.
- Svilen Kanev et al., "Profiling a Warehouse-Scale Computer," *ISCA* 2015. The origin of the "datacenter tax" — the fleet-wide cost of low-level primitives like allocation, memcpy, hashing, and serialization.
- Luiz André Barroso, Urs Hölzle, Parthasarathy Ranganathan, *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*, 3rd ed., Morgan & Claypool, 2018.

- John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2017 — the standard reference underpinning the rest of this volume.
- Brendan Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed., Addison-Wesley, 2020, for the measurement discipline (`perf`, flame graphs, the USE method) this chapter insists on.

Chapter 2 — The Modern CPU: Pipelines, Out-of-Order, Speculation

What this chapter covers. Chapter 1 argued for mechanical sympathy: the machine has a grain, and code that runs with the grain is often an order of magnitude faster than code that fights it. This chapter opens the box and looks at the engine directly. A modern server core — a Graviton4 Neoverse V2, an AMD Zen 5, an Intel Redwood/Granite Rapids core — is a deeply pipelined, superscalar, out-of-order, speculative machine that bears almost no resemblance to the sequential fetch-execute loop most engineers carry in their heads. It fetches, decodes, and *renames* instructions; it executes them out of program order the instant their inputs are ready; it guesses the outcome of branches long before it knows them; and it retires results back into program order so that, from the outside, the illusion of sequential execution holds. Understanding this machine is what lets you reason about why one loop runs at four instructions per cycle and a functionally identical loop runs at one — and why, at fleet scale, that ratio is a line item on your cloud bill.

Learning goals:

- Trace an instruction through the classic five-stage pipeline and explain how pipelining buys throughput without reducing latency.
- Classify the three hazard families — structural, data, control — and state the mechanism by which each stalls a pipeline and how hardware mitigates it.
- Explain why IPC greater than one is possible (superscalar issue) and what caps instruction-level parallelism in real code.
- Describe the out-of-order engine's pieces — register renaming, the scheduler and reservation stations, execution ports, and the reorder

buffer — and how dataflow execution with in-order retirement reconciles speed with correctness.

- Explain branch prediction, the cost of a misprediction, and why data-dependent branches are expensive while predictable ones are nearly free.
- Reason about how the OoO engine hides memory latency, and what memory-level parallelism buys.
- Place CISC/RISC and micro-ops correctly, and know why the distinction rarely matters to backend throughput — but why ARM in the datacenter does.
- Understand SMT/Hyper-Threading as resource sharing, when it helps and when it hurts, and its security and capacity-planning implications for multi-tenant clouds.

The naive model, and why it is wrong

Most engineers' mental model of a CPU is a loop: fetch the instruction at the program counter, decode it, execute it, write the result, advance the program counter, repeat. This model is pedagogically useful and operationally false for any core you will actually deploy on. It is false in three compounding ways. The CPU does not finish one instruction before starting the next (it *pipelines*). It works on more than one instruction per stage (it is *superscalar*). And it does not execute instructions in the order you wrote them (it is *out-of-order* and *speculative*). Each of these is a throughput optimization layered on the previous, and each introduces a class of hazard that the hardware must paper over to preserve the illusion of sequential execution your program depends on.

The through-line of the entire chapter is a single tension: **program order is a correctness contract; execution order is a performance decision.** The core's job is to violate execution order as aggressively as it can get away with while making the outside world — memory, other cores, the architectural register file — believe program order was honored. Everything that follows is machinery for having it both ways.

Two definitions anchor the discussion. The **clock** is the core's heartbeat; a 3 GHz core ticks three billion times per second, and a *cycle* is one tick. **IPC** — instructions per cycle — is the average number of instructions the core completes (retires) per tick. Performance is, to first order,

```
instructions_per_second = clock_frequency × IPC
```

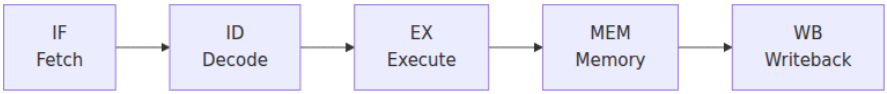
Frequency is bounded by physics and thermals and has barely moved in fifteen years (server parts sit around 2.5–3.8 GHz sustained). IPC is where architecture earns its keep, and where your code either cooperates or does not. A modern wide core can retire 4–8 instructions per cycle in the best case and well under 1 in the worst; the same silicon, the same frequency, a 5–10× swing driven entirely by how the code interacts with the pipeline, the predictors, and the caches.

The classic five-stage pipeline

Start with the textbook RISC pipeline, because every real machine is an elaboration of it. An instruction passes through five stages:

1. **Fetch (IF)** — read the instruction bytes from the instruction cache at the program counter.
2. **Decode (ID)** — interpret the bytes into an operation and operands; read source registers.
3. **Execute (EX)** — perform the ALU operation, or compute a memory address, or resolve a branch.
4. **Memory (MEM)** — for loads and stores, access the data cache.
5. **Writeback (WB)** — write the result back into the register file.

Without pipelining, each instruction occupies the entire datapath for all five stages before the next begins: five cycles per instruction, one instruction at a time. Pipelining overlaps them. Once instruction *i* leaves Fetch and enters Decode, instruction *i+1* enters Fetch. In steady state, all five stages are busy on five different instructions at once, and the pipeline retires one instruction per cycle.



The crucial insight is that pipelining does **not** reduce the *latency* of any single instruction — that instruction still takes five stages, and in a deeper pipeline it takes more. What pipelining buys is *throughput*: the rate at which instructions complete. This is the assembly-line analogy, and it is exact. A car still takes hours to build; the factory ships one every

few minutes because many cars are in progress at once. The distinction between latency and throughput recurs at every level of a distributed system — a request's latency and a service's throughput are governed by different things — and it starts here, in the pipeline.

Pipelining also lets each stage be simpler and therefore faster, which raises the clock. A stage that does one-fifth of the work can settle in one-fifth of the time, so the clock can tick roughly five times faster than an unpipelined design of the same logic. This is why pipeline *depth* and clock frequency are coupled, a relationship we return to below.

Hazards: where the assembly line jams

The assembly-line illusion holds only if every instruction is independent of the ones ahead of it. Real code is full of dependencies, and dependencies create **hazards** — situations where naively advancing the pipeline would produce a wrong answer. There are three families.

Structural hazards arise when two instructions need the same hardware resource in the same cycle — a single memory port that both a fetch and a load want, or a single divider that two divides contend for. The fix is either duplication (separate instruction and data caches, the "Harvard" split, so fetch and load never collide) or arbitration (one instruction waits). Modern cores spend enormous area duplicating resources specifically to eliminate structural hazards.

Data hazards arise when an instruction needs the result of one still in flight. The canonical case is *read-after-write* (RAW), a true dependency: `b = a + 1; c = b + 1` — the second add needs `b`, which the first add has not yet written back. In the naive pipeline the second instruction would read a stale `b` from the register file. The classic mitigation is **forwarding** (also called bypassing): wire the ALU output directly back to the ALU input so the result is available to the next instruction the cycle after it is computed, without waiting for writeback. Forwarding eliminates most RAW stalls but not all — a load feeding a dependent instruction still costs a cycle (the *load-use* delay), because the loaded value is not available until the MEM stage completes.

The other two data-hazard types are *false* dependencies and matter enormously later: **write-after-read** (WAR) — an instruction writes a register that an earlier instruction still needs to read — and **write-after-write** (WAW) — two instructions write the same register and the later

write must win. Neither reflects a real flow of data; both are artifacts of *naming* — the program reused an architectural register. Register renaming, discussed below, dissolves them entirely.

Control hazards arise from branches. Until a conditional branch resolves in EX, the fetch stage does not know which instructions to fetch next — the fall-through path or the branch target. In a five-stage pipeline that is a two-to-three cycle bubble per branch; in a fifteen-stage pipeline it is catastrophic. This is the hazard that motivates branch prediction, the second-biggest idea in the chapter.

Hazard type	Cause	Example	Primary mitigation
Structural	Two instructions need the same hardware unit in the same cycle	Fetch and load both want the single memory port	Duplicate resources (split I/D cache), multiple execution ports
Data — RAW (true)	Instruction reads a register a prior in-flight instruction writes	`add b,a,1; add c,b,1`	Forwarding/bypass; stall on load-use; OoO scheduling
Data — WAR/WAW (false)	Reuse of a register name creates an ordering constraint with no real data flow	`add a,b,c; add b,d,e` (WAR on `b`)	**Register renaming** (eliminates entirely)
Control	Next fetch address unknown until branch resolves	Any conditional branch or indirect jump	Branch prediction + speculative execution; flush on mispredict

Pipeline depth is a trade-off, not a free lunch

If splitting work into more, smaller stages raises the clock, why not make the pipeline fifty stages deep? Intel tried something close: the Pentium 4 "Prescott" core ran a ~31-stage pipeline chasing clock frequency, reached the high-3-to-4 GHz range in the mid-2000s — and was a thermal and performance disappointment relative to its shorter-pipeline successors. The reason is the cost of getting it wrong.

Every stage in flight when a branch mispredicts or an exception fires must be **flushed** — thrown away — and refilled from the correct path. A deeper pipeline has more instructions in flight, so each flush discards more work and takes more cycles to refill. Deep pipelines multiply the misprediction penalty. The Pentium 4's branch mispredict penalty was roughly 20–30+ cycles; the shorter Core-derived pipelines that replaced it paid far less per mispredict and clocked lower, and won decisively on real workloads.

Modern server cores settle around 14–20 pipeline stages — deep enough to sustain high clocks, shallow enough that a mispredict costs on the order of 15–20 cycles rather than 30+. The number is a designed compromise between frequency and flush cost, and it explains why raw GHz stopped being a useful proxy for performance around 2005. Depth is one axis of the "why has single-thread performance plateaued" story; the ILP wall, next, is the other.

Superscalar: IPC greater than one

A scalar pipeline retires at most one instruction per cycle — IPC ≤ 1. A **superscalar** core has multiple copies of the pipeline's back-end resources and can fetch, decode, issue, and retire *several* instructions per cycle. A wide modern core fetches on the order of 4–8 instructions per cycle, decodes similarly, and has 8–12+ execution units it can dispatch to in parallel. IPC greater than one is not exotic; it is the entire point of a modern core, and the reason a 3 GHz chip vastly outperforms a 3 GHz chip from 2004.

For a superscalar core to actually sustain IPC > 1, it must find, in each window of the instruction stream, multiple instructions that can execute simultaneously — instructions with no true data dependency between them. This property of a program is its **instruction-level parallelism (ILP)**. Consider:

```
a = x + y      # (1)
b = p + q      # (2)
c = a + b      # (3)
```

Instructions (1) and (2) are independent and can execute in the same cycle on two different ALUs. Instruction (3) depends on both and must wait. The best achievable here is two cycles for three instructions — IPC

1.5 — bounded by the dependency chain (1)/(2) → (3). No amount of hardware width executes (3) before its inputs exist. This is the fundamental limit: **ILP is a property of the program's dataflow graph**, and the critical path through that graph is a hard floor on latency that width cannot lower.

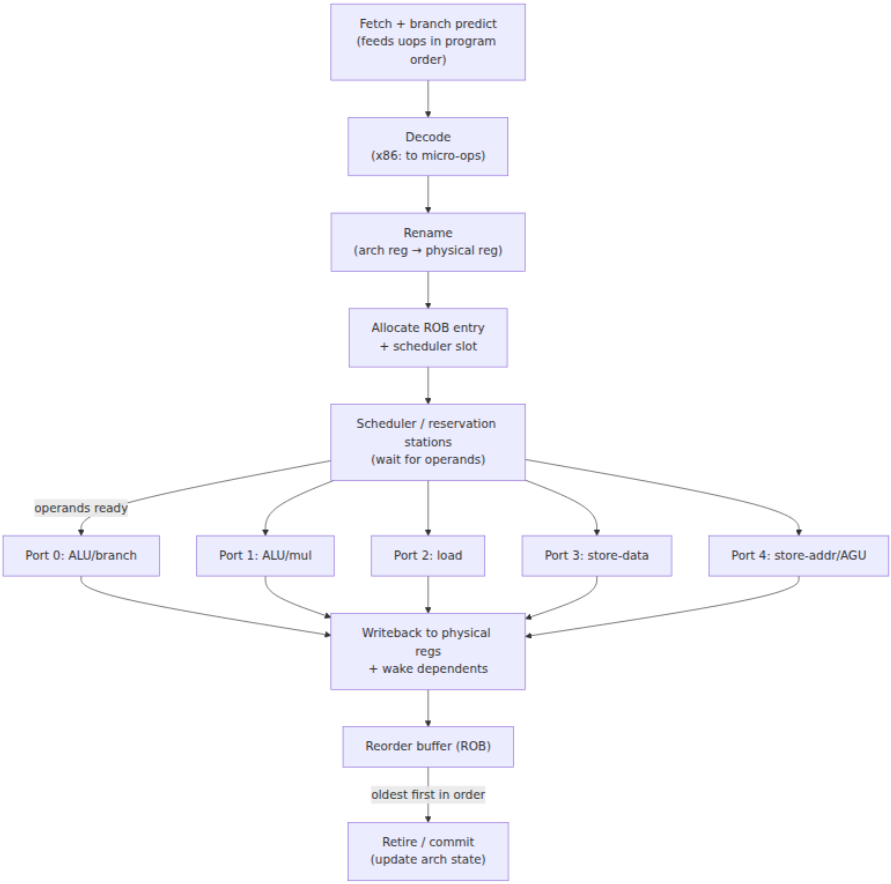
Real programs have limited, bursty ILP. Long dependency chains (pointer chasing, accumulator loops), unpredictable branches, and memory stalls all starve the execution units. Classic studies (Wall, *Limits of Instructional-Level Parallelism*, 1991, and much subsequent work) found that with realistic prediction and window sizes, sustainable ILP for general-purpose integer code sits in the low single digits — which is precisely why cores are 4–8 wide rather than 32 wide. Beyond a point, adding width yields nothing because the code has no more independent work to feed it. This is the **ILP wall**, and it is one of the reasons the industry pivoted from making single cores wider to putting many cores on a die — pushing the parallelism problem up to the software, where you now solve it with threads, goroutines, and, at the largest scale, a fleet of machines.

To extract even that low-single-digit ILP, an in-order superscalar is not enough. Consider a load that misses the L1 cache: in an in-order machine, that load and *every instruction behind it* stall for the tens or hundreds of cycles the miss takes to resolve — even instructions that do not depend on the load at all. The independent work is *right there* in the stream, but in-order execution cannot reach past the stalled instruction to run it. Recovering that work is the job of out-of-order execution.

Out-of-order execution: the heart of the machine

Out-of-order (OoO) execution is the single most important idea in this chapter and the one most often garbled. The core principle is **dataflow**: an instruction executes as soon as its input operands are available and an execution unit is free, *regardless of its position in program order*. Instructions that are ready run; instructions waiting on a slow load or a long dependency chain sit aside and let others pass. Then — and this is what preserves correctness — results are committed back to architectural state **in program order**. Execute out of order for speed; retire in order for correctness. That reconciliation is what the whole OoO apparatus exists to perform.

Walk through the pieces. The conceptual lineage is Tomasulo's algorithm (IBM System/360 Model 91, 1967); you do not need the full algorithm, but you must get the pieces right.



Front end — fetch, decode, in program order. Instructions enter in program order. On x86 they are decoded into one or more fixed-format internal operations called **micro-ops** (uops); on ARM/RISC-V the instructions are already close to uop-like but still get cracked into internal ops. The front end also runs the branch predictor (next section) to keep fetching down the predicted path without waiting for branches to resolve. Everything downstream operates on uops.

Register renaming — killing false dependencies. The architectural register file that the ISA exposes is small: x86-64 has 16 general-purpose

architectural registers, AArch64 has 31. Real cores have a much larger **physical register file** — hundreds of registers. At rename, each uop's destination architectural register is mapped to a *fresh* physical register, and its sources are mapped to whichever physical registers currently hold those architectural values. This single mechanism eliminates WAR and WAW hazards **completely**: because every write goes to a brand-new physical register, two instructions that both "write RAX" write different physical registers and can proceed independently; there is no false ordering constraint left. Only *true* (RAW) dependencies survive renaming, because those reflect real data flow — a source that maps to a physical register some earlier uop will produce. Renaming is why the tiny architectural register file of x86 is not the bottleneck you might fear: the visible names are few, but the hardware has plenty of storage and hands out fresh names constantly.

Dispatch to the scheduler / reservation stations. Renamed uops are allocated an entry in the reorder buffer (below) and a slot in the **scheduler** — a pool of waiting uops, historically organized as **reservation stations** attached to execution ports. A uop sits in its scheduler slot until all its source operands are ready. Operands become ready as producing uops finish and *broadcast* their results (the "wake-up"); a waiting uop watching for that physical register captures the value and becomes eligible to issue. This is the dataflow engine: readiness, not program position, gates execution.

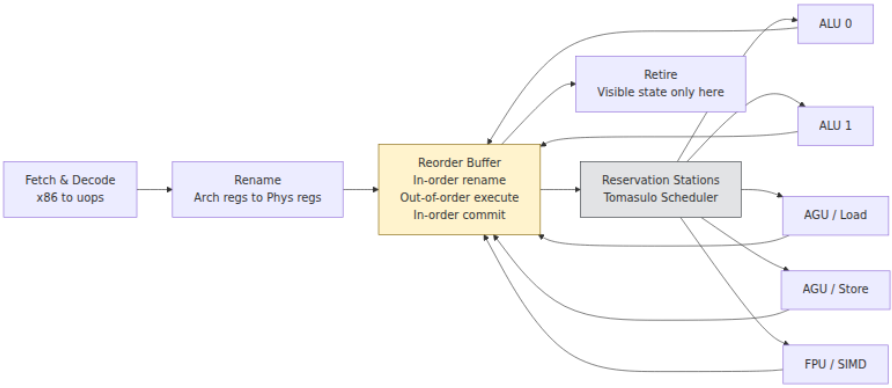
Execution ports and units. When a uop is ready and its port is free, the scheduler **issues** it to an **execution port**. A port is a dispatch lane to a cluster of functional units — integer ALUs, the multiplier, the load unit, the store-address and store-data units, the floating-point/SIMD units. A wide core has on the order of 8–12 ports; the mix (how many can do loads, how many can do FP multiply-add) directly shapes what code patterns saturate the machine. Two independent adds go to two ALU ports in the same cycle; that is superscalar issue in action. Port contention — for example, code that is all loads on a machine with two load ports — is a real and measurable throughput ceiling that a good profiler will surface.

Writeback and wake-up. A finished uop writes its result to its physical register and broadcasts that the register is ready, waking any scheduler entries waiting on it. Those dependents can now issue, possibly the very

next cycle. The chain of produce → wake → consume is the pipeline's inner clockwork.

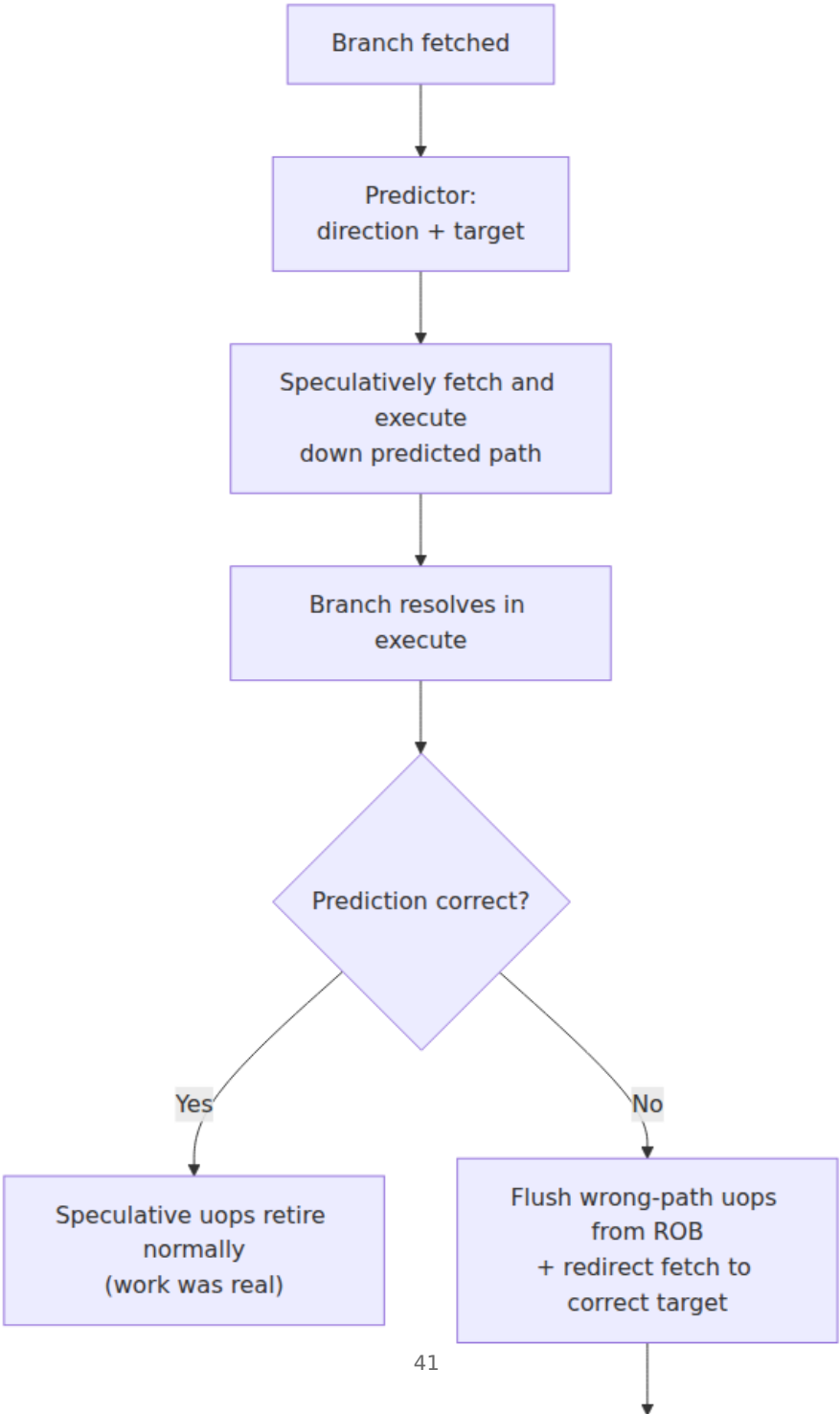
The reorder buffer (ROB) — in-order retirement. Every uop, at allocation, is assigned an entry in the **reorder buffer**, a FIFO in program order. Uops complete execution out of order and mark their ROB entry done, but they **retire** — commit their results to architectural state and become irrevocable — strictly from the head of the ROB, oldest first, in program order. Retirement is where the out-of-order world is folded back into the sequential contract. It is also where speculation is resolved: if an instruction at the ROB head raises an exception, or a branch turns out mispredicted, everything younger in the ROB is discarded before it can affect architectural state. The ROB is the ledger that lets the core run ahead recklessly and still present a clean, in-order history. Its size — a few hundred entries on modern cores — bounds the **instruction window**: how far ahead of a stalled instruction the core can look for independent work. A bigger window hides more latency (it can find more to do while a cache miss resolves) at the cost of area and power.

To make the dataflow idea concrete, reconsider the cache-missing load. In the OoO machine the load issues, misses, and goes off to memory — but it does *not* block the pipeline. The core keeps fetching, renaming, and dispatching younger instructions into the window; any that do not depend on the missing load execute freely, out of order, while the miss is outstanding. Only the load's actual dependents wait. When the data arrives, the load completes, wakes its dependents, and — because retirement is in order — the whole sequence still commits as though nothing had been reordered. The stall that would have frozen an in-order core for a hundred cycles is, on the OoO core, largely *hidden* behind useful work. This is the mechanism by which the memory hierarchy of Chapter 3 becomes tolerable at all.



Branch prediction and speculation

Return to the control hazard. A conditional branch's direction is not known until it executes, but the front end must keep fetching every cycle or the whole superscalar machine starves. Waiting for each branch would cost the pipeline-depth penalty on *every branch*, and branches are roughly one instruction in five in typical code — the machine would spend most of its life fetching bubbles. The solution is to **guess**: the core predicts the direction (taken/not-taken) and the target of each branch and speculatively fetches and executes down the predicted path, before it knows whether the guess was right.


Branch fetched
Predictor:
direction + target
Speculatively fetch and
execute
down predicted path
Branch resolves in
execute
Prediction correct?
Yes
No
Speculative uops retire
normally
(work was real)
Flush wrong-path uops
from ROB
+ redirect fetch to
correct target

Speculatively executed instructions run through the OoO engine like any others, writing physical registers and sitting in the ROB — but they **cannot retire** until the branch they depend on resolves. If the prediction was correct, that speculative work was real work; those uops retire in order and nothing was wasted. If it was wrong, every wrong-path uop is squashed from the ROB, the physical registers they claimed are reclaimed, and fetch is redirected to the correct target. The penalty is a full pipeline refill — on modern server cores commonly cited as roughly **15–20 cycles**, though the exact figure varies by microarchitecture and by where the mispredict is detected, so treat any single number as approximate. At 2–4 uops per cycle, a mispredict throws away on the order of 40–80 uops of potential work. Branches are common enough that predictor accuracy is one of the largest levers on real IPC.

How the prediction is made

Modern predictors are **history-based** and startlingly good — well over 95%, often above 99%, accuracy on typical code. The core idea is that branch behavior is highly correlated with history: a given branch tends to go the same way (loop back-edges are taken almost every iteration), and, more powerfully, a branch's outcome is often correlated with the *pattern* of recent branches leading up to it. Predictors exploit both:

- **Local history** — the recent taken/not-taken pattern of *this* branch. A branch alternating T,N,T,N is perfectly predictable from its own history.
- **Global history** — the pattern of the last *N* branches system-wide, capturing correlations like "if the bounds check above was taken, this branch is not."

State-of-the-art designs are **TAGE-class** predictors (TAgged GEometric history length; Seznec and Michaud). Conceptually, TAGE keeps several tables indexed by hashes of the program counter combined with **geometrically increasing history lengths** — one table keyed on the last few branches, another on the last dozen, another on the last several dozen, and so on. A prediction is taken from the table with the *longest* matching history that has a confident entry, on the theory that a longer matching context is a more specific, more reliable predictor. Shorter-history tables provide fallbacks when the long-history context has not been seen before. You do not need the update math; the load-bearing intuition is: **modern predictors learn long, precise correlations**

across the recent branch stream, and they are extremely accurate when a branch's outcome is a learnable function of history. Cores also keep a separate **Branch Target Buffer** for the *target* address (needed for indirect branches and calls) and a **Return Address Stack** for returns, since predicting the target is a distinct problem from predicting the direction.

Why predictable branches are free and data-dependent ones are not

The predictor is only as good as the pattern it can learn. A branch whose outcome is a learnable function of history — a loop condition, a null check that is almost always false, a bounds check that almost always passes — is predicted correctly nearly every time and costs essentially nothing; the speculative work is real work. A branch whose outcome is *effectively random from the predictor's viewpoint* — a comparison against unsorted, unpredictable data — cannot be learned. It mispredicts roughly half the time, and each mispredict costs the full flush.

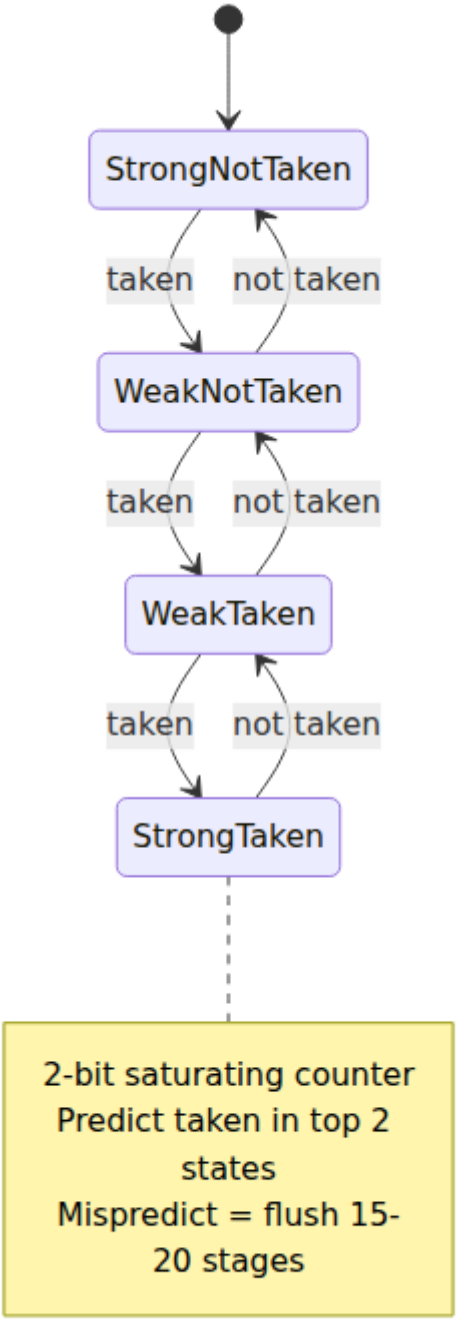
This is the origin of one of the most-cited performance lessons in practice, developed in depth in Chapter 8: iterating over an array and branching on `if (data[i] > threshold)` runs dramatically faster when `data` is **sorted** than when it is random, even though the instruction count is identical. Sorted, the branch is a long run of not-taken followed by a long run of taken — trivially predictable. Random, it is a coin flip the predictor cannot beat, and the mispredict penalty dominates the loop. Nothing about the arithmetic changed; the predictor's success rate did. The engineering responses — making branches predictable, or eliminating them with **branchless** code (conditional moves, masking, `SIMD`) so there is nothing to mispredict — are Chapter 8's subject. The point here is *why* they work: they attack the misprediction penalty at its root.

Speculation as a security boundary: Spectre and Meltdown

Speculation executes instructions that may be on the wrong path and are later squashed. Architecturally, squashed instructions leave no trace — that is the whole design. But they leave *microarchitectural* traces: a speculatively executed load can pull a line into the cache, and that cache state persists after the squash. The **Spectre** and **Meltdown** family of vulnerabilities (disclosed January 2018) weaponize exactly this. An

attacker trains the predictor so the core speculatively executes a load it should not — reading, transiently, memory it is not architecturally allowed to read — and encodes that secret into the cache footprint, then recovers it with a cache-timing side channel (Chapter 3 covers the timing mechanism). The architectural result is correctly rolled back; the microarchitectural side effect is not. Meltdown exploited speculation past a permission check to read kernel memory; Spectre-v1 exploited speculation past a bounds check; Spectre-v2 exploited mistrained indirect-branch prediction to steer speculation into attacker-chosen gadgets.

The deep lesson — pursued in Chapter 10 on hardware isolation, and connected to multi-tenant trust in the supply-chain volume (Book 6, cloud-native security) — is that **speculation broke the assumption that architectural isolation implies physical isolation.** The mitigations (retpolines and later hardware IBRS/eIBRS for branch-target injection, kernel page-table isolation for Meltdown, microcode updates, and speculation barriers) cost real performance and reshaped how kernels and hypervisors defend tenant boundaries. For a backend engineer the operational takeaway is concrete: the mitigations are not free, they landed as measurable throughput regressions across fleets, and on shared hardware the boundary between your workload and a co-tenant's is enforced in part by microarchitectural controls you do not see.


StrongNotTaken
taken
not taken
WeakNotTaken
taken
not taken
WeakTaken
taken
not taken
StrongTaken
2-bit saturating counter
Predict taken in top 2 states
Mispredict = flush 15-20 stages

Memory and the CPU: hiding latency

The OoO engine's greatest single service is hiding memory latency. Chapter 3 details the hierarchy; the number that matters here is the gap: an L1 hit is a few cycles, a last-level-cache miss to DRAM is on the order of a couple hundred cycles. At 3 GHz, a couple hundred cycles is ~70 nanoseconds during which an in-order core would be frozen. The OoO core instead runs ahead.

Loads and stores flow through dedicated **load/store units** behind their execution ports. Stores are special: a store's value must not become visible until the store retires (it might be on a mispredicted path), so stores are buffered in a **store buffer** and drained to cache only at retirement. That buffer enables **store-to-load forwarding**: a later load to an address a pending store has just written gets the value straight from the store buffer rather than waiting for it to reach cache. The store buffer is also, not incidentally, a primary source of the memory-reordering behavior that Chapter 4's memory-consistency discussion is about — it is why a store followed by a load to a different address can appear reordered to other cores.

The performance-critical capability is **memory-level parallelism (MLP)**: the core can have *many* outstanding cache misses in flight at once. Because non-dependent loads issue out of order, the core can send several independent misses to memory concurrently and overlap their latencies, rather than paying for them one after another. A structure of **miss-status handling registers** (MSHRs) tracks the outstanding misses; their count bounds MLP. This is why the *access pattern* dominates memory-bound performance far more than raw bandwidth: a pointer-chasing linked list serializes misses (each load's address depends on the previous load's result — a dependency chain the OoO engine cannot break), so latencies add up; an array traversal exposes many independent misses the core overlaps, so latencies hide behind one another. Same bytes moved, wildly different throughput, entirely because of how much MLP the pattern exposes. Hardware **prefetchers** amplify this further by detecting sequential and strided patterns and fetching lines before they are demanded — which is precisely why linear scans are so much friendlier than random access. Chapter 8 turns all of this into concrete data-structure guidance.

Instruction sets: CISC, RISC, and why the uop settles the argument

x86 is a **CISC** (complex instruction set): variable-length instructions, many addressing modes, instructions that both access memory and compute. ARM and RISC-V are **RISC** (reduced): fixed-length (AArch64 is 32-bit-wide instructions), load/store architectures where only explicit loads and stores touch memory. For decades this distinction was framed as a fundamental performance divide. Inside a modern core, it has largely dissolved, and the reason is the **micro-op**.

An x86 core does not execute x86 instructions directly. Its decoders **crack** each variable-length x86 instruction into one or more fixed-format internal uops, and everything past decode — rename, schedule, execute, retire — operates on those uops, which look much like RISC operations. A `add [rax], rbx` (load-add-store) becomes a load uop, an add uop, and a store uop. So beneath the ISA, an x86 core and an ARM core are the *same kind of machine*: wide, out-of-order, superscalar, speculative, uop-based. Cores also cache decoded uops in a **uop cache** so hot loops skip the expensive x86 decode entirely, blunting the one place CISC decoding genuinely costs more (variable-length decode is harder to parallelize than fixed-length).

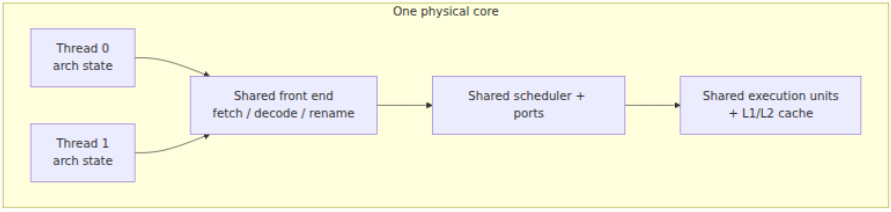
The practical consequence for backend engineers is that **the ISA is rarely the thing determining your service's throughput.** Both families are OoO superscalar underneath; both have good predictors and deep windows. What differs and *does* matter is not RISC-vs-CISC in the abstract but the concrete microarchitecture, the memory system, the core count, the frequency, and — increasingly — the price and power of the specific part.

Which is why **ARM in the datacenter is not a footnote.** AWS Graviton (Neoverse cores), Ampere Altra, Google Axion, Microsoft Cobalt, and NVIDIA Grace are real, deployed, general-purpose server parts, and the driver is economics, not ISA elegance: competitive per-core performance at meaningfully better performance-per-watt and lower price, which at fleet scale is the whole game. For most managed-runtime and compiled backend workloads, porting is close to a recompile-and-retest (JVM, Go, Node, .NET, Python all run natively on AArch64), with the usual caveats about native dependencies, hand-written assembly or SIMD intrinsics, and x86-specific assumptions. The migration story is a cost story, and it

belongs in the same conversation as instance sizing and reservation strategy, not in a discussion of instruction encodings.

SMT / Hyper-Threading: sharing one core

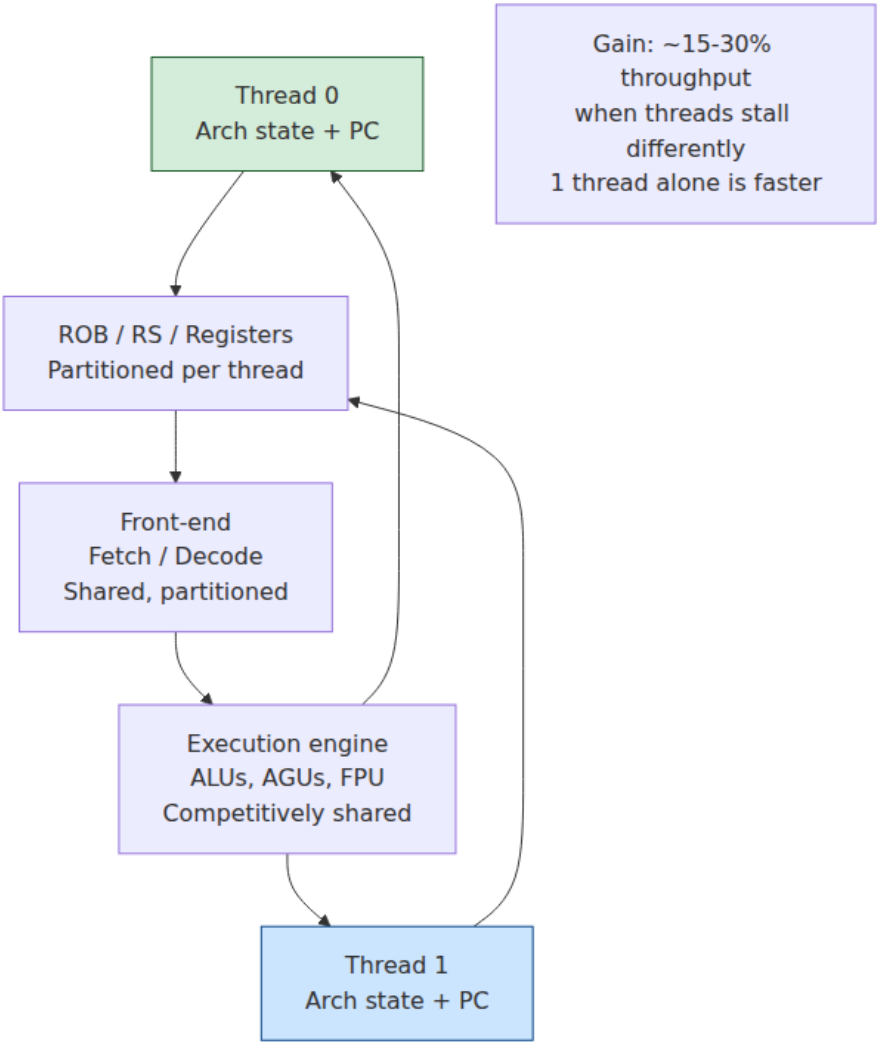
A wide OoO core is frequently *underutilized* by a single thread: dependency chains and cache misses leave execution ports idle even with a deep window. **Simultaneous multithreading (SMT)** — Intel's brand is Hyper-Threading — exploits those idle resources by running *two* (sometimes more) threads on one physical core at once. Each thread has its own architectural state (registers, program counter), but they **share** the physical execution resources: the ports, the schedulers, the caches, the TLBs, and — critically — they compete for slots in the ROB and physical register file.



SMT **helps** when a single thread cannot keep the ports busy — which is most of the time for latency-bound, memory-stalling backend workloads. While thread 0 waits on a cache miss, thread 1's independent instructions fill the idle ports; the core's aggregate throughput rises. Typical real-world gains are modest and workload-dependent — commonly in the ballpark of 10–30% additional throughput per core, occasionally more, essentially never the 2× a naive "two threads" count would suggest, because the two threads are sharing one core's finite execution capacity, not doubling it.

SMT **hurts** when a single thread already saturates the shared resources. Two threads that both hammer the FP units, or that between them thrash a shared L1/L2 or the ROB, contend rather than complement: each runs slower than it would alone, and total throughput can even *drop*. Cache-sensitive workloads are the classic losers — two threads halve the effective per-thread cache. This is why SMT is a throughput optimization with a **latency and predictability cost**: a thread's performance now depends on what its sibling is doing, which makes tail latency noisier and harder to reason about.

SMT is also a **security** surface. Because the two threads share microarchitectural resources, one can observe the other's behavior through timing. **L1TF** (L1 Terminal Fault) and the **MDS** family (Microarchitectural Data Sampling — RIDL, Fallout, ZombieLoad, 2019) let a thread sample data flowing through shared microarchitectural buffers on the sibling thread. The mitigations are heavy: some environments disable SMT entirely for untrusted multi-tenancy, and cloud schedulers use **core scheduling** / **gang scheduling** so that only sibling threads from the *same* trust domain share a core. For a backend engineer this is not abstract — it is why some security-sensitive fleets run with Hyper-Threading off, eating the throughput loss to close the side channel.



Distributed-systems lens: IPC is a line item

Everything above is single-core mechanics, but you operate fleets, and at fleet scale these mechanics become economics and SLOs.

IPC is money. A service that runs at IPC 1.0 instead of 2.0 needs roughly twice the cores to serve the same request rate — twice the instances, twice the bill, twice the power. Frequency is fixed by the part you rented;

IPC is the dimension your code controls. At a few instances this is noise; across tens of thousands of cores it is a budget line, and it is why performance engineering on hot paths (Chapter 8, and the performance volume) has a direct, computable ROI. "Cycles, not instructions" is the mindset: a profiler that shows instruction counts can lie, because two functions with equal instruction counts can differ 5× in cycles depending on cache misses and mispredicts. Measure cycles and IPC, not instruction count.

Your vCPU is probably a hyperthread. On most public clouds a "vCPU" is one SMT thread, and two vCPUs on one instance may be the two siblings of a single physical core, sharing all the execution resources above. This reframes capacity planning: a 2-vCPU instance is not two independent cores, and CPU-bound work will not scale linearly across its vCPUs. It also reframes **noisy neighbors**: in multi-tenant environments a co-tenant's thread sharing your physical core competes for your ports and pollutes your caches, and you see it as unexplained latency variance you cannot fix from inside your VM. Instance types that guarantee full physical cores (or that pin/isolate cores) exist precisely to sell predictability back to workloads that need it.

ARM migration is a fleet-cost decision. Graviton-class parts change the performance-per-dollar and performance-per-watt of the whole fleet. Because the ISA rarely gates backend throughput, the migration is usually gated by toolchain and dependency portability, not by any inherent x86 advantage — which makes it a platform-engineering project with a clear financial payoff, evaluated with the same rigor as any other capacity decision.

Warmup, prediction training, and tail latency. The predictors, the caches, and (for managed runtimes) the JIT all **learn** — they need to see the workload before they are fast. A cold process mispredicts branches it will later predict perfectly, runs interpreted before it is JITed, and misses caches it will later hit. This is the microarchitectural half of the **cold-start** problem: a freshly scheduled serverless function or a just-started pod pays a warmup tax that a long-running, steady-state service does not. It is also why **microbenchmarks mislead**: without warmup, you measure the cold, untrained machine; with unrealistic warmup, you train the predictor on a pattern production never sees and measure a fiction. Benchmark methodology — warm up, use realistic data distributions so the predictor faces production-like branch patterns, measure many

iterations, report distributions not means — is not pedantry; it is the difference between a number that predicts production and one that does not.

Frequency scaling is real and it moves your numbers. Turbo, thermal throttling, and power-cap governors mean the "3 GHz" part runs at a frequency that depends on how many cores are busy, the workload's instruction mix (heavy AVX-512 code often clocks *lower* to stay in the power envelope), the ambient thermals, and the governor policy. Two runs of the same code on the same instance can differ because the second ran at a lower turbo bin. For repeatable measurement, pin frequency where you can; for production reasoning, know that your effective clock is a runtime variable, not a datasheet constant.

Reading the counters. The hardware exposes performance counters that make all of this observable. On Linux, `perf stat` reports the essentials:

```
perf stat -e cycles,instructions,branches,branch-misses,\
cache-references,cache-misses ./your_service --bench
```

```
     45,231,509,884      cycles
     58,004,112,340      instructions      #    1.28  insn per cycle
      9,880,441,203      branches
        512,003,918      branch-misses     #    5.18% of all branches
      2,140,556,001      cache-references
        401,229,847      cache-misses      #   18.75% of all cache-refs
```

Read it as a diagnosis. **IPC 1.28** on a core capable of 4+ says the pipeline is stalling most cycles — the question is why. A **5% branch-miss rate** is high (well-predicted code is under 1–2%) and points at data-dependent branches worth making predictable or removing. **18% cache misses** point at the memory system (Chapter 3) and access patterns (Chapter 8). To attribute stalls precisely, top-down methodologies (`perf`'s `topdown` events, Intel VTune, AMD uProf) break each cycle into *retiring*, *bad speculation*, *front-end bound*, and *back-end bound*, telling you whether you are losing cycles to mispredicts, to fetch, or to memory. The performance volume (Volume 2) develops this into a full workflow; the point here is that the abstract machine of this chapter is directly measurable, and the counters are how you turn "it feels slow" into "IPC 1.28, back-end bound on L2 misses in `parseRecord`."

Key takeaways

- The real core pipelines (throughput without lower latency), is superscalar (IPC can exceed one), and executes out of order and speculatively. The naive fetch-execute loop is the wrong mental model for any server CPU.
- Performance ≈ frequency × IPC. Frequency is largely fixed; IPC is where code cooperates or fights, and a 5–10× IPC swing on identical silicon is entirely realistic.
- Three hazard families jam the pipeline: structural (shared units), data (RAW is real; WAR/WAW are false names), and control (branches). Renaming eliminates false dependencies; forwarding and OoO scheduling attack real ones; prediction attacks control hazards.
- The OoO engine is dataflow with an in-order commit: rename to a large physical register file (killing WAR/WAW), wait in the scheduler until operands are ready, issue to execution ports, and retire in program order from the reorder buffer. The ROB size bounds how far ahead the core sees to hide stalls.
- Branch prediction (TAGE-class, history-based, >95% typical) makes control-heavy code fast; a misprediction costs a full pipeline flush, roughly 15–20 cycles (hedge it). Predictable branches are nearly free; data-dependent, unpredictable branches are expensive — the root of the sorted-vs-unsorted phenomenon and the case for branchless code (Chapter 8).
- Speculation is the root of Spectre/Meltdown: squashed instructions leave microarchitectural traces (cache state) that break the assumption that architectural isolation implies physical isolation — a multi-tenant concern (Chapter 10; Book 6).
- The OoO engine hides memory latency by running ahead; memory-level parallelism (many outstanding misses) is why access pattern beats raw bandwidth, and why pointer chasing is slow.
- CISC vs RISC mostly does not matter to backend throughput — both are OoO superscalar under a uop layer — but ARM in the datacenter (Graviton and peers) is a real performance-per-dollar and per-watt decision.
- SMT shares one core between threads: it helps hide stalls, hurts under resource contention, adds tail-latency noise, and opens side channels (L1TF, MDS). Your cloud "vCPU" is usually one such thread.

- At fleet scale, IPC is cost, oversubscribed vCPUs and noisy neighbors are shared-core contention, warmup and prediction training shape cold-start tail latency, and turbo/thermal scaling makes your effective clock a runtime variable. `perf stat` makes all of it measurable.

Further reading

- John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. (Morgan Kaufmann, 2017) — the standard reference; Chapters 3 (ILP, OoO, speculation) and Appendix C (pipelining) cover this chapter's core material rigorously.
- Agner Fog, *The Microarchitecture of Intel, AMD and VIA CPUs* and *Instruction Tables* — https://www.agner.org/optimize/ — the definitive open reference on real port layouts, uop breakdowns, and latencies per microarchitecture.
- R. M. Tomasulo, "An Efficient Algorithm for Exploiting Multiple Arithmetic Units," *IBM Journal of Research and Development*, 1967 — the origin of register renaming and reservation-station scheduling.
- André Seznec and Pierre Michaud, "A case for (partially) TAgged GEometric history length branch prediction" (TAGE), *Journal of Instruction-Level Parallelism*, 2006 — the basis of modern branch predictors.
- David W. Wall, "Limits of Instruction-Level Parallelism," DEC WRL Research Report 93/6, 1993 — the classic empirical study of how much ILP real programs actually contain.
- Kocher et al., "Spectre Attacks: Exploiting Speculative Execution," and Lipp et al., "Meltdown: Reading Kernel Memory from User Space" — the 2018 disclosures; see also https://meltdownattack.com/.
- van Schaik et al. (RIDL) and the MDS disclosures — https://mdsattacks.com/ — for the SMT-shared-buffer side channels (ZombieLoad, RIDL, Fallout).
- Ulrich Drepper, "What Every Programmer Should Know About Memory," 2007 — dated in specifics but excellent on how the OoO core interacts with the memory hierarchy.
- Intel, *64 and IA-32 Architectures Optimization Reference Manual*, and Arm, *Neoverse V2 Software Optimization Guide* — vendor

microarchitecture and optimization references, including the top-down performance methodology.

- Brendan Gregg, *Systems Performance*, 2nd ed. (Addison-Wesley, 2020) — practical `perf`, PMCs, and CPU performance analysis on Linux at production scale. ```

Chapter 3 — The Memory Hierarchy and Caches

What this chapter covers. Of all the hardware topics a backend engineer can internalize, the memory hierarchy pays the highest dividend. The reason is a single, decades-old, still-widening fact: **the processor can compute far faster than main memory can feed it.** Every trick modern hardware plays — caches, prefetchers, out-of-order execution (Chapter 2), speculation — exists to paper over that gap. Programs that respect the hierarchy run an order of magnitude faster than algorithmically identical programs that ignore it, on the same silicon, with the same instruction count. This chapter explains *why* the gap exists, *how* the cache hierarchy hides it, and *how to write code and design data structures* that keep the fast paths hot. It closes by extending the same locality principle outward: past the die, past the socket, out to SSDs, remote caches, and other regions — because the memory hierarchy does not stop at the chip's edge, and neither does the reasoning.

Learning goals — after this chapter you should be able to:

- Explain the **memory wall** — the growing latency gap between CPU and DRAM — and why caches, not faster DRAM, are the industry's answer.
- Reason quantitatively about the **cache hierarchy** (L1/L2/L3/DRAM), with order-correct sizes and latencies, and distinguish inclusive from exclusive caches.
- Explain the **cache line** as the atomic unit of data movement (typically 64 bytes) and derive most cache-friendly-code advice from that one fact.
- Define **temporal** and **spatial locality**, and connect them to the two structural properties of real programs that make caches work.

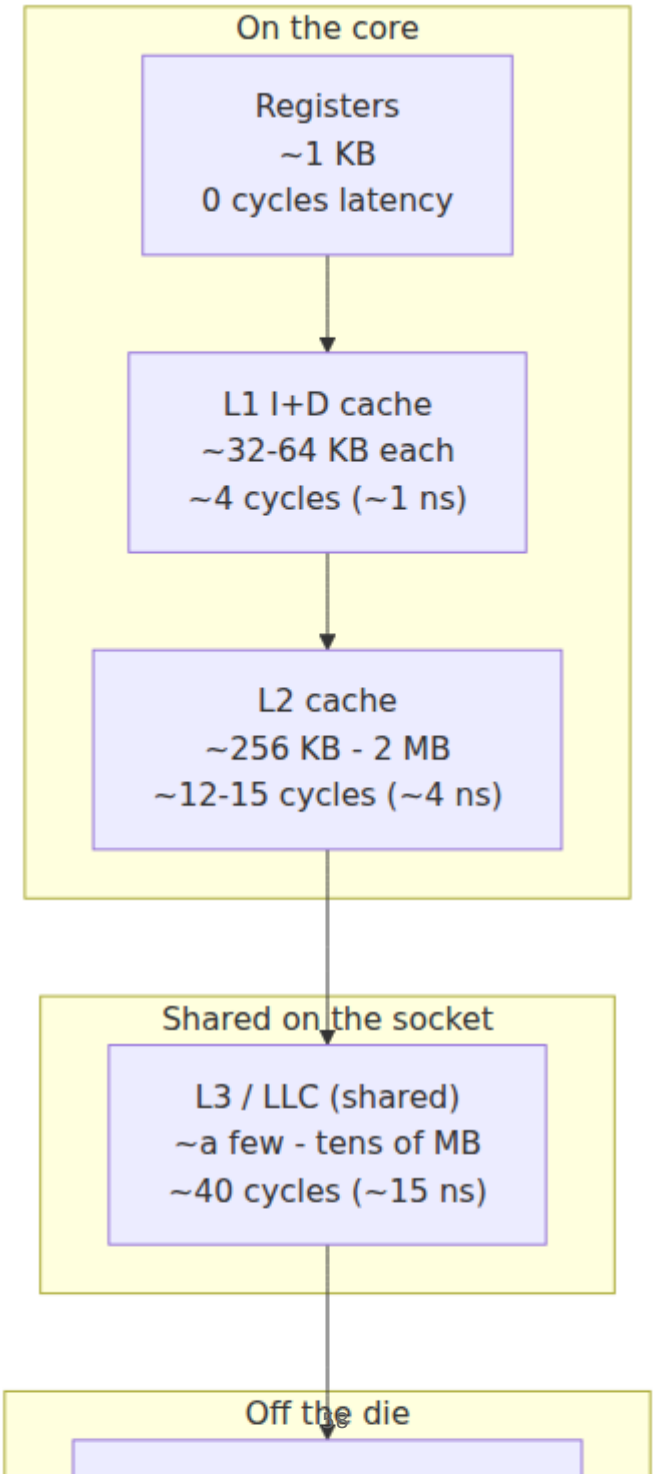
- Decompose a physical address into **tag / index / offset**, and explain **set-associative** mapping, the **3 C's** (compulsory, capacity, conflict), and the eviction and **write policies** a cache implements.
- Explain **hardware prefetching**, why sequential access is dramatically faster than random, and why pointer-chasing defeats the prefetcher.
- Explain the **TLB** as the "other cache," why TLB misses trigger page walks, and how huge pages relieve TLB pressure.
- Apply all of it: **data-oriented design**, struct-of-arrays vs array-of-structs, cache blocking, and why arrays beat linked lists — and see the same ideas govern **distributed caching tiers**.

The memory wall: the fact that motivates everything

Start with the numbers, because the argument is entirely about numbers. A modern server core retires several instructions per cycle at a few GHz. Call it a ~0.3 ns cycle time. In that cycle the core can do real arithmetic — an add, a multiply, a compare. Now ask that core to read a value from main memory (DRAM) that is not cached anywhere. The round trip is on the order of **~100 ns**. At ~0.3 ns per cycle, 100 ns is **several hundred cycles**. The exact figure is vendor-, generation-, and configuration-dependent — 60–120 ns is a reasonable spread for a local DRAM access on contemporary server parts — but the order of magnitude is stable and it is the point: **a DRAM miss costs hundreds of instructions' worth of time.** A core that stalls on memory is a Ferrari idling at a red light.

This gap is not an accident of one bad generation; it is a structural divergence that has widened for decades. CPU throughput (clock times instruction-level parallelism) improved far faster than DRAM *latency*. DRAM *bandwidth* has grown respectably — each generation moves more bytes per second — but the time to service a single, latency-bound, dependent access has barely improved. The industry has a name for the cumulative result: the **memory wall**. The gap between how fast a core can consume data and how fast memory can deliver a single random datum grew until memory latency, not compute, became the dominant cost for most workloads. Backend software is squarely a "most workloads": chasing pointers through a request graph, walking a hash map, deserializing a payload — all latency-bound, all memory-bound.

Why not just build faster DRAM? Because the physics and economics trade off against each other. Fast memory (SRAM, the stuff caches are made of) uses ~6 transistors per bit, is expensive, power hungry, and physically large per bit — you cannot afford gigabytes of it. Dense memory (DRAM, one transistor plus one capacitor per bit) is cheap and huge but slow, and it sits across a bus, off the die, sometimes across a socket (Chapter 6). You cannot have capacity and latency in the same technology at an acceptable price. So the industry did the only thing it could: it built a **hierarchy** — a small amount of very fast memory close to the core, backed by progressively larger and slower tiers — and bet that programs access memory in a way that lets small fast tiers capture most of the traffic. That bet is called *locality*, and it almost always pays.


On the core
Registers
~1 KB
0 cycles latency
L1 I+D cache
~32-64 KB each
~4 cycles (~1 ns)
L2 cache
~256 KB - 2 MB
~12-15 cycles (~4 ns)
Shared on the socket
L3 / LLC (shared)
~a few - tens of MB
~40 cycles (~15 ns)
Off the die

Notice the shape: as you move away from the core, capacity grows by roughly 10-100x per step and latency grows by a similar factor. Each tier is a cache for the tier below it. The core's job — and the compiler's, and yours — is to arrange that the overwhelming majority of accesses are satisfied by the tiers near the top.

The hierarchy, tier by tier

The following table gives order-correct figures for a contemporary x86 or ARM server core. Treat every number as an approximation whose **order of magnitude is the durable fact**; the exact cycle counts and sizes vary by vendor (Intel, AMD, ARM designs differ), by generation, and by SKU.

Tier	Typical size (per core unless noted)	Typical latency	Notes
Registers	Dozens of named regs, ~1 KB total	0 extra cycles	Named operands; the compiler allocates these
L1 data (L1d)	~32-64 KB	~4-5 cycles (~1 ns)	Split from L1 instruction cache; per core
L1 instr (L1i)	~32-64 KB	~4-5 cycles	Feeds the front end (Chapter 2)
L2 (unified)	~256 KB - 2 MB	~12-15 cycles (~4 ns)	Per core on most designs
L3 / LLC	~a few to tens of MB	~40 cycles (~10-20 ns)	**Shared** across cores on a socket
DRAM (local)	GBs to TBs	~60-120 ns	Off-die; non-uniform under NUMA (Chapter 6)
Remote DRAM	—	~1.5-2x local DRAM	Another socket's memory (Chapter 6)

A few structural points that matter more than the exact numbers:

L1 is split, lower levels are unified. L1 is two caches: L1i for instructions and L1d for data. They are separate because the front end fetching instructions and the load/store units fetching data would

otherwise contend for the same port every cycle. From L2 down, one unified cache holds both.

L3 is shared; L1/L2 are private. Each core has its own L1 and (usually) L2. The last-level cache (LLC, usually L3) is shared by all cores on the socket. This sharing is what makes the LLC the coordination point for **cache coherence** — the protocol that keeps per-core caches consistent when multiple cores touch the same line — which is the entire subject of Chapter 4. It is also where **false sharing** does its damage (Chapter 8).

Latency compounds on a miss. The latencies above are not additive in the happy path — a hit in L1 costs ~4 cycles, full stop. But a *miss* pays the cost of checking each level it misses in before finding the data. An L1 miss that hits in L2 costs roughly the L2 latency; an access that misses all the way to DRAM has paid the tag checks along the way and then eats the ~100 ns. This is why miss *rates* at each level, not just averages, drive real performance.

Inclusive vs exclusive caches

When a line lives in L1, does a copy also occupy L2 and L3? That is the inclusion policy, and designs differ:

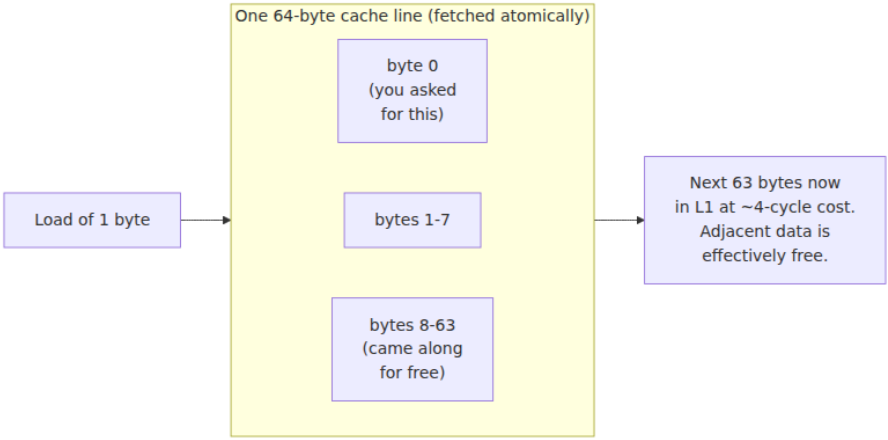
- **Inclusive** (historically common on Intel LLCs): every line in an upper level is *also* present in the level below. This wastes some capacity (the same bytes are stored twice) but simplifies coherence: to know whether *any* core has a line, a snoop only has to check the shared LLC. If it is not in the LLC, it is in no private cache.
- **Exclusive** (common on many AMD designs): a line lives in *exactly one* level, so aggregate capacity is the sum of the levels. This maximizes effective cache size but complicates coherence and eviction (evicting from L2 may require inserting into L3).
- **Non-inclusive / NINE** (neither strictly inclusive nor exclusive): a middle ground many modern designs use.

You rarely tune for inclusion policy directly, but it explains otherwise-surprising numbers — e.g., why an "N MB L3" does not always behave like N MB of *additional* room, and why coherence traffic patterns differ between vendors.

The cache line: the atomic unit of data movement

Here is the single most load-bearing fact in this chapter. **Caches do not move bytes. They move lines.** A cache line is a fixed-size, aligned block of memory — on essentially every current mainstream CPU, **64 bytes** — and it is the smallest unit the cache tracks, fetches, and evicts.

When your code reads one `int` at address `0x...40`, the hardware does not fetch 4 bytes. It fetches the entire aligned 64-byte line containing that address (here, `0x...40` through `0x...7F`) from wherever it currently lives, and installs the whole line in L1. The consequences ripple through everything:



- **Accessing one byte costs you 64.** Bandwidth and cache capacity are consumed in 64-byte units. A struct with one hot 4-byte field still drags its 60 neighboring bytes into cache.
- **Adjacency is free — that is spatial locality made physical.** If you access byte 0 and will soon access byte 8, the second access is already in L1 because the whole line arrived together. Data structures that place things-used-together next to each other get this for free.
- **Alignment matters.** A value that straddles a 64-byte boundary lives in *two* lines, so touching it can cost two cache accesses (and two misses). Aligning hot structures to line boundaries avoids split accesses.

- **False sharing is a cache-line artifact.** If two cores write two *different* variables that happen to share one 64-byte line, the coherence protocol treats it as contention over one line and ping-pongs it between cores, even though the variables are logically independent. This is entirely a consequence of the line being the unit of coherence, and it is dissected in Chapter 8.

Internalize this: **almost every piece of cache-friendly-code advice reduces to "make good use of each 64-byte line you pull in."** Pack the bytes you will use together; do not pull in lines you will touch once; do not let two threads fight over one line.

Locality: why the bet pays off

Caches work because real programs are not random-access machines over their whole address space. They exhibit two kinds of *locality of reference*:

- **Temporal locality:** if you access a location, you are likely to access *the same* location again soon. Loop induction variables, the top of a hot stack frame, a frequently dereferenced object, a configuration struct read on every request — all reused within a short window. A cache exploits temporal locality automatically: once a line is in, subsequent accesses hit.
- **Spatial locality:** if you access a location, you are likely to access *nearby* locations soon. Iterating an array, reading successive fields of a struct, walking a contiguous buffer. A cache exploits spatial locality by fetching whole lines — the neighbors ride along.

These are not laws of nature; they are empirical properties that emerge from how we write code: loops, sequential data structures, function-local working sets, and the stack. The cache hierarchy is a bet that *your program has locality*, and for well-written code it wins overwhelmingly. The corollary is the engineer's lever: **cache-friendly code is code engineered to maximize both kinds of locality** — reuse data while it is still hot (temporal), and lay data out so that things-used-together are physically adjacent (spatial).

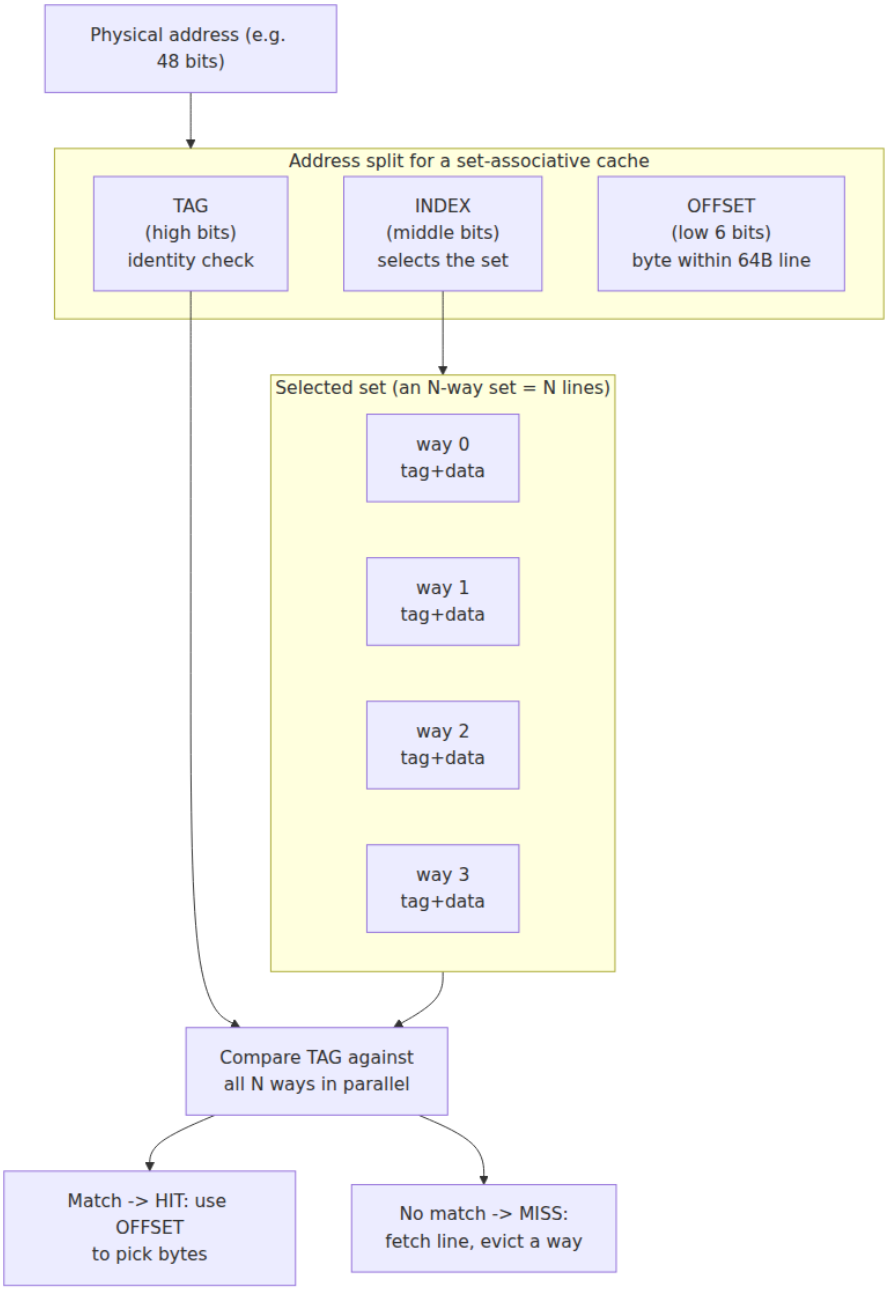
The **working set** is the vocabulary for reasoning about this: the set of memory a piece of code touches over a given time window. If the working set of a hot loop fits in L1, the loop runs at L1 speed. If it fits in L2 but not L1, at L2 speed. If it spills to DRAM, the loop runs at DRAM speed

and no amount of micro-optimization elsewhere will save it. Much of practical performance work is *shrinking the working set of the hot path until it fits in a fast tier* — a theme we will see again, at the fleet level, when sizing an in-memory store.

Cache organization: how an address finds its line

A cache must answer one question quickly: for a given address, is the line present, and if so where? It cannot afford to search all of itself on every access. The solution is to slice the address into three fields and use them to look up a small number of candidate slots.

Take a physical address and a cache with 64-byte lines. The low bits index *within* a line; the middle bits select a *set*; the high bits are the *tag* that confirms identity:


Physical address (e.g. 48 bits)
Address split for a set-associative cache
TAG (high bits) identity check
INDEX (middle bits) selects the set
OFFSET (low 6 bits) byte within 64B line
Selected set (an N-way set = N lines)
way 0 tag+data
way 1 tag+data
way 2 tag+data
way 3 tag+data
Compare TAG against all N ways in parallel
Match -> HIT: use OFFSET to pick bytes
No match -> MISS: fetch line, evict a way

The math is concrete. With **64-byte lines**, the low **6 bits** are the offset (2^6 = 64). If the cache has **S sets**, the next **log2(S)** bits are the index. Everything above is the tag. Worked example: a 32 KB, 8-way set-associative L1d with 64-byte lines has 32768 / 64 = 512 lines total, divided into 512 / 8 = **64 sets**. So offset = 6 bits, index = log2(64) = 6 bits, and the tag is the remaining high bits. Given an address, the hardware masks out bits [11:6] to pick 1 of 64 sets, reads all 8 ways in that set, compares their stored tags against bits [47:12] in parallel, and on a match uses bits [5:0] to select the requested bytes.

The knob here is **associativity** — how many ways (candidate slots) a set has — and it defines three points on a spectrum:

- **Direct-mapped (1-way):** each address maps to exactly one slot. Fast and cheap to build (one tag compare), but two hot addresses that map to the same slot evict each other on every access, even if the rest of the cache is empty. Pathological for certain stride patterns.
- **Fully associative (N-way over the whole cache):** a line may go in *any* slot. No conflict misses, but every access must compare against every tag — too expensive for anything but tiny caches (the TLB is often built this way; see below).
- **Set-associative (the real answer):** a compromise. The cache is divided into sets; a line maps to exactly one set (by its index bits) but may occupy any of the N ways within that set. Real caches are typically 4-way to 16-way. This keeps tag comparison cheap (N compares) while giving each address N places to live, which drastically cuts conflicts versus direct-mapped.

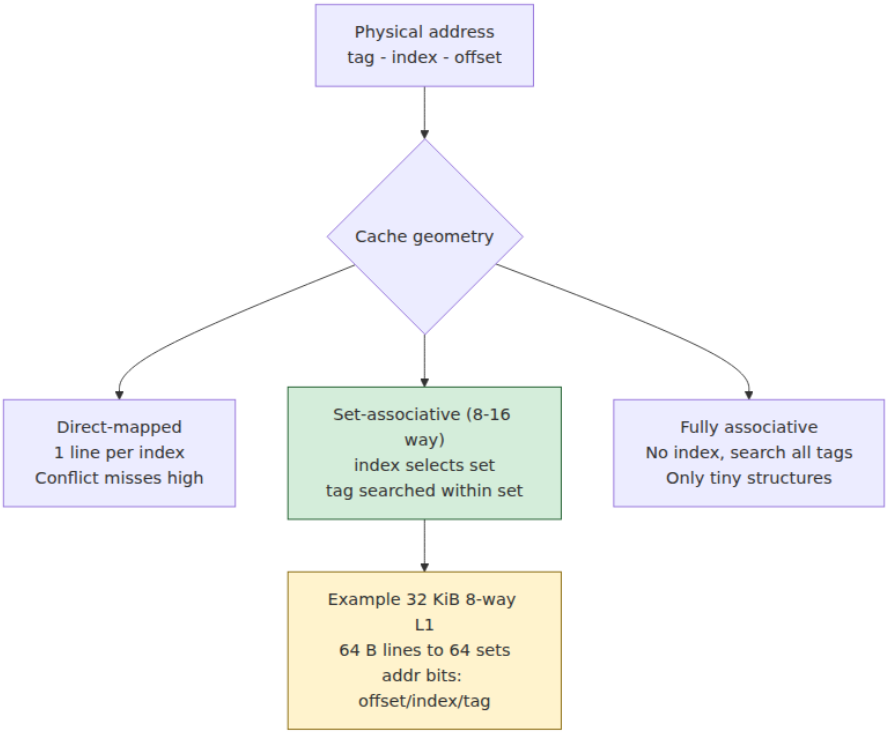
The 3 C's: a taxonomy of misses

Every cache miss falls into one of three categories — the classic **3 C's** — and naming the category tells you what to do about it:

Miss type	Cause	How to reduce it
Compulsory	First-ever access to a line ("cold" miss); it was never in cache	Prefetching; larger lines; unavoidable for truly first touches

Miss type	Cause	How to reduce it
Capacity	Working set exceeds cache size; lines evicted before reuse	Shrink the working set; blocking/tiling; better algorithms
Conflict	Too many hot lines map to the same set; evicted despite free capacity	More associativity; change data layout/stride to spread across sets

The distinction is practically important. A **capacity** problem means your hot data is simply too big for the tier — the fix is to make it smaller or process it in cache-sized chunks (blocking). A **conflict** problem means data that *would* fit is colliding on a few sets because of an unlucky stride (e.g., iterating a large 2D array by a power-of-two column stride can map every access to the same set) — the fix is to change the layout or padding so accesses spread across sets. **Compulsory** misses are the cost of touching data for the first time; prefetching is how the hardware hides them.

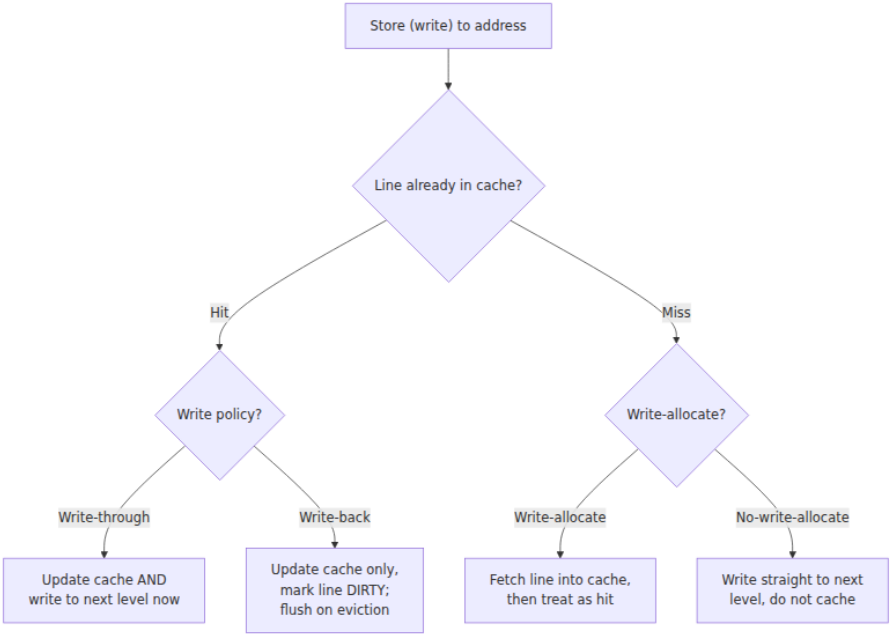


Cache operations: hits, evictions, and writes

On every memory access the cache does a lookup as above. A **hit** returns data at that tier's latency. A **miss** must fetch the line from a lower tier and install it — and if the target set is full, it must first **evict** an existing line to make room.

Eviction / replacement policy. True LRU (evict the least-recently-used way) is the theoretical ideal for exploiting temporal locality, but tracking exact recency across many ways is expensive in hardware. Real caches use **pseudo-LRU** approximations (tree-based PLRU, or aging/NRU schemes) that get most of LRU's benefit with a few bits of state per set. You cannot control the policy, but knowing it is *recency-biased* tells you the mental model: recently touched lines survive; lines you stopped touching get evicted — which is exactly why temporal locality (reuse soon) pays.

Write policies. What happens on a *store* is governed by two orthogonal decisions:



Dimension	Option	Behavior
On a write hit	**Write-through**	Write updates this level and propagates to the next level immediately. Simple; more traffic.
On a write hit	**Write-back**	Write updates only this level; the line is marked **dirty**. The dirty line is written to the next level lazily, on eviction. Far less write traffic; standard for L1/L2/L3.
On a write miss	**Write-allocate**	Fetch the missing line into cache first, then perform the write (pairs naturally with write-back).
On a write miss	**No-write-allocate**	Write to the next level without caching the line (pairs naturally with write-through).

Modern data caches are overwhelmingly **write-back + write-allocate**, because write-through's traffic is ruinous when writes are frequent and re-read. The **dirty bit** is the key piece of state: it marks a line whose cached copy is newer than memory, so eviction knows it must be flushed rather than simply dropped. Dirty lines are also central to coherence (Chapter 4): a modified line held privately by one core is the thing other cores must be prevented from reading stale.

One practical consequence: even *write-only* patterns pull data into cache. Writing a large buffer you will never read back still consumes cache (via write-allocate) and evicts useful lines. For exactly this case, ISAs provide **non-temporal / streaming stores** (e.g., x86 `MOVNT*`) that bypass the cache and write straight to memory, preserving the cache for data with real reuse.

Prefetching: why sequential access flies and random access crawls

A compulsory miss stalls the core for ~100 ns. If the hardware could *predict* which line you will need next and fetch it *before* you ask, the stall would be hidden. That is exactly what a **hardware prefetcher** does, and it is the reason sequential access is often 5-10x faster than random access even when the same number of bytes is touched.

Each core has one or more prefetchers watching the stream of cache accesses for patterns:

- **Next-line / sequential prefetchers** notice you are walking forward and fetch the following line(s) ahead of demand.
- **Stride prefetchers** detect constant-stride patterns — accessing every 200th byte, say — and project the stride forward, fetching ahead along the arithmetic progression.

When your access pattern is predictable, the prefetcher runs ahead of the core, and by the time the core issues the demand access the line is already in (or on its way to) L1/L2. The ~100 ns latency is overlapped with useful work and effectively disappears. This is why **iterating an array is nearly as fast as the CPU can consume it**: the prefetcher turns a sequence of would-be compulsory misses into hits.

Now consider **random access** — hash tables with poor locality, and especially **pointer chasing** through a linked structure where each node's address is only known *after* dereferencing the previous node. The prefetcher sees no pattern (the addresses are unpredictable), so it cannot run ahead. Worse, the accesses are *dependent*: the core literally cannot issue the next load until the current one returns, because the current load produces the address of the next. There is nothing to overlap. Every node is a fresh ~100 ns compulsory miss, fully exposed, serialized. A linked list of a million nodes scattered across the heap is a million serialized DRAM round trips: tens of milliseconds of pure stall for data an array would stream in microseconds. **This is the mechanism behind "arrays beat linked lists,"** and we will return to it as a design rule.

Software can also *hint* prefetches. Compilers emit prefetch instructions (x86 `PREFETCHT0/T1/T2/NTA`), and C/C++ expose `__builtin_prefetch`. These are worth reaching for only in narrow cases — e.g., when you know the address several iterations ahead but the hardware cannot infer it (walking an index array into a large table). Used carelessly they hurt: a mistimed prefetch evicts a useful line or wastes bandwidth. The reliable, general strategy is not to hint the prefetcher but to **give it a pattern it can follow**: sequential and constant-stride access.

The TLB: the "other cache"

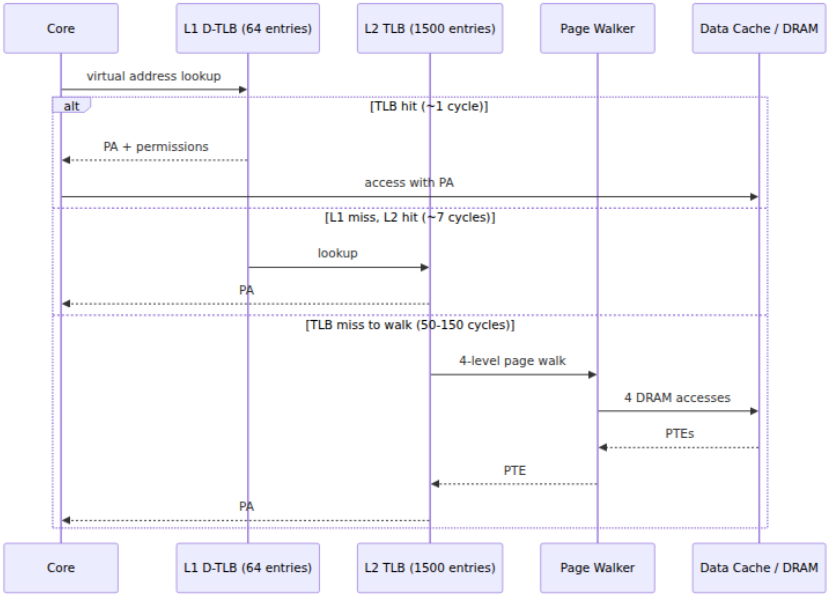
There is a second cache in the critical path of every memory access, and backend engineers routinely forget it exists: the **Translation Lookaside**

Buffer. Under virtual memory (Volume 2), the addresses your program uses are *virtual*; the hardware must translate each to a *physical* address before the cache hierarchy can be consulted. That translation walks a multi-level **page table** in memory — potentially several dependent DRAM accesses just to find where your data lives. If every load paid a page walk, virtual memory would be unaffordable.

The TLB caches recent virtual-to-physical translations, keyed by page. On a **TLB hit**, translation is effectively free and overlapped with the cache lookup. On a **TLB miss**, the hardware performs a **page walk** — traversing the page-table levels (four levels for standard x86-64 4 KB pages, so up to four dependent memory accesses, themselves possibly cache misses) to find the translation, then fills the TLB. A page walk can cost tens to well over a hundred cycles, sometimes more if the page-table entries are themselves not cached.

The TLB is small — often on the order of dozens to a few thousand entries, split into L1 and L2 TLBs and frequently split for instructions vs data, and commonly fully or highly associative because it is tiny. Its **reach** — how much memory it can map without a miss — is entries times page size. With 4 KB pages, even a 1500-entry TLB reaches only ~6 MB. A backend service with a multi-gigabyte heap, touching memory sparsely (a large hash map, a big in-process cache), can miss the TLB constantly even while its *data* fits in the CPU caches. TLB pressure becomes the bottleneck, invisible to anyone only watching cache miss rates.

The standard fix is **huge pages** (2 MB or 1 GB on x86-64) instead of 4 KB pages. Each huge-page TLB entry maps 512x or 262144x more memory, so the same number of TLB entries reaches vastly more of the heap and page walks (when they happen) traverse fewer levels. This is why databases, JVMs, and large in-memory stores expose huge-page / `MADV_HUGEPAGE` options and why Linux offers transparent huge pages. The mechanics of paging, page tables, and huge pages belong to Volume 2 (Virtual Memory); the point here is architectural: **the TLB is a cache with the same hit/miss/reach economics as the data caches, it sits on every access, and huge pages are to the TLB what good locality is to the data cache.**



Measuring what actually happens

None of this is guesswork you have to do in your head. The CPU exposes **hardware performance counters** that report exactly how the hierarchy is behaving, and on Linux `perf` reads them. A first pass:

```
# Overview: IPC plus L1 and last-level cache miss rates for a run
perf stat -e cycles,instructions,\
L1-dcache-loads,L1-dcache-load-misses,\
LLC-loads,LLC-load-misses,\
dTLB-loads,dTLB-load-misses \
  ./my-service --benchmark
```

A trimmed, illustrative output:

```
     1,204,551,239,004      cycles
       842,113,908,551      instructions      #    0.70  insn per cycle
       310,442,118,900      L1-dcache-loads
        61,884,203,110      L1-dcache-load-misses   #  19.9% of all L1-
dcache accesses
        58,110,442,001      LLC-loads
        41,203,118,774      LLC-load-misses         #  70.9% of all LL-
cache accesses
        22,441,905,003      dTLB-loads
         3,110,884,220      dTLB-load-misses        #  13.9% of all
dTLB accesses
```

Read it as a diagnosis. **IPC of 0.70** on a core capable of 3-4 is the signature of a memory-bound workload — the core is stalling, not computing. A **~20% L1 miss rate** feeding a **~71% LLC miss rate** says a large fraction of accesses fall all the way to DRAM: a working set that does not fit and poor locality. The **~14% dTLB miss rate** flags TLB pressure worth trying huge pages against. To find *where* in the code, `perf record` / `perf report` attributes misses to functions and source lines; `perf c2c` specifically hunts cache-line contention (false sharing) — the subject of Chapter 8. Intel's Top-down Microarchitecture Analysis (TMA), surfaced by tools like `toplev`, classifies stalls into front-end/back-end/memory-bound buckets so you can confirm you are memory-bound before optimizing for it. Volume 2's chapters on observability go deeper; the discipline to carry from here is: **measure the hierarchy directly — do not infer cache behavior from wall-clock time alone.**

Cache-friendly design: the payoff

Everything above converges on a small set of design rules. These are the highest-leverage, lowest-glamour performance techniques in backend engineering, and they follow mechanically from cache lines, locality, and prefetching.

Prefer sequential over random access

Restated as a rule because it is the most important one: lay out data so the hot path walks it *forward*, in order, and let the prefetcher do its job. Sequential access converts compulsory misses into hidden latency; random access exposes every one. When you must access randomly (hashing, indexing), at least make each random access *land in a line whose other bytes you will also use*, so one miss pays for more work.

Arrays beat linked lists (and pointer graphs)

An array of N elements occupies N contiguous, aligned lines. Iterating it is sequential: the prefetcher streams it, and each 64-byte line delivers several elements. A linked list of the same N elements is N separately-allocated nodes scattered across the heap, each holding a pointer to the next. Iterating it is pointer chasing: unpredictable *and* dependent, so every node is an exposed, serialized DRAM miss — and part of each fetched line is wasted on the `next` pointer itself.

The lesson generalizes to any pointer-heavy structure: trees of individually heap-allocated nodes, hash maps with per-entry allocations, graphs of objects. When traversal performance matters, prefer **contiguous, index-based** layouts: array-backed structures, flat arrays with integer indices instead of pointers, arena/pool allocation that packs nodes together, B-trees (wide, cache-line-sized nodes) over binary trees, open-addressing hash tables (probe within a line) over chained ones. The algorithmic complexity may be identical; the constant factor from cache behavior is not.

Struct-of-arrays vs array-of-structs (hot/cold splitting)

Suppose you process a million records but the hot loop only reads two fields out of twenty. In the natural **array-of-structs (AoS)** layout, the twenty fields of each record are contiguous, so each 64-byte line you fetch is mostly the eighteen *cold* fields you did not want — you burn bandwidth and cache capacity on data the loop never touches.

```
// Array-of-structs: hot loop touches 2 of 20 fields, but each line
// is dominated by cold fields. Poor line utilization.
struct Record { int id; float score; /* + 18 cold fields */ };
struct Record records[N];
for (int i = 0; i < N; i++) total += records[i].score;   // strided
over cold fields

// Struct-of-arrays: hot fields are contiguous. Each fetched line is
// all 'score' values. Sequential, dense, prefetchable.
struct Records {
    int   id[N];
    float score[N];   // the hot loop streams this array with perfect
line utilization
    /* cold fields in their own arrays */
};
for (int i = 0; i < N; i++) total += records.score[i];   // pure
sequential stream
```

Struct-of-arrays (SoA) stores each field in its own contiguous array. Now the hot loop over `score[]` touches only lines full of `score` values — every byte fetched is useful, and the access is perfectly sequential. This is the same idea as **hot/cold field splitting**: separate the frequently accessed ("hot") fields from the rarely accessed ("cold") ones so the hot working set is dense and small. Columnar databases and analytics engines are SoA taken to its logical end — storing by column so scans touch only the columns queried — which is *why* columnar formats dominate analytical workloads. SoA also feeds SIMD (Chapter 7): contiguous same-type values vectorize cleanly.

The trade-off is real: SoA hurts when you need *all* fields of *one* record at once (now they are scattered across N arrays), and it complicates code. Choose the layout that matches the *access pattern of the hot path*, not the one that models the domain most naturally.

Pack, align, and shrink

- **Pack** structures to eliminate padding waste, but be aware that packing hot and cold fields into one line can *reduce* line utilization — sometimes deliberate padding to separate them is better. Order fields by access, not by whim.
- **Align** hot, frequently-accessed structures to 64-byte boundaries so they do not straddle two lines (`alignas(64)` in C++, `#[repr(align(64))]` in Rust). For data written by multiple threads, pad to a full line to prevent false sharing (Chapter 8).

- **Shrink** the working set. Smaller types (int32 vs int64), bit-packing, interning, and dropping fields the hot path never reads all pull the working set toward a fast tier. Halving a struct can double how much of it fits in L2.

Blocking / tiling for cache

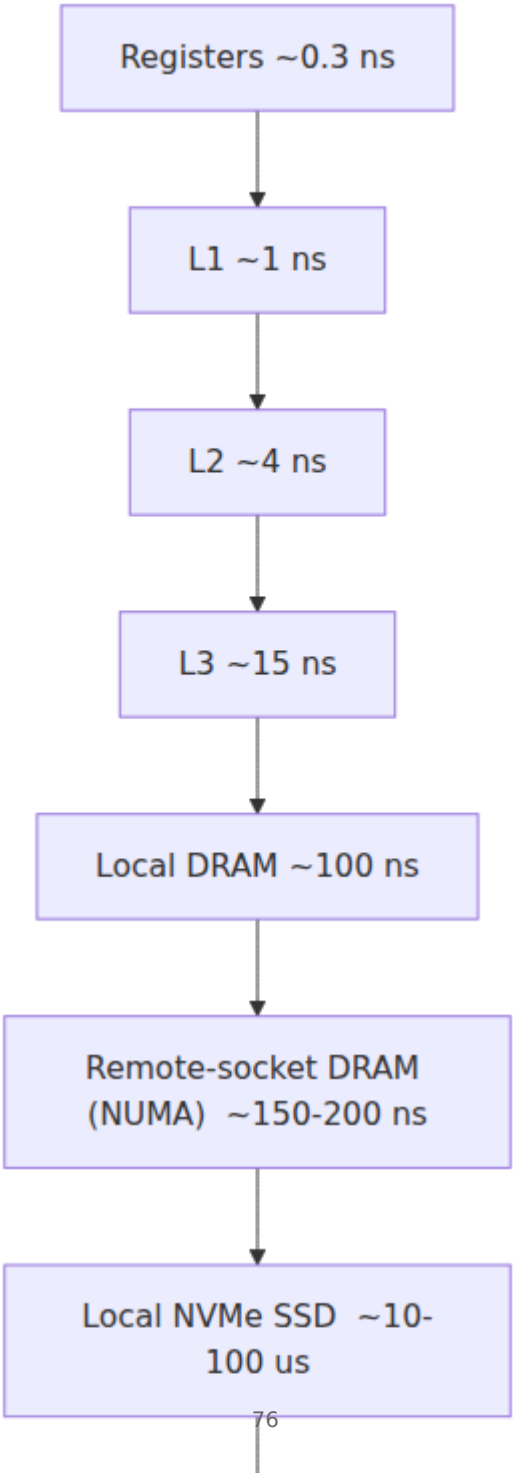
When a computation's working set is larger than a cache but can be decomposed, restructure it to operate on cache-sized **blocks** so each block's data is loaded once and fully reused before moving on. The textbook case is dense matrix multiply: the naive triple loop streams entire rows and columns, evicting data before it is reused (a capacity-miss disaster on large matrices). **Tiled** matrix multiply processes small sub-blocks that fit in cache, reusing each loaded block many times, and can run several times faster for the identical FLOP count. The general principle — *restructure the loop so the inner iterations reuse a cache-resident block* — applies far beyond linear algebra: batch processing, join algorithms, and any nested iteration over large data.

Data-oriented design

Step back and the through-line is **data-oriented design**: design around *how data is accessed and transformed*, not around abstract objects. Ask what the hot path reads, in what order, how often; then lay out memory so that path is sequential, dense, and small. It is a deliberate inversion of object-oriented instinct (one rich object per entity, pointers everywhere), and in performance-critical code it routinely delivers order-of-magnitude wins because it aligns the software with the machine the CPU actually is: a fast core starved by a slow memory, rescued by lines, locality, and prefetching.

Distributed-systems lens: the hierarchy does not stop at the die

Everything in this chapter is a special case of one principle: **data movement dominates cost and latency, so keep data close to where it is used, and organize tiers so the fast/near tier absorbs most of the traffic.** That principle does not care whether the "tier" is an SRAM cache or a datacenter in another region. The memory hierarchy is the innermost few rungs of a ladder that extends all the way out to cross-region replication, and the same reasoning governs every rung.


Registers ~0.3 ns
L1 ~1 ns
L2 ~4 ns
L3 ~15 ns
Local DRAM ~100 ns
Remote-socket DRAM (NUMA) ~150-200 ns
Local NVMe SSD ~10-100 us

Look at the latency scale: each rung is ~10-1000x slower than the one above, exactly like the on-die hierarchy — just extended by twelve orders of magnitude from registers to cross-region. A backend architecture *is* a memory hierarchy design problem wearing different words:

- **Distributed caches are just another tier.** Redis or Memcached in front of a database is the same move as L3 in front of DRAM: a smaller, faster store that absorbs the hits so the slow tier below is spared. The hit-rate math is identical, and so is the failure mode — a cold cache or a poor hit rate exposes the slow tier's latency to every request, the distributed analogue of a working set that does not fit. **CDNs** are the same idea pushed to the network edge: cache content near the user because the "latency" of the far tier is now propagation delay across the planet.
- **"Working set fits in cache" governs in-memory store sizing.** Sizing a Redis cluster or an in-process cache is precisely the L1-vs-DRAM question at fleet scale: does the hot working set fit in RAM, or does it spill to disk/DB and pay the slow tier on every miss? The eviction policy you pick (LRU/LFU) is the same recency/frequency reasoning as a CPU's pseudo-LRU, and a poorly chosen one thrashes the tier exactly as a bad stride thrashes a cache set. Provisioning service memory is working-set analysis: give the hot path enough fast tier that it does not fall to the slow one.
- **Data locality is the distributed spatial locality.** Co-locating a service with its data (same rack, same AZ, same region), partitioning so related data lands on the same shard, and denormalizing so one read fetches everything a request needs — these are spatial locality and struct-of-arrays, re-expressed. "One round trip that fetches everything the request needs" is "one cache line that holds everything the loop touches," scaled up. The N+1 query problem is pointer chasing across the network: a chain of dependent, latency-exposed round trips where a single batched (sequential) fetch would do.
- **The multiplication effect.** A cache inefficiency that costs one core a few nanoseconds per request is multiplied by every request, every core, every host in the fleet, every hour. At scale, a data-layout change that improves LLC hit rate a few points can translate into materially fewer servers and lower power — the fleet-wide version of the same arithmetic. Data movement is the dominant cost at every

scale, so the payoff of reducing it compounds as you multiply the machines.

- **Even "DRAM" is not uniform — NUMA.** On a multi-socket server (Chapter 6), a core's access to memory attached to *another* socket is meaningfully slower than to its local memory — remote DRAM is itself a slower tier. The flat "~100 ns DRAM" is already an approximation the moment you have more than one socket, and thread/memory placement (NUMA affinity) becomes a locality decision at the host level. The hierarchy has non-uniformity baked in before you even leave the box.

The engineer who has internalized cache lines, locality, and working sets already owns the mental model for the whole distributed stack. It is hierarchies of tiers with growing latency, and the game is always the same: keep the working set in the fast tier, move data in useful-sized chunks, and make access patterns predictable enough that the system can run ahead of demand. The constants change by twelve orders of magnitude; the reasoning does not change at all.

Key takeaways

- The **memory wall** is the motivating fact: a DRAM miss costs ~100 ns = hundreds of cycles, the gap has widened for decades, and it makes most backend workloads memory-bound. Caches exist to hide DRAM latency because faster DRAM at capacity is not economical.
- The hierarchy — registers → L1 (~4 cyc) → L2 (~12 cyc) → shared L3 (~40 cyc) → DRAM (~100 ns) — grows ~10-100x in size and latency per step; each tier caches the one below. Order-correct numbers matter; exact figures are vendor/generation-dependent.
- The **cache line (64 bytes)** is the atomic unit of data movement, and almost all cache-friendly advice follows from it: accessing one byte fetches 64, adjacency is free (spatial locality), and false sharing is a line artifact.
- Caches exploit **temporal** (reuse soon) and **spatial** (nearby soon) locality. Cache-friendly code maximizes both; keep the hot **working set** small enough to fit a fast tier.
- An address splits into **tag / index / offset**; **set-associative** mapping balances conflict misses against hardware cost. Misses are **compulsory / capacity / conflict**, and the category dictates the fix.

Real caches use **write-back + write-allocate**, pseudo-LRU eviction, and **dirty** bits.

- **Hardware prefetchers** hide compulsory misses on sequential/strided access, which is why sequential access is far faster than random. **Pointer chasing** is unpredictable *and* dependent — every node is an exposed, serialized DRAM miss — which is why **arrays beat linked lists**.
- The **TLB** is the "other cache," on every access; misses trigger expensive **page walks**, and **huge pages** extend TLB reach for large heaps.
- Practical wins: sequential over random, arrays over pointer graphs, **struct-of-arrays / hot-cold splitting**, packing/alignment, shrinking the working set, and **blocking/tiling** — the essence of **data-oriented design**. Measure it directly with `perf` counters, not wall clock.
- The hierarchy **extends past the die**: distributed caches, CDNs, and data-locality decisions are the same locality principle at larger constants. "Working set fits in cache" sizes Redis and service RAM; N+1 queries are network pointer-chasing; the payoff of reducing data movement multiplies across the fleet.

Further reading

- John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. (Morgan Kaufmann, 2017) — Chapter 2 and Appendix B are the canonical, quantitative treatment of the memory hierarchy, the 3 C's, associativity, and write policies.
- Ulrich Drepper, "What Every Programmer Should Know About Memory" (2007), Red Hat — https://people.freebsd.org/~lstewart/articles/cpumemory.pdf — long, detailed, and still the best single deep dive on cache mechanics and cache-friendly code, despite its age.
- Denning, "The Locality Principle" (Communications of the ACM, 2005) — the foundational articulation of locality and working sets.
- Wulf and McKee, "Hitting the Memory Wall: Implications of the Obvious" (ACM SIGARCH Computer Architecture News, 1995) — the paper that named the memory wall.

- Intel 64 and IA-32 Architectures Optimization Reference Manual — https://www.intel.com/sdm — the authoritative source for prefetcher behavior, cache organization, non-temporal stores, and TLB details on Intel parts (vendor-specific but rigorous).
- Brendan Gregg, *Systems Performance*, 2nd ed. (Addison-Wesley, 2020), and https://www.brendangregg.com — practical guidance on `perf`, PMCs, and diagnosing memory-bound workloads on Linux.
- Agner Fog, "Optimizing software in C++" and the microarchitecture manuals — https://www.agner.org/optimize/ — detailed, measured latency/throughput data across CPU generations.
- "Data-Oriented Design" — Richard Fabian, *Data-Oriented Design* (2018), https://www.dataorienteddesign.com/dodbook/ — the design philosophy of laying out data for the cache, with worked examples.
- Igor Ostrovsky, "Gallery of Processor Cache Effects" — https://igoro.com/archive/gallery-of-processor-cache-effects/ — short, empirical demonstrations of line size, associativity, and false sharing you can reproduce.
- Denis Bakhvalov, *Performance Analysis and Tuning on Modern CPUs* — https://github.com/dendibakh/perf-book — a modern, free treatment of Top-down analysis, `perf`, and memory-hierarchy optimization.

Chapter 4 — Cache Coherence and Hardware Memory Consistency

What this chapter covers. Chapter 3 gave every core its own private, fast caches and showed why that hierarchy is the difference between a core that computes and a core that stalls. But private caches create a problem the moment there is more than one core: the *same* line of memory can sit in several cores' caches at once, and when one core writes, every other copy is instantly, silently wrong. **Cache coherence** is the set of protocols by which the cores conspire to keep their private caches consistent, so that the many-cache reality still presents software with the illusion of a single, shared memory. This chapter explains that machinery — MESI and its variants, snooping versus directories — and then confronts its cost: coherence traffic, true and false sharing, cache-line ping-ponging, and

why an innocuous-looking pair of counters can run ten times slower than it should. From there it climbs to the harder, subtler topic that coherence is *not*: **memory consistency** — the order in which one core's writes to *different* locations become visible to another core, where store buffers, weak memory models, x86-TSO versus ARM, and memory fences live. It closes on the observation that makes all of it worth a backend engineer's time: hardware memory consistency and distributed-systems consistency are the same theory twelve orders of magnitude apart, and the engineer who understands one already understands the other.

Learning goals — after this chapter you should be able to:

- State the **coherence problem** precisely and give the two invariants (write propagation and write serialization) that any coherent system must satisfy.
- Explain **snooping** versus **directory-based** coherence, and why the choice tracks system size — bus snooping for a few cores, directories for many-core and multi-socket (Chapter 6).
- Walk the **MESI** state machine cold: the four states, the transitions on local and remote reads and writes, how an invalidate protocol propagates writes, and how a read of a Modified line gets the freshest data. Place **MESIF** (Intel) and **MOESI** (AMD) as the real-world variants.
- Diagnose **true sharing** (contention on genuinely shared data) and **false sharing** (distinct data colliding on one 64-byte line), quantify the latter's cost, and fix it with padding and alignment (Chapter 8).
- Explain how **atomic read-modify-write** operations (CAS, fetch-add) are implemented on top of coherence — the x86 `LOCK` prefix, ARM's LL/SC — and why contended atomics are a scalability killer.
- Draw the **sharp line between coherence and consistency**: coherence is per-location agreement; consistency is the ordering of operations across *different* locations.
- Compare **sequential consistency**, **x86-TSO**, and **ARM/POWER weak ordering**; explain the store buffer as the root cause of store→load reordering; and place **fences** (`mfence`, `dmb`) and acquire/release semantics as the tools that restore the ordering software needs — the hardware substrate that Volume 4's software memory models compile down to.

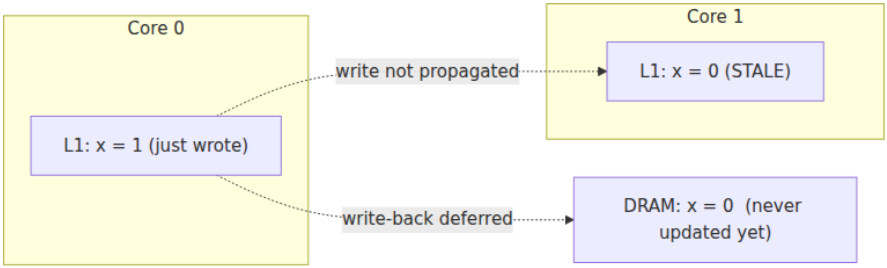
- See the exact analogy between hardware and distributed consistency — coherence ≈ single-object linearizability, consistency ≈ multi-object ordering, Lamport underneath both (Volume 6).

The coherence problem

Return to the machine Chapter 3 built. Each core has a private L1 (split instruction/data) and usually a private L2; the L3, or last-level cache, is shared across the socket. The unit of everything is the 64-byte **cache line**. Now put two cores to work on the same data.

Core 0 reads the line containing a variable `x` (say `x = 0`), fetched from DRAM through L3 into core 0's L1. Core 1 reads `x` too and gets its own copy in its own L1 — harmless, both copies are identical and reading changes nothing. Now core 0 executes `x = 1`, writing into *its* L1 copy. Core 0's L1 now says `x = 1`; core 1's L1 still says `x = 0`. If nothing intervenes, core 1 reads `0` forever — from its point of view the value is right there in L1, so why go to DRAM? Worse, under the write-back policy of Chapter 3, core 0's new value never even reaches DRAM until the line is evicted, so *neither* other caches *nor* memory reflect the write.

This is the **cache coherence problem**: private caches let the same location have multiple independent copies, and an uncoordinated write makes the others stale. Left unmanaged, a multiprocessor would present not shared memory but N disagreeing memories, and no shared-memory program — no lock, no queue, no `volatile` flag — could work.



A memory system is **coherent** if it upholds two invariants:

1. **Write propagation.** A write to a location by one core eventually becomes visible to every other core. No copy stays stale indefinitely.
2. **Write serialization.** All writes to *a single location* are seen by *all* cores in the *same order*. If core A observes the sequence `x=1` then

`x=2`, no other core may observe `x=2` then `x=1`. There is one global order of writes *per location*, and everyone agrees on it.

Note carefully what coherence does *not* promise: nothing about the ordering of writes to *different* locations. That is the domain of memory consistency, the second half of this chapter, and conflating the two is the single most common confusion in this area. Hold the line: **coherence is a per-location guarantee.** Each location behaves, on its own, as if there were one true copy that all cores read and write in a single agreed order — even though the bytes are physically smeared across a dozen caches.

Coherence is implemented in hardware, transparently, on every commodity multicore. Software does not opt in. The interesting questions are *how* the hardware does it and what it *costs*, because the cost lands directly on the performance of every concurrent data structure a backend service runs.

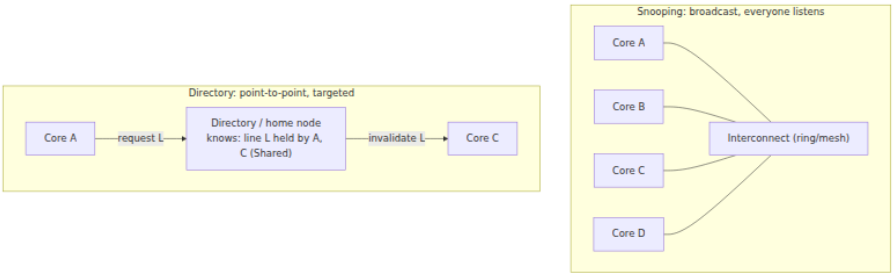
Snooping versus directories

To keep copies consistent, the caches must communicate: when a core wants to write a line, the other cores holding that line must find out and drop or update their copies. There are two architectural families for that communication, and the choice tracks system scale.

Snooping (broadcast) coherence. Historically the caches shared a common bus, and every cache controller *snoops* — watches — every transaction on it. When core 0 wants exclusive ownership of a line to write, it broadcasts an invalidation for that address; every other cache checks whether it holds the line and, if so, invalidates its copy. Reads work the same way: a read request is broadcast, and whichever cache holds the freshest copy (or memory) responds. Snooping is simple and low-latency for small systems because the broadcast *is* the coordination — everyone hears everything. Its fatal flaw is bandwidth: broadcast traffic grows with the number of participants, and a shared bus becomes a bottleneck. Modern chips do not use a literal shared bus but a ring or mesh interconnect with a snoop filter; the logical model — coherence requests are broadcast (or filtered-broadcast) and observed by all — is still snooping, and it dominates within a single socket's handful-to-dozens of cores.

Directory-based coherence. Instead of broadcasting to everyone, the system keeps a **directory**: metadata, distributed alongside memory,

recording *which caches hold each line and in what state*. To write a line, a core consults the directory (a point-to-point message to the line's home node), which knows exactly which cores have copies and sends invalidations only to *those* cores — not a broadcast. This trades latency and complexity (an extra indirection, a directory to maintain) for scalability: coherence traffic is proportional to the number of *sharers* of a line, not the number of cores in the machine. Directories are how coherence scales to many-core chips and, crucially, across **sockets**. On a multi-socket server the inter-socket links (Intel UPI, AMD Infinity Fabric) carry directory-based coherence traffic, and the non-uniform cost of that traffic is exactly the NUMA effect Chapter 6 develops: touching a line owned by another socket's cache means a cross-socket coherence round trip.



The two families are not mutually exclusive. Large systems are hierarchical: snooping (or a snoop filter) within a socket, directories between sockets. The mental model to keep is that coherence is a *messaging protocol between caches* whose cost is the messages it must send — a framing that is the through-line to the distributed-systems lens at the end of the chapter. It is not a metaphor. Coherence *is* a consensus protocol implemented in silicon.

MESI in depth

The protocol that decides what messages to send and when is a state machine. Each cache line, in each cache, carries a few bits of coherence state. The canonical protocol — the one to know cold, because the variants are deltas on it — is **MESI**, named for its four states.

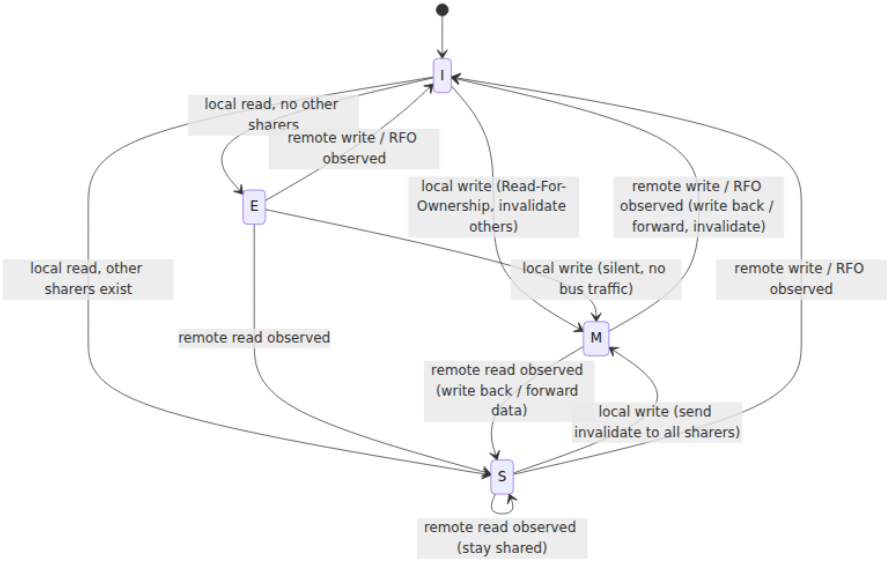
State	Meaning	Other caches may hold it?	Memory up to date?	Can read?	Can write without a bus transaction?
M — Modified	This cache has the only copy, and it is dirty (differs from DRAM).	No	No — this cache owns the truth	Yes	**Yes** — already exclusive and dirty
E — Exclusive	This cache has the only copy, and it is clean (matches DRAM).	No	Yes	Yes	**Yes** — silently transitions to M
S — Shared	This cache has a copy; others may too. Clean.	Yes (possibly)	Yes	Yes	**No** — must invalidate others first
I — Invalid	No valid copy here.	—	—	No (miss)	No

The four states encode two orthogonal bits of information: **is this copy exclusive or shared?** and **is it clean or dirty?** Modified is exclusive+dirty; Exclusive is exclusive+clean; Shared is shared+clean; Invalid is no copy. (There is no "shared+dirty" state in plain MESI — that is exactly the state MOESI adds, below.)

The **E** state is what makes MESI better than the older MSI protocol. When a core reads a line no one else holds, it lands in **E**, not S. If it then writes, it transitions **E → M silently**, no bus transaction, because it already holds the line exclusively — the common case for thread-local data: read then write, no coherence traffic. Had the read landed in S, the write would have needed a bus transaction to invalidate copies that do not even exist. E makes the uncontended case free.

The transitions

The state machine is driven by two kinds of events: **local** events (this core's loads and stores) and **remote** events (coherence messages observed from other cores — another core's read, another core's read-for-ownership/write). The essential transitions:



Read the machine event by event.

Local read of an Invalid line (a read miss). If no other cache holds the line, the data comes from memory (or L3) and the line goes to **E**. If other caches *do* hold it, the reader lands in **S** and any E holder demotes to S. If a remote cache held the line in **M**, that cache supplies the freshest data — writing it back and both settling in S, or forwarding it — because DRAM is stale. This is how a read of a modified line gets the latest value: the protocol routes the read to the owner of the dirty copy, not to stale memory.

Local write of an Invalid or Shared line. To write, the core must own the line exclusively. It issues a **Read-For-Ownership (RFO)**: a request that both fetches the data (if needed) *and* invalidates every other copy, dropping every other holder to **I**. Only once all copies are gone does the writer transition to **M** and store. This is the **invalidate protocol**: a write

does not push the new value to other caches (that would be an *update* protocol, rare in practice); it *removes* the other copies, forcing them to re-fetch on next access. Invalidate wins because one invalidation message is cheaper than broadcasting every new value, and because it serializes writes.

Local write of an Exclusive line. No one else has a copy, so no invalidation is needed: **E → M silently**. The cheap path E exists to enable.

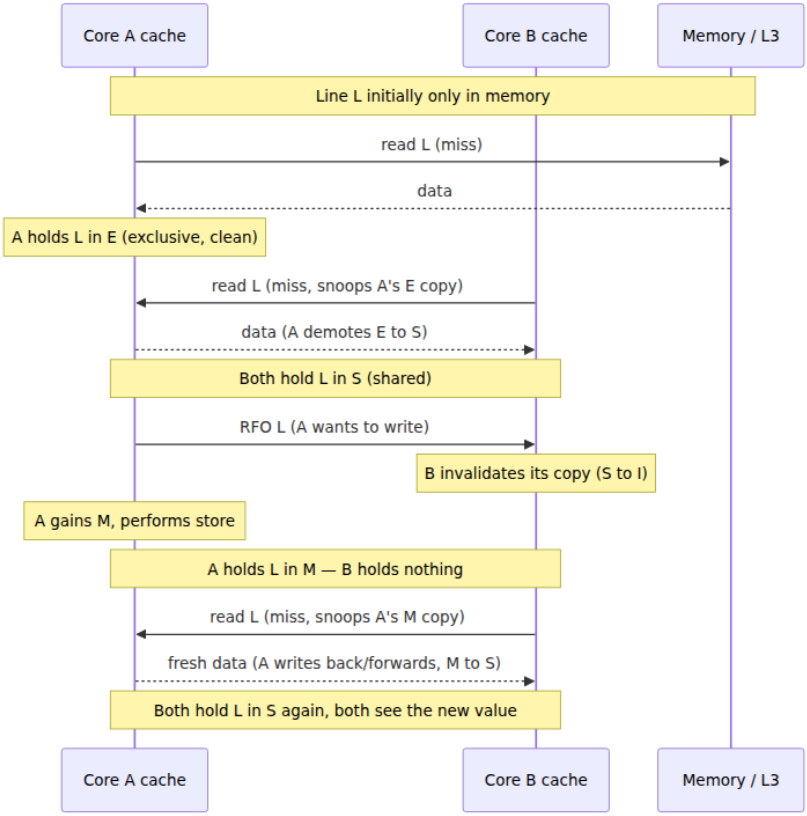
Remote read observed while in M or E. The line must demote: **M → S** (writing back or forwarding the dirty data first, so the reader gets the fresh value) or **E → S**. Now two caches share it, clean.

Remote write / RFO observed. This cache must **invalidate**: M/E/S → **I**. If it was M, it first supplies its dirty data (the new writer needs the current value). Afterward the remote core owns the line in M and this core has nothing.

Write serialization — coherence invariant 2 — falls out of this structure: to write a line you must first gain exclusive ownership via RFO, and only one core can hold M at a time. The ownership hand-offs impose a single global order on the writes to that line, and every core observes writes in that order because it only ever reads the line by acquiring a coherent copy from the current owner.

A concrete scenario: two readers, one writer

Trace the canonical sequence — two cores read a line, then one writes it — through MESI, because this is the pattern under every shared flag, every reference count, every spinlock.



Step by step: A reads L, finds no other holder, caches it in **E**. B reads L, snoops A's copy, A demotes to **S**, both hold L in **S**. A writes L: because L is Shared, A issues an **RFO**, B invalidates (**S → I**), A transitions to **M** and stores. When B next reads L it misses, snoops A's **M** copy, A supplies the fresh data and demotes to **S**, and B gets the new value. Coherence held: B's stale copy was invalidated *before* A's write completed, and B's re-read was routed to the current owner, never to stale memory.

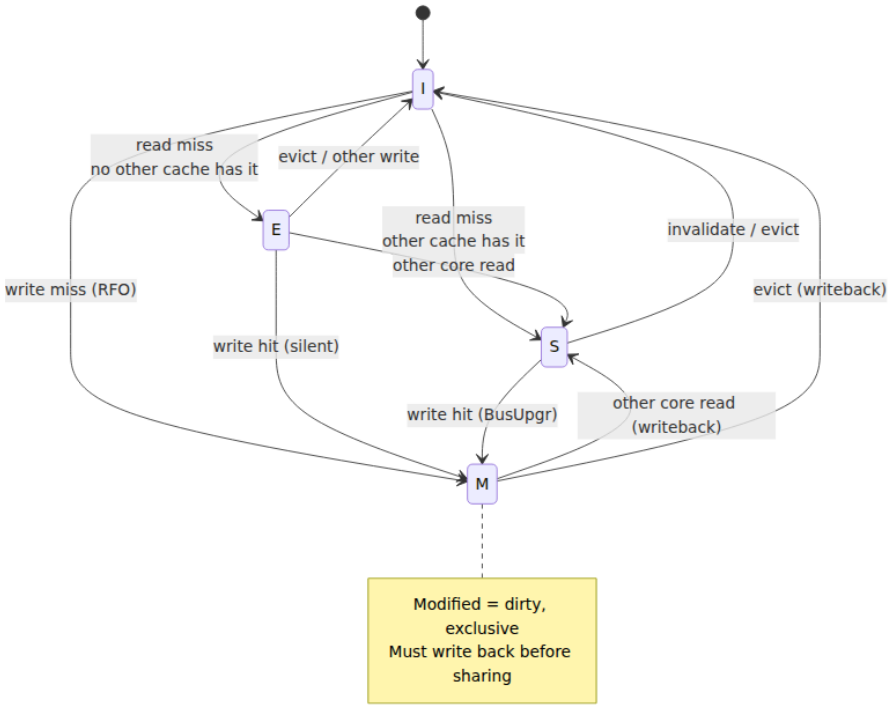
Every arrow crossing between caches is a coherence message with real latency — tens of nanoseconds within a socket, more across sockets. Run once, this pattern is invisible; run in a tight loop on contended data, it is the dominant cost, and the next section is about exactly that.

MESIF and MOESI: the real-world variants

Commodity chips extend MESI with a fifth state to handle the case plain MESI handles clumsily: what happens when a Shared line, held clean by several caches, is requested by yet another cache? In MESI, either memory supplies it (slow) or the responders race (redundant). Both major vendors add a state to designate a single responder, but they solve slightly different problems.

- **MESIF (Intel).** Adds **F — Forward.** Among several caches sharing a clean line, exactly one holds it in **F**; that cache is the designated forwarder that responds to new read requests. The others stay in S and stay silent. This makes cache-to-cache transfer of *clean, shared* data fast (one nearby cache forwards it, rather than a slow memory fetch) while avoiding the redundant responses of naive MESI. F is essentially "Shared, and I'm the one who answers."
- **MOESI (AMD).** Adds **O — Owned.** O is the "shared *and* dirty" state that plain MESI lacks. In MESI, transitioning M → S requires writing the dirty data back to memory so the shared copies are clean. MOESI lets the owner keep the line **dirty** while *also* sharing it: the O holder has the authoritative (dirty) copy, supplies it to readers cache-to-cache, and defers the write-back. Other sharers are in S; memory stays stale until the O line is evicted. This avoids a memory write-back on every M → shared transition — valuable when producer/consumer patterns pass dirty data core-to-core repeatedly.

Both extensions exist to make **cache-to-cache data transfer** the fast path, because in a busy multicore the freshest copy of a contended line usually lives in *another core's cache*, not in memory. You will not program to F or O, but knowing they exist explains why "the data was in another core's cache" is a distinct, often cheaper cost than "the data was in DRAM." For the rest of the chapter, MESI is the model; MESIF/MOESI are the production refinements.



The cost of coherence

Coherence is correct and automatic, but not free, and its cost concentrates precisely where multiple cores touch the same cache lines — which is where concurrent backend code lives. Two phenomena dominate: true and false sharing. Same mechanism (coherence traffic on a contended line), completely different cures.

True sharing and cache-line ping-ponging

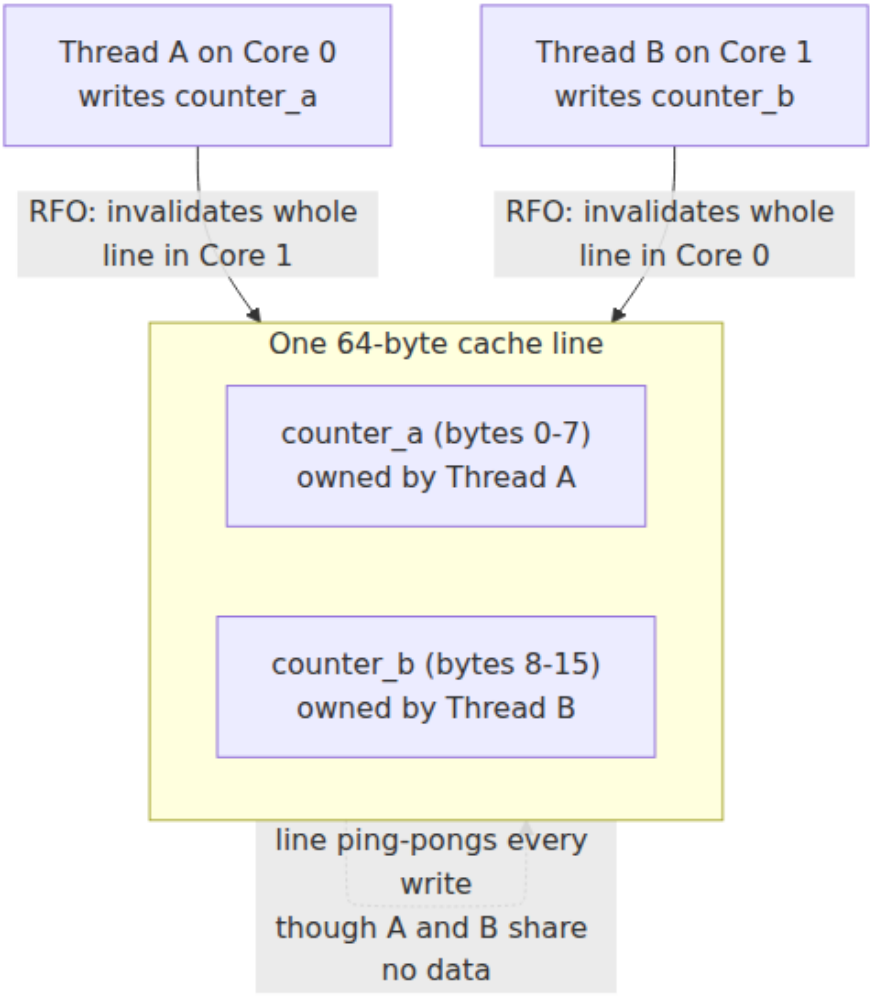
True sharing is contention on data the cores genuinely share. A global counter incremented by every thread, a shared work-queue head, a spinlock word, a reference count on a hot object: these are single locations that many cores read and write. Each write requires an RFO that invalidates every other holder; each subsequent read by another core re-fetches the line from the current owner. The line bounces from cache to cache — **cache-line ping-ponging** — and every bounce is a

coherence round trip. The line spends its life in transit, owned by no core long enough to be useful.

The scalability consequence is stark. A single shared counter under a fetch-add from N cores does not run N times faster; it runs *slower than one core*, because coherence traffic grows with N while useful work does not. The line can be in **M** in only one cache at a time, so the cores serialize on the ownership hand-off *plus* pay interconnect latency the single-core case never pays. This is the mechanical-sympathy face of a deep truth: **shared mutable state does not scale.** The cure is to *stop sharing* — per-thread counters summed on read (sharded counters), thread-local accumulation, combining trees, or partitioning so each core owns its slice. Volume 4 develops the software patterns (`LongAdder`-style structures, per-CPU data); the hardware reason they exist is on this page.

False sharing: the subtle killer

False sharing catches good engineers because the code *looks* contention-free. Two threads update two *different* variables — no shared state in the program's logic — but the variables happen to sit on the *same 64-byte cache line*. Coherence operates on lines, not variables (Chapter 3: the line is the atomic unit), so even though the threads never touch the same datum, every write by one thread invalidates the *whole line* in the other's cache, including the byte the other cares about. The hardware cannot tell the writes are logically independent; it sees two cores writing the same line and ping-pongs it between them exactly as if they were truly sharing.



Make it concrete. Two threads, each hammering its own counter:

```
struct counters {
    long a;   // updated only by thread A
    long b;   // updated only by thread B
};            // sizeof == 16 bytes, both fields on ONE cache line

struct counters ctr;

void *thread_a(void *_) { for (long i = 0; i < 1'000'000'000; i++)
ctr.a++; return 0; }
void *thread_b(void *_) { for (long i = 0; i < 1'000'000'000; i++)
ctr.b++; return 0; }
```

Logically these threads are independent — `a` and `b` are never both touched by anyone. Run them on two cores and the program is often **5–10x slower** than running the same two loops on data in separate lines, and sometimes slower than running the two loops *sequentially on one core*. Every `ctr.a++` on core 0 issues an RFO that invalidates the line in core 1, so core 1's next `ctr.b++` misses and must re-acquire the line, which invalidates core 0's copy, and so on. The line ping-pongs a billion times. No lock, no atomic, no shared variable — just an accident of memory layout, turning two embarrassingly parallel loops into a coherence storm.

The fix is to force the two variables onto **different cache lines** by padding and aligning:

```
struct counters {
    alignas(64) long a;   // start of its own 64-byte line
    alignas(64) long b;   // start of a different 64-byte line
};                        // sizeof == 128 bytes; a and b never share a
line
```

With each counter on its own line, core 0's writes to `a` never invalidate the line holding `b`. Both counters live permanently in their owner's L1 in **M** state, the loops hit no coherence traffic, and the program runs at full speed and scales linearly. The cost is 112 wasted bytes — a trade every performance-sensitive concurrent data structure makes deliberately. C++ standardizes the line size hint as `std::hardware_destructive_interference_size` (the granularity to *separate* to avoid false sharing) and `std::hardware_constructive_interference_size` (the granularity to *pack together* for true sharing benefit); the JVM offers `@Contended` (with `-XX:-RestrictContended`); Go and Rust libraries pad hot per-CPU structures the

same way. Chapter 8 treats alignment and struct layout as a first-class design concern; false sharing is its most expensive failure mode.

False sharing is nasty in production because it is invisible in the source: no shared state, no lock contention, nothing a code review flags. It shows up only under profiling — a high `mem_load_l3_hit_retired.xsnp_hitm` (cross-core snoop hit on a modified line, "HITM") on Intel `perf` is the fingerprint — or as a mysterious refusal to scale past a couple of cores. Canonical sightings: adjacent fields of a hot struct written by different threads; an array indexed by thread ID (`counts[thread_id]++` with 8-byte elements packs eight threads' counters into one line); the head and tail pointers of a concurrent queue on the same line, so producers and consumers fight over it. The defense is a habit: any field written by one thread and sitting near a field written by another is a false-sharing candidate — separate them by a line.

Atomic operations at the hardware level

Coherence keeps individual reads and writes consistent, but concurrent algorithms need more: an **atomic read-modify-write (RMW)** — read a value, compute a new one, write it back, with *no other core able to intervene* in between. `compare-and-swap` (CAS), `fetch-and-add`, `exchange`, and `test-and-set` are the primitives every lock, every lock-free queue, every reference count is built on. They are not free coherence rides; they are the most expensive operations on the coherence fabric, and understanding why is the key to understanding the scalability ceiling of any lock-based or lock-free design.

How an atomic RMW uses coherence. To perform, say, `fetch_add(&x, 1)` atomically, the core must hold the line containing `x` in **M** (exclusive, dirty) for the entire duration of read-compute-write, and it must prevent any other core from stealing the line in the middle. There are two hardware strategies:

- **x86: the `LOCK` prefix.** `lock xadd`, `lock cmpxchg`, `lock inc` acquire the line exclusively (RFO to M) and hold it locked against snoops for the RMW's duration, so no other core observes or modifies it mid-operation. When the operand fits within one cache line (the normal case) this is a **cache lock** — cheap-ish, local to the coherence fabric. Only a split operand straddling two lines falls back to a **bus lock** that locks the whole memory subsystem — catastrophically slow, and why

you keep atomics naturally aligned. Modern Intel parts fault or heavily penalize such split-lock operations to protect against a noisy neighbor holding the bus.

- **ARM / RISC / POWER: load-linked / store-conditional (LL/SC).** Rather than locking, these architectures use an *optimistic* pair. `LL` (load-linked / load-exclusive, `LDXR` on ARM) reads the location and sets a monitor on the line. Code computes the new value. `SC` (store-conditional, `STXR`) writes it back *only if* no other core touched the line since the LL — otherwise SC fails and the code retries the whole sequence in a loop. The hardware monitor is driven by coherence: a remote RFO to the monitored line clears the reservation, making the SC fail. This is lock-free at the ISA level and maps naturally onto weakly-ordered machines. ARMv8.1's LSE atomics (`LDADD`, `CAS`, `SWP`) add true single-instruction atomics on top, which scale better than LL/SC retry loops under contention.

Why atomics are expensive has two components that map onto the two halves of this chapter. First, **coherence**: an atomic RMW *must* acquire the line exclusively (M), so it can never be satisfied by a shared copy the way a plain read can — every atomic is at least an RFO, and a coherence transfer if another core holds the line. Second, **ordering**: atomics carry memory-ordering semantics (below), and enforcing them constrains the store buffer and the out-of-order engine (Chapter 2), draining pipelines and blocking reordering the core would otherwise do for speed. An uncontended atomic on a line already in M costs a few to a couple dozen cycles (mostly ordering); a *contended* atomic — the line bouncing between cores — costs a full cross-core coherence round trip *per operation*, tens to hundreds of cycles, and serializes the cores on the line.

Contended atomics are a scalability killer for the same reason true sharing is: they force the line to ping-pong and add ordering fences on top. A spinlock hammered by 32 cores spends almost all its cycles moving the lock line between caches, not working. This is why modern concurrency avoids centralized atomics: back-off, MCS/CLH queue locks that spin on *local* memory instead of a shared word, per-CPU data with no cross-core atomics, and lock-free structures engineered to minimize contended CAS on the hot path. Volume 4 covers those designs; the hardware reason they are necessary — an atomic is a coherence exclusive-acquire plus an ordering fence, both exploding under contention — is here.

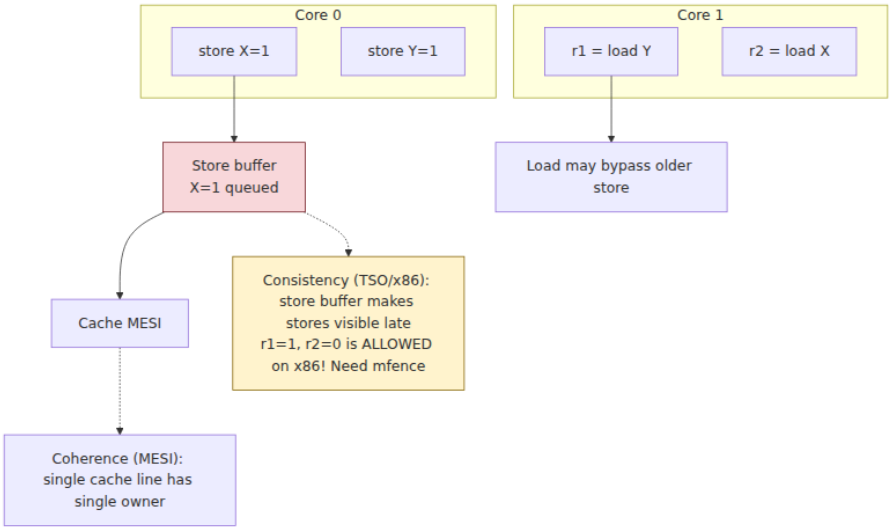
Coherence is not consistency

Now the pivot, and the most important conceptual distinction in the chapter. Everything so far — MESI, snooping, invalidation, atomics acquiring lines — is about a **single memory location** and keeping its many cached copies in agreement. That is **coherence**. But concurrent programs care about a second, harder question that coherence says *nothing* about: given writes to *different* locations, in what order do other cores observe them?

Make the distinction sharp with the two questions:

	Coherence	**Consistency (memory ordering)**
Scope	A single location	Multiple, different locations
Question	Do all cores agree on the order of writes to `x`?	Do all cores agree on the *relative* order of a write to `x` and a write to `y`?
Guaranteed by	Cache coherence protocol (MESI) — always, on all commodity hardware	The memory **consistency model** (SC, TSO, weak) — varies by architecture
Analogy (Vol 6)	Single-object linearizability	Multi-object / cross-key ordering

The classic program that exposes the gap: two locations `x` and `y`, both initially 0; core 0 runs `x = 1; r1 = y;` and core 1 runs `y = 1; r2 = x;`. Coherence guarantees each location behaves sanely on its own. But it does *not* forbid the outcome `r1 == 0 && r2 == 0`: both cores read the *old* value of the other's variable, as if each store had not happened when the other's load ran. On a sequentially consistent machine that is impossible (no interleaving of the four operations produces it); on real x86, ARM, and POWER hardware, **it happens** — and coherence is not violated, because coherence never promised anything about the *cross-location* ordering of `x`'s write relative to `y`'s read. That is a *consistency* question, answered by the memory model. Coherence is necessary but not sufficient: a machine can be perfectly coherent and still reorder operations across locations in ways that break naive concurrent code. The rest of the chapter is about that reordering.



Memory consistency models

A **memory consistency model** is the contract between the hardware and software specifying which reorderings of memory operations (to different locations) the hardware is allowed to perform — that is, which outcomes a multithreaded program can legally observe. It is the formal answer to "what can another core see, and in what order?"

Sequential consistency: the intuitive ideal

Sequential consistency (SC), defined by Leslie Lamport in 1979, is the model programmers intuitively assume. Its definition: the result of any execution is the same as if all cores' operations were executed in *some single sequential order*, and each core's operations appear in that order *in the order the program issued them*. In plain terms: pick some interleaving of all the threads' memory operations that respects each thread's program order; the machine behaves as if that interleaving actually happened. No operation appears to move before an earlier operation from the same thread.

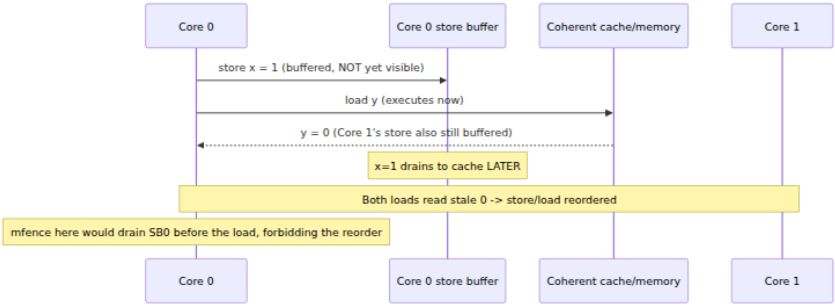
SC makes the `r1==0 && r2==0` outcome impossible, and it is what you *wish* the machine gave you. The problem is performance. SC effectively forbids the store buffer and most of the reordering the out-of-order engine (Chapter 2) wants to do, forcing each memory operation to

become globally visible before the next begins; a store that misses in cache would stall every subsequent operation, including independent ones, for the full coherence latency. No mainstream CPU implements SC in hardware, for the same reason no database defaults to serializable isolation: the strongest, most intuitive model is the slowest, so designers relax it and hand software the tools to recover strictness where it matters.

Why hardware reorders: the store buffer

The single most important source of relaxation on modern hardware is the **store buffer** (Chapter 2's write path). When a core executes a store, it does not wait for the store to reach the cache and propagate coherently — that could take hundreds of cycles on a miss. Instead it drops the store into a **store buffer**, a small FIFO of pending writes, and moves on immediately. The store retires from the buffer to the L1 cache in the background, once the line is owned. This is essential for performance: it lets the core keep executing past a store that missed, overlapping the store's coherence cost with later work.

But it breaks SC in a specific way. Consider a core that executes `store x = 1` then `load y`. The store sits in the buffer, not yet visible to other cores, while the load of `y` — a *different* location — executes and completes. So from another core's perspective, this core's **load of `y` happened before its store to `x` became visible** — a **store→load reorder**. That is exactly the mechanism behind `r1==0 && r2==0`: each core's store is stuck in its buffer while its load reads the other's stale value. The store buffer is not a bug but a universal optimization, and its consequence — loads can appear to overtake earlier stores — is the defining relaxation of x86's memory model.



x86-TSO versus ARM/POWER: strong versus weak

Real architectures sit on a spectrum from SC (strongest, unimplemented) to very weak. Two points on that spectrum matter for backend work.

x86-TSO (Total Store Order) is the memory model of Intel and AMD x86 — *relatively strong*. Its one significant relaxation is the store buffer's: **a load may be reordered before an earlier store to a different location** (store→load). Everything else is kept in order: store→store is preserved (stores become visible in program order — hence "Total Store Order"), and so are load→load and load→store. The store buffer is FIFO and drains in order, and a core sees its *own* stores immediately (store forwarding). TSO is close enough to SC that much racy-but-lucky x86 code happens to work — a trap when the same code is ported to ARM.

ARM and POWER are weakly ordered. They relax *almost all* orderings — store→store, load→load, load→store, and store→load can all be reordered — subject only to single-thread data dependencies and per-location coherence. Two independent stores can become visible in the opposite order from program order; two independent loads can complete out of order. This gives the hardware far more reordering freedom (and simpler, lower-power machinery), at the cost of demanding software insert explicit ordering wherever it needs order. POWER is weaker still (it does not even guarantee a single global store order). The practical upshot is the porting hazard: **code accidentally correct on x86-TSO because TSO only reorders store→load will break on ARM Graviton**, where load→load and store→store reorder freely. As ARM servers went mainstream (Chapter 2), this became a real source of bugs surfacing only on the ARM fleet.

Reordering allowed?	Sequential Consistency	x86-TSO	ARM / POWER (weak)
Store → Store	No	No	**Yes**
Load → Load	No	No	**Yes**
Load → Store	No	No	**Yes**
Store → Load (diff. loc.)	No	**Yes**	**Yes**

Reordering allowed?	Sequential Consistency	x86-TSO	ARM / POWER (weak)
Own store visible to self early	—	Yes (store forwarding)	Yes
Single global store order	Yes	Yes	Not guaranteed (POWER)

Two takeaways from the table. First, *every* real model allows store→load reordering, because every real machine has a store buffer — even the strong x86. Second, the gap between TSO and weak is *which other* reorderings are allowed, and it is large: three additional reordering freedoms that weak machines have and x86 does not. Portable concurrent code cannot rely on the model; it must state its ordering requirements explicitly, which is what fences and the language memory models do.

Memory barriers and fences

If the hardware reorders memory operations for speed, software needs a way to say "not here — this ordering is load-bearing." That tool is the **memory barrier** (fence): an instruction that constrains the reordering of memory operations around it. Fences are how a program recovers exactly as much ordering as it needs and no more, paying the performance cost only at the specific points where correctness demands it.

x86 fences. Because x86-TSO already preserves most orderings, x86 needs few fences. The important one is `mfence` (full fence): it prevents any memory operation before it from reordering with any after it, and in particular **drains the store buffer** — all buffered stores become globally visible before any later load executes. `mfence` is precisely what forbids the store→load reorder, closing the `r1==0 && r2==0` hole. `sfence` / `lfence` order stores/loads respectively and are rarely needed on TSO for ordinary code (they appear around non-temporal streaming stores, which are *not* TSO-ordered, and in speculation control). Note that a `lock`-prefixed atomic *also* acts as a full fence — which is why atomics carry an ordering cost on top of coherence, and why you rarely need a separate `mfence` beside one.

ARM fences. Weak ordering means ARM needs fences far more often. `DMB` (Data Memory Barrier) orders memory accesses around it (scoped, e.g. `DMB ISH`). `DSB` (Data Synchronization Barrier) is stronger — it blocks

execution until prior accesses *complete*. `ISB` flushes the pipeline for self-modifying code and system-register changes. ARMv8 also provides ordered load/store variants — **load-acquire (`LDAR`)** and **store-release (`STLR`)** — that bake the common ordering into the memory instruction, usually cheaper and more precise than a standalone `DMB`.

Acquire and release semantics are the ordering vocabulary you will actually think in, because they are what language memory models expose (Volume 4). They are *one-way* barriers, which is what makes them cheaper than a full fence:

- A **load-acquire** guarantees that no memory operation *after* it (in program order) is reordered *before* it. It is the read side of a handoff: once you acquire, everything the producer did before its release is visible to you. Reads/writes cannot "leak up" above an acquire.
- A **store-release** guarantees that no memory operation *before* it is reordered *after* it. It is the write side: everything you did before the release is published atomically-visibly to whoever acquires. Reads/writes cannot "leak down" below a release.

Together they implement the **publish/subscribe** pattern at the heart of every lock and lock-free handoff: a producer writes data, then does a store-release on a flag; a consumer does a load-acquire on the flag, and if it sees the flag set it is guaranteed to see all the data the producer wrote *before* the release. Acquire/release is strictly weaker than a full fence — it orders one direction, not both — which is why it is the default: it buys exactly the ordering a handoff needs without the two-directional store-buffer drain of `mfence`/`DMB`. On x86-TSO, acquire/release loads and stores are often *free* (plain `mov`) because TSO already provides the ordering; on ARM they compile to `LDAR`/`STLR`. Same source, different cost, because the memory model differs.

This is the hardware substrate for Volume 4. High-level languages do not make you write `mfence` or `DMB`. They define a *software* memory model — `std::memory_order_acquire`/`release`/`seq_cst` in C++11, the happens-before edges of the Java Memory Model, Go's `sync/atomic`, Rust's `Ordering` — and the *compiler* lowers those abstractions to the right fences per target: the same acquire/release C++ source emits nothing on the load path on x86-TSO but `LDAR`/`STLR` on ARM, the minimum barriers that honor the language's promised ordering. Volume 4 is entirely about that upper layer — data races, happens-before, the C++/

Java/Go memory models, lock-free algorithms — and it *compiles down to the fences on this page*. When Volume 4 says a release-store synchronizes-with an acquire-load, the machine underneath is doing store-buffer discipline and `STLR` / `LDAR` or `mov`. Coherence and consistency are the floor; the software memory model is the language you reason in on top of it.

Distributed-systems lens

The reason this chapter belongs in a backend engineer's education is not that you will hand-tune MESI transitions. It is that **the entire problem — and its entire theory — is the distributed systems problem, one abstraction layer down.** The parallels are not analogies of convenience; they are the same mathematics applied at different scales, and seeing that is a genuine unlock.

Coherence is distributed consensus in hardware. Cores with private caches that must agree on the current value of a location are *replicas* that must agree on the current value of an object. The coherence protocol — request ownership, invalidate other copies, serialize writes through a single owner — is a replication/consensus protocol. RFO-and-invalidate is a single-leader scheme: to write you must become the leader (M owner) for that line, and only one exists at a time — exactly how a single-leader replicated log serializes writes to a key. The directory is a metadata service tracking which replicas hold which objects in which state, the role a placement service plays in a distributed store. Cross-socket coherence traffic on UPI/Infinity Fabric (Chapter 6) *is* cross-replica coordination latency, and NUMA is the hardware's version of "some replicas are in another datacenter."

The consistency models are literally the same models. This is the highlight. The hierarchy of hardware memory models maps one-to-one onto the hierarchy of distributed consistency models, and Lamport is underneath *both*:

Hardware (this chapter)	Distributed systems (Volume 6)	What it guarantees
Cache coherence (per location)	**Linearizability** of a single object	One object behaves as one copy with a real-time-consistent total order of ops

Hardware (this chapter)	Distributed systems (Volume 6)	What it guarantees
Sequential consistency (Lamport 1979)	**Sequential consistency** (same name, same author)	A single total order consistent with each thread's/client's program order; no real-time guarantee across objects
x86-**TSO** / weak ordering	**Causal / bounded-staleness** models	Some orderings preserved (causal, or store order), others relaxed for performance
No ordering / racy	**Eventual consistency**	Writes propagate eventually; readers may see stale/reordered values until they converge

This is no coincidence of naming: Lamport's 1979 paper "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs" *defined* SC for hardware, and the identical definition is the SC distributed systems use. Linearizability (Herlihy and Wing, 1990) is SC *plus* a real-time ordering constraint, and it is precisely what coherence provides per location: a single location, viewed across all cores, is a *linearizable register*. The `r1==0 && r2==0` litmus test is the hardware twin of a distributed anomaly where two clients each read the other's key before the other's write propagates — same shape, same cause (relaxed cross-object ordering), same fix (a synchronization point: a fence in hardware, a coordination round trip in the distributed system).

Coordination is expensive — at every scale. The chapter's cost story is the distributed cost story. An atomic RMW acquires a line exclusively and fences: a coordination round trip. Contended atomics collapse throughput because coordination serializes the cores — exactly as a distributed system routing every write through one consensus group collapses because coordination serializes the clients. False sharing is *accidental* coordination — two parties forced to coordinate because they landed on the same coherence unit though they share no logic — whose distributed twin is two unrelated keys hashed to the same partition. The fix is identical in spirit: **change the layout so independent things stop sharing a unit of coordination** — pad to separate cache lines; re-shard so hot keys land on different partitions.

Which yields the unifying principle, the mechanical-sympathy version of the distributed mantra: **shared mutable state is the enemy of scale, in silicon exactly as across a network.** The cures are the same: partition so each worker owns its slice (per-core data ↔ sharding); prefer local to shared state (`LongAdder` ↔ CRDTs); minimize writes that must coordinate (lock-free hot paths ↔ fewer cross-partition transactions); accept the weakest consistency the problem tolerates (acquire/release instead of `seq_cst` ↔ causal/eventual instead of linearizable). A backend engineer who has internalized why 32 cores hammering one atomic run slower than one core has *already internalized* why a system funneling every write through one leader does not scale — and the reason to prefer share-nothing architectures (Volume 7) is the reason to prefer per-CPU data structures (Volume 4). The constants differ by twelve orders of magnitude; the theory — Lamport's — does not change at all.

Key takeaways

- **Coherence** keeps the many cached copies of a *single* location in agreement, guaranteeing **write propagation** (writes eventually become visible) and **write serialization** (all cores see writes to one location in the same order). It is automatic on all commodity hardware.
- **MESI** is the canonical protocol: **M**odified (exclusive dirty), **E**xclusive (exclusive clean — enables the silent E→M write with no bus traffic), **S**hared (clean, possibly-shared, write needs an invalidate), **I**nvalid. Writes use **Read-For-Ownership** to invalidate other copies (invalidate protocol); a read of a Modified line is routed to the dirty owner, not stale memory. **MESIF** (Intel, adds Forward) and **MOESI** (AMD, adds Owned = shared+dirty) optimize cache-to-cache transfer.
- **Snooping** (broadcast, all caches observe) suits a socket's cores; **directories** (targeted, track sharers) scale to many-core and cross-socket, where they are the NUMA coherence traffic of Chapter 6. Coherence is a message-passing consensus protocol in hardware.
- **True sharing** (genuinely shared hot data) and **false sharing** (distinct variables colliding on one 64-byte line) both cause **cache-line ping-ponging** — the line bounces between cores, one coherence round trip per bounce. False sharing is invisible in source and can cause **5–10x slowdowns**; fix it by **padding/aligning** hot

per-thread fields to separate cache lines (Chapter 8). The `perf` fingerprint is cross-core snoop hits on modified lines (HITM).

- **Atomic RMW** (CAS, fetch-add) must acquire the line **exclusively** (M) and enforce ordering: x86 `LOCK`-prefix (cache lock, also a full fence), ARM **LL/SC** or LSE atomics. They cost coherence *plus* ordering, and **contended atomics serialize cores** — the scalability killer behind per-CPU data and queue locks (Volume 4).
- **Coherence ≠ consistency.** Coherence is per-location; **consistency** is the ordering of operations to *different* locations as seen by other cores. The `r1==0 && r2==0` litmus test is legal on real hardware without violating coherence.
- **Sequential consistency** is the intuitive, unimplemented ideal. Real hardware relaxes it for speed, driven mainly by the **store buffer**, which lets a **load overtake an earlier store to a different location**. **x86-TSO** relaxes *only* store→load; **ARM/POWER** relax store→store, load→load, and load→store as well — so racy code that survives on x86 can break on ARM Graviton.
- **Fences** restore ordering where needed: x86 `mfence` (drains the store buffer), ARM `DMB` / `DSB`, and the one-way **acquire/release** semantics (`LDAR` / `STLR`) that implement publish/subscribe handoffs cheaply. This is the substrate that **Volume 4's software memory models** (C++/Java/Go/Rust atomics) compile down to.
- **The distributed-systems parallel is exact.** Coherence ≈ single-object **linearizability**; hardware consistency models ≈ distributed consistency models (SC/TSO/weak ↔ sequential/causal/ eventual), **Lamport** underneath both. Atomics-are-expensive ↔ coordination-is-expensive; false sharing ↔ accidental co-partitioning; the cure in both worlds is **stop sharing the unit of coordination** — partition, per-core/per-node state, share-nothing (Volumes 6 and 7).

Further reading

- Leslie Lamport, "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs," *IEEE Transactions on Computers*, 1979 — the paper that defined sequential consistency; the common ancestor of hardware and distributed consistency.

- Maurice Herlihy and Jeannette Wing, "Linearizability: A Correctness Condition for Concurrent Objects," *ACM TOPLAS*, 1990 — defines linearizability; the per-object guarantee coherence provides.
- Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, David A. Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd ed. (Morgan & Claypool, 2020) — the definitive, rigorous modern treatment of both topics together; the single best source for this chapter's material.
- Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. (Morgan Kaufmann, 2017), Chapter 5 — coherence protocols, snooping vs directory, consistency models.
- Sewell, Sarkar, Owens, Nardelli, Myreen, "x86-TSO: A Rigorous and Usable Programmer's Model for x86 Multiprocessors," *Communications of the ACM*, 2010 — the formal, verified model of x86 memory ordering; the authority for the TSO claims here.
- Paul E. McKenney, *Is Parallel Programming Hard, And, If So, What Can You Do About It?* — https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html — deep, practical coverage of memory barriers, RCU, per-CPU data, and real weak-memory hardware.
- Ulrich Drepper, "What Every Programmer Should Know About Memory" (2007) — https://people.freebsd.org/~lstewart/articles/cpumemory.pdf — Section 6 covers coherence, atomics, and false sharing with measurements.
- Preshing on Programming, "Memory Barriers Are Like Source Control Operations," "Acquire and Release Semantics," and the weak-vs-strong-ordering posts — https://preshing.com/ — the clearest informal explanations of acquire/release and hardware reordering for practitioners.
- Intel 64 and IA-32 Architectures Software Developer's Manual, Vol. 3A, "Memory Ordering," and the Arm Architecture Reference Manual (ARMv8-A), "Memory model" — https://www.intel.com/sdm and https://developer.arm.com/documentation/ — authoritative per-architecture ordering rules, `LOCK`, `mfence`, `DMB` / `DSB`, `LDAR` / `STLR`.
- Hans-J. Boehm and Sarita Adve, "Foundations of the C++ Concurrency Memory Model," *PLDI 2008* — the bridge from these hardware models to the language memory models of Volume 4.

- Herb Sutter, "Eliminate False Sharing," Dr. Dobb's, 2009 — a practitioner walk-through of false sharing and padding fixes with numbers.

Chapter 5 — Storage Hardware: HDD, SSD, NVMe, and Persistent Memory

What this chapter covers. Chapter 3 built a hierarchy of tiers with growing latency — registers, caches, DRAM — and stopped at the edge of the box. This chapter goes one tier further down, to the tier where data actually *survives*: persistent storage. Everything a backend engineer cares about preserving — the write-ahead log, the committed transaction, the Kafka partition, the object in the bucket — ultimately lands on a spinning platter, a NAND flash cell, or (briefly, historically) a persistent-memory module. And the *physics* of those media is not a curiosity: it is the hidden premise behind almost every design decision in a data system. Append-only logs, LSM trees, B-tree node sizes, the obsession with `fsync`, the difference between a "committed" write and a durable one — none of these make sense until you understand what the hardware underneath is actually doing. This chapter explains how each storage technology works mechanically, what performance and durability guarantees it really provides, and how those guarantees propagate up into the correctness of distributed systems. It is, deliberately, the hardware chapter that sets up Volume 5 (Databases).

Learning goals — after this chapter you should be able to:

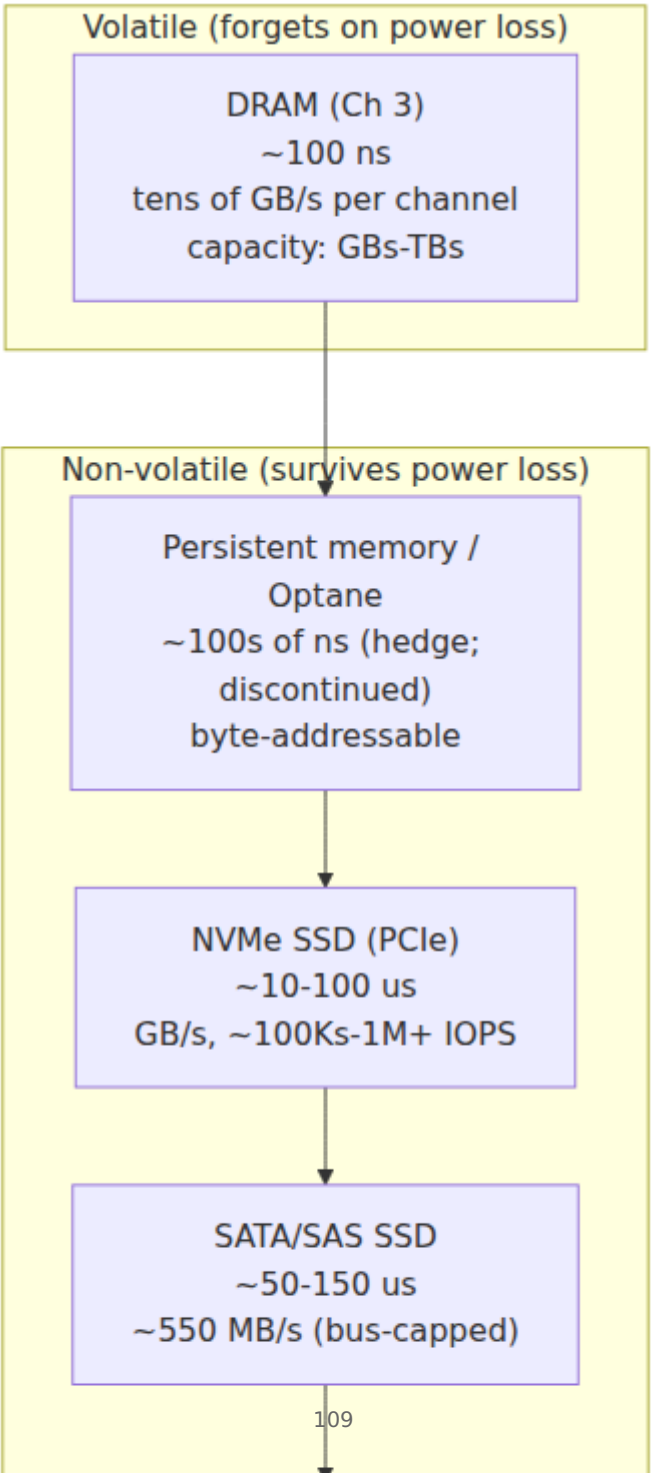
- Place **storage in the hierarchy** below DRAM, and reason about the latency / bandwidth / persistence / cost trade-offs that separate the tiers by orders of magnitude.
- Explain **HDD mechanics** — seek time and rotational latency — and derive from them why random I/O is catastrophic on disk and why storage engines evolved toward **sequential, append-only** access.
- Explain **NAND flash**: cells (SLC/MLC/TLC/QLC), the page/block asymmetry, **erase-before-write**, and the **Flash Translation Layer** — wear leveling, garbage collection, write amplification, and the endurance limits (DWPD/TBW) they exist to manage.

- Explain why **NVMe over PCIe** displaced SATA/AHCI: deep parallel queues, low latency, high IOPS, and how NVMe-oF disaggregates storage across a network.
- Describe **persistent memory** (Intel Optane / 3D XPoint) accurately, including its byte-addressable programming model, its durability/ordering challenges, and its **discontinuation**.
- Reason precisely about **durability**: what "the write is persistent" actually requires, the path from application through page cache and drive cache to media, and where `fsync`, FUA, and **power-loss protection** actually make the guarantee.
- Use the right **performance metrics** — IOPS vs bandwidth vs latency, queue depth, random vs sequential — and connect them to cloud storage abstractions (provisioned IOPS, network-attached disks).

The storage tier: below DRAM, and persistent

Chapter 3's hierarchy had a floor of DRAM at ~100 ns. Storage sits below that floor and differs from every tier above it in one categorical way that dwarfs all the quantitative differences: **it is non-volatile.** DRAM forgets everything the instant power drops; storage remembers. That single property is why storage exists, and also why it is slow — the mechanisms that make a bit *stick* (magnetizing a platter region, trapping charge on a floating gate) are inherently costlier to read and write than flipping an SRAM cell or refreshing a DRAM capacitor.

The result is a chasm. Where the memory hierarchy spans registers (sub-nanosecond) to DRAM (~100 ns) — three orders of magnitude — the jump from DRAM to storage adds several more. A local NVMe SSD read is on the order of **tens of microseconds**; a random HDD read is **milliseconds**. From an L1 hit (~1 ns) to an HDD seek (~10 ms) is roughly **seven orders of magnitude** — the difference between one second and four months. It is why data systems are, at bottom, the art of not going to storage — and of going there cheaply when forced to.


Volatile (forgets on power loss)
DRAM (Ch 3)
~100 ns
tens of GB/s per channel
capacity: GBs-TBs
Non-volatile (survives power loss)
Persistent memory /
Optane
~100s of ns (hedge;
discontinued)
byte-addressable
NVMe SSD (PCIe)
~10-100 us
GB/s, ~100Ks-1M+ IOPS
SATA/SAS SSD
~50-150 us
~550 MB/s (bus-capped)

The table below gives order-correct figures. As in Chapter 3, treat every number as an approximation whose **order of magnitude is the durable fact**; exact values depend on the specific part, generation, and workload. What matters is the *shape*: each row down is roughly an order of magnitude slower and cheaper per byte than the row above, and the persistence column is the reason any of the lower rows exist at all.

Tier	Read latency (typical)	Bandwidth (typical)	Random IOPS (typical)	Persistent?	$/GB (relative)
DRAM	~100 ns	tens of GB/s per channel	n/a (byte-addressable)	No	highest
Persistent memory	~100s of ns (hedge)	several GB/s	very high	Yes	high (historic)
NVMe SSD (PCIe)	~10-100 us	~3-14 GB/s (Gen3-Gen5 x4)	~100Ks to 1M+	Yes	medium
SATA/ SAS SSD	~50-150 us	~550 MB/s (SATA III cap)	~50K-100K	Yes	medium-low
HDD	~5-15 ms (random)	~100-260 MB/s (sequential)	~75-200	Yes	lowest

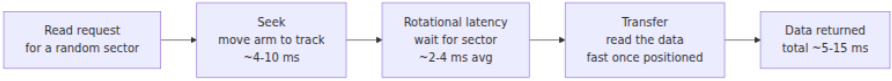
Read the IOPS column against the latency column and a pattern jumps out that governs the rest of this chapter: **the gap between random and sequential access widens as the medium gets slower.** On DRAM, random vs sequential differs by a small factor (prefetching, row buffers). On an SSD, random 4K reads are slower than sequential but still respectable. On an HDD, random access is *two to three orders of magnitude* worse than sequential, because a spinning disk must physically move to reach a random location. Storage physics does not just make storage slow; it makes *access-pattern discipline* the single highest-leverage decision in a storage engine's design.

HDDs: mechanical latency and the tyranny of the seek

A hard disk drive is the last electromechanical component in a modern server. It matters even though flash has displaced it for hot data — because the design lessons HDDs burned into database engineering are still with us, and because HDDs remain the economical choice for cold, bulk, and archival storage where capacity per dollar dominates.

An HDD stores bits as magnetized regions on the surfaces of rigid **platters** that spin on a **spindle** at a constant rate — commonly 5,400, 7,200, or (in enterprise parts) 10,000 or 15,000 RPM. A stack of read/write **heads**, one per surface, is mounted on an **actuator arm**. To read or write a specific sector, the drive must do two mechanical things in sequence:

1. **Seek.** The actuator swings the head to the correct radial track. Moving the arm and letting it settle takes, on average, **~4-10 ms** (a full stroke across all tracks is longer; a short seek to an adjacent track is shorter — the "average seek time" on the datasheet is a mean over random track pairs).
2. **Rotational latency.** Once the head is on the right track, it must wait for the platter to spin the desired sector underneath it. On average this is half a revolution. At 7,200 RPM a full revolution takes ~8.3 ms, so average rotational latency is **~4.2 ms**. At 15,000 RPM it is ~2 ms. This is why RPM is on the spec sheet: it directly sets the rotational-latency floor.



Add seek and rotation and a single **random** access costs on the order of **5-15 ms** before a single useful byte moves. Invert that and you get the brutal headline number: a 7,200-RPM drive services only **~75-150 random IOPS**. That is not a typo and it has not meaningfully improved in decades, because it is bounded by mechanics, not electronics — arms and platters obey Newton, not Moore.

Now contrast **sequential** access. Once the head is positioned, the platter streams data under it continuously; adjacent sectors require no seek and no extra rotational wait. Sequential throughput is therefore governed by areal density and RPM, and lands at **~100-260 MB/s** on modern drives

— respectable, and *thousands* of times more efficient per byte than random access. This is the single most important fact about the HDD:

> On a hard disk, **sequential access is not a little faster than random access; it is catastrophically, order-of-magnitudes faster.** A workload of small random reads can leave a drive delivering well under 1 MB/s of useful data while its head thrashes; the same drive reads sequentially at 200 MB/s.

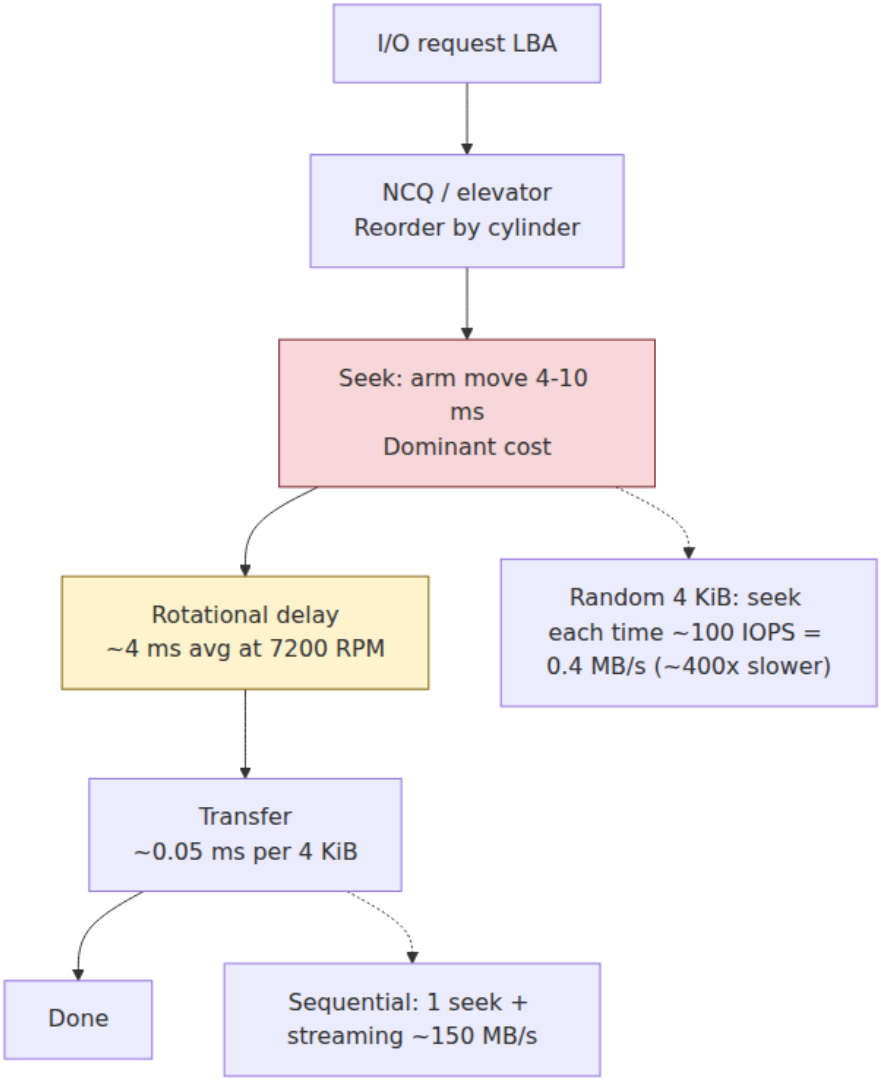
What the HDD taught databases

This one asymmetry shaped the storage-engine designs you study in depth in Volume 5. Every classic technique is, at root, a strategy to turn random I/O into sequential I/O:

- **Append-only logs and write-ahead logging (WAL).** Writing new data by *appending* to the end of a file keeps the head in one place, converting a stream of updates into one long sequential write. A transaction commits by appending its record to the WAL — sequential, fast — and the expensive random updates to the main data structures happen later, in batch. Volume 5's durability chapters and Volume 10's log-structured systems (Kafka) both descend from this idea.
- **B-trees with large nodes.** The B-tree (and its disk-oriented variant the B+ tree) exists to minimize the number of random seeks per lookup by making each node a large, contiguous block — historically sized to a disk block or a small multiple of it — so that one seek retrieves a high-fanout node. A tree with fanout in the hundreds reaches billions of keys in three or four seeks. The node is large *specifically because the seek is the cost and the transfer is nearly free once you are positioned.*
- **Log-structured designs.** Taken to the limit, the log-structured filesystem and the log-structured merge-tree (LSM) treat *all* writes as sequential appends and reorganize in the background. On an HDD this is close to optimal: you only ever pay for sequential writes plus periodic sequential compaction.

The enduring lesson generalizes past HDDs. Chapter 3 showed sequential memory access beats random because of prefetching; storage multiplies the stakes: **sequential beats random at every tier, and the slower**

the tier, the more extreme the penalty. Design access patterns for the slowest tier your data touches.



SSDs: NAND flash and the erase-before-write problem

A solid-state drive has no moving parts. It stores bits as trapped electric charge in **NAND flash** cells, and it reads them electronically. That alone

removes seek and rotational latency and is why SSDs deliver random reads three to four orders of magnitude faster than an HDD. But NAND flash has its own deeply peculiar physics, and that peculiarity — not raw speed — is what a backend engineer must understand, because it drives write behavior, tail latency, endurance, and cost across a fleet.

Cells: the density-vs-endurance trade-off

A NAND cell stores charge on a floating gate (or a charge-trap layer in modern 3D NAND). How many *bits* you store per cell is a design choice that trades density against speed and endurance:

Type	Bits/ cell	Voltage levels	Relative density	Relative endurance (P/E cycles, hedge)	Relative speed
SLC	1	2	1x	~50K-100K	fastest
MLC	2	4	2x	~3K-10K	fast
TLC	3	8	3x	~1K-3K	medium
QLC	4	16	4x	~100-1K	slowest

The physics is intuitive once stated: to store n bits in a cell you must distinguish 2^n distinct charge levels. More levels means finer voltage margins, so reads and (especially) writes take longer and more care, and the cell tolerates fewer erase cycles before the insulation degrades and the margins collapse. SLC is fast and durable but expensive per bit; QLC is cheap and dense but slow to write and short-lived. Most mainstream drives today are TLC; QLC targets read-heavy, cost-sensitive bulk storage. (Modern capacity comes largely from stacking layers vertically — "3D NAND," now well past 200 layers — rather than from shrinking cells, which had hit reliability limits.) The endurance numbers above are deliberately hedged ranges; the ordering (SLC $\gg$ MLC $\gg$ TLC $\gg$ QLC) is the durable fact.

Pages and blocks: the asymmetry that drives everything

Here is the property that makes flash unlike any storage that came before it. NAND is organized into two different units for two different operations:

- A **page** is the unit of **read and write (program)** — commonly ~4-16 KB.
- A **block** is the unit of **erase** — a block contains many pages (often 128-256+), so a block is on the order of a few megabytes.

And the rule that follows is the crux of the entire chapter:

> You can **read** a page and you can **write (program)** an *empty* page, but you **cannot overwrite a page in place.** To rewrite a page you must first **erase the entire block** it belongs to — and erase only works at block granularity.

Flash cells are programmed by pushing charge onto the floating gate one page at a time, but charge can only be *removed* in bulk, a whole block at once. So "modify these 4 KB" cannot be done directly. If the drive naively erased the whole block to change one page, it would (a) destroy the other 200-odd valid pages in that block and (b) burn a precious erase cycle for a tiny change. No drive does that. Instead, every SSD interposes a layer of indirection.

The Flash Translation Layer (FTL)

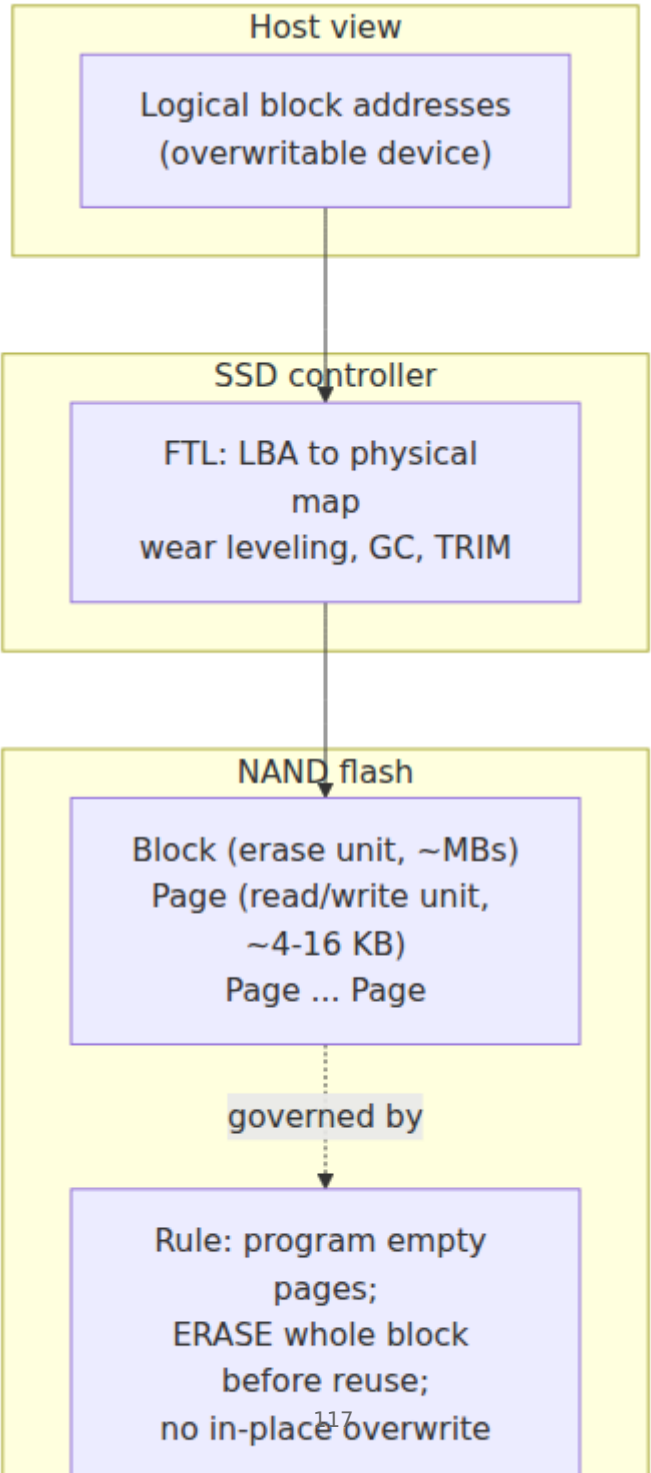
The **FTL** is firmware running on the SSD's controller that presents a normal, overwritable block device to the host while hiding all of NAND's quirks. It is, in effect, a tiny log-structured storage engine embedded in the drive. It does several jobs at once.

Logical-to-physical mapping and out-of-place writes. The host addresses the drive by *logical* block address (LBA); the FTL keeps a mapping table from LBAs to *physical* NAND locations. When the host "overwrites" LBA 42, the FTL does **not** touch the old physical page — it writes the new data to a fresh, already-erased page elsewhere, remaps LBA 42 to it, and marks the old page **stale**. Overwrite-in-place becomes write-elsewhere-and-remap, which is why an SSD's random-write pattern, as seen by the NAND, is actually sequential appends into erased blocks — the FTL is log-structuring for you.

Wear leveling. Because each block tolerates only a limited number of program/erase (P/E) cycles, the FTL spreads writes evenly across all blocks — rotating writes and occasionally relocating cold data — so the whole device ages uniformly rather than wearing out a few hot blocks while the rest stay fresh.

Garbage collection (GC). Over time, blocks fill with a mix of valid and stale pages. To reclaim space the FTL must produce *fully erased* blocks. It picks a victim block, copies its still-valid pages into a fresh block, then erases the victim. That copy step is the catch: **to free space, the drive must itself write data** — internal writes the host never asked for.

TRIM. When a filesystem deletes a file, the underlying LBAs are free from the host's point of view, but the SSD has no way to know that — as far as the FTL knows, that data might still be needed. The **TRIM** command (SATA) / **Deallocate** (NVMe) lets the OS tell the drive "these LBAs are now garbage." This lets GC treat those pages as stale immediately instead of dutifully copying dead data during collection, which improves both performance and endurance. A filesystem that does not issue TRIM slowly degrades an SSD's write performance.


Host view
Logical block addresses
(overwritable device)
SSD controller
FTL: LBA to physical
map
wear leveling, GC, TRIM
NAND flash
Block (erase unit, ~MBs)
Page (read/write unit,
~4-16 KB)
Page ... Page
governed by
Rule: program empty
pages;
ERASE whole block
before reuse;
no in-place overwrite

Write amplification, endurance, and the p99 tail

Two consequences of the FTL matter enormously to a backend engineer.

Write amplification. Because of out-of-place writes and GC's copy-valid-pages step, the NAND physically writes *more* bytes than the host sent. The ratio is the **write amplification factor (WAF)**: bytes written to NAND ÷ bytes written by the host. A WAF of 3 means one host-write becomes three flash-writes. WAF rises when the drive is full (fewer free blocks, GC runs more often and copies more valid data per erase) and when the workload is small random writes (which scatter stale pages across many blocks, so GC finds few fully-stale blocks to reclaim cheaply). Drives fight this with **over-provisioning** — hidden spare capacity (often ~7% to 28%) the FTL uses as GC breathing room. This is why enterprise SSDs quote *usable* capacity below their raw NAND, and why keeping an SSD below ~80% full materially improves both its speed and its lifespan.

Endurance. Because cells wear out and WAF multiplies host writes, an SSD has a finite write budget, quoted two ways on the datasheet:

- **TBW (terabytes written)** — total host bytes the drive is warranted to absorb over its life.
- **DWPD (drive writes per day)** — how many times you may overwrite the *entire* capacity every day for the warranty period (typically 5 years). A 2 TB drive at 1 DWPD tolerates 2 TB/day; at 3 DWPD, 6 TB/day.

Read-optimized QLC drives may be rated well below 1 DWPD; write-intensive enterprise drives reach 3-10 DWPD. Sizing a database or Kafka fleet means checking that your *actual* write rate, times real-world WAF, fits under the drives' DWPD — or you replace drives on an unplanned schedule.

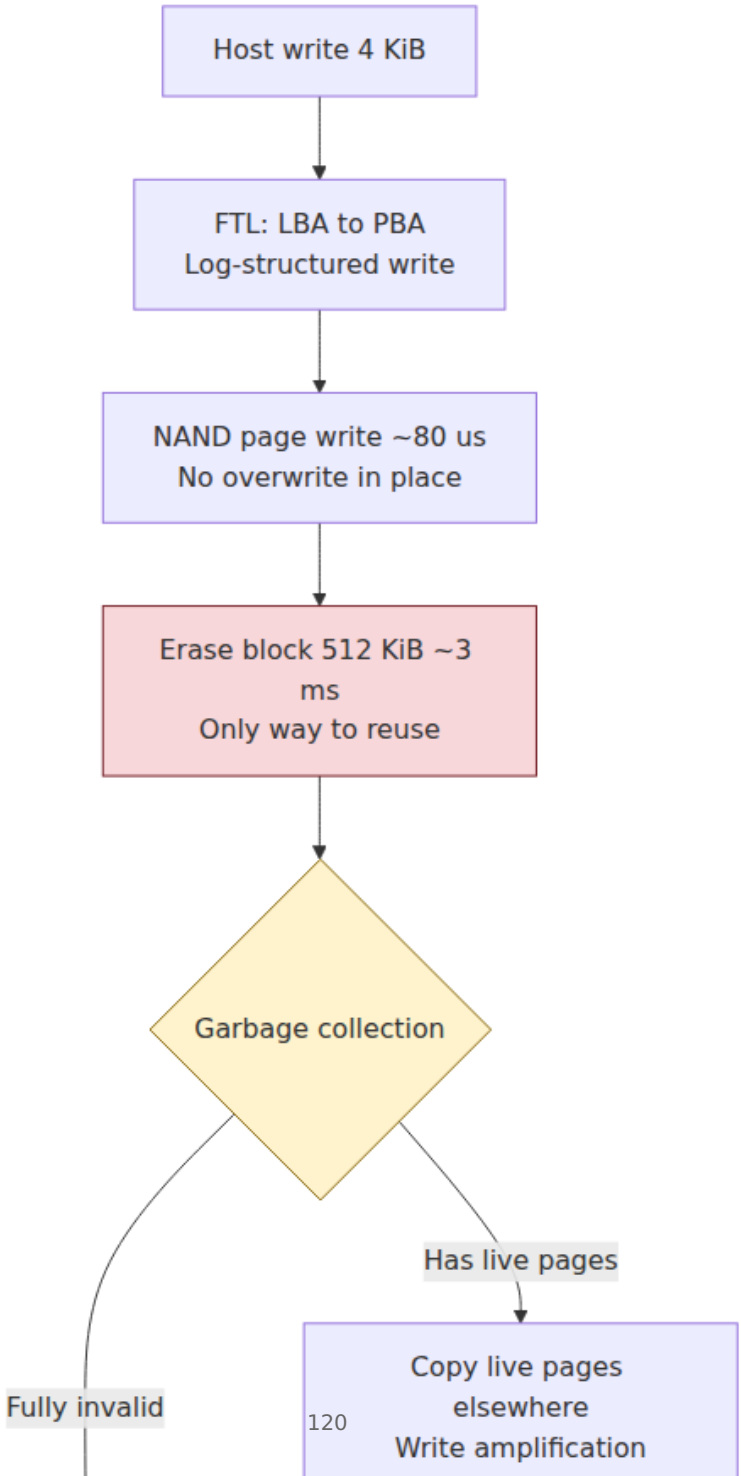
GC-induced tail latency. This is the one that bites backend engineers in production and rarely appears in benchmarks. Garbage collection runs *concurrently with host I/O*, contending for the same NAND channels and controller. A read or write that arrives while the drive is mid-GC on the block it needs can be delayed far beyond the drive's nominal latency. The average latency looks great; the **p99 / p999 tail** spikes, correlated with write pressure and how full the drive is. If your service has a tight tail-latency SLO and its p999 mysteriously worsens under write-heavy load or as disks fill, SSD garbage collection is a prime suspect. Enterprise drives

with better GC algorithms and more over-provisioning exist largely to tame this tail; it is a real, recurring cause of production latency incidents.

What the SSD changed for database design

The HDD's lesson was "sequential ⩾ random, avoid seeks at all costs." The SSD rewrites part of that lesson and reinforces another part:

- **Random *reads* became cheap.** With no seek and no rotation, a random read is roughly as fast as a sequential read (both ~tens of microseconds on NVMe). This relaxes decades of index design that existed purely to avoid random reads; a query doing a handful of random index lookups is no longer a disaster.
- **Writes still are not free — but for a new reason.** The HDD penalized random writes for seeks; the SSD penalizes them for write amplification and GC. The *conclusion* — prefer large, sequential, batched writes — survives; the *mechanism* changed from mechanics to erase semantics.
- **This reshaped the LSM-vs-B-tree debate.** B-trees do in-place updates (small random writes), which on flash means more write amplification and GC pressure. LSM trees buffer writes in memory, flush them as large sequential runs, then compact in the background — a pattern that maps cleanly onto flash's love of big sequential writes, at the cost of read amplification and its own compaction WAF. Neither is universally better; the trade-off is the durable/latency/write-amplification balance Volume 5 dissects. The point for now: **the medium's write physics is a first-class input to that decision.**


Host write 4 KiB
FTL: LBA to PBA
Log-structured write
NAND page write ~80 us
No overwrite in place
Erase block 512 KiB ~3 ms
Only way to reuse
Garbage collection
Has live pages
Copy live pages elsewhere
Write amplification
Fully invalid

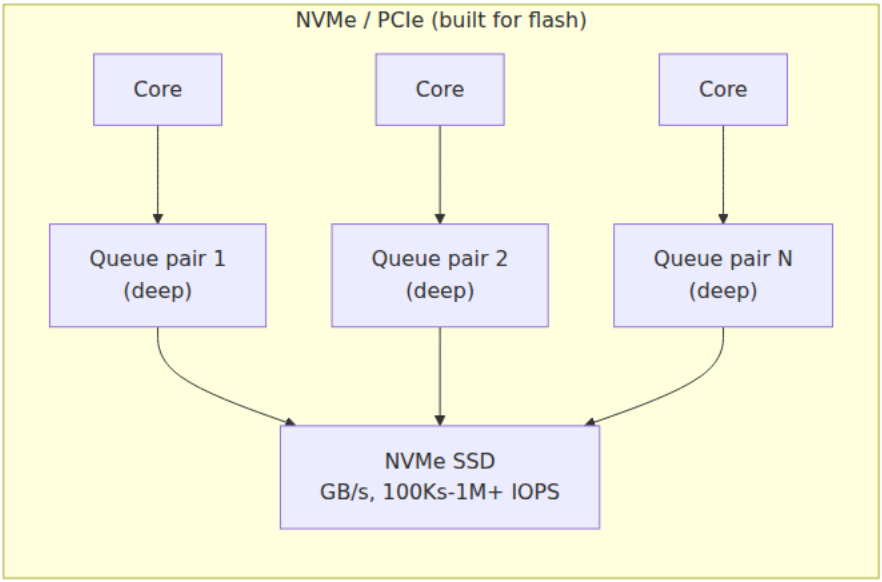
NVMe: the interface catches up to the media

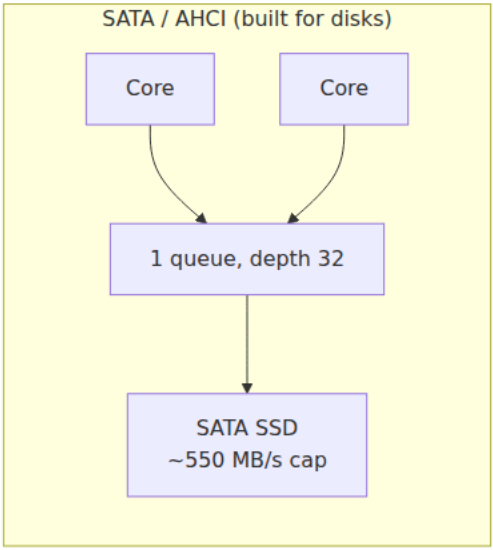
For years the SSD's real bottleneck was not the flash — it was the *cable*. Early SSDs shipped on the **SATA** interface with the **AHCI** protocol, both designed in the era of spinning disks. That legacy showed in two crippling limits:

- **Bandwidth.** SATA III tops out at 6 Gb/s, ~550 MB/s of usable throughput. A SATA SSD can saturate that with sequential reads and then simply stop scaling — the flash could go faster, the wire could not.
- **Parallelism.** AHCI was built for a device that could only do one thing at a time (a disk with one head). It exposes **a single command queue with depth 32**. That is fine for a mechanical drive that cannot service more than a couple of hundred IOPS anyway. It is absurd for flash, which is internally a massively parallel array of NAND dies across many channels and *wants* thousands of requests in flight.

NVMe (Non-Volatile Memory Express) is the storage protocol designed for flash from scratch, and it speaks over **PCIe** directly to the CPU rather than through a legacy storage controller. Its two defining moves are the mirror image of AHCI's two limits:

- **PCIe bandwidth.** NVMe rides PCIe lanes. A Gen3 x4 link delivers ~3.5 GB/s, Gen4 x4 ~7 GB/s, Gen5 x4 ~14 GB/s — roughly an order of magnitude above SATA and rising each PCIe generation. The wire is no longer the bottleneck.
- **Massive parallelism.** NVMe supports up to **65,535 I/O queues, each up to 65,536 entries deep**. In practice a driver creates one submission/completion queue pair *per CPU core*, so cores issue I/O to the drive without contending on a shared lock or a single shallow queue. This matches the flash's internal parallelism and lets a single NVMe SSD sustain hundreds of thousands to over a million IOPS.



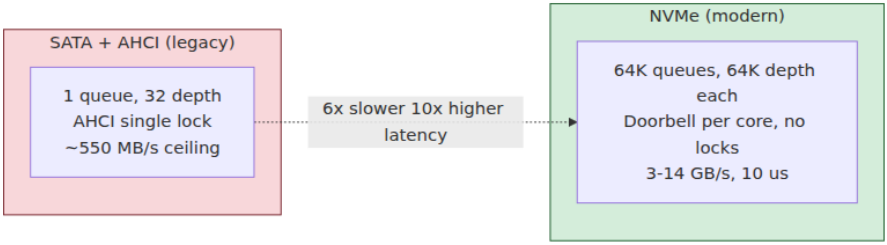


The latency picture follows. An NVMe SSD read is on the order of **~10-100 µs** end to end — the NAND itself contributes ~10-20 µs; the rest is the driver, queueing, and PCIe. That is still ~100x slower than

DRAM, but ~100x *faster* than an HDD seek, and low enough that the *software* overhead (system calls, interrupts, context switches) becomes a visible fraction of the total. This is why modern high-performance storage stacks cut per-I/O software cost with `io_uring` (batched, low-syscall async I/O), polling instead of interrupts at very high IOPS, and userspace drivers like **SPDK** that bypass the kernel to talk to the NVMe queues directly. When the device answers in microseconds, kernel overhead you cheerfully ignored in the HDD era becomes the bottleneck.

NVMe over Fabrics: disaggregating storage across the network

NVMe's command model is not intrinsically tied to a local PCIe slot. **NVMe over Fabrics (NVMe-oF)** carries the same queue-based protocol over a network transport — RDMA (RoCE/InfiniBand), Fibre Channel, or plain TCP — so a host can drive a *remote* NVMe device almost as if it were local, adding only the network round trip (tens of microseconds on a tuned RDMA fabric). This is the enabling technology for **disaggregated storage**: instead of stranding flash inside each compute server (where it is often underutilized), the datacenter pools NVMe drives in dedicated storage nodes and lets any compute node mount capacity from the pool over the fabric. Compute and storage scale independently; a failed compute node does not take its data with it; utilization rises. This is the hardware substrate under a lot of modern cloud block storage and "compute-storage separation" database architectures, and it is a direct line to the distributed-systems discussion below and to Volume 12's treatment of cloud infrastructure.



Persistent memory: the tier that almost was

For a few years it looked as though the wall between "memory" and "storage" might come down. **Intel Optane** persistent memory (built on **3D XPoint** technology, co-developed with Micron) shipped as **DCPMM**

modules that plugged into DDR4 DIMM slots and sat on the **memory bus** alongside DRAM — but retained their contents across power loss. The promise was extraordinary: **byte-addressable persistence at near-DRAM speed.**

That phrase describes a genuinely different kind of device. It is **byte-addressable**: unlike a block device (SSD/HDD) reached in ≥512-byte sectors through a driver and a system call, persistent memory is addressed with ordinary CPU **load and store instructions**, byte by byte, through the memory controller — no I/O stack. It is **persistent**: after a power cut, the bytes are still there. And it ran at **near-DRAM speed** — latency higher than DRAM but far below flash, on the order of **hundreds of nanoseconds** (hedge; it varied by access pattern and read vs write). Densities exceeded DRAM's, so a single server could hold multiple terabytes.

Optane offered two operating modes. In **Memory Mode**, it acted as a large *volatile* main-memory pool with DRAM as a cache in front of it — cheap capacity, persistence not exposed to software. In **App Direct Mode**, applications saw a separately addressable, durable region and managed persistence explicitly — the mode that actually let a database keep its structures directly in persistent memory.

The hard part: durability and ordering

App Direct mode surfaced a subtle correctness problem that is worth understanding even though the product is gone, because it is the same problem `fsync` solves for block devices, just moved up into the CPU's cache hierarchy. When you execute a store to a persistent-memory address, the write does **not** immediately reach the persistent media. It lands first in the CPU's **store buffers and caches** (Chapter 3 and Chapter 4), which are volatile. If power drops while the data is still in a cache line, it is lost — even though it was "written to persistent memory." So making a persistent- memory write *durable* required an explicit sequence:

1. **Flush the cache line** to the memory controller with an instruction like `CLWB` (cache-line write-back) or `CLFLUSHOPT`.
2. **Fence** with `SFENCE` to order the flush before subsequent operations.
3. Rely on the platform's **Asynchronous DRAM Refresh (ADR) /** enhanced ADR guarantee, which promised that data reaching the

memory controller's write pending queue would be flushed to media on power failure (backed by capacitors/reserve energy).

Getting this ordering right — so a crash never leaves a half-updated persistent structure — is exactly as hard as crash-consistent on-disk formats, minus the block granularity. Intel's **PMDK** (Persistent Memory Development Kit) gave programmers transactional primitives over this model: powerful, and genuinely difficult to use correctly.

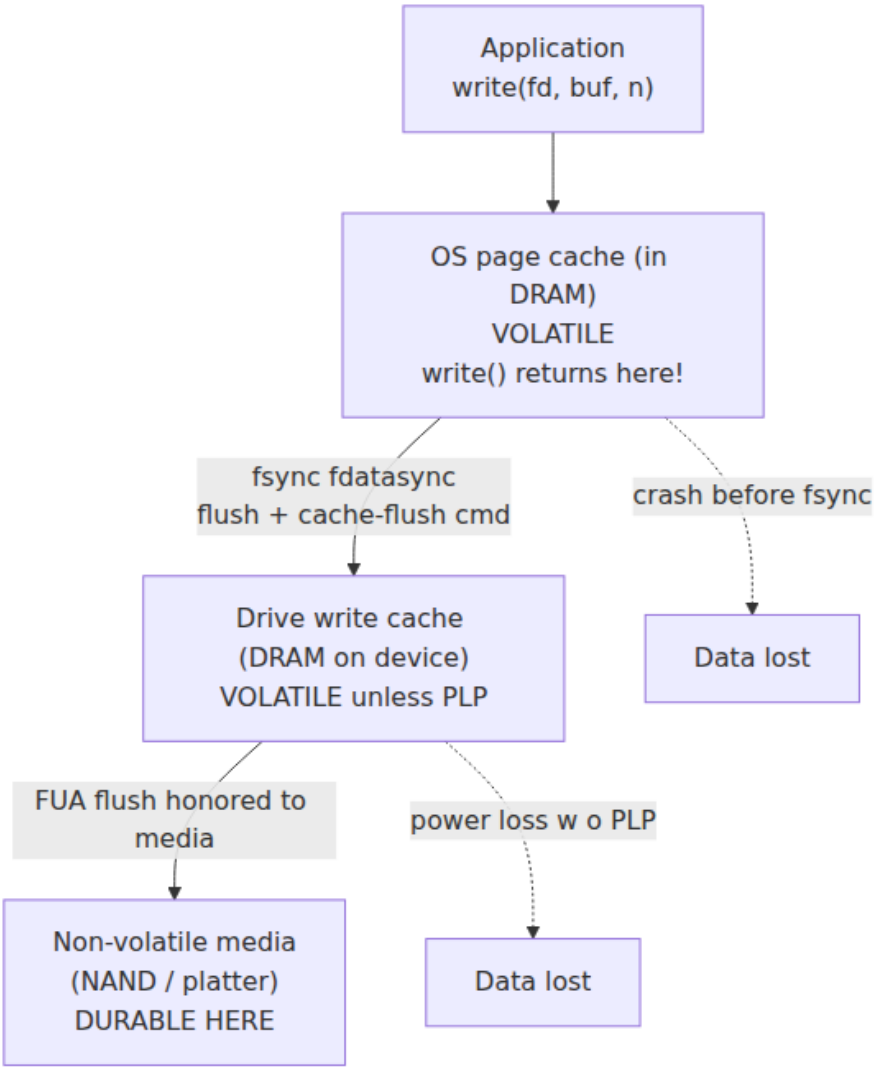
Status: discontinued

Despite the promise, adoption stayed limited — the programming model was demanding, it required specific Intel platforms, the price/capacity story never clearly beat DRAM-plus-fast-SSD, and little software was rewritten for App Direct mode. **Micron exited the 3D XPoint business and sold its fab in 2021, and Intel announced it was winding down the Optane business in 2022.** The product line is discontinued; treat any Optane figures here as historical.

The *concept* — non-volatile, byte-addressable memory on or near the memory bus — has not gone away. The industry's current vehicle for memory expansion and pooling is **CXL (Compute Express Link)**, a cache-coherent interconnect over PCIe that lets large, possibly persistent, memory tiers attach to a host and be shared across hosts. Whether byte-addressable persistence returns as a mainstream tier is open, but the design problems Optane exposed — cache-flush durability, crash-consistent in-memory structures, a tier between DRAM and SSD — are real and will recur. For now the mainstream persistent tier is the NVMe SSD, and durability there flows through the block I/O stack — the next section.

Durability: what "the write is persistent" actually costs

This is the section a backend engineer must not skim, because it is where correctness lives. A distributed database, a message queue, a consensus log all promise "once I acknowledge your write, it will survive a crash." That promise is only as good as the weakest link in the physical path the write takes — and that path has more volatile stages than most engineers realize. Consider `write()` appending a record to a file:



Walk the stages:

Stage 1 — the page cache (volatile). By default, `write()` copies your bytes into the kernel's **page cache** in DRAM and returns *immediately*, reporting success. Nothing has touched the disk; the kernel flushes dirty pages *eventually*, on its own schedule. Lose power in that window and the "successful" write is gone. `write()` returning is **not** durability; it is a

promise to try later. (`O_DIRECT` bypasses the page cache, sending data toward the device — but that still does not guarantee it reached non-volatile media; see stage 2.)

Stage 2 — the drive's write cache (volatile, unless protected). Almost every SSD and HDD has its own **DRAM write cache**, and may **acknowledge a write while it still sits in that volatile on-device cache**, before it is committed to NAND or platter. This lets the drive batch and reorder writes — and is a **data-loss trap** on power failure. This is the stage most engineers forget.

Stage 3 — non-volatile media. Only when the bytes are actually written to NAND cells or magnetized onto the platter are they durable. Everything above this line is volatile.

Making it durable: fsync and friends

To force the write down through the volatile stages, the application must call **`fsync(fd)`**, which does two things: it flushes the file's dirty page-cache pages to the device, *and* issues a **cache- flush command** (SATA `FLUSH CACHE`, NVMe `Flush`) telling the drive to push its volatile write cache to non-volatile media, waiting for confirmation. Only after `fsync` returns has the write plausibly reached durable media. **`fdatasync`** is a cheaper variant that flushes the file *data* plus only the metadata needed to read it back (e.g., a size change), skipping non-essential updates like `mtime` — which is why databases that manage their own file layout prefer it. The **FUA (Force Unit Access)** flag is an alternative: it tells the drive a specific write must reach media before acknowledgment, bypassing the volatile cache for that write.

`fsync` is **slow** — it turns an asynchronous, batchable operation into a synchronous round trip that blocks until the media confirms. That is precisely why databases *obsess* over it. The write-ahead log's job is to make the commit-path `fsync` as cheap as possible: one sequential append, one `fdatasync`, and the transaction is durable — while the expensive random updates to the main structures flush lazily and replay from the log after a crash. **`fsync` on the WAL is the moment a transaction becomes durable, and its latency is a floor on commit latency.** Volume 5's durability chapters are largely a study of amortizing this one system call — group commit (batching many transactions into one `fsync`), commit pipelining, and `synchronous_commit`-style knobs are all responses to how expensive stage-2-and-3 durability is.

Power-loss protection: the enterprise-vs-consumer trap

There is one more subtlety that has caused real, catastrophic data-loss incidents. Some drives — notably **enterprise SSDs** — contain **power-loss protection (PLP)**: on-board **capacitors** holding enough energy to flush the volatile write cache to NAND if power is cut mid-write. On such a drive, data acknowledged into the drive's cache is effectively safe even without waiting for it to reach NAND, because the capacitors guarantee it will get there. This makes `fsync` dramatically faster (the drive can honor a flush from its protected cache) *and* keeps the durability guarantee intact.

Consumer SSDs typically have no PLP — their volatile cache really is volatile. Worse, some consumer drives (and misconfigured systems) have historically **ignored or lied about cache-flush commands** to win benchmarks, acknowledging a flush without committing to media. On such hardware `fsync` returns, your database believes the write is durable, and a power cut proves otherwise. This is the durability trap:

> Building a database or replicated log on **consumer SSDs without power-loss protection** — or on any device that does not honor flush/FUA — means your durability guarantee is a fiction. The layer above did everything right (`fsync`, WAL, replication) and the hardware silently broke the contract.

The table summarizes where durability is and is not guaranteed:

Layer	Location	Volatile?	Made durable by
Application buffer	Process DRAM	Yes	`write()` (moves it to page cache)
OS page cache	Kernel DRAM	Yes	`fsync` / `fdatasync` (flush to device)
Drive write cache (no PLP)	On-device DRAM	Yes	flush/FUA reaching media; **lost on power cut**
Drive write cache (with PLP)	On-device DRAM	Effectively safe	capacitors flush to NAND on power loss
NAND / platter	Non-volatile media	No	this *is* durability

Reasoning about storage performance

With the mechanisms in hand, here are the metrics an engineer reasons with.

Latency, IOPS, and bandwidth are three views of the same device, and they trade off.

- **Latency** — time to service one operation. The right metric for a single dependent read (e.g., an index lookup on the critical path). Report it as a *distribution*, not a mean: p50, p99, p999. As the SSD GC discussion showed, the tail is where storage hurts a latency-sensitive service.
- **IOPS** — operations per second, usually quoted for small (4 KB) random ops. The right metric for workloads dominated by many small scattered accesses (OLTP, key-value). Note IOPS and latency are linked through concurrency (Little's Law): sustained IOPS ≈ queue depth ÷ per-op latency.
- **Bandwidth (throughput)** — bytes per second, usually quoted for large sequential ops. The right metric for scans, backups, log replay, analytics. A device can be bandwidth-rich but IOPS-poor or vice versa; ask which one your workload needs.

Queue depth. A single outstanding request exposes the full per-op latency and leaves the device's internal parallelism idle. NVMe SSDs reach their headline IOPS only at **high queue depth** — many requests in flight so the drive keeps all its NAND channels busy. This is why benchmarks specify queue depth (e.g., "4K random read, QD32"), why async I/O (`io_uring`, SPDK) matters for full throughput, and why a synchronous, one-`fsync`-at-a-time write path leaves most of the drive unused. But depth trades against latency: deeper queues raise throughput *and* per-op latency (requests wait behind others). Choosing a queue depth is choosing a point on the throughput-vs-latency curve.

Random vs sequential — still real, less extreme. On flash the random/sequential gap shrank dramatically from the HDD's 100-1000x but did not vanish. Sequential still wins: large sequential writes are far friendlier to the FTL (less write amplification, easier GC), and sequential reads benefit from read-ahead and larger transfers. The HDD-era instinct to batch and sequentialize is still correct on flash; it is just no longer life-or-death for reads.

Cloud storage inherits — and taxes — the physics

Most backend engineers no longer touch a physical drive; they provision an abstraction. But the abstraction does not repeal the physics — it repackages it and often adds a network tax on top. Two families matter (Volume 12 covers them in depth):

- **Network-attached block storage** — AWS **EBS**, GCP **Persistent Disk**, Azure Managed Disks. These present a block device, but the bytes live on storage nodes reached over the datacenter network (conceptually the disaggregated/NVMe-oF model above). Consequences that surprise people: (1) you buy performance as **provisioned IOPS and/or throughput**, decoupled from capacity — a small fast volume or a large slow one, and you pay for the IOPS. (2) A **network-latency tax**: a network-attached volume's latency (often ~sub-millisecond to low milliseconds) is higher than a *local* NVMe SSD's tens of microseconds, because every I/O crosses the network and a replication layer. (3) The volume is typically **replicated** behind the scenes, which is why it survives instance failure — and part of the latency cost. For the lowest latency, clouds also offer **local instance/ephemeral NVMe** — physically attached, microsecond-class, but **not durable across instance stop/termination**. The classic mistake is putting a database's WAL on ephemeral local disk for speed and discovering after an instance replacement that "local" meant "gone."
- **Object storage** — **S3**, GCS, Azure Blob. A different abstraction entirely: an HTTP key-value store of immutable objects, with high latency (tens of milliseconds), effectively unbounded bandwidth and capacity, and very high durability. Not a block device and not `fsync`-able per byte; you get whole-object PUT/GET semantics. Modern "lakehouse" and disaggregated databases lean on object storage as the cheap, durable bottom tier and cache hot data on faster local NVMe — the memory-hierarchy pattern of Chapter 3, re-expressed across the cloud.

The lesson: know *which* physics your abstraction is standing on. "Provisioned IOPS," "the network- storage latency tax," "local NVMe is fast but ephemeral," and "object storage is durable but high-latency and coarse-grained" are all direct descendants of the mechanisms in this chapter.

Distributed-systems lens: storage physics is data-system physics

Everything above was single-node. The payoff is that these mechanisms propagate directly into the correctness and cost of distributed data systems.

Append-only and log-structured designs are storage physics made architecture. Kafka (Volume 10) stores each partition as an append-only sequence of segment files and gets much of its throughput from never doing random writes — riding the same "sequential ⩾ random" fact the HDD taught, with cheap sequential consumer reads for the same reason. LSM stores (Volume 5) exist because buffering writes and flushing them as large sequential runs is the pattern flash rewards. The log is not just a convenient abstraction; it is the shape of write that storage hardware is fastest at, at every tier.

Durability is the foundation under consensus correctness. This is the sharpest connection in the chapter. Consensus protocols — Raft, Paxos, Viewstamped Replication (Volume 6) — are proven correct on the assumption that when a node says "I have persisted this log entry," the entry is actually durable. Raft's guarantee that a committed entry is never lost depends on a majority of nodes having *durably* stored it before the leader considers it committed. If a node acknowledges a log append that was still sitting in a volatile drive cache — because `fsync` was skipped for speed, or the drive lacked power-loss protection and lied about a flush — then a correlated power event (a rack losing power) can erase "committed" entries from a majority at once, and the protocol's safety proof no longer holds. **A "committed" write that was never actually durable breaks Raft/Paxos.** The hardware-level durability mechanisms — `fsync`, FUA, capacitor-backed PLP — are not an implementation detail beneath consensus; they are the physical layer the entire correctness argument rests on. Real systems have lost data exactly here, by trusting an acknowledgment the hardware never earned.

The durability-vs-latency trade-off is a core design axis. Because `fsync` is slow, every replicated data system chooses where to sit on a spectrum: `fsync` on every write (maximally durable, slowest); batching writes and flushing periodically (faster, with a window of un-flushed data at risk on crash); or replicating to N nodes' memory instead of any single disk (fast, durable only against uncorrelated failures — vulnerable to

correlated power loss). Group commit, `synchronous_commit=off` modes, and "flush every N ms" knobs are all points on this axis, and choosing among them *is* choosing a durability guarantee — trading acknowledged-write latency against the data-loss window on failure.

Write amplification and endurance are fleet-economics problems. On one machine, WAF and DWPD are curiosities; across thousands of write-heavy nodes they set the drive-replacement rate and a real slice of the storage bill. A compaction strategy that halves write amplification does not just run faster — it doubles drive lifetime across the fleet. This is why database and streaming teams at scale measure device-level write amplification, not just application write rate.

Disaggregation and the network-storage tax reshape architecture. NVMe-oF and cloud block storage let compute and storage scale independently and let a database survive the loss of any compute node — at the cost of a network round trip on every I/O. Newer "compute-storage separation" databases accept higher per-I/O latency in exchange for elastic, durable storage, then claw it back by caching hot data on local NVMe and DRAM. It is Chapter 3's hierarchy at datacenter scale, with the same rule: keep the working set in the fast local tier; go to the slow, shared, durable tier as rarely as possible.

The through-line from Chapter 3 holds all the way down: hierarchies of tiers with growing latency and growing persistence, where the game is always to keep hot data in the fast tier, move data in large sequential chunks, make access patterns predictable, and — new at this tier — be precise about the exact moment a write becomes durable, because distributed correctness is built on that moment.

Key takeaways

- **Storage is the persistent tier below DRAM**, adding several orders of magnitude on top of the memory hierarchy — from an L1 hit (~1 ns) to an HDD seek (~10 ms) is ~7 orders of magnitude. Non-volatility is the reason storage exists and the reason it is slow.
- **HDDs pay seek (~4-10 ms) + rotational latency (~2-4 ms) per random access** — only ~75-200 random IOPS but ~100-260 MB/s sequential. This 100-1000x random/sequential gap drove databases toward append-only logs, WAL, and large-node B-trees: sequential ≫ random, more extreme than any tier above.

- **NAND's defining rule is erase-before-write**: read/write per page (~4-16 KB), erase per block (~MBs), no in-place overwrite. The **FTL** hides this with out-of-place writes, an LBA→physical map, **wear leveling**, **garbage collection**, and **TRIM**.
- Cells trade density for endurance/speed (**SLC > MLC > TLC > QLC**). Endurance is finite (**DWPD/ TBW**) and **write amplification** (GC + out-of-place writes) multiplies host writes; over- provisioning and staying below ~80% full mitigate both. **GC contends with host I/O and spikes p99/p999 tail latency** under write pressure — a real production issue averages hide.
- SSDs made **random reads cheap** (relaxing index design); writes still favor large sequential batches (now for erase semantics, not seeks), reshaping the **LSM-vs-B-tree** trade-off (Volume 5).
- **NVMe over PCIe** replaced SATA/AHCI's single depth-32 queue and 550 MB/s cap with up to 64K deep queues and multi-GB/s bandwidth, reaching 100Ks-1M+ IOPS at ~10-100 µs — low enough that software overhead matters (`io_uring`, SPDK). **NVMe-oF** extends the protocol over a network for disaggregated storage.
- **Persistent memory** (Optane / 3D XPoint) offered byte-addressable persistence at ~100s of ns with a cache-flush durability model (`CLWB`/`SFENCE`/ADR), but is **discontinued** (Micron exited 2021, Intel wound down Optane 2022); the concept lives on around CXL. Treat its figures as historical.
- **Durability requires reaching non-volatile media, not just returning from `write()`** (which only reaches the volatile page cache). `fsync`/`fdatasync` flushes to the device and issues a cache-flush; the drive's own DRAM cache is **volatile unless it has capacitor-backed power-loss protection**. Consumer SSDs without PLP (or drives that lie about flushes) silently break the durability contract.
- **Cloud storage inherits the physics**: provisioned IOPS decoupled from capacity, a network-latency tax on network-attached volumes (EBS/PD), the durability-vs-ephemerality trap of local instance NVMe, and object storage as a durable-but-high-latency, coarse-grained bottom tier (Volume 12).
- **Distributed-systems consequence:** append-only/LSM/Kafka exist because of storage write physics; `fsync` + power-loss protection are the physical foundation of consensus correctness (a "committed"

write that was never durable breaks Raft/Paxos, Volume 6); write amplification/endurance are fleet-cost problems; and durability-vs-latency is a core data-system design axis (Volume 5).

Further reading

- John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. (Morgan Kaufmann, 2017) — the storage/memory appendices give the quantitative treatment of disk mechanics, flash, and dependability.
- NVM Express base specification — https://nvmexpress.org/specifications/ — the authoritative source for the NVMe queueing model, command set, Flush/FUA semantics, and NVMe-oF transports.
- Cai, Ghose, Haratsch, Luo, Mutlu, "Error Characterization, Mitigation, and Recovery in Flash-Memory- Based Solid-State Drives" (Proceedings of the IEEE, 2017) — a rigorous survey of NAND cell physics, wear, and FTL techniques.
- "The Unwritten Contract of Solid State Drives" — Jun He et al., EuroSys 2017 — how SSD internals (GC, mapping, write amplification) leak into application performance, and what software must do to cooperate.
- Patterson, Gibson, Katz, "A Case for Redundant Arrays of Inexpensive Disks (RAID)" (SIGMOD 1988) — foundational on disk performance/reliability trade-offs; still the mental model for storage redundancy.
- "Ext4, btrfs, and the others" / kernel documentation on the block layer and cache flushing — https://www.kernel.org/doc/html/latest/block/ and the `fsync(2)` / `open(2)` man pages — the precise Linux semantics of the page cache, `O_DIRECT`, `fdatasync`, and barrier/flush behavior.
- Ramnatthan Alagappan et al., "Protocol-Aware Recovery for Consensus-Based Storage" (FAST 2018) and Pillai et al., "All File Systems Are Not Created Equal: On the Complexity of Crafting Crash- Consistent Applications" (OSDI 2014) — how real `fsync` / crash-consistency assumptions interact with (and break) storage-backed distributed protocols.
- SNIA (Storage Networking Industry Association) NVM Programming Model and persistent-memory documentation — https://

www.snia.org/ — the standardized programming model behind persistent memory, including flush/fence durability semantics.

- Intel, "Intel Optane Persistent Memory" product documentation and the 2022 wind-down announcements — background on 3D XPoint, App Direct vs Memory Mode, and the discontinuation; PMDK at https://pmem.io/ for the (still-instructive) persistent-memory programming model.
- Brendan Gregg, *Systems Performance*, 2nd ed. (Addison-Wesley, 2020) and https://www.brendangregg.com — practical measurement of disk latency distributions, IOPS, queueing, and `fio` / `biolatency` for characterizing real storage devices.
- AWS EBS and GCP Persistent Disk documentation — https://docs.aws.amazon.com/ebs/ and https://cloud.google.com/compute/docs/disks — the actual provisioned-IOPS, throughput, durability, and local-vs-network-attached semantics of cloud block storage (Volume 12).

Chapter 6 — Multi-Socket Systems and NUMA

What this chapter covers. Chapters 3 and 4 treated main memory as a single flat tier: past the last-level cache lies "DRAM," a uniform pool every core reaches at roughly the same ~100 ns cost. On a laptop, a phone, or a small cloud instance that model is accurate enough to reason with. On the machines that run serious backend workloads — dual- and quad-socket servers, and increasingly even large *single*-socket chiplet parts — it is a lie, and an expensive one. Memory on these systems is physically partitioned into **NUMA nodes**, each welded to a particular socket or memory controller. A core reading its *local* node's memory pays the familiar DRAM latency; a core reading a *remote* node's memory must first cross an inter-socket interconnect, and pays noticeably more — commonly on the order of 1.5–2x the local latency, at lower bandwidth. This chapter explains how memory became non-uniform, what the hardware actually looks like, how the OS lets you control placement, and how to design services that respect node locality instead of scattering threads and data at random across a box. The organizing idea, developed in the closing lens, is that **a NUMA machine is a distributed system**

inside the chassis — nodes with local memory joined by a network — and that the same locality, partitioning, and coordination-minimizing instincts you already apply across services apply unchanged, just at nanosecond scale.

Learning goals — after this chapter you should be able to:

- Explain why **UMA** (uniform memory access over a shared bus) stopped scaling and why modern servers are **NUMA**, with memory partitioned into per-socket nodes.
- Describe the physical anatomy of a NUMA system: sockets, integrated memory controllers, DIMMs per node, and the inter-socket interconnect (**Intel UPI/QPI**, **AMD Infinity Fabric**).
- Reason quantitatively (with correct hedging) about **local vs remote** memory latency and bandwidth, and read **NUMA distance** from the topology.
- Explain **sub-NUMA clustering** and **chiplet NUMA** — why one socket can be several NUMA nodes — and how cache coherence (Chapter 4) scales across sockets via directories.
- Explain the OS's NUMA policies: **first-touch** allocation, explicit binding (`numactl`, `mbind`, `set_mempolicy`), **interleaving**, and NUMA-aware scheduling.
- Use `numactl`, `numastat`, and `lscpu` to observe topology and diagnose remote-access problems.
- Apply the engineer's playbook: per-node data partitioning, first-touch initialization, thread/memory affinity, avoiding cross-node shared mutable state, and deciding when to **pin to one node** vs span.
- Connect all of it to cloud instance sizing (Volume 12) and to distributed-systems design writ small.

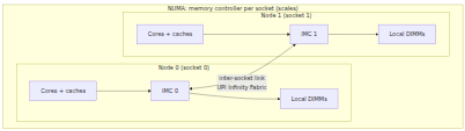
From uniform to non-uniform: why the shared bus died

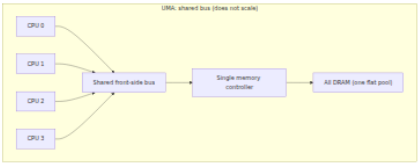
For a long stretch of computing history, multiprocessor memory really was uniform. Several CPUs sat on a **shared front-side bus**, and beyond that bus was a single **memory controller** (in the "north bridge" chipset) fronting all of DRAM. Every processor was electrically equidistant from every byte of memory; an access cost the same regardless of which CPU issued it. This is **UMA — Uniform Memory Access** — and it is the

model of memory most engineers still carry in their heads. It is simple to program: memory is one flat pool, and where a thread runs has no bearing on how fast it reaches its data.

UMA has a fatal scaling property: the shared bus and the single memory controller are a **centralized, contended resource**. Every CPU's every DRAM access funnels through the same wires and the same controller. Add processors and you add demand on a fixed-bandwidth channel that all of them must arbitrate for. Bus contention rises, effective per-core bandwidth falls, and electrical constraints (a shared multi-drop bus cannot clock arbitrarily fast as you hang more devices off it) cap how far the design goes. Two or four cores on a shared bus is fine. Dozens of cores, each capable of issuing memory requests every few nanoseconds, saturate any single controller. The bottleneck is structural, not a matter of building a faster bus — it is the same centralization problem you would recognize instantly if someone proposed routing every microservice's database traffic through one shared connection.

The industry's answer was to **decentralize the memory system**, and it arrived in mainstream x86 when the memory controller moved *onto the CPU die* (Intel calls it the **integrated memory controller**, IMC; AMD integrated it with Opteron in the mid-2000s and Intel with Nehalem in 2008). Once each socket has its *own* memory controller and its *own* directly-attached DIMMs, aggregate memory bandwidth scales with socket count: two sockets have two controllers and two sets of DIMMs, roughly doubling bandwidth versus one. But this decentralization has an unavoidable consequence. A core on socket 0 can reach socket 0's DIMMs directly through socket 0's controller — fast. To reach socket 1's DIMMs, its request must travel *across a link between the sockets*, be serviced by socket 1's controller, and the data must travel back. That extra hop is real distance and real latency. Memory is no longer uniform: it is **NUMA — Non-Uniform Memory Access**. The performance of a memory access now depends on *which node* holds the data relative to *which core* is asking.





The trade is deliberate and, on balance, overwhelmingly worth it: you accept that *some* accesses are slower (remote) in exchange for far higher *aggregate* bandwidth and the ability to scale past what any single controller could feed. NUMA is not a defect to be engineered away; it is the price of a memory system that scales to many cores. The engineer's job is not to eliminate non-uniformity — you cannot — but to arrange that the *overwhelming majority of accesses are local*. That sentence is the whole chapter in miniature, and you will notice it is the identical instinct as "keep the working set in the fast tier" from Chapter 3, now applied one rung out.

The hardware: sockets, controllers, and the interconnect

A concrete two-socket server makes the anatomy tangible. Each **socket** holds one physical processor package: many cores, their private L1/L2 caches, a shared last-level cache, and an **integrated memory controller** driving several **DDR channels**, each populated with DIMMs. The memory hung off socket 0's controller is **node 0**; the memory off socket 1's controller is **node 1**. A **NUMA node** is precisely this pairing of *a set of cores* with *the memory local to them* — a locality domain.

The two sockets are joined by a dedicated **cache-coherent inter-socket interconnect**:

- **Intel** uses **UPI (Ultra Path Interconnect)** on modern Xeon (Skylake-SP onward, ~2017), which superseded the older **QPI (QuickPath Interconnect)** introduced with Nehalem. UPI is a point-to-point, packetized, coherent link; a socket typically has multiple UPI links so that in two- and four-socket topologies every socket can reach every other with few hops.
- **AMD** uses **Infinity Fabric**, the same coherent fabric that also stitches together the multiple chiplets *inside* an EPYC package (more on that below). Between sockets it carries coherent memory and cache traffic; AMD markets the inter-socket links as xGMI ("global memory interconnect").

This interconnect is the defining piece of hardware in a NUMA system, and the single most important thing to understand about it is that it is a **shared, finite, saturable resource** — a network link. Every remote memory access, every cache line that must be shipped between sockets

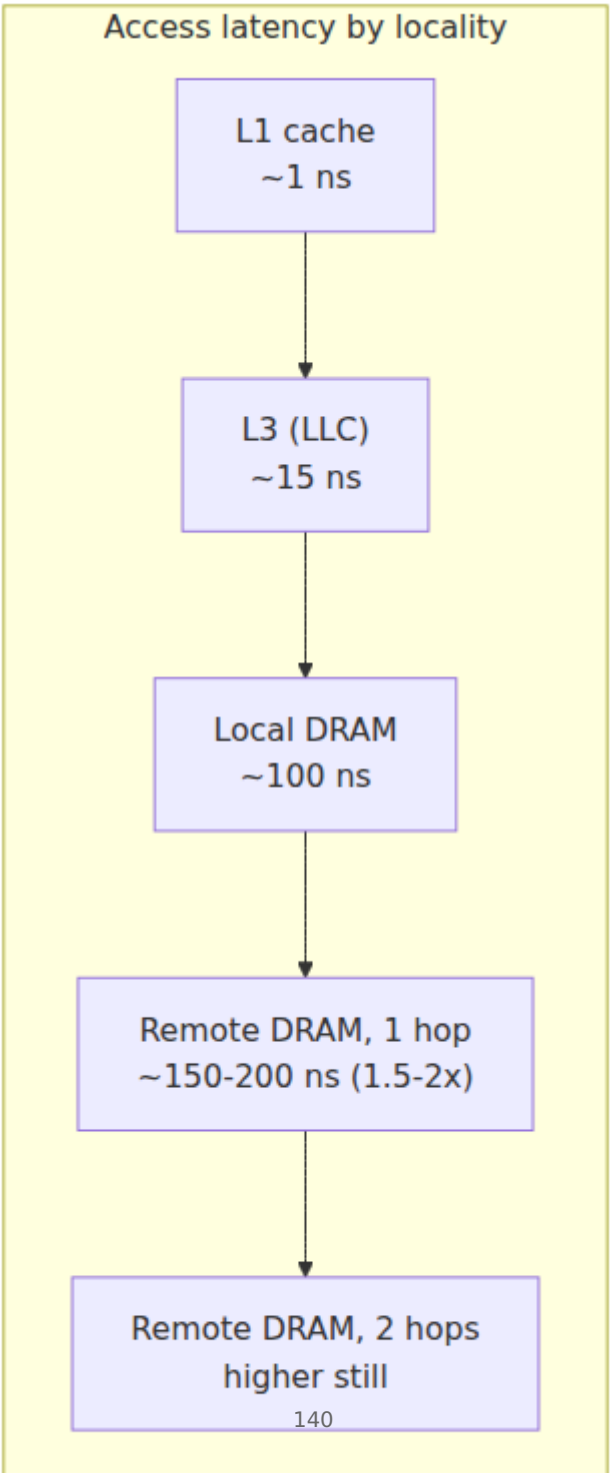
to maintain coherence (Chapter 4), every atomic operation on a line another socket owns — all of it crosses these links. Its bandwidth is high but bounded, and it is *shared* by all cores on both sockets. A workload that generates heavy cross-socket traffic can saturate the interconnect, at which point *even local-looking* performance degrades because coherence and remote traffic are queued behind each other on a congested link. Hold onto that framing: the interconnect is a network, and networks congest.

Local vs remote: the numbers, hedged

The performance signature of NUMA is the gap between local and remote access. The exact figures are vendor-, generation-, topology-, and BIOS-configuration-dependent, so treat these as order-of-magnitude guidance whose *ratios* are the durable fact:

Property	Local (same node)	Remote (across one interconnect hop)
Load latency	~80–120 ns (Ch. 3 DRAM)	**~1.5–2x local** (roughly ~120–200+ ns)
Sustained read bandwidth	Full local channel BW	**Lower** — capped by interconnect, often well below local
Multi-hop remote (4-socket)	—	Worse still: 2 hops cost more than 1

Two clarifications the ratio alone hides. First, the **latency** penalty (~1.5–2x) is the number most often quoted, but the **bandwidth** penalty can matter more for streaming workloads: remote bandwidth is throttled by the interconnect's capacity, which is typically lower than the aggregate local DRAM bandwidth of a node, and it is shared with coherence traffic. A bandwidth-bound job pulling data from a remote node can run several times slower than the same job pulling locally, not merely 1.5–2x. Second, in topologies larger than two sockets, not all "remote" is equal: on a four-socket box wired so that some socket pairs are directly linked and others are reached via an intermediate socket, a **two-hop** remote access is slower than a one-hop remote access. Non-uniformity has *gradations*, which is why the topology exposes a distance *matrix*, not a single local/remote bit.


Access latency by locality
L1 cache
~1 ns
L3 (LLC)
~15 ns
Local DRAM
~100 ns
Remote DRAM, 1 hop
~150-200 ns (1.5-2x)
Remote DRAM, 2 hops
higher still

NUMA distance and topology

Firmware describes the topology to the OS through an ACPI table — the **SLIT (System Locality Information Table)** — which encodes a **distance matrix** between nodes. By convention the distance from a node to itself is **10** (representing "1.0x"), and remote distances are expressed relative to it: a common value for one-hop remote is around **21** (roughly 2.1x), with larger numbers for multi-hop. These are *relative* figures the firmware advertises, not measured nanoseconds, but they give the OS scheduler and allocator a cost model for placement decisions. You can read the matrix directly:

```
$ numactl --hardware
available: 2 nodes (0-1)
node 0 cpus: 0 1 2 3 ... 31   (and their SMT siblings)
node 0 size: 257843 MB
node 0 free: 201120 MB
node 1 cpus: 32 33 34 35 ... 63
node 1 size: 258048 MB
node 1 free: 245880 MB
node distances:
node   0   1
  0:  10  21
  1:  21  10
```

The diagonal `10`s are local; the off-diagonal `21`s say "remote memory on this box costs about 2.1x local." A four-socket machine would show a 4x4 matrix, potentially with a mix of `21` (directly linked) and higher (multi-hop) entries. This table *is* the machine's locality map, and every NUMA tuning decision is ultimately about keeping accesses on the cheap diagonal.

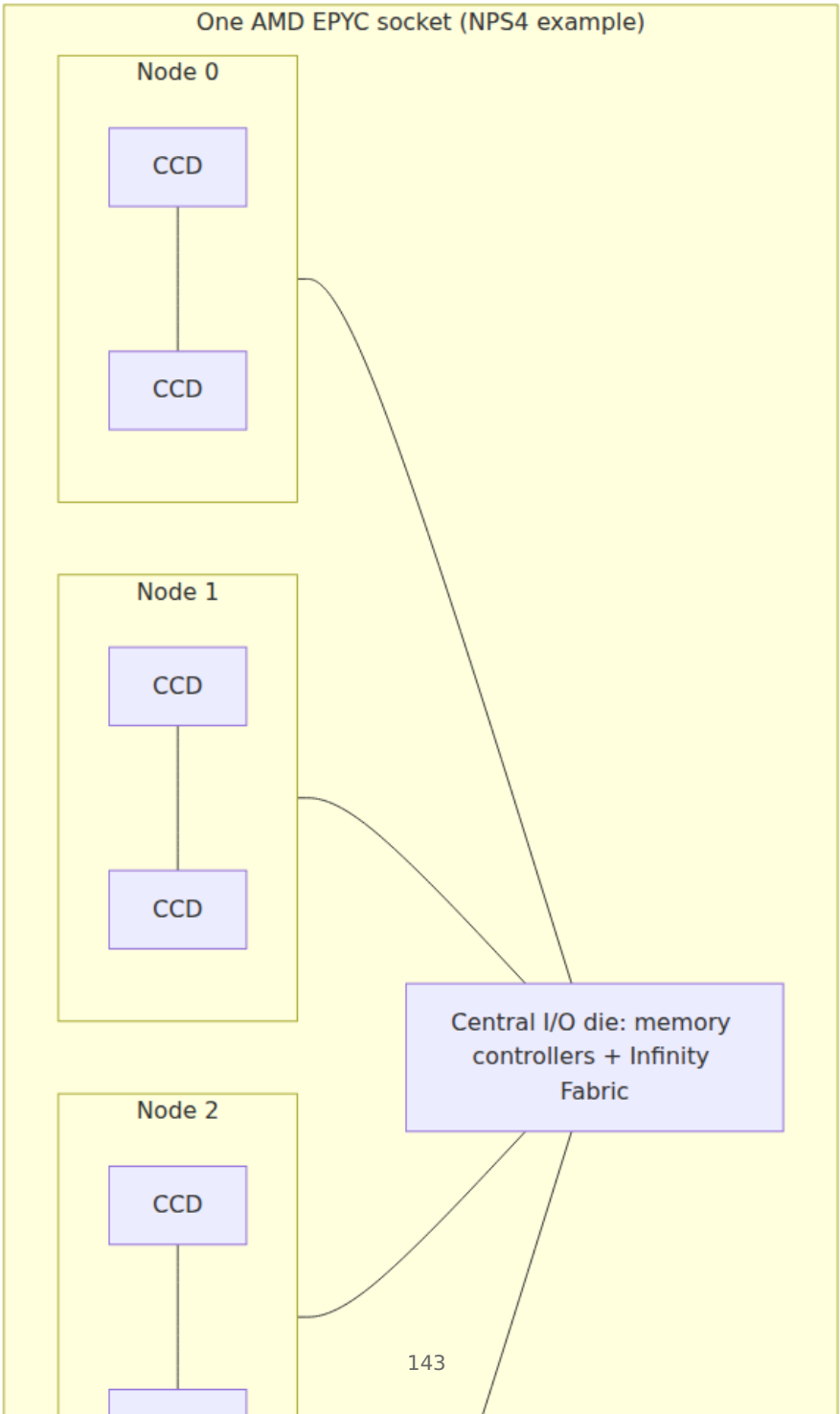
Sub-NUMA clustering and chiplet NUMA: one socket, several nodes

The clean "one socket = one node" picture is now the exception on high-core-count parts. Two hardware realities split a single socket into multiple NUMA nodes.

Sub-NUMA Clustering (SNC) — Intel's feature (its predecessor was Cluster-on-Die) — deliberately partitions one physical socket into two or more NUMA nodes in firmware. A big socket internally has its LLC and memory controllers distributed across a mesh of cores; a core on one side of the die reaches "its" nearby LLC slices and memory controller faster

than the ones on the far side. SNC exposes those internal locality domains as separate NUMA nodes so the OS can keep threads near the LLC and memory channels physically closest to them, shaving latency. The cost is that the socket now presents multiple smaller nodes with the same local/remote asymmetry *within the package*.

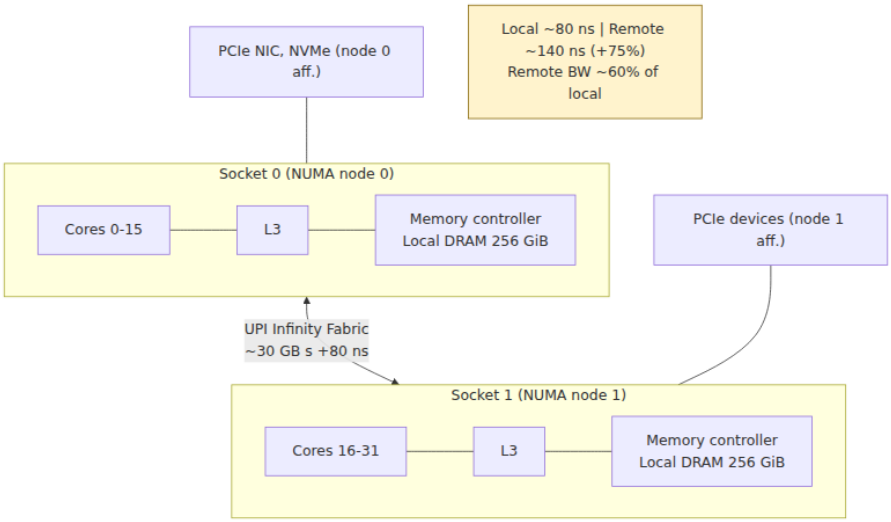
Chiplet NUMA is even more fundamental, and AMD EPYC is the canonical example. An EPYC package is not a single monolithic die; it is several **core complex dies (CCDs)** plus a central **I/O die**, all joined by on-package **Infinity Fabric**. The memory controllers live on the I/O die. Depending on the generation and the **NPS (Nodes Per Socket)** firmware setting, a single EPYC socket can present as one NUMA node (NPS1, memory interleaved across all the socket's channels) or be split into 2 or 4 nodes (NPS2 / NPS4), each node binding a subset of CCDs to the memory channels physically nearest them. The first-generation EPYC (Naples) was even more explicitly NUMA: four dies each with their own memory controller meant **four NUMA nodes in a single socket** by default. The lesson is blunt: **you can have NUMA effects on a one-socket machine.** "It's a single-socket box, so NUMA doesn't apply" is a mistake on modern high-core-count parts — check `numactl --hardware`, do not assume.


One AMD EPYC socket (NPS4 example)
Node 0
CCD
CCD
Node 1
CCD
CCD
Node 2
CCD
Central I/O die: memory controllers + Infinity Fabric

Coherence across sockets: the directory scales, snooping does not

Chapter 4 established cache coherence (MESI and its cousins) as the protocol that keeps per-core caches consistent. On a single socket, small core counts can maintain coherence by **snooping** — broadcasting "who has this line?" and letting every cache answer. Broadcast does not scale across sockets: flooding the interconnect with a snoop for every miss would saturate it instantly. Large NUMA systems therefore lean on **directory-based coherence** (Chapter 4): the system tracks, for each line, which nodes hold a copy, so a coherence action can be sent *only to the nodes that matter* rather than broadcast to all. Intel implements this with **home agents** and **snoop filters / directory** state associated with each memory region; AMD's Infinity Fabric carries coherence with probe-filtering to avoid needless broadcasts (earlier Opterons called this "HT Assist").

The performance consequence for the backend engineer is the crucial part: **coherence traffic crosses the interconnect.** When a line is modified on socket 0 and read on socket 1, the protocol must ship the line (and its ownership) across the link — a cross-socket cache-line transfer that costs far more than a local one. A cache line that ping-pongs between sockets because two nodes are writing the same data is the NUMA-scale version of false sharing (Chapter 8), and every bounce is an interconnect round trip. This is why *cross-node shared mutable state is the cardinal NUMA sin*: it converts what should be local cache activity into interconnect congestion, and it does so invisibly — the code looks like an ordinary memory write.



Why it matters for backend performance

Now make it concrete for the services you actually run. Consider a large in-memory workload — a database buffer pool, a Redis-like cache, a JVM with a big heap, a search index — on a two-socket box. Nothing about the *code* mentions NUMA. But every one of its memory accesses is either local or remote, and the ratio between them is set by two placement decisions that, left to chance, go badly:

1. **Where the process's memory pages physically live** (which node's DIMMs hold them).
2. **Which node each thread runs on** at the moment it touches that memory.

If a thread pinned nowhere in particular runs on socket 1 and hammers data that happens to sit in socket 0's DIMMs, *every access is remote*: 1.5–2x latency, reduced bandwidth, and interconnect traffic. Do this across dozens of threads whose memory and execution are scattered independently across both nodes, and you get **remote accesses everywhere** — a workload paying the remote penalty on a large fraction of its memory traffic, plus an interconnect saturated by the resulting cross-socket transfers and coherence probes. The service's throughput and tail latency degrade, sometimes severely, for reasons *invisible in the*

application code and invisible to anyone measuring only CPU utilization: the cores look busy, but they are stalling on remote memory.

The magnitude is not academic. Large in-memory services on multi-socket hardware have been measured losing double-digit percentages of throughput to naive NUMA placement, recovered by nothing more than pinning memory and threads to the same node. The classic pathology is a service that **allocates its big data structures on one thread at startup** (say, a main thread on node 0 that builds the whole heap or buffer pool) and then serves requests from a **pool of worker threads spread across both nodes**. By default (first-touch, below), all that memory landed on node 0. Now half the workers — those scheduled on node 1 — do *all* their work against remote memory, forever. The fix is a placement decision, not an algorithm change.

There is also a second-order effect worth naming. The interconnect is shared, so NUMA problems are *coupling* problems: a bandwidth-hungry, badly-placed job on one node can congest the interconnect and degrade a well-behaved job on the other node that merely needs occasional coherence traffic to get through. Non-uniformity plus a shared link means one tenant's remote-access storm is another tenant's latency spike — a theme that returns under the cloud and distributed-systems lenses.

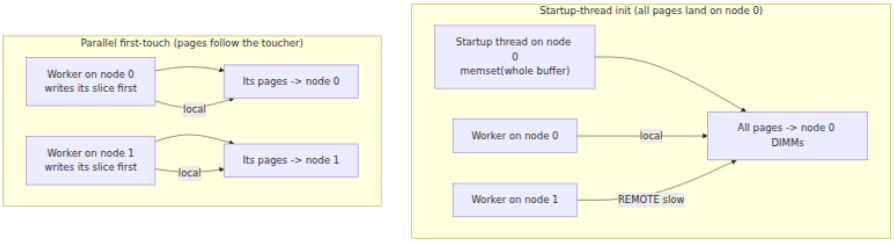
The OS and NUMA: policies, placement, and tools

The hardware presents non-uniform memory; the operating system decides *where pages go* and *where threads run*. This is the Volume 2 (Operating Systems) side of the story — memory management and scheduling — but the mechanisms are essential here because they are the levers a backend engineer actually pulls.

First-touch: the default that surprises people

Linux's default memory policy is **first-touch** (the local-allocation policy, `MPOL_DEFAULT` / `MPOL_LOCAL`). When your program calls `malloc` / `mmap`, no physical page is allocated yet — the virtual mapping exists but is not backed by DRAM. The physical page is allocated lazily on the **first write (the first "touch")**, and — this is the key rule — it is allocated on the NUMA node of **the CPU that performed that first touch**, provided that node has free memory.

The consequence is subtle and constantly trips people up: **it is not the thread that *allocates* the memory that determines placement, it is the thread that first *touches* it.** A common bug: a program `malloc`s and then `memset`s a huge buffer on its startup thread (node 0), then hands slices of that buffer to worker threads across both nodes. First-touch already placed *every* page on node 0 during the `memset`; the node-1 workers are now permanently remote. The corresponding *correct* pattern — and the single most important NUMA idiom in practice — is **first-touch initialization**: have each worker thread initialize (first-write) the memory *it* will subsequently use, from the CPU it will run on, so each page is placed on that worker's local node.



This is why HPC and database codebases parallelize their array initialization loops with the *same* thread/data decomposition they use for the compute loop: it is not to speed up the initialization, it is to place each page on the node that will own it. First-touch is a good default precisely because, if you *do* initialize data on the thread that will use it, it does the right thing automatically. It only bites when allocation and use are separated across nodes.

Explicit placement: numactl, mbind, set_mempolicy

When first-touch is not enough, Linux exposes explicit control at two granularities.

Whole-process, from the outside: `numactl` launches a process under a chosen policy without touching its code:

```
# Pin a service's CPUs AND memory to node 0: run only on node 0's
cores,
# allocate only from node 0's DIMMs. The "mini-machine" recipe.
numactl --cpunodebind=0 --membind=0  ./my-service

# Prefer node 1 for allocation but allow spillover if it fills up.
numactl --preferred=1  ./my-service

# Interleave all allocations across both nodes (bandwidth play, below).
numactl --interleave=all  ./my-service
```

In-process, from code: the `libnuma` library wraps the underlying syscalls — **`set_mempolicy(2)`** sets the calling thread's default policy for future allocations, **`mbind(2)`** sets a policy for a specific address range, and **`move_pages(2)`** migrates already-placed pages to chosen nodes. These let a service that understands its own data structures place them deliberately: bind the per-node shards to their nodes, interleave a shared read-only table, and so on. Thread placement is the complementary half, set with **`sched_setaffinity(2)`** / `pthread_setaffinity_np` (or `taskset`), pinning threads to the CPUs of the node whose memory they use.

The three placement policies and when to reach for each:

Policy	Mechanism	Effect	Use when
First-touch	Default (`MPOL_LOCAL`) + init on the using thread	Page placed on the node of the first writer	The common, correct default — *if* you initialize data on the thread that uses it
Bind	`numactl --membind` / `mbind(MPOL_BIND)`	Allocations forced onto a specific node (fails/spills if full)	Per-node share-nothing partitioning; pin a latency-sensitive service to one node
Preferred	`numactl --preferred` / `MPOL_PREFERRED`	Prefer a node, but fall back to others if it is full	You want locality but cannot tolerate allocation failure on pressure

Policy	Mechanism	Effect	Use when
Interleave	`numactl --interleave` / `MPOL_INTERLEAVE`	Pages round-robined across nodes	A single large structure accessed uniformly by all nodes; **bandwidth** over latency

Interleaving deserves a special note because it inverts the usual goal. Binding maximizes locality: great when a thread mostly hits its own node's data. But some workloads have one big shared structure that *every* thread on *every* node pounds — no partitioning makes it local to everyone. Binding it to one node makes that node's controller and the interconnect a hotspot; the other node's accesses are all remote and all funnel through one controller. **Interleaving** spreads the structure's pages evenly across nodes, so its *aggregate* bandwidth demand is served by *all* the memory controllers in parallel and no single node is the bottleneck. You trade guaranteed-local latency (now roughly half the accesses are remote by construction) for balanced, aggregated bandwidth. Interleave is a bandwidth optimization for shared, uniformly-accessed data; bind is a latency optimization for partitionable data. Choosing between them is choosing which resource is scarce.

NUMA-aware scheduling and automatic balancing

The scheduler's half of the job is to run threads on the node holding their memory. The Linux scheduler is NUMA-aware: it prefers to keep a task on its "home" node and is reluctant to migrate it across nodes, because migration turns local accesses remote. Linux also ships **automatic NUMA balancing** (`kernel.numa_balancing`), which samples a task's memory accesses and *reactively* migrates pages toward the node running the task, or the task toward its pages, trying to converge on locality without any application involvement. It helps unmanaged workloads, but it is a heuristic operating with delay and overhead: it reacts after the remote accesses have already happened, its page-fault-driven sampling costs cycles, and it can thrash on workloads whose access pattern shifts. For a service you control and care about, **explicit placement beats relying on the balancer** — the balancer is the safety net for code that didn't bother, not a substitute for a service that knows its own data. (The scheduler and memory-management internals are Volume 2, Operating

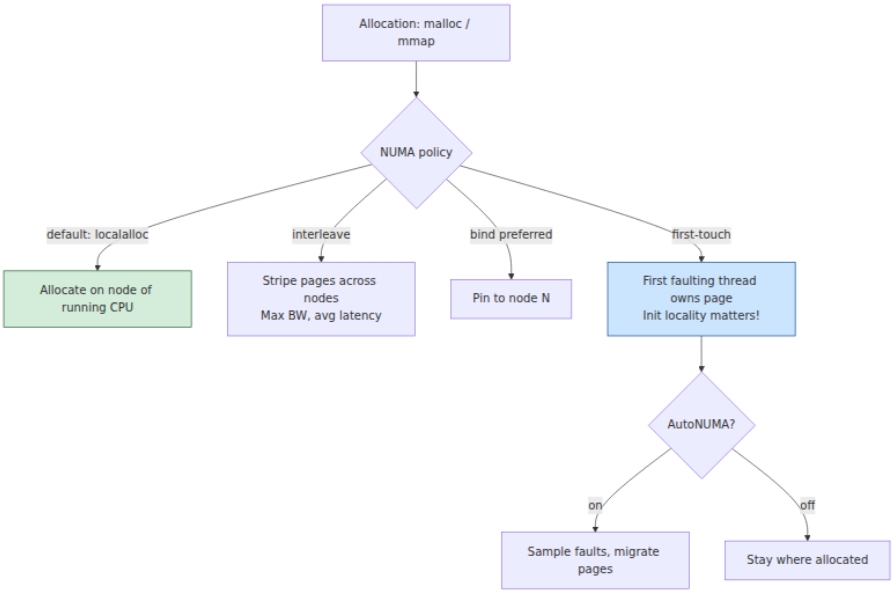
Systems; here the point is that placement is a first-class, controllable property, not something to leave to chance.)

Observing NUMA behavior

You cannot tune what you cannot see. The essential tools:

Tool / source	What it tells you
`numactl --hardware`	Node count, cores and memory per node, and the **distance matrix**
`lscpu`	NUMA node → CPU mapping (`NUMA node0 CPU(s): ...`), sockets, cores, SMT layout
`numastat`	Systemwide per-node hit/miss counters: `numa_hit`, `numa_miss`, `numa_foreign`
`numastat -p <pid>`	Per-process memory distribution across nodes — *is this process's memory scattered?*
`/proc/<pid>/numa_maps`	Per-mapping node placement for a process
`perf stat` / PMU events	Local vs remote DRAM access counters (e.g. offcore/uncore memory events)

The `numastat` counters are the diagnosis. **`numa_hit`** is allocations that landed on the intended (local) node; **`numa_miss`** is allocations that wanted a node but had to spill elsewhere; **`numa_foreign`** is allocations intended for another node that landed here. A healthy, well-placed service shows `numa_hit` dominating. High `numa_miss` / `numa_foreign` means memory pressure is forcing spillover across nodes — you are getting remote placement not by choice but because the target node was full. `numastat -p <pid>` on a suffering service that shows its memory split roughly 50/50 across nodes, when it *should* be partitioned, is the smoking gun for the "scattered allocation" pathology above.



Designing for NUMA: treat each node like a mini-machine

The techniques converge on one mental model: **treat each NUMA node as a small, self-contained machine — its own cores, its own memory — and design so that work stays inside its node.** This is the same share-nothing instinct you already apply to distributed systems, and it produces the same playbook.

Partition data per node (share-nothing). Give each node its own shard of the data and its own pool of threads that work only that shard. A connection or request is routed to a node, and everything it touches — its buffers, its slice of the index, its scratch memory — lives on that node's DIMMs and is worked by that node's cores. No thread reaches across the interconnect in the common path. This is exactly horizontal sharding (Volumes 5 and 7), applied to sockets: partition so that the hot path is node-local, and cross-node work becomes the rare exception rather than the pervasive default.

First-touch initialization. As established, allocate *and initialize on the using thread*, from the CPU that will run it, so each page is placed local. Parallelize your initialization with the same decomposition as your

compute. This one habit prevents the most common real-world NUMA regression.

Pin threads to their memory (affinity). Partitioning only pays off if threads *stay* on their node. Pin worker threads to the CPUs of the node whose data they own (`sched_setaffinity`, `taskset`, or a runtime's affinity API), so the scheduler cannot drift them across the interconnect and strand them from their memory. Memory affinity and thread affinity are two halves of one decision; setting one without the other is half a fix.

Avoid cross-node shared mutable state. A hot, mutable data structure written by threads on *both* nodes is the worst case: every write potentially bounces a cache line across the interconnect (Chapter 4 coherence, now cross-socket) and serializes on that link. Where you need shared state, prefer per-node replicas reconciled occasionally, per-node counters summed on read, sharded locks, or read-mostly data that can be replicated to each node. This is *identical* reasoning to minimizing cross-shard transactions in a distributed database — coordination across nodes is expensive, so design it out of the hot path.

Interleave what is genuinely shared and uniformly accessed. For the residual large structure that truly cannot be partitioned and is hammered uniformly by all nodes, interleave it for bandwidth rather than binding it to one node and creating a hotspot.

When to pin to one node vs span

A recurring practical decision: should a service **span** all nodes or **pin to one**?

- **Pin the whole service to a single node** (`numactl --cpunodebind=N --membind=N`) when its working set *fits* in one node's memory and its throughput needs fit within one node's cores and bandwidth. You get guaranteed locality — essentially *zero* remote accesses — with trivial configuration and no code changes. This is the right default for a latency-sensitive service small enough to fit a node, and it is common to run **one service instance per NUMA node** on a big box (two sockets → two pinned instances), turning one physical server into N independent single-node machines fronted by a load balancer. That "one instance per node" pattern is the cleanest way to use a multi-socket box: it makes the share-nothing partitioning explicit at the

process boundary and sidesteps in-process NUMA complexity entirely.

- **Span nodes with careful internal partitioning** when the service genuinely needs more memory or cores than one node provides — a database whose buffer pool must be larger than one node's DIMMs, or a throughput target exceeding one node's capacity. Now you must do the per-node partitioning, first-touch, and affinity work *inside* the process, because you cannot avoid using both nodes.
- **Ignore NUMA entirely** when there is only one node — small cloud instances, single-socket parts that present as NPS1 — or when the workload is not memory-bound enough for the remote penalty to matter. The cost of NUMA-tuning is real (complexity, rigidity); spend it only where the box is actually multi-node and the workload actually feels it. Always verify with `numactl --hardware` first; do not tune NUMA on a machine that has one node, and do not ignore it on a "single socket" that turns out to be four nodes.

Cloud and virtualization: NUMA you didn't ask for

Most backend engineers meet NUMA not on bare metal they racked but on **large cloud instances**, and the virtualization layer adds a twist worth understanding (developed further in Volume 12, Cloud Infrastructure).

The large instance types — the ones with many vCPUs and hundreds of gigabytes of RAM — are backed by **multi-socket physical hosts**, and their NUMA topology is (for the biggest sizes) **exposed to the guest**. Inside a big VM, `numactl --hardware` shows multiple nodes because the hypervisor has presented a virtual NUMA (**vNUMA**) topology that mirrors the underlying hardware. It mirrors it precisely because *hiding* it would be worse: if the guest OS and its NUMA-aware applications (databases especially) believed memory was uniform when it was not, their placement decisions would be actively wrong. So for large VMs the hypervisor exposes vNUMA and then works to keep each virtual node's memory and vCPUs backed by the *same physical node* — a placement problem the hypervisor solves with the same first-touch/affinity logic, one level down.

Several practical consequences follow:

- **vCPU and memory placement is the hypervisor's job, and it can get it wrong.** If the hypervisor scatters a VM's vCPUs and memory

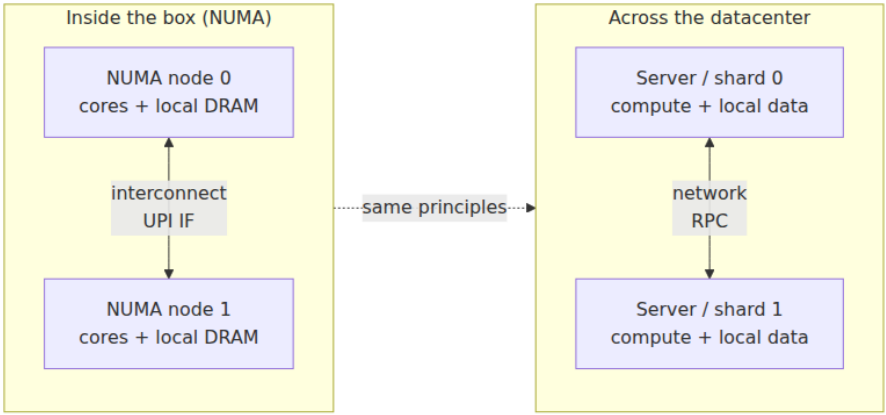
across physical nodes — or migrates a VM's vCPUs without moving its memory — the guest suffers remote accesses it cannot even see the cause of, because from inside the guest the topology *looks* consistent. This is why the largest instance types are often sized to align with whole sockets or whole hosts: aligning the VM to the physical NUMA boundaries removes the hypervisor's opportunity to split it badly.

- **"NUMA-aware" instance sizing.** Choosing an instance that fits within a *single* physical NUMA node (fewer vCPUs, node-sized memory) sidesteps guest-side NUMA entirely — the whole VM is one node, and the in-guest tuning above becomes unnecessary. When you *do* need a multi-node instance, size it to whole nodes/sockets rather than an awkward fraction, so the hypervisor can place it cleanly.
- **Pinning services to sockets.** On a large multi-node VM, the same "one instance per node" pattern applies inside the guest: run one instance of the service per vNUMA node, each pinned, rather than one giant process spanning all of them.
- **The noisy-neighbor × NUMA interaction.** In multi-tenant clouds, VMs from different customers share a physical host and its interconnect. A co-tenant generating heavy cross-node memory traffic congests the *shared interconnect and memory controllers*, degrading your VM's memory latency and bandwidth even though your placement is perfect — noisy-neighbor contention expressed through the NUMA fabric. It is invisible from inside your guest and is one reason latency-sensitive workloads pay for dedicated hosts or whole-socket isolation.

A note on where this is heading: **CXL (Compute Express Link)** attached memory (Volume 12, and touching Chapter 5's persistent-memory story) appears to the OS as *memory-only NUMA nodes* — nodes with capacity but no local CPUs, sitting at a higher latency tier than even remote DRAM. The NUMA abstraction is becoming the OS's general framework for *any* non-uniform memory, not just multi-socket DRAM, which makes the reasoning in this chapter more central over time, not less.

Distributed-systems lens: a distributed system inside the box

Here is the frame that makes everything above cohere, and it is worth stating as strongly as it deserves: **a NUMA machine is a distributed system, and the interconnect is its network.** Look at the ingredients. There are multiple **nodes**, each with its own **local memory**. The nodes are joined by a **network** (the interconnect) with **finite, shared, saturable bandwidth**. Accessing another node's data means sending a request across that network and paying **non-uniform latency**: local is cheap, remote is 1.5–2x, multi-hop remote is worse. Keeping data consistent across nodes requires a **coordination protocol** (cache coherence) whose messages traverse the network. Substitute "server" for "node," "datacenter network" for "interconnect," and "RPC" for "remote access," and you have described a distributed system in every particular. The constants are nanoseconds instead of milliseconds; the structure is identical.



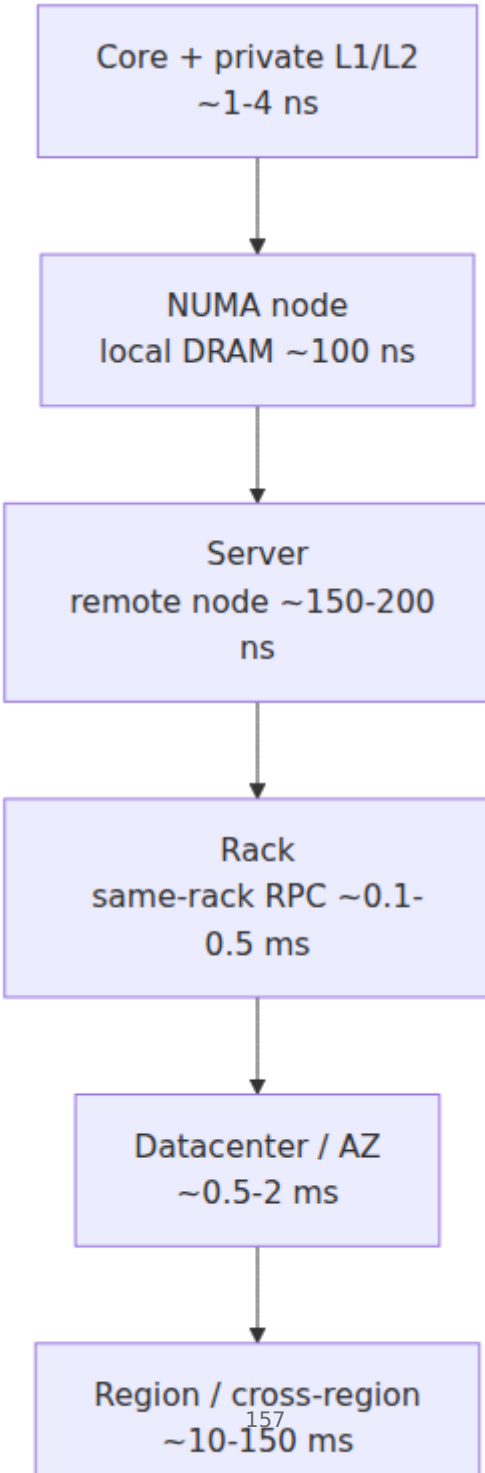
Because the structure is identical, the *design principles* transfer without modification — and a backend engineer who already reasons about distributed data locality *already owns the NUMA mental model*:

- **Data locality — keep computation near its data.** The prime directive of NUMA (run a thread on the node holding its memory) is the prime directive of distributed systems (run computation where its data lives; do not do remote reads on the hot path). A remote memory access is a remote read. First-touch initialization is "provision the data on the node that will serve it." The instinct not to

make a synchronous cross-region call in a request path is the *same* instinct as not scattering a thread away from its memory.

- **Partitioning / sharding — share-nothing per node.** Per-node data partitioning is horizontal sharding (Volumes 5 and 7). "Treat each NUMA node like a mini-machine with its own data and threads" is exactly "each shard is an independent unit that owns its slice." The "one service instance per NUMA node" pattern is literally running one shard per node, load-balanced — sharding applied to sockets.
- **Minimize cross-node coordination.** Cross-socket coherence traffic is cross-node RPC. A cache line ping-ponging between sockets is a chatty distributed coordination protocol saturating the network. The fix is the same at both scales: design out shared mutable state on the hot path, prefer per-node replicas and per-node counters, avoid the distributed-transaction-equivalent of a line owned by two sockets. Minimizing cross-shard transactions and minimizing cross-node coherence are the same discipline.
- **Affinity / placement-aware scheduling.** Pinning threads to the node with their data is locality-aware scheduling — the same idea as scheduling a task on the machine (or rack, or AZ) that holds its data, as data-locality schedulers do for data-parallel jobs. Affinity is placement, and placement is a locality decision at every scale.
- **The shared network is the scarce, coupling resource.** The interconnect saturates and couples tenants exactly as a datacenter network does; a noisy neighbor's cross-node traffic degrades yours through the shared fabric. "Watch the interconnect as a bottleneck" is "watch the network as a bottleneck." Both reward keeping traffic local so the shared link stays uncongested.

The deepest observation is that **the hierarchy is fractal.** Locality governs every level, with the same structure and the same rules, differing only in the numerical constants:


Core + private L1/L2
~1-4 ns
NUMA node
local DRAM ~100 ns
Server
remote node ~150-200
ns
Rack
same-rack RPC ~0.1-
0.5 ms
Datacenter / AZ
~0.5-2 ms
Region / cross-region
~10-150 ms

At every rung there is a *near* tier that is fast and cheap and a *far* tier reached across some network that is slower and shared, and at every rung the winning move is the same: **keep the working set and the coordination local; cross the boundary rarely, in useful-sized chunks; and treat the link between tiers as a scarce resource to be conserved.** Chapter 3 drew this ladder from registers to cross-region and argued the reasoning does not change across it. NUMA is one specific rung of that ladder — the rung just past local DRAM — and it is the rung where "distributed system" first becomes literally true *inside a single machine*. An engineer who thinks in data locality across services is not learning a new idea when they learn NUMA; they are recognizing an old one, applied at the hardware level, at nanosecond scale.

Key takeaways

- **UMA does not scale:** a shared bus and single memory controller are a centralized bottleneck. Moving the memory controller onto each socket scales aggregate bandwidth but makes memory **non-uniform** (**NUMA**): local access is fast, remote (across the interconnect) is slower.
- A **NUMA node** pairs a set of cores with their local memory. Sockets are joined by a coherent interconnect — **Intel UPI/QPI**, **AMD Infinity Fabric** — which is a **shared, saturable network**. Remote latency is commonly **~1.5–2x** local; remote **bandwidth** is lower and can hurt streaming workloads more than the latency ratio suggests. Firmware advertises a **distance matrix** (SLIT; local=10, one-hop remote ≈21).
- **One socket can be several NUMA nodes:** Intel **Sub-NUMA Clustering** and AMD **chiplet/NPS** topologies (EPYC's CCDs + I/O die) create intra-package non-uniformity. Never assume "single socket = single node" — check `numactl --hardware`.
- Coherence at scale is **directory-based** (Chapter 4); coherence traffic **crosses the interconnect**, so **cross-node shared mutable state** (a line ping-ponging between sockets) is the cardinal NUMA sin.
- Naive placement scatters a service's memory and threads across nodes, making a large fraction of accesses remote and saturating the interconnect — a double-digit performance loss **invisible in application code and CPU-utilization metrics**.

- **First-touch** is the default: a page is placed on the node of the thread that first *writes* it. **First-touch initialization** — initialize data on the thread/CPU that will use it — is the single most important NUMA idiom. Beyond it: **bind** (force a node), **preferred** (prefer with spillover), and **interleave** (spread for bandwidth). Bind optimizes latency for partitionable data; interleave optimizes bandwidth for shared, uniformly-accessed data.
- Control placement with `numactl`, `mbind` / `set_mempolicy` / `move_pages`, and thread affinity (`sched_setaffinity` / `taskset`). Observe with `numactl --hardware`, `lscpu`, `numastat` (`numa_hit` / `numa_miss` / `numa_foreign`), and `numastat -p <pid>`. Linux **automatic NUMA balancing** is a safety net, not a substitute for explicit placement.
- The design playbook: **partition per node (share-nothing)**, **first-touch init**, **thread+memory affinity**, **avoid cross-node shared mutable state**, **interleave the genuinely shared**. **Pin to one node** when the working set fits (often "one instance per node"); **span with internal partitioning** only when it must not; **ignore NUMA** on single-node boxes.
- Large **cloud instances are multi-socket NUMA** with **vNUMA** exposed to the guest; size instances to whole nodes/sockets, pin per node, and beware the **noisy-neighbor × interconnect** interaction. **CXL** memory extends the NUMA model to memory-only tiers.
- **The lens:** a NUMA machine is a **distributed system inside the box** — nodes, local memory, a shared network, non-uniform latency, a coordination protocol. Data locality, sharding, minimizing cross-node coordination, and affinity scheduling apply unchanged. The hierarchy is **fractal** (core→node→server→rack→DC→region); only the constants differ.

Further reading

- Christoph Lameter, "NUMA (Non-Uniform Memory Access): An Overview," *ACM Queue*, 2013 — https://queue.acm.org/detail.cfm?id=2513149 — a clear, authoritative overview of NUMA hardware, Linux policies, and tuning from a longtime Linux memory-management maintainer.
- Ulrich Drepper, "What Every Programmer Should Know About Memory" (2007), Red Hat — https://people.freebsd.org/~lstewart/

articles/cpumemory.pdf — Part 5 covers NUMA support, node distances, and first-touch/`libnuma` in depth.

- `numactl(8)`, `numastat(8)`, `mbind(2)`, `set_mempolicy(2)`, `move_pages(2)`, and `sched_setaffinity(2)` man pages — the authoritative reference for the policies and syscalls; `numa(7)` and `numa(3)` document the Linux NUMA model and `libnuma` API.
- The Linux kernel NUMA documentation — https://www.kernel.org/doc/html/latest/mm/numa.html and the automatic NUMA balancing / scheduler NUMA docs — for the memory-policy and balancing internals (see also Volume 2, Operating Systems).
- Intel 64 and IA-32 Architectures Optimization Reference Manual and the relevant Xeon uncore/UPI documentation — https://www.intel.com/sdm — for UPI, Sub-NUMA Clustering, home agents, and directory coherence on Intel parts.
- AMD, "AMD EPYC Processor Memory and NUMA (Nodes Per Socket) Tuning Guides" and the Zen/EPYC architecture documentation — https://www.amd.com/en/developer.html — for Infinity Fabric, CCD/I/O-die topology, and NPS configuration (vendor-specific but rigorous).
- Brendan Gregg, *Systems Performance*, 2nd ed. (Addison-Wesley, 2020), and https://www.brendangregg.com — practical methodology for observing memory locality, `numastat`, and PMU-based local/remote access analysis on Linux.
- Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. (Morgan Kaufmann, 2017) — Chapter 5 covers distributed/shared-memory multiprocessors, directory coherence, and the scaling arguments behind NUMA.
- CXL Consortium specifications — https://computeexpresslink.org — for CXL-attached memory and its presentation as memory-only NUMA nodes (see also Volume 12, Cloud Infrastructure, and Chapter 5).

Chapter 7 — Data Parallelism: SIMD, Vectorization, and GPUs for Backend

---

What this chapter covers. Chapters 2 through 6 pursued one form of parallelism: doing *different* things at once. Instruction-level parallelism

overlaps unrelated instructions inside one core; multicore and NUMA (Chapters 2, 6) run different threads on different cores; the whole distributed-systems enterprise runs different requests on different machines. This chapter is about the other axis entirely — doing the *same* thing to *many pieces of data* at once. When a query engine sums a billion-row column, when a JSON parser scans a gigabyte payload, when a model scores a batch of embeddings, the work is not a diverse mix of tasks; it is one operation applied uniformly across a mountain of homogeneous elements. That shape is **data parallelism**, and hardware exploits it with dedicated machinery: **SIMD** vector units inside every modern CPU core, and **GPUs** — accelerators built almost entirely out of it. For a backend engineer these are no longer exotic. Vectorized execution is why modern OLAP engines are fast; GPU inference is a standard serving tier. The organizing idea, developed in the closing lens, is that data parallelism scales *the throughput of a single node* — the orthogonal axis to distribution, which scales *across* nodes — and that the same hard lessons you know from distributed systems (data movement dominates; locality is everything; batching amortizes fixed cost; measure before you believe) reappear at the level of vector lanes and PCIe links.

Learning goals — after this chapter you should be able to:

- Distinguish **data parallelism** (SIMD/SIMT) from **task parallelism** (ILP, multicore), and place both in **Flynn's taxonomy**.
- Explain CPU **SIMD**: vector registers, lanes, and the ISA progression **SSE (128-bit) → AVX/AVX2 (256-bit) → AVX-512 (512-bit)**, plus **ARM NEON** and length-agnostic **SVE/SVE2**.
- State the **throughput win** (N elements per instruction) and the **constraints** (contiguous, aligned, uniform op, no divergent control flow) that decide whether a loop vectorizes.
- Reason about how you *actually* get SIMD: **auto-vectorization** and why it is fragile, **intrinsics**, and portable **SIMD libraries** — and why hand-vectorization is sometimes unavoidable.
- Name the backend workloads that are SIMD wins: **columnar/ vectorized analytics**, **simdjson**-style parsing, compression, hashing, filtering, and ML inference.
- Explain **GPU architecture**: thousands of simple cores, the **SIMT** model, **warps/wavefronts**, the memory hierarchy, and **latency-**

hiding via massive oversubscription — and the host/device model where **PCIe transfer cost** often dominates.

- Use **arithmetic intensity** and the **roofline** (Chapter 1) to decide when an accelerator helps.
- Situate **TPUs, FPGAs, and DPUs** in the industry's move to **domain-specific, heterogeneous compute**.
- Connect all of it to the distributed reality: vectorized-per-node + sharded-across-nodes OLAP, and data-parallel-within-GPU + distributed-across-GPUs ML training.

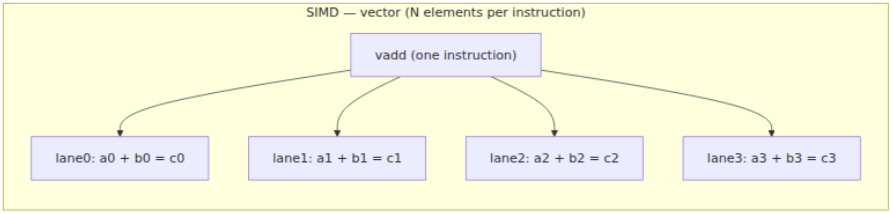
Two axes of parallelism, and Flynn's taxonomy

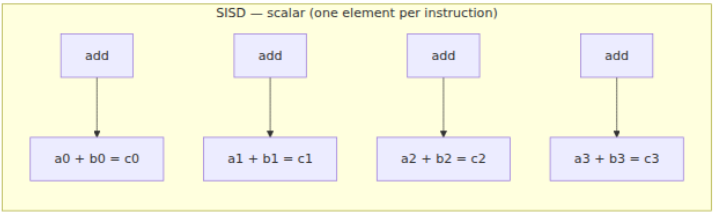
Every parallel machine answers two independent questions: how many *instruction streams* are in flight, and how many *data streams* each acts on. Michael Flynn's 1966 taxonomy names the four combinations, and though it is coarse it remains the cleanest framing for this chapter:

- **SISD** — Single Instruction, Single Data. The classical scalar processor: one instruction operates on one datum. This is the mental model most code is written against, even on hardware that is nothing like it anymore.
- **SIMD** — Single Instruction, Multiple Data. One instruction operates on a *vector* of data elements simultaneously. This is what a CPU vector unit does, and the topic of the first half of this chapter.
- **MISD** — Multiple Instruction, Single Data. Different operations on the same datum. Essentially a theoretical curiosity; systolic fault-tolerant designs are sometimes shoehorned into it. Ignore it.
- **MIMD** — Multiple Instruction, Multiple Data. Independent instruction streams over independent data. This is multicore, multi-socket (Chapter 6), and every cluster of machines you have ever operated.

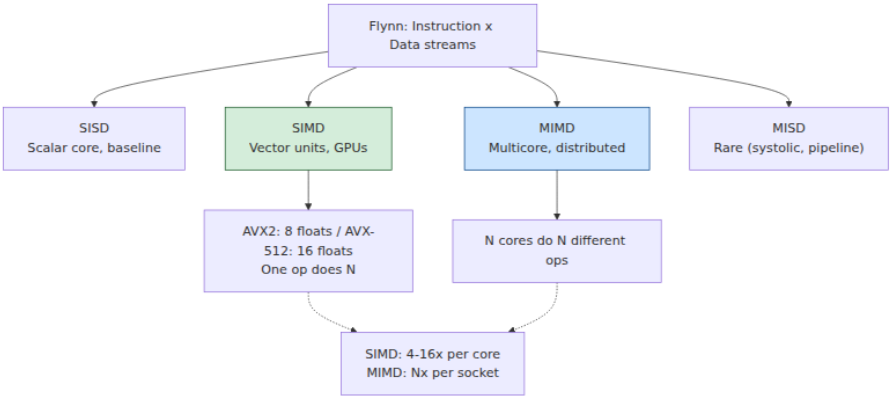
The crucial contrast is **SIMD vs MIMD**, because it is the contrast between the two ways to parallelize. The ILP of Chapter 2 (out-of-order, superscalar issue) and the multicore/multi-socket parallelism of Chapters 2 and 6 are **task parallelism**: the hardware finds or the programmer creates *independent work*, and runs pieces of it concurrently, each piece potentially different. Its cost is coordination — dependencies, synchronization, cache coherence (Chapter 4), scheduling. **Data**

parallelism is the opposite bargain. It requires that the work be *homogeneous* — the same operation over many elements — and in exchange it eliminates most per-element overhead: one instruction fetch, one decode, one dependency check amortized over 8, 16, or thousands of elements. When a workload fits, data parallelism is the most *energy- and silicon-efficient* parallelism there is, which is exactly why the industry has poured a decade of transistor budget into wider vectors and bigger GPUs.





SISD does four adds to add four pairs; SIMD does one `vadd` that produces four results in the same latency a scalar add takes. Widen the vector and the ratio grows. That is the entire idea. Everything else is the engineering required to keep the lanes full and the constraints satisfied.



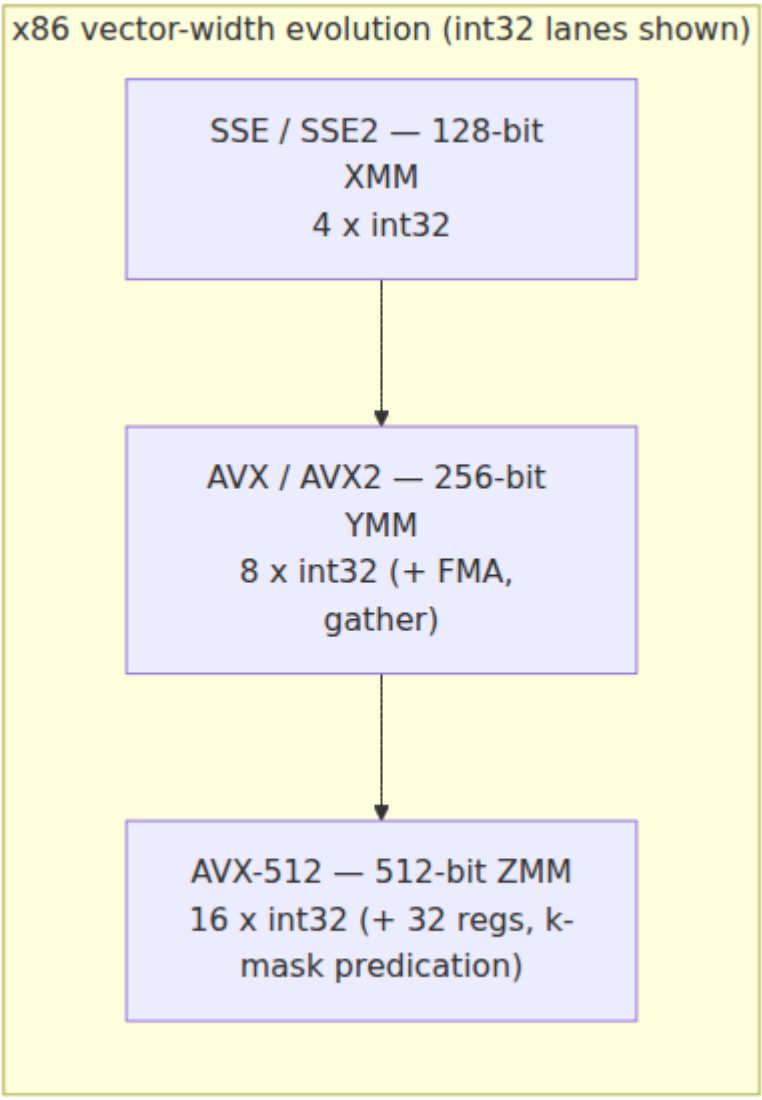
CPU SIMD: vector registers, lanes, and the ISA ladder

A SIMD instruction operates on a **vector register** — a wide register partitioned into equal-width **lanes**, each holding one element. The register width is fixed by the ISA; the element width is chosen by the instruction. A 256-bit register is 8 lanes of 32-bit integers, or 4 lanes of 64-bit doubles, or 32 lanes of 8-bit bytes. A single "add packed 32-bit integers" instruction reads two such registers, adds lane 0 to lane 0, lane 1 to lane 1, and so on across all lanes independently, and writes the packed result — the classic **SIMD lane** picture: parallel, non-interacting pipes, one instruction driving all of them.

The x86 SIMD story is a straight ladder of doubling widths, each rung adding registers and instructions while keeping the last:

- **MMX** (1997) — the false start: 64-bit, integer-only, aliased onto the x87 floating-point registers so you could not use both at once. Effectively dead; mentioned only because its descendants inherited the "packed" vocabulary.
- **SSE / SSE2 / SSE3 / SSE4** (1999 onward) — **128-bit** `XMM` registers. SSE added packed single-precision float; SSE2 added packed double and packed integers, making 128-bit SIMD general-purpose. A 128-bit register is **4 × float32**, **2 × float64**, or **4 × int32** (16 × int8). SSE is the baseline: every x86-64 CPU has SSE2, so compilers can assume it unconditionally.
- **AVX** (2011, Sandy Bridge) — **256-bit** `YMM` registers, initially for floating point, with a cleaner three-operand (non-destructive) encoding via the VEX prefix. **AVX2** (2013, Haswell) extended the 256-bit operations to *integers*, and added **FMA** (fused multiply-add: `a*b+c` in one rounding step, doubling peak FLOP/s for dot-product-shaped math) and **gather** (load from non-contiguous addresses). A 256-bit register is **8 × float32** or **8 × int32**.
- **AVX-512** (Xeon Phi "Knights Landing" 2016; mainstream server with Skylake-SP 2017) — **512-bit** `ZMM` registers, **32** of them (up from 16), and, most importantly, **eight mask registers** (`k0` – `k7`) for *per-lane predication*. A 512-bit register is **16 × float32**, **16 × int32**, or **8 × float64/int64**. AVX-512 is not one extension but a *family* of subsets — F (foundation), CD, BW, DQ, VL, VNNI (integer dot-product for inference), BF16, and more — and different CPUs implement

different subsets, which makes "does this machine have AVX-512" a genuinely fiddly question at deployment time.



The **mask registers** deserve emphasis because they attack SIMD's core weakness. Ordinary SIMD demands *uniform* operation: every lane does the same thing. Real code has conditionals — "add only where the value is positive." Without masks you must compute both branches for all lanes

and blend the results. AVX-512's per-lane masks let one instruction write only the lanes the mask selects, so vectorized code can express `if` without fully abandoning the vector. SVE and modern GPUs use predication for the same reason; it is one of the defining features of contemporary SIMD.

On the **ARM** side the design philosophy diverged:

- **NEON** (Advanced SIMD) — fixed **128-bit** registers, mandatory in AArch64. Structurally like SSE: four float32 lanes, wide vocabulary of packed integer/float ops. Every Apple Silicon, Graviton, and mobile ARM core has it.
- **SVE / SVE2** (Scalable Vector Extension, ARMv8-A/ARMv9) — the interesting departure. SVE is **vector-length agnostic (VLA)**: the *code does not encode the vector width*. The hardware chooses an implementation width — any multiple of 128 bits from 128 up to 2048 — and the *same binary* runs correctly on all of them, driven by a `WHILELT`-style predicate loop that handles whatever the remaining element count is. Fujitsu's A64FX (the Fugaku supercomputer) implements 512-bit SVE; other parts implement 128 or 256. SVE also bakes in per-lane predication and gather/scatter. **SVE2** generalizes the instruction set to cover the media/DSP workloads NEON handled, positioning it as NEON's successor. RISC-V's **Vector extension (RVV)** takes the same length-agnostic path — a deliberate rejection of x86's "add a new fixed width and a new ISA every few years" churn.

ISA	Register width	int32 lanes	float32 lanes	Notes
SSE / SSE2	128-bit	4	4	x86-64 baseline; always present
AVX / AVX2	256-bit	8	8	AVX2 adds 256-bit integer, FMA, gather
AVX-512	512-bit	16	16	32 regs, k-mask predication; subset zoo
ARM NEON	128-bit (fixed)	4	4	Mandatory on AArch64

ISA	Register width	int32 lanes	float32 lanes	Notes
ARM SVE / SVE2	128–2048, VLA	width-dependent	width-dependent	Length-agnostic; one binary, many widths

The throughput arithmetic is simple and seductive: at 256 bits you process 8 × int32 per vector instruction, at 512 bits you process 16. A tight loop that saturates the vector unit can approach an **N×** speedup over scalar, where N is the lane count — and FMA doubles it again for multiply-accumulate math. The catch is the word *saturate*. Getting there requires satisfying constraints that ordinary code routinely violates, and understanding those constraints is the difference between a headline number and a disappointment.

The constraints: what SIMD demands of your data and control flow

SIMD is fast because it is rigid. Four requirements dominate.

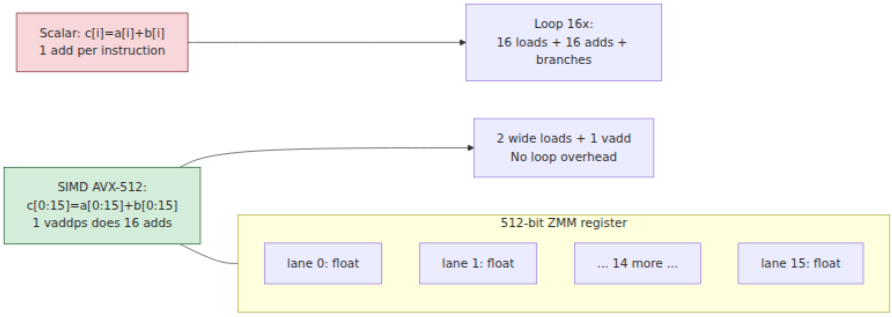
Contiguity and layout. A vector load pulls N adjacent elements from memory in one shot. That is cheap only if the N elements you want *are* adjacent. An **array of structs** (`struct { float x, y, z; } pts[]`) scatters each field across memory, so vectorizing "sum all x" means gathering every third-ish element — either a slow gather instruction or a manual shuffle. A **struct of arrays** (`float xs[]; float ys[]; float zs[]`) puts all the x's contiguous, and the vector load is trivial. This **AoS → SoA** transformation is the single most impactful thing you can do to make data SIMD-friendly, and it is exactly the same instinct as **columnar storage** in analytics (below): store like with like so the machine can stream it.

Alignment. Vector loads are fastest (historically, mandatory) when the address is aligned to the vector width. Modern hardware tolerates unaligned vector loads with a small penalty and a large one when a load straddles a cache line (Chapter 3), but alignment still matters for peak throughput and is one of the things the compiler cannot always prove.

Uniform operation. Every lane runs the same op. A loop body that does different arithmetic depending on the element does not map onto one instruction.

No divergent control flow. This is the killer. A branch inside the loop — `if (x[i] > threshold)` — asks different lanes to do different things, which SIMD cannot express directly. The escape hatches are **predication/masking** (compute all lanes, mask off the unwanted results — wasted work, but branch-free) and, on wide branches, giving up and staying scalar. Data-dependent early exit, function calls with side effects, and anything that can throw all defeat vectorization.

These four are not incidental — they *define* what "data-parallel" means. A workload that satisfies them is SIMD-friendly; one that cannot is not, no matter how much compute it burns.



How you actually get SIMD

Owning an AVX-512 CPU does not make your code use it. There are four routes, in ascending order of effort and control.

Auto-vectorization is the compiler recognizing a vectorizable loop and emitting vector instructions on its own (`-O2` / `-O3` with `-march=native` or an explicit `-mavx2` ; LLVM's LoopVectorizer, GCC's tree vectorizer). When it works it is free. The problem is that **it is fragile and fails silently**, and the reasons it fails are precisely the constraints above dressed up as compiler-provability questions:

- **Aliasing.** If two pointers *might* overlap, writing through one could change what the other reads, so the compiler must assume a loop-carried dependency and refuse to vectorize. C's `restrict` (Rust's ownership model, Fortran's non-aliasing-by-default) is the promise that unlocks this — and its absence is the most common silent failure.

- **Alignment and trip count unknown at compile time** force the compiler to emit peeling/remainder code and may make it decide vectorization is not worth it.
- **Loop-carried dependencies** — each iteration reads the previous iteration's result (a running sum written back to the same accumulator naively, pointer-chasing, prefix computations) — are unvectorizable as written.
- **Function calls, potential exceptions, complex control flow, `break` on a data condition** — all block it.
- **Reductions and floating-point reassociation.** Vectorizing a sum means adding elements in a different order, and floating-point addition is not associative (Chapter 9), so the result changes in the last bits. A strict compiler will *not* reorder FP math unless you pass `-ffast-math` / `-ffp-contract` or use a reduction pragma — meaning correct-by-default compilers leave FP reductions scalar unless you opt in.

The practical consequence: **auto-vectorization is a bonus, not a plan.** It reliably handles simple, `restrict`-annotated, statically-sized loops over contiguous arrays, and it evaporates the moment the code gets interesting. You must *verify* it happened — read the assembly, check the compiler's vectorization remarks (`-Rpass=loop-vectorize` / `-fopt-info-vec`), or measure — because the compiler will not warn you that it silently gave up and left 8× on the table. Treating "the optimizer will vectorize it" as a guarantee is one of the most common performance misconceptions in backend code.

Intrinsics are the explicit route: functions that map one-to-one onto vector instructions (`_mm256_add_epi32`, `vaddq_s32`), letting you hand-write the exact SIMD you want in C/C++/Rust while the compiler still handles register allocation and scheduling. This is how the fast libraries are built. It is powerful and *non-portable* — AVX2 intrinsics do not run on ARM, and 256-bit code does not automatically become 512-bit — so real projects carry per-ISA code paths selected at runtime by CPU feature detection (CPUID). It is also verbose and easy to get subtly wrong.

Portable SIMD libraries paper over the ISA fragmentation. Google's **Highway** (C++), Rust's `std::simd` / the `wide` and `packed_simd` ecosystem, C++'s experimental `std::simd`, and Intel's ISPC (a SIMD-oriented language) let you write vector logic once against an abstract

vector type and compile it to NEON, AVX2, AVX-512, or SVE, often with runtime dispatch built in. This is the sweet spot for most teams that need explicit SIMD without maintaining four hand-written kernels.

The reason **hand-vectorization survives** despite decades of compiler work is that the highest-value SIMD routines — parsing, decoding, the inner loops of a database engine — use algorithmic transformations (clever shuffles, table lookups via `pshufb`, branch-free state machines) that no auto-vectorizer will ever discover, because they require *rethinking the algorithm* to be data-parallel, not merely widening an existing scalar loop.

The AVX-512 frequency caveat

One deployment-relevant wrinkle. On several **older Intel server generations** (notably Skylake-SP and Cascade Lake, roughly 2017–2019), executing heavy AVX-512 (and to a lesser degree AVX2) instructions drew enough power that the core **downclocked** — sometimes across the whole socket — to stay within thermal and voltage limits. The effect: sprinkling a little AVX-512 into an otherwise scalar or lightly-threaded workload could *slow down* the surrounding non-vector code by dropping the clock, occasionally making the "optimization" a net loss at the application level. This drove real guidance at the time (Cloudflare and others documented it) to be cautious about AVX-512 in mixed workloads. The important hedges: it was **always workload- and SKU-dependent**, it was most acute on those specific generations, and **newer parts (Ice Lake onward, and AMD's Zen 4 AVX-512) reduced or largely eliminated the penalty**. It is a caution to *measure* on your actual hardware, not a blanket "avoid AVX-512." (Consumer Alder Lake, incidentally, shipped with AVX-512 fused off after early enabling — another reason to feature-detect rather than assume.)

Backend uses of SIMD

SIMD stopped being a graphics/HPC niche and became load-bearing backend infrastructure. The pattern is always the same: a workload that is *bulk, homogeneous, and column-shaped* gets rewritten to process many elements per instruction.

Vectorized query execution — the reason modern OLAP is fast. The defining architectural choice of ClickHouse, DuckDB, and the Apache

Arrow ecosystem is that they process data in **columnar batches**, not row-at-a-time. Classic databases execute a query one tuple at a time through a tree of operators (the "Volcano" iterator model): for each row, call `next()`, chase virtual functions, evaluate the predicate. The per-row interpreter overhead swamps the actual work. **Vectorized execution** — pioneered by **MonetDB/X100** (Boncz, Zukowski, Nes, CIDR 2005) — flips this: operators consume and produce *vectors* of thousands of values from a single column, so the inner loop is a tight, branch-light, cache-friendly, and **auto-/hand-vectorizable** pass over contiguous, same-type data. A `WHERE price > 100` over a column becomes a SIMD compare producing a bitmask; a `SUM` becomes a SIMD reduction; a filter becomes a mask-and-compact. This is why "vectorized execution" is the marquee feature of every fast analytical engine — the word *vectorized* is literally SIMD. Columnar storage (Volume 5, on data systems) and SIMD are symbiotic: columns give you the contiguous, uniform-type layout that SIMD demands.

JSON and string parsing — simdjson. Parsing looks inherently sequential and branchy — the last place you would expect SIMD. **simdjson** (Langdale and Lemire, 2019) showed otherwise, parsing JSON at multiple gigabytes per second by recasting parsing as data-parallel *stages*: use SIMD to classify all bytes at once (find every quote, brace, backslash, whitespace via packed compares), build a structural-character bitmap branch-free, and only then walk structure. The lesson generalizes: many "sequential" text problems — UTF-8 validation, CSV parsing, base64, escaping — have branch-free SIMD reformulations, and the libraries that implement them (simdjson, and Arrow's and the browsers' adoption of the same techniques) are now standard dependencies.

Compression, encoding, hashing, checksums. LZ4/Zstd matching, bit-packing and delta encoding in columnar formats (Parquet/ORC), base64, CRC32 and hashing all have SIMD implementations; x86 even exposes a hardware `CRC32` instruction and carry-less multiply (`PCLMULQDQ`) that GCM-mode AES and hashing exploit. These sit in the hot path of storage engines and network stacks.

Search, filtering, and set operations. Scanning a buffer for a byte or pattern (SIMD `memchr`/substring search — the guts of fast log grep and packet inspection), intersecting sorted posting lists in a search index, and bitmap operations (Roaring bitmaps) are all vectorized.

ML inference on CPU. Not every model needs a GPU. Quantized (int8) inference, embedding similarity (dot products at scale), and classical ML lean on FMA and AVX-512's **VNNI** int8 dot-product instructions; libraries like oneDNN and the inner loops of ONNX Runtime and llama.cpp are heavily hand-vectorized.

Backend workload	What SIMD does	Representative implementations
Columnar / OLAP query execution	Batched column filters, aggregations, joins	ClickHouse, DuckDB, Arrow (MonetDB/X100 lineage)
JSON / text parsing	Branch-free byte classification, structural scan	simdjson, Arrow, browser parsers
Compression / encoding	Match search, bit-packing, delta, base64	LZ4, Zstd, Parquet/ORC codecs
Hashing / checksums / crypto	CRC32, GCM, vectorized hashing	hardware CRC32/ PCLMULQDQ, xxHash
Search / filtering	memchr, substring, posting-list intersect, bitmaps	ripgrep, search indexes, Roaring
CPU ML inference	int8/FP dot products, FMA, VNNI	oneDNN, ONNX Runtime, llama.cpp

The through-line: SIMD gives you a **per-core throughput multiplier** on data-parallel work, entirely inside one thread, on hardware you already rent. It is the cheapest performance on this list — no new machines, no accelerator, no network — which is exactly why the fast systems chase it.

GPUs: data parallelism as the whole machine

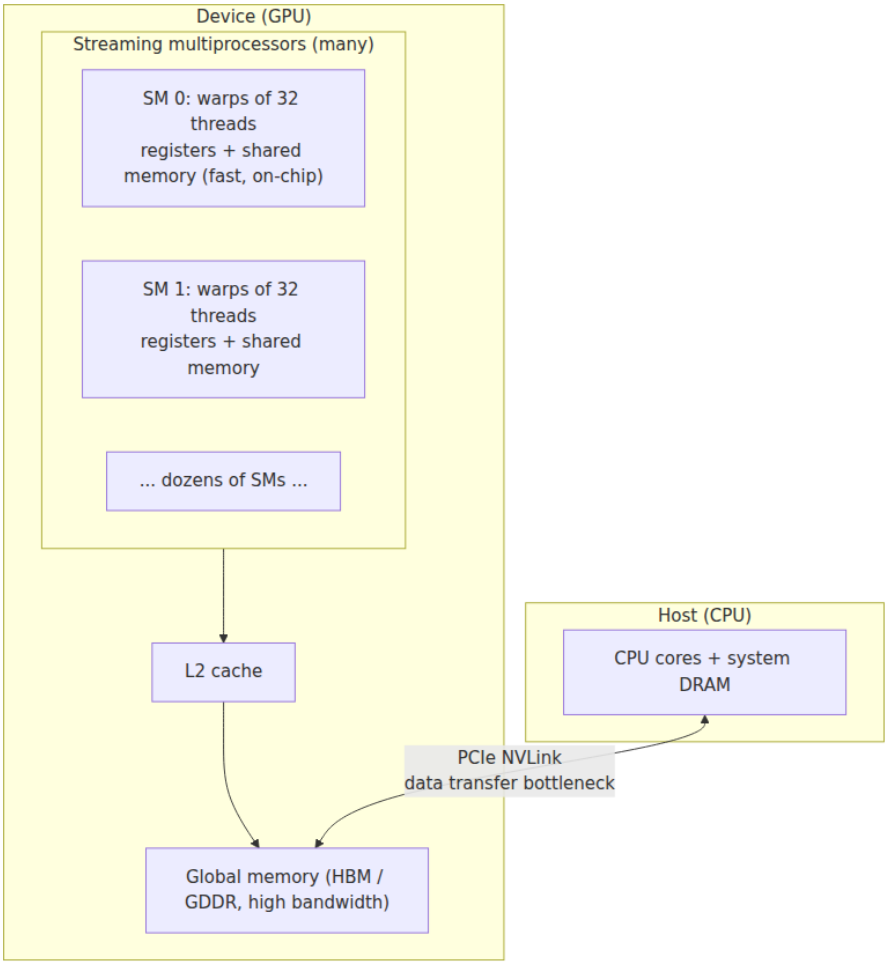
A CPU is a latency machine: a handful of enormously complex cores (Chapter 2 — out-of-order, deep speculation, big caches) built to finish *one* instruction stream as fast as possible, with SIMD bolted on the side. A **GPU** inverts every priority. It is a **throughput machine**: thousands of small, simple arithmetic units, minimal per-core control logic, shallow speculation, and a design that assumes you have *so much* independent data-parallel work that it can hide latency not by avoiding stalls but by

having thousands of other operations ready to run whenever one stalls. Where a CPU spends transistors making one thread fast, a GPU spends them making ten thousand threads *coexist*.

The SIMT execution model

GPUs execute in a model NVIDIA calls **SIMT — Single Instruction, Multiple Thread**. You write a **kernel** as if it were scalar code for a single thread; you launch it across a huge grid of threads (millions is routine); and the hardware groups threads into fixed bundles — **warps** of **32 threads** on NVIDIA, **wavefronts** of **64** (or 32 on RDNA) on AMD — that execute **in lockstep**: all 32 threads of a warp run the *same* instruction at the same time on their own data. That lockstep bundle *is* a SIMD vector; SIMT is SIMD with a friendlier programming model, where the hardware manages the lanes and per-lane masking for you instead of making you write explicit vector code.

The consequence is that GPUs have the *same* fundamental constraint as CPU SIMD, wearing a different name: **warp divergence**. If threads in a warp take different sides of a branch (`if (tid % 2)`), the hardware must execute *both* paths with the inactive lanes masked off — serializing the divergent regions and wasting the masked lanes. Divergent, branchy code on a GPU is exactly as pathological as divergent control flow in CPU SIMD, and for the identical reason. GPUs love code where every thread in a warp does the same thing to adjacent data.



The memory hierarchy and latency hiding

A GPU's memory hierarchy mirrors the CPU's tiers (Chapter 3) but with different proportions:

- **Registers** — per-thread, enormous in aggregate (a GPU has hundreds of KB to MBs of register file per SM so thousands of threads can each keep state resident). Fastest.
- **Shared memory** — a small, fast, *software-managed* scratchpad per SM (tens to ~100+ KB), shared by the threads of a block. It is the GPU programmer's most important tool: you *explicitly* stage data

from global memory into shared memory so a block of threads can reuse it, turning many slow global accesses into one. It plays the role of a programmer-controlled L1.

- **L1 / L2 caches** — hardware caches, L2 shared across the device.
- **Global memory** — the large off-chip DRAM (**HBM** on datacenter parts, **GDDR** on consumer), with **very high bandwidth** (well into the TB/s range on modern datacenter GPUs) but also high latency — many hundreds of cycles.

The defining trick is **latency hiding through oversubscription**. A CPU hides memory latency with big caches and out-of-order execution. A GPU mostly does neither; instead, when a warp issues a global-memory load and stalls waiting hundreds of cycles, the SM's scheduler instantly switches to *another ready warp*, and another, keeping the arithmetic units busy while dozens of warps' loads are in flight. This only works if you give it enough parallelism to hide the latency — high **occupancy**, many resident warps per SM. Too few threads, or too much per-thread register/shared-memory pressure limiting how many warps fit, and the latency stops being hidden and the GPU stalls. The GPU's throughput is *contingent on massive oversubscription*; feed it a small problem and its cores sit idle waiting on memory, no faster than (often slower than) a CPU.

One more memory subtlety that trips up every GPU newcomer: **coalescing**. When the 32 threads of a warp each load from global memory, the hardware is fastest when their addresses are *contiguous* — thread 0 reads word 0, thread 1 word 1 — so the accesses **coalesce** into a few wide memory transactions. Strided or scattered per-thread accesses de-coalesce into many transactions and tank effective bandwidth. This is the GPU version of the CPU's contiguity/AoS-vs-SoA rule, at warp granularity, and it is the same lesson Chapter 3 taught about cache lines: **spatial locality decides your bandwidth.**

The host/device model and the transfer wall

A GPU is a separate device across a bus. The programming model is **host** (CPU) and **device** (GPU) with *separate memories*, and the workflow is: **copy input from host DRAM to device global memory → launch the kernel → copy results back**. Those copies traverse **PCIe** (roughly ~32 GB/s each way on PCIe 4.0 x16, ~64 GB/s on PCIe 5.0 x16) — or, on tightly-coupled systems, **NVLink** at several times that — and *that link is*

very often the bottleneck. On-device HBM bandwidth is measured in TB/s; PCIe is an order of magnitude slower. If a kernel does only a little arithmetic per byte transferred, the PCIe copies dominate the wall-clock time and the GPU's compute never gets to shine.

This is precisely the Chapter 1 / Chapter 3 lesson restated one level out: **data movement dominates.** The same instinct that says "don't chase pointers across DRAM" and "keep the working set in cache" says, at the GPU boundary, "don't ship data across PCIe for a trivial computation." The winning patterns keep data *resident* on the GPU across many kernels (train/infer in place rather than round-tripping), overlap transfer with compute (async copies on streams), and batch enough work that the transfer is amortized. A naive "copy array over, do one cheap op, copy it back" GPU offload is almost always *slower* than just doing it on the CPU — the classic disappointment.

When GPUs win, and when they lose — arithmetic intensity and the roofline

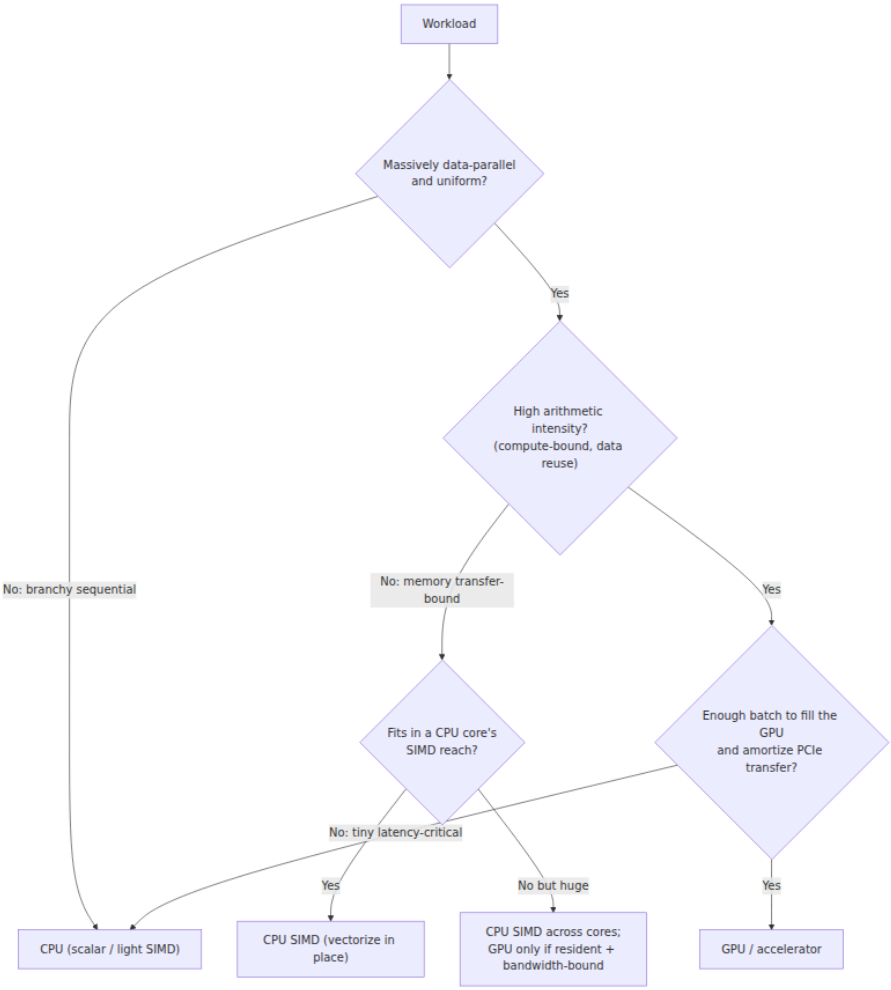
The single best predictor of whether an accelerator helps is **arithmetic intensity**: FLOPs performed per byte of memory traffic (Chapter 1's **roofline** model — Williams, Waterman, Patterson, 2009). A workload with **low arithmetic intensity** (few operations per byte — a vector add, a filter, a copy) is **memory-bound**: its speed is capped by bandwidth, and a GPU's edge shrinks to its (real, but bounded) HBM-bandwidth advantage, easily erased by the PCIe transfer. A workload with **high arithmetic intensity** (many operations per byte — dense matrix multiply, convolution, attention, where each loaded value is reused across many multiply-adds) is **compute-bound**, and here the GPU's thousands of FMA units and dedicated **tensor/matrix cores** deliver order-of-magnitude wins. This is not a coincidence that GPUs dominate deep learning: neural network training and inference are dense linear algebra, the highest-arithmetic-intensity workload in mainstream computing, and the accelerators were co-designed with it.

The honest ledger:

GPUs win when the work is *massively data-parallel* (millions of independent elements), *high arithmetic intensity* (compute-bound, reuses data on-chip), *uniform* (little warp divergence), *batchable* (enough work to fill thousands of cores and hide latency), and *transfer-amortizable* (data stays resident, or compute per transferred byte is high). Dense

linear algebra, deep learning, large-scale image/video processing, some Monte Carlo and scientific simulation, and increasingly analytical scans over GPU-resident columns.

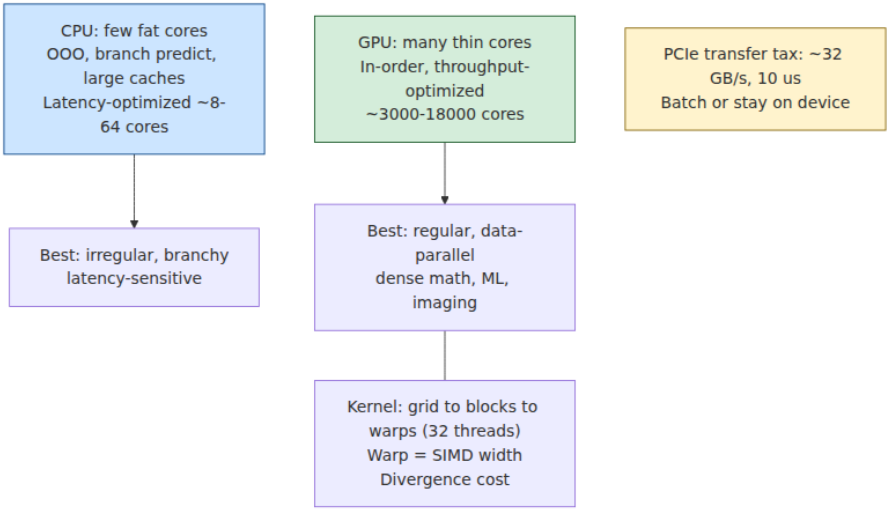
GPUs lose when the work is *branchy/divergent* (warps serialize), *latency-sensitive* with tiny inputs (a single small request pays full PCIe + launch overhead to compute almost nothing — the GPU's throughput is irrelevant when there is no bulk to throughput), *low arithmetic intensity / transfer-bound* (PCIe eats the benefit), *pointer-chasing or irregular* (no coalescing, no parallelism), or simply *small* (not enough work to amortize any of the fixed costs). Most of a typical request/response backend — parse, branch, call services, serialize — is exactly this shape, which is why the CPU still runs the world.



CUDA, ROCm, and the software reality

The dominant GPU programming stack is **CUDA** (NVIDIA), a C++-based model plus a deep library ecosystem — cuBLAS, cuDNN, and the frameworks (PyTorch, TensorFlow, JAX) that sit on top and are what most backend engineers actually touch. AMD's answer is **ROCm** with **HIP**, a near-source-compatible CUDA analog; cross-vendor efforts include **SYCL/ oneAPI** and the older **OpenCL**. For most backend work you will never write a kernel: you will call a framework that dispatches optimized kernels for you, and your job is the *systems* problem — feeding the GPU,

batching, memory management, and keeping the expensive device busy. The kernel-level details above matter not because you will write them but because they explain *why* your GPU service behaves the way it does when a batch is too small or a transfer too frequent.



GPUs for backend engineering

For most backend teams, the GPU arrives as a **serving tier**, not a library call. The dominant use is **ML inference** — serving a model (an LLM, a ranking model, an embedding model, a vision model) behind an RPC — with training the other major consumer. The engineering shape of a GPU inference service is distinct enough to enumerate, because it is where architecture meets operations.

Batching is the core lever. A GPU serving one request at a time is almost entirely idle — a single inference does not fill thousands of cores, and it pays full fixed overhead. So inference servers (Triton, TGI, vLLM, TensorFlow Serving) **batch** many concurrent requests into one kernel launch, trading a little latency (waiting a few milliseconds to accumulate a batch) for a large throughput gain (the GPU processes the batch nearly as fast as one request). This is a direct **throughput-vs-latency** trade-off, and tuning the batch window and size is the central operational knob. **Dynamic/continuous batching** (as in vLLM's paged-attention scheduler for LLMs) refines this by adding and retiring sequences from the running batch each step. The lesson is the same one batching teaches everywhere

in distributed systems (Volume 6): **amortize fixed cost over many items**, here the fixed cost being kernel launch and the accelerator's appetite for parallelism.

The economics are unusual. Datacenter GPUs are expensive and, in the current AI cycle, supply- constrained; a GPU that sits at 10% utilization is burning money at a rate that dwarfs CPU waste. This inverts the usual backend priority: for a GPU tier, *keeping the accelerator saturated* often matters more than shaving tail latency, because the marginal cost structure is dominated by the device. Hence aggressive batching, model **quantization** (int8/FP8/FP4 to fit more model and more batch in memory and cut compute), **KV-cache** management for LLMs, multi-tenancy (MIG partitioning to run several small models on one physical GPU), and autoscaling policies driven by GPU utilization and queue depth rather than CPU. The transfer/latency trade-offs from the architecture section become SLO decisions: keep model weights resident (never reload across PCIe per request), keep the batch on-device, and accept the batching latency as the price of affordable throughput.

Beyond inference, GPUs show up in **GPU-accelerated databases and analytics** (HeavyDB/OmniSci, RAPIDS cuDF, and GPU-accelerated Spark) that run scans, joins, and aggregations on GPU-resident columns — the same vectorized-execution idea from earlier, pushed onto SIMT hardware — and in general acceleration of data-parallel batch jobs. The caveat is always the transfer wall: GPU analytics wins when data can live on the GPU across the workload, and loses when every query must stream fresh data across PCIe.

Other accelerators: the heterogeneous datacenter

SIMD and GPUs are two points on a spectrum whose far end is *hardware built for one job*. Hennessy and Patterson, in their 2018 Turing Award lecture "A New Golden Age for Computer Architecture," argued that with Moore's Law slowing and Dennard scaling dead (Chapter 1), the future of performance is **domain-specific architectures** — trading general-purpose flexibility for enormous efficiency on a target workload. The datacenter is now visibly **heterogeneous**, and a backend engineer should recognize the players:

- **TPUs (Tensor Processing Units)** — Google's ML accelerators, built around a large **systolic array** matrix-multiply unit. A systolic array

streams data through a grid of multiply-accumulate cells so each loaded value is reused across many operations without returning to memory — a hardware embodiment of maximizing arithmetic intensity. TPUs (and the broader class of **dedicated inference/training chips** — AWS **Inferentia/Trainium**, and various startups' parts) trade GPU generality for higher efficiency on the narrow domain of dense neural-network math.

- **FPGAs (Field-Programmable Gate Arrays)** — reconfigurable logic you program into a custom circuit for your workload. Used for line-rate network processing, custom compression/crypto, and low-latency specialized pipelines (financial exchanges, Microsoft's Catapult/Bing and SmartNIC work). Extremely efficient for the fixed function they are configured to; costly to develop and slower-clocked than ASICs.
- **DPUs / SmartNICs (Data Processing Units)** — programmable NICs (NVIDIA **BlueField**, AWS **Nitro**, Intel IPUs) that offload networking, storage, encryption, and virtualization from the host CPUs, freeing those cores for tenant work and providing hardware isolation. In cloud infrastructure the DPU is where a growing share of the "infrastructure tax" now runs (Volume 12).

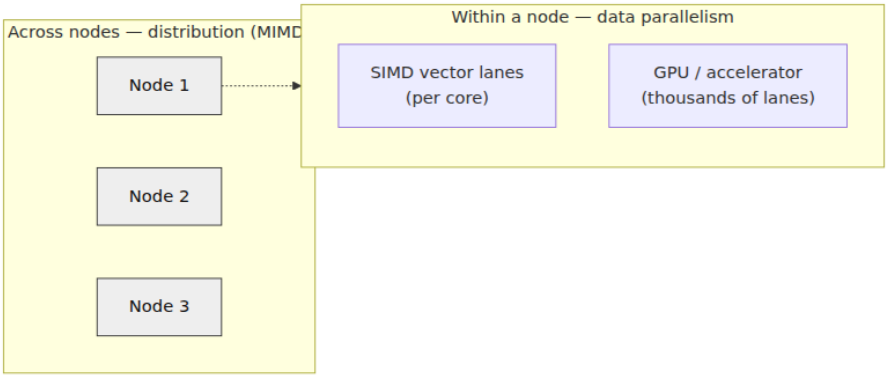
The unifying trend: as general-purpose scaling slows, performance increasingly comes from **matching the silicon to the workload**. For the backend engineer this means the compute fabric under a large service is no longer "a pile of x86 cores" but a *portfolio* — CPUs with SIMD, GPUs, and domain-specific accelerators — and choosing among them, and moving data efficiently between them, becomes a first-class design problem.

The distributed-systems lens: two axes, one set of lessons

Step back and the whole chapter is about a *second axis of scaling*, orthogonal to the one the rest of your career has been about.

Distribution scales *across* nodes; data parallelism scales *within* one. Sharding, replication, and service decomposition add machines to handle more load — the horizontal axis. SIMD and GPUs make each machine do more per unit time — the vertical, per-node throughput axis.

They are independent and multiplicative, and the systems that define the state of the art in bulk data processing **combine both**.



Distributed columnar OLAP is the canonical union. A ClickHouse or distributed-DuckDB or Spark deployment **shards** data across nodes (the distributed axis — partition, scatter the scan, gather-merge the aggregates) *and* runs **vectorized SIMD execution** on each node's local columns (the data-parallel axis). Query speed is the product: `nodes × per-node-vector-throughput`. Neglect either axis and you leave most of the machine on the floor. And the two axes echo each other — columnar layout gives SIMD its contiguous input *and* gives the distributed layer clean partition boundaries; the same "store like with like, minimize movement" instinct serves both.

ML training is the sharpest example of the same fractal. Training a large model is **data-parallel within each GPU** (SIMT over a batch), **data-parallel across GPUs in a node** (split the batch or the model across 8 accelerators over NVLink), *and* **distributed across nodes** (data-parallel and model/tensor/ pipeline-parallel training over the datacenter network, synchronizing gradients with all-reduce). At every level it is the *same* distributed-systems problem: **partition the work, minimize the coordination and the data you move, hide the communication behind computation.** All-reduce across nodes is the network-scale version of warp coalescing; the gradient-synchronization step is a barrier with the same coordination cost as any distributed consensus round, which is why interconnect bandwidth (NVLink, InfiniBand) is as load-bearing to training throughput as FLOPs. The partitioning and coordination trade-offs you know from Volume 6 apply *unchanged*; only the constants shift.

Data movement dominates — at every level. This chapter's most portable lesson is one you already hold from Chapter 1: the bottleneck is usually *moving the data*, not *computing on it*. It recurs at every scale, and the mitigation is always **locality**:

Level	The "network"	The dominating cost	The lesson
Vector unit	Memory ↔ SIMD registers	Non-contiguous / unaligned loads	AoS → SoA; keep data packed
GPU	PCIe host ↔ device	Transfer, launch overhead	Keep data resident; batch; amortize
Multi-GPU / node	NVLink	Inter-GPU sync	Overlap comms with compute
Cluster	Datacenter network	All-reduce / shuffle	Locality-aware partitioning

It is the same picture Chapter 6 drew for NUMA — a distributed system inside the chassis — extended outward until it becomes the literal datacenter. PCIe is to the GPU what the inter-socket link is to NUMA and what the network is to a cluster: the scarce, saturable channel whose traffic you must minimize.

Accelerators are a tier in the fleet. Operationally, a GPU inference service is another service — but one whose economics (expensive, scarce, saturation-sensitive) push its design toward aggressive batching, utilization-driven autoscaling, and careful cost accounting (Volumes 7, 11, 12). The heterogeneous datacenter means capacity planning is no longer a single fungible pool of cores but a portfolio of CPU, GPU, and specialized silicon, each matched to the workloads whose shape it fits — and the engineer's job includes knowing *which shape a workload is*.

The engineer's mental model

Compress everything above into a decision procedure you can run in your head.

1. **Is the workload data-parallel?** Same operation, many homogeneous elements, contiguous or contiguous-izable, little data-dependent branching? If yes, SIMD/GPU is on the table. If it is

branchy, sequential, pointer-chasing, or a diverse mix of tasks, it is not — keep it scalar and parallelize with threads/services instead.

2. **How much data, and what is the arithmetic intensity?** Small data or low FLOPs-per-byte → stay on the CPU, and reach for **SIMD** (auto-vectorization verified, or intrinsics/portable-SIMD for the hot loops). Large data *and* high arithmetic intensity (data reused on-chip, compute-bound) → an **accelerator** can deliver order-of-magnitude wins.
3. **Does the data movement pay for itself?** For a GPU, the PCIe transfer and launch overhead must be amortized by enough compute or enough batch, with data kept resident where possible. If the workload is transfer-bound, the accelerator will disappoint no matter its peak FLOPs. **Roofline first, FLOPs second.**
4. **Batch for throughput.** Both SIMD (fill the lanes) and GPUs (fill the cores, hide latency) reward processing many elements per invocation. Right-size the batch to trade latency for throughput deliberately.
5. **Measure — do not assume.** The compiler silently un-vectorizes; AVX-512 can downclock on old parts; a GPU offload can be net-negative when transfer-bound. Read the assembly, check vectorization remarks, profile the kernel, watch utilization. Every number in this chapter is a *hypothesis about your hardware* until you have measured it on your hardware.

Hold onto the framing: this is the *other axis*. Everything you know about scaling out — partition, localize, batch, minimize movement, coordinate as little as possible — applies just as forcefully to scaling *up* the throughput of a single node with vectors and accelerators. The lanes are just narrower and the network is just shorter.

Key takeaways

- **Two axes of parallelism.** *Task parallelism* (ILP, multicore, distribution) runs *different* work concurrently and pays a coordination cost; *data parallelism* (SIMD/SIMT) runs the *same* operation over *many elements* and, when the workload fits, is the most silicon- and energy-efficient parallelism there is. Flynn: SISD/SIMD/MISD/MIMD; the live contrast is SIMD vs MIMD.

- **CPU SIMD is a per-core throughput multiplier.** Vector registers hold N lanes; one instruction processes all N. x86: **SSE 128-bit (4×int32) → AVX/AVX2 256-bit (8×, +FMA/gather) → AVX-512 512-bit (16×, +32 regs, +k-mask predication)**. ARM: **NEON 128-bit fixed**; **SVE/SVE2 length-agnostic** (one binary, 128–2048-bit implementations), like RISC-V RVV.
- **SIMD is fast because it is rigid.** It demands **contiguous, aligned** data, a **uniform operation**, and **no divergent control flow**. **AoS → SoA** and columnar layout are the highest-leverage transformations; masks/predication are the escape hatch for conditionals.
- **Getting SIMD is on you. Auto-vectorization** is a fragile bonus that fails *silently* on aliasing, unknown alignment/trip-count, loop-carried dependencies, calls, and FP reassociation — *verify it*. **Intrinsics** and **portable libraries** (Highway, `std::simd`, ISPC) are the reliable routes; hand-vectorization survives because the best kernels need algorithmic rethinking no compiler will do.
- **AVX-512 caveat:** older Intel server parts (Skylake/Cascade Lake) could **downclock** under heavy AVX-512, sometimes making it a net loss in mixed workloads; newer parts (Ice Lake+, AMD Zen 4) largely fixed this. Feature-detect and measure.
- **SIMD is load-bearing backend infrastructure: vectorized query execution** (ClickHouse/DuckDB/Arrow, from MonetDB/X100) is *why modern OLAP is fast*; **simdjson** parses JSON at GB/s; compression, hashing, search, and CPU ML inference all lean on it.
- **GPUs are throughput machines.** Thousands of simple cores; **SIMT** groups threads into **warps (32) / wavefronts (64)** executing in lockstep — SIMD with a scalar programming model, and the **same divergence penalty**. Latency is hidden by **massive oversubscription**, not caches. Memory: registers → software-managed **shared memory** → L1/L2 → high-bandwidth **HBM/GDDR global memory**; **coalesced** access is the warp-level contiguity rule.
- **The transfer wall dominates.** Host/device with separate memory; input and output cross **PCIe** (~32–64 GB/s), an order of magnitude below on-device HBM. Naive copy-in/compute/copy-out is often slower than the CPU. Keep data **resident**, overlap transfer with compute, **batch**.
- **Arithmetic intensity (roofline) decides.** High FLOPs/byte + compute-bound + massive uniform parallelism

- batchable → GPU wins (dense linear algebra, deep learning). Branchy, latency-sensitive, small, or transfer-bound → CPU wins. Roofline first, peak FLOPs second.
- **GPUs reach backends as a serving tier**, mostly **ML inference**. **Batching** is the core lever (throughput vs latency); the **economics** (expensive, scarce, saturation-sensitive) make *keeping the accelerator busy* the priority — hence quantization, KV-cache management, MIG multi-tenancy, and utilization-driven autoscaling.
- **Heterogeneous compute is the trend** (Hennessy-Patterson's "new golden age"): **TPUs** (systolic arrays), **FPGAs**, **DPUs/SmartNICs**, and dedicated inference chips trade generality for domain efficiency as general-purpose scaling slows.
- **The lens:** data parallelism scales *within* a node; distribution scales *across* nodes — orthogonal and multiplicative. Real bulk systems combine both (**sharded + vectorized OLAP**; **SIMT-within-GPU + distributed-across-GPUs training**), and **data movement dominates at every level** — SIMD registers, PCIe, NVLink, the network — with **locality** the universal mitigation.

Further reading

- Michael J. Flynn, "Some Computer Organizations and Their Effectiveness," *IEEE Transactions on Computers*, 1972 (and the 1966 proceedings) — the original taxonomy (SISD/SIMD/MISD/MIMD).
- Samuel Williams, Andrew Waterman, David Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, 2009 — the arithmetic-intensity/roofline model underpinning every accelerator decision here (see also Volume 1, Chapter 1).
- Peter Boncz, Marcin Zukowski, Niels Nes, "MonetDB/X100: Hyper-Pipelining Query Execution," CIDR 2005 — https://www.cidrdb.org/cidr2005/papers/P19.pdf — the seminal vectorized-execution paper behind modern OLAP (see also Volume 5, on data systems).
- Geoff Langdale and Daniel Lemire, "Parsing Gigabytes of JSON per Second," *The VLDB Journal*, 2019 — https://arxiv.org/abs/1902.08318 — simdjson and the branch-free SIMD parsing technique; see also https://github.com/simdjson/simdjson.

- Intel 64 and IA-32 Architectures Software Developer's and Optimization Reference Manuals — https://www.intel.com/sdm — the authoritative reference for SSE/AVX/AVX2/AVX-512 semantics, encodings, and optimization guidance (including AVX frequency behavior).
- Arm, "Arm Architecture Reference Manual" and the SVE/SVE2 programming guides — https://developer.arm.com/documentation — for NEON, and the vector-length-agnostic SVE/SVE2 model.
- Google Highway (portable SIMD) — https://github.com/google/highway — and Rust `std::simd` (https://doc.rust-lang.org/std/simd/) — practical portable-SIMD libraries with runtime dispatch.
- NVIDIA, "CUDA C++ Programming Guide" — https://docs.nvidia.com/cuda/cuda-c-programming-guide/ — the definitive description of the SIMT model, warps, the memory hierarchy, coalescing, and host/device transfer; AMD ROCm/HIP docs (https://rocm.docs.amd.com) for the cross-vendor analog.
- John L. Hennessy and David A. Patterson, "A New Golden Age for Computer Architecture," *Communications of the ACM*, 2019 (2017 Turing Award lecture) — the case for domain-specific/heterogeneous accelerators.
- Norman P. Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit," ISCA 2017 — https://arxiv.org/abs/1704.04760 — the TPU systolic-array design and its efficiency argument.
- Woosuk Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP 2023 — https://arxiv.org/abs/2309.06180 — the vLLM continuous-batching/KV-cache work that exemplifies GPU-inference serving (see also Volumes 7 and 12).
- Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. (Morgan Kaufmann, 2017), Chapters 4 (data-level

parallelism: vector, SIMD, GPU) and 7 (domain-specific architectures) — the rigorous textbook treatment of everything in this chapter.

Chapter 8 — Performance Anti-Patterns: False Sharing, Branch Misprediction, Cache Thrashing

---

What this chapter covers. Chapters 2 through 4 built the machine: a deeply pipelined, speculating, out-of-order core (Chapter 2); a multi-level cache hierarchy that decides whether that core computes or stalls (Chapter 3); and the coherence protocol that keeps many private caches consistent, at a cost (Chapter 4). Those chapters were theory. This one is the payoff: the concrete, recurring, hardware-level performance anti-patterns that bite production backend code, how to *recognize* each one from its symptom, how to *measure* it with CPU performance counters instead of guessing, and how to *fix* it. The throughline is the observation from Chapter 1 that most performance problems surviving an algorithmic pass are mechanical-sympathy problems — two implementations with identical big-O complexity routinely differ by 10x, and the difference is almost always cache misses, branch mispredictions, false sharing, memory-bandwidth saturation, atomic contention, or allocator pressure. Each of these has a signature you can see in `perf`, and a matching fix. The chapter ends where every real optimization must: a measurement workflow, because you cannot fix what you have not measured, and modern hardware is far too counterintuitive to optimize by inspection.

Learning goals — after this chapter you should be able to:

- Explain why "same big-O, 10x different performance" is the normal case, and why the cause is mechanical rather than algorithmic — a data-movement or control-flow problem, not an operations-count problem.
- Recognize the six anti-patterns — **pointer chasing / poor locality**, **false sharing**, **branch misprediction**, **memory-bandwidth saturation**, **atomic/lock contention**, and **allocation/GC pressure** — from code shape *and* from their `perf`-counter fingerprint.

- Apply the matching fix for each: contiguous layout and struct-of-arrays and blocking; cache-line padding; branchless code and predictable branches; reducing data movement; sharding shared writes into per-core state; and arenas, pools, and value types.
- Use `perf stat`, `perf record`, `perf top`, `perf c2c`, cachegrind, and flame graphs to identify the bottleneck *class*, then re-measure to confirm the fix.
- Reason about why every one of these effects *multiplies across a fleet*: a hot-loop cache-miss pattern on 10,000 cores is 10,000 cores of waste, false sharing and atomic contention are the single-node face of distributed coordination overhead, and p99 tail latency is frequently one of these effects firing on one unlucky node.

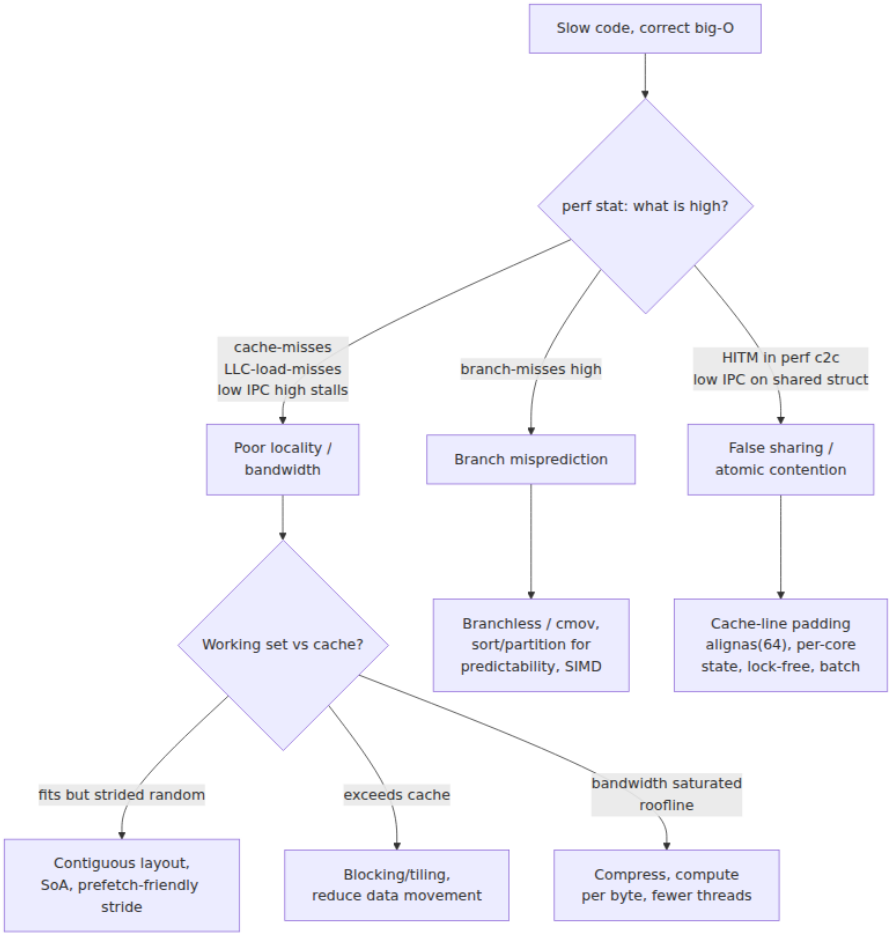
The shape of the problem: same big-O, 10x apart

A senior engineer's instinct, faced with a slow endpoint, is to reach for complexity: find the accidental $O(n^2)$, the missing index, the N+1 query. That instinct is correct and you should exhaust it first. But there is a large, important class of performance problems that *survives* the algorithmic pass — the profile is flat, the complexity is right, the query plan is clean, and the code is still three, five, ten times slower than it should be. Those are hardware-effect problems. The operation count is fine; the *cost per operation* is not, because the operations are stalling the pipeline, missing cache, bouncing cache lines between cores, or waiting on memory bandwidth.

The reason this is the normal case and not an edge case is the latency hierarchy of Chapter 1. An L1 hit is a few cycles; a last-level-cache miss to DRAM is on the order of a couple hundred cycles — two orders of magnitude. A correctly predicted branch is nearly free; a mispredicted one costs the full pipeline depth, hedged at roughly 15–20 cycles on a modern core. A cache line that ping-pongs between two cores' L1 caches costs a coherence round trip — tens to low hundreds of cycles — *every time*. None of these costs appears in your source code, in your big-O analysis, or in a code review. They are properties of the *data layout* and *access pattern*, which the source deliberately hides. Two loops that do the same arithmetic can differ by 10x purely in how they touch memory.

This is why the discipline of this chapter is inseparable from measurement. You cannot look at a loop and know its IPC (instructions

per cycle), its cache-miss rate, or its branch-misprediction rate. Modern cores are too speculative, too out-of-order, too prefetch-driven for the human eye to model. The professional stance is: form a hypothesis about the bottleneck *class*, confirm it with a performance counter that is diagnostic of that class, apply the matching fix, and re-measure. The rest of this chapter is organized as a catalog of classes — each with its code shape, its mechanism, its counter, and its fix — followed by the workflow that ties them together.



Cache-unfriendly access patterns

Chapter 3 established the cache's operating assumptions: it moves memory in 64-byte lines, it rewards **temporal** locality (reuse the same data soon) and **spatial** locality (use nearby data soon), and its hardware prefetchers stream ahead of *sequential* access. Every cache anti-pattern is a violation of one of those assumptions.

Pointer chasing: the linked structure tax

Consider summing a value across a linked list versus an array. Same O(n), wildly different hardware behavior.

```
// Linked list: each node is a separate allocation, scattered across
the heap.
struct Node { int value; struct Node *next; };
long sum_list(struct Node *head) {
    long s = 0;
    for (struct Node *n = head; n; n = n->next)   // n->next is a data-
dependent load
        s += n->value;
    return s;
}

// Array: contiguous, prefetch-friendly.
long sum_array(const int *a, size_t n) {
    long s = 0;
    for (size_t i = 0; i < n; i++)                // a[i], a[i+1] on
the same/next line
        s += a[i];
    return s;
}
```

`sum_array` touches memory in strict address order. The prefetcher recognizes the stream and pulls the next lines into L1 *before* the loop asks for them; sixteen `int`s share one 64-byte line, so fifteen of every sixteen accesses are guaranteed L1 hits. The loop is bound by arithmetic, not memory, and runs near the core's peak IPC.

`sum_list` is the opposite. Each node was allocated independently and lives at an unpredictable address, so `n->next` is a **data-dependent load**: the CPU cannot compute the next address until the current node arrives from memory. The prefetcher has no stride to lock onto. Worst case — nodes in random heap order, working set larger than the last-level cache — *every* `n = n->next` is a cache miss to DRAM. The core issues one load,

stalls a couple hundred cycles waiting for it, follows the pointer, stalls again. This is **pointer chasing**, and it defeats every latency-hiding trick the core has: out-of-order execution cannot proceed past a load whose *address* isn't known, so the deep reorder window sits idle. The same logic indicts naive binary trees, hash tables with per-node chaining, and any "graph of small heap objects" layout. The fingerprint is low IPC (often well under 1.0), a high `cache-misses` count, and a high fraction of `stalled-cycles-backend` — the core is waiting on memory, not computing.

The fix is layout, not algorithm. Store the nodes in an array and use indices instead of pointers; allocate from an arena so logically adjacent nodes are physically adjacent; or, where the structure permits, abandon the linked form for a flat array. A B-tree beats a binary search tree in practice not because of asymptotics but because packing many keys per cache-line-sized node turns $\log_2(n)$ scattered misses into far fewer wide, cache-friendly reads.

Array-of-structs vs struct-of-arrays

The most common locality mistake in backend code is loading whole objects to read one field. Take a million entities and sum one field.

```
// Array of Structs (AoS): the intuitive layout.
struct Order {                  // sizeof ~= 64 bytes: one full cache
line
    uint64_t id;
    double    price;            // the only field we need
    uint32_t  qty, flags;
    char      customer[32];
};
struct Order orders[1'000'000];

double total_aos(void) {
    double t = 0;
    for (size_t i = 0; i < 1'000'000; i++)
        t +=
orders[i].price;  // touches price, drags in the whole 64B line
}
```

Every iteration needs 8 bytes (`price`) but pulls a full 64-byte line into cache — 56 bytes of `id`, `qty`, `flags`, and `customer` that the loop never reads. You are running the memory system at one-eighth of its useful bandwidth, and if `Order` were larger than a line, each element would cost

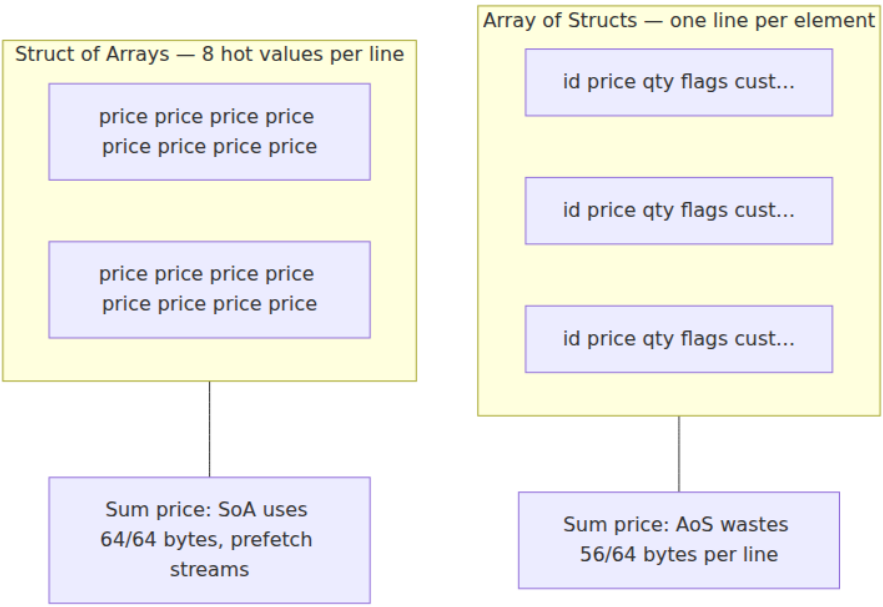
multiple misses. The **struct-of-arrays** (SoA) layout stores each field in its own contiguous array:

```
// Struct of Arrays (SoA): one array per field.
struct Orders {
    uint64_t id[1'000'000];
    double    price[1'000'000];   // the hot field, densely packed
    uint32_t  qty[1'000'000];
    uint32_t  flags[1'000'000];
    char      customer[1'000'000][32];
};

double total_soa(const struct Orders *o) {
    double t = 0;
    for (size_t i = 0; i < 1'000'000; i++)
        t += o->price[i];         // eight useful doubles per line,
prefetch streams
}
```

Now the `price` array is dense: eight `double`s per line, no wasted bytes, a perfect prefetch stream, and — as a bonus — a layout the compiler can auto-vectorize into SIMD (Chapter 7). The speedup on a field-sum over cold data is often ~4–8x, tracking the ratio of struct size to field size. This is the core idea of **data-oriented design**: organize memory around the *access pattern* of the hot loop, not around the conceptual "object."

You rarely need to flip the *entire* schema. **Hot/cold splitting** captures most of the win: keep the few frequently-accessed fields together and dense, and push the rarely-touched fields (the `customer` blob, audit metadata) into a separate structure reached by index. The hot loop then streams the hot fields with near-perfect line utilization.



Large strides and working sets that exceed cache

Two more locality traps. First, **strided access** that steps across lines defeats the prefetcher and wastes each line. The textbook case is iterating a 2D array in the wrong order:

```
// Column-major traversal of a row-major array: stride = N *
sizeof(elem).
for (size_t j = 0; j < N; j++)
    for (size_t i = 0; i < N; i++)
        s += a[i][j];   // consecutive iterations jump N elements — a
new line each time
```

Each inner-loop access lands on a different cache line (and, for large `N`, a different page, stressing the TLB too). Swapping the loop order to `a[i][j]` with `j` innermost restores sequential access and can be several times faster with no change to the arithmetic.

Second, and more insidious, is the **working set exceeding cache** — **cache thrashing** in the capacity sense. As long as the data a loop repeatedly touches fits in a cache level, reuse is free. The moment the working set grows past that level's capacity, every reuse becomes a capacity miss to the next level down, and throughput falls off a cliff. This

is why a matrix multiply written as three naive nested loops collapses once the matrices no longer fit in cache: each element of one operand is re-read n times, but by the time it is needed again it has been evicted. The fix is **blocking** (**tiling**): restructure the loops to operate on sub-blocks small enough that the block's working set fits in cache, so each loaded element is fully reused before eviction.

```
// Tiled matrix multiply: operate on BxB blocks that fit in L1/L2.
for (size_t ii = 0; ii < N; ii += B)
  for (size_t jj = 0; jj < N; jj += B)
    for (size_t kk = 0; kk < N; kk += B)
      for (size_t i = ii; i < ii+B; i++)
        for (size_t j = jj; j < jj+B; j++) {
          double c = C[i][j];
          for (size_t k = kk; k < kk+B; k++)   // A[i][k], B[k][j]
blocks stay resident
            c += A[i][k] * B[k][j];
          C[i][j] = c;
        }
```

Blocking is the single most important loop transformation for bandwidth-bound numerical code, and the same principle — "size the unit of work to the cache" — reappears in database join algorithms (cache-conscious hash joins) and in log-structured merge tuning. The fingerprint of a capacity-miss problem is that performance is fine for small inputs and degrades sharply at the input size where the working set crosses a cache boundary; `perf stat` shows `LLC-load-misses` climbing and IPC falling as size grows.

False sharing: the canonical multicore bug

False sharing is the most subtle anti-pattern in this chapter because it is *invisible in the source*. The code has no shared variable — two threads touch two *different* variables — and yet they contend, because coherence operates on 64-byte lines, not on variables (Chapter 4). If two independent variables happen to land on the same cache line, and two cores write them, the line ping-pongs between the cores' caches under the MESI protocol as if the writes genuinely conflicted. Each write must first invalidate the other core's copy and pull the line into Modified state; the other core's next write does the same in reverse. The line bounces, one coherence round trip per write, and a workload that should scale linearly with cores instead gets *slower* as you add them.

The canonical example is per-thread counters packed into an array:

```
// BAD: NTHREADS counters in a contiguous array — several per cache
line.
long
counters[NTHREADS];              // 8 bytes each: 8 counters share one
64B line

void *worker(void *arg) {
    int id = *(int*)arg;
    for (long i = 0; i < 1'000'000'000; i++)
        counters[id]++;              // thread id's "own" counter —
but shares a line
    return NULL;
}
```

Every thread updates a counter no other thread reads. There is no logical sharing, no lock, no data race — the program is correct. But `counters[0]` through `counters[7]` occupy one line, so eight threads hammering "their own" counters are all writing the *same line*. The line ping-pongs eight ways, and the loop can run an order of magnitude slower than the single-threaded version — adding cores makes it worse. This exact pattern hides in per-thread statistics arrays, sharded rate-limiter buckets, worker-pool state structs, and any `struct` whose fields are written by different threads.

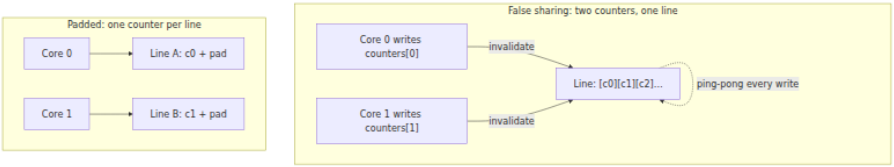
The fix is to force each thread's hot data onto its own cache line by **padding** or **alignment**:

```
// GOOD: each counter alone on its own 64-byte cache line.
struct alignas(64) PaddedCounter {    // C++; C11: _Alignas(64)
    long value;
    char pad[64 - sizeof(long)];      // fill the rest of the line
};
struct PaddedCounter counters[NTHREADS];

void *worker(void *arg) {
    int id = *(int*)arg;
    for (long i = 0; i < 1'000'000'000; i++)
        counters[id].value+
+;         // now the only live datum on its line
    return NULL;
}
```

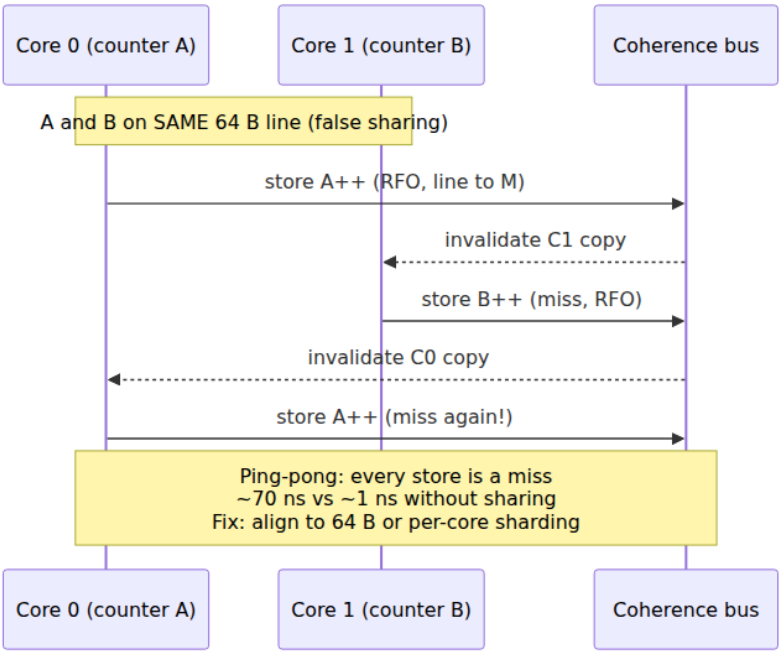
With each counter alone on its line, no two threads ever share a line, the coherence traffic vanishes, and the workload scales with cores as

intended. The recovery is frequently an order of magnitude on the contended microbenchmark. The cost is 64 bytes per counter of otherwise-wasted memory — a trade almost always worth making for genuinely hot, thread-private data.



Languages give you first-class mechanisms for this. C++17 exposes `std::hardware_destructive_interference_size` (the padding size to *avoid* false sharing) and `std::hardware_constructive_interference_size` (to deliberately co-locate). Java has `@Contended` (JDK 8+, gated behind `-XX:-RestrictContended`), which the JVM honors by padding the annotated field. Rust programmers reach for `crossbeam`'s `CachePadded<T>`. Go has no annotation, so you pad structurally with an unused `[64]byte` or `_ [8]uint64` field, or by giving each goroutine a fully separate allocation. The Linux kernel uses `____cacheline_aligned` throughout its per-CPU data for exactly this reason.

Two caveats. First, some hardware prefetches lines in *pairs* (an "adjacent-line" prefetcher, 128-byte effective granularity), so on those parts padding to 128 bytes can be safer for the hottest data — measure. Second, false sharing is a *write* problem: read-only shared data on one line is fine, since multiple caches can hold a line in Shared state simultaneously. It is concurrent *writes* to distinct variables on one line that trigger the ping-pong.



Branch misprediction: when the pipeline guesses wrong

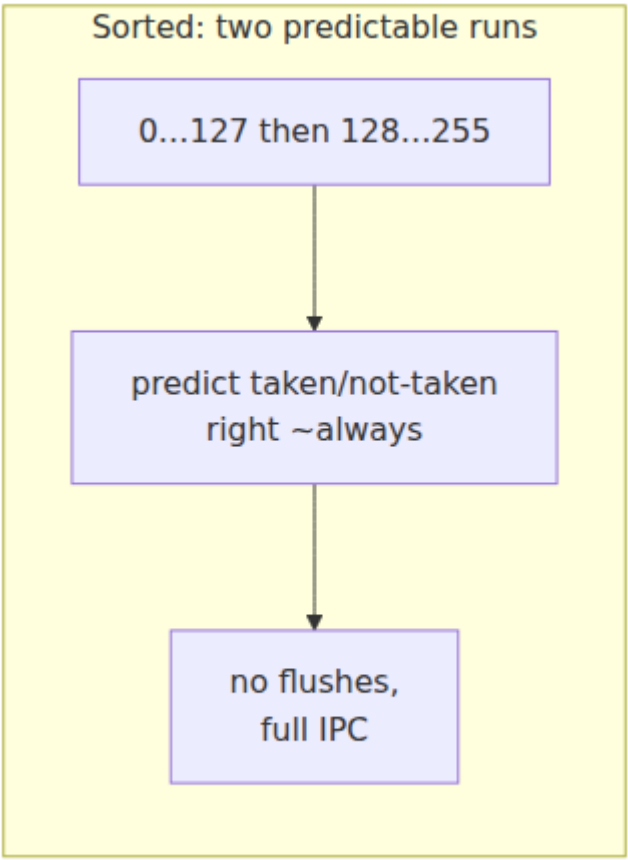
Chapter 2's core is deeply pipelined and speculative: to keep the pipeline full it *predicts* the direction of each conditional branch and executes speculatively down the predicted path. Modern predictors are astonishingly good — well over 95% accurate on typical code — because most branches are highly biased (loop back-edges taken n−1 times, error checks almost never taken). But a branch whose direction is *data-dependent and effectively random* is unpredictable in principle: the predictor is right about half the time, and each miss flushes the speculative work and refills the pipeline, costing roughly the pipeline depth — hedged at ~15–20 cycles on current cores.

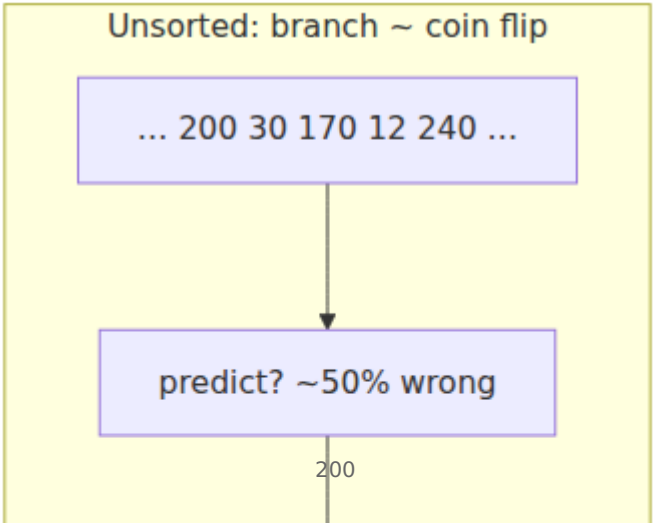
The famous illustration — the most-viewed performance question on Stack Overflow — is summing the elements of an array that exceed a threshold, and discovering that *sorting the array first* makes the loop several times faster:

```
int data[N];                // values in [0, 255]
// ... fill with random values ...
// std::sort(data, data + N); // <-- with this line, the loop below is
often ~3-6x faster

long sum = 0;
for (int reps = 0; reps < 100000; reps++)
    for (int i = 0; i < N; i++)
        if (data[i] >= 128)   // the branch whose predictability
changes everything
            sum += data[i];
```

The arithmetic is identical whether or not the array is sorted — same number of comparisons, same number of additions, same O(N) per pass. What changes is the *predictability of the branch*. On **unsorted** random data, `data[i] >= 128` is true about half the time in no pattern; the predictor cannot do better than a coin flip, mispredicts roughly half the branches, and eats a pipeline flush each time. On **sorted** data, the branch is false for the entire first run of small values and then true for the entire tail — two long, perfectly predictable runs with a single transition. The predictor is right essentially always, the pipeline never flushes, and the loop runs at full speed. The visible several-fold speedup is *entirely* the elimination of branch mispredictions; `perf stat` confirms it as a collapse in `branch-misses` between the two runs.


Sorted: two predictable runs
0...127 then 128...255
predict taken/not-taken
right ~always
no flushes,
full IPC

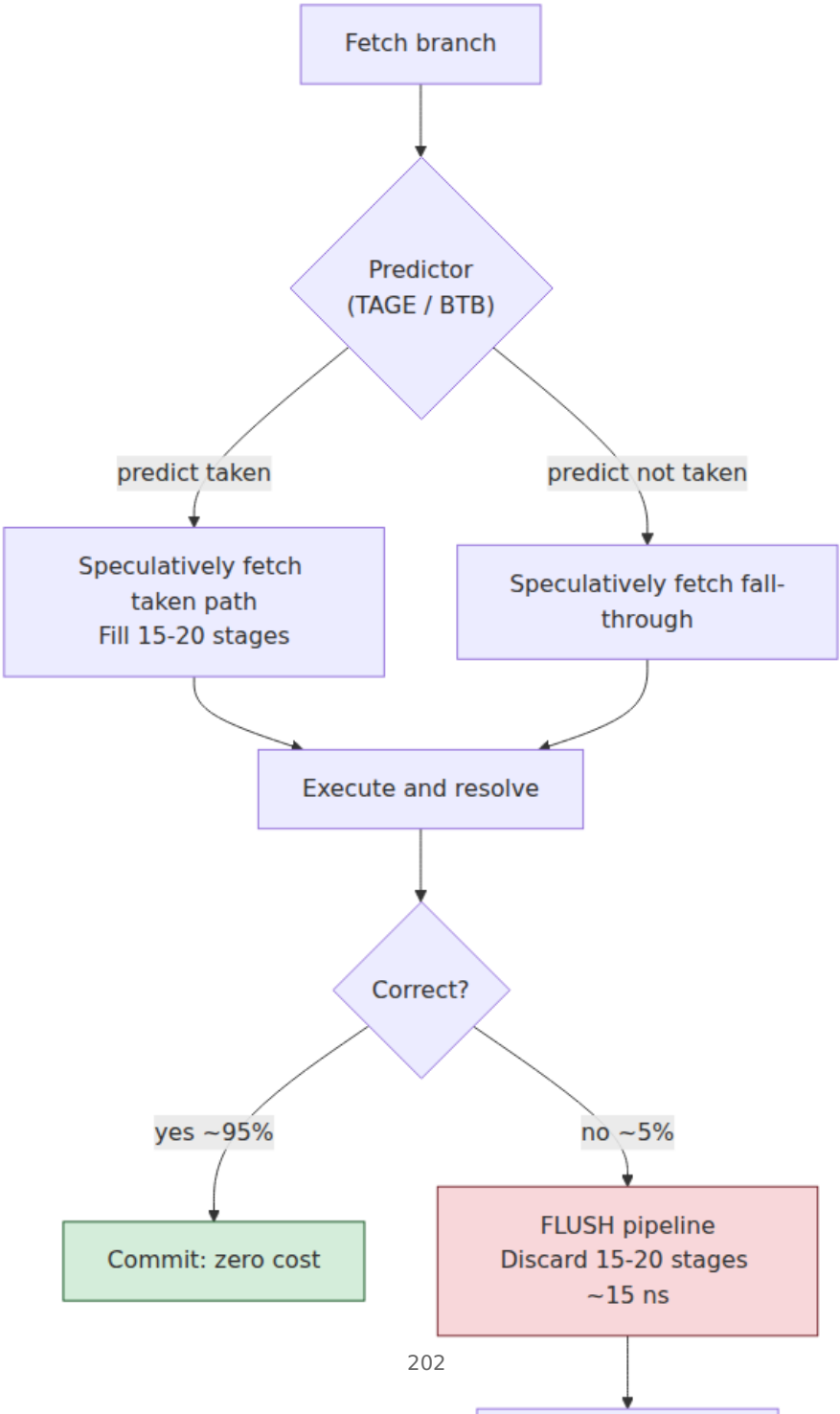

Unsorted: branch ~ coin flip
... 200 30 170 12 240 ...
predict? ~50% wrong

There are three fixes, and *sorting the data is rarely the real one* — you sorted here only to make the branch predictable, which is a strange thing to pay O(n log n) for. The direct fix is to remove the unpredictable branch from the hot loop entirely:

```
// Branchless: compute the mask, multiply. No conditional to
mispredict.
for (int i = 0; i < N; i++)
    sum += data[i] * (data[i] >= 128);   // (cond) is 0 or 1; no branch
// or, encouraging a conditional move:
    sum += (data[i] >= 128) ? data[i] : 0;
```

The comparison `data[i] >= 128` produces a 0/1 value with no control-flow branch; the CPU executes a straight-line sequence, often lowering the ternary to a **conditional move** (`cmov`) that selects a result without a jump. There is nothing to mispredict, so the branchless version runs at the *sorted* speed on *unsorted* data. The third fix is **SIMD** (Chapter 7): vector compare-and-blend instructions evaluate the predicate across a whole vector lane-wise with no branches at all, and the compiler will often auto-vectorize the branchless form.

A critical caveat, because branchless code is frequently *misapplied*: converting a branch to a `cmov` removes the misprediction penalty but also removes the predictor's ability to *skip work* on the common path. A well-predicted branch is nearly free and lets the core avoid the untaken side entirely; forcing both sides to execute (as a `cmov` does) can be *slower* when the branch was predictable. So branchless transformations pay off only for genuinely unpredictable branches in genuinely hot loops. Everywhere else, leave the branch — it is cheaper, and it is more readable. As always: measure `branch-misses`, confirm the branch is both hot and unpredictable, and only then reach for branchless or SIMD.


Fetch branch
Predictor
(TAGE / BTB)
predict taken
predict not taken
Speculatively fetch
taken path
Fill 15-20 stages
Speculatively fetch fall-
through
Execute and resolve
Correct?
yes ~95%
no ~5%
Commit: zero cost
FLUSH pipeline
Discard 15-20 stages
~15 ns

Memory-bandwidth saturation

The locality section optimized for *latency* — avoiding misses. But a class of workloads is not latency-bound at all; it is **bandwidth-bound**. It streams so much data through the cores that the limiting resource is the sustained bytes-per-second the memory system can deliver, not the miss latency of any single access. Once a workload is bandwidth-bound, the counterintuitive consequence is that *adding threads does not help* — every core is already waiting on a shared, saturated memory channel, so more cores just contend for the same fixed bandwidth. Throughput plateaus (or dips, once coherence and contention overhead grow) no matter how many cores you throw at it.

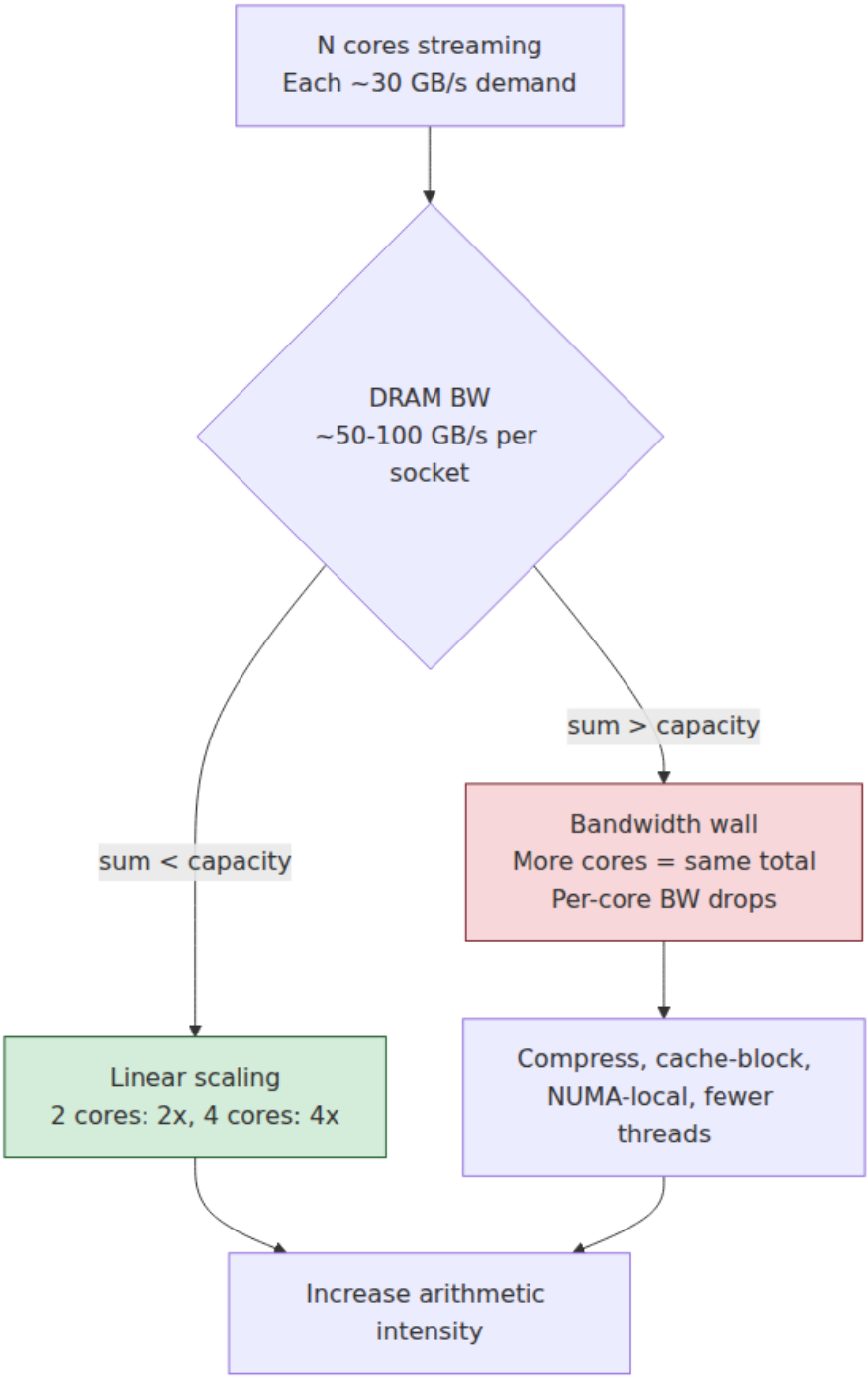
The tool for reasoning about this is the **roofline model** (Chapter 1). Plot achievable performance against **arithmetic intensity** — FLOPs (or useful work) per byte moved from memory. Low-intensity kernels (a vector add: two reads, one write, one add — a fraction of an op per byte) sit under the sloped "bandwidth roof" and are limited by memory bandwidth; high-intensity kernels (a well-blocked matrix multiply that reuses each loaded byte many times) reach the flat "compute roof." The model tells you, before you write a line of optimization, whether your kernel *can* be sped up by faster compute (it is under the compute roof) or only by *moving less data* (it is under the bandwidth roof).

For a bandwidth-bound kernel the fixes all reduce data movement per unit of work — i.e., raise arithmetic intensity:

- **Better locality / blocking** so loaded bytes are reused before eviction (the SoA and tiling fixes above) — this converts a bandwidth problem into a compute problem.
- **Compression** — smaller types (`float32` over `float64`, packed integers, dictionary encoding) move fewer bytes per element, so more elements fit per line and per second of bandwidth. Trading a little decode compute for a lot less memory traffic is almost always a win when bandwidth-bound.
- **Compute more per byte** — fuse passes so data loaded once is used for several operations instead of being re-streamed for each (loop fusion, operator fusion in query and ML engines).
- **Fewer threads, right placement** — because more threads cannot beat the bandwidth wall, and on a NUMA machine (Chapter 6) a thread pulling from a *remote* socket's memory pays extra

interconnect latency and consumes shared interconnect bandwidth. Pin threads to the node that owns their data.

The fleet consequence is direct: a bandwidth-bound service is one where the standard scaling reflex — add vCPUs — buys nothing, because the bottleneck is the memory subsystem the vCPUs share. Recognizing bandwidth saturation (flat throughput vs. thread count, high `LLC-load-misses` streaming to DRAM, and — where available — memory-controller bandwidth counters near their ceiling) prevents the expensive mistake of scaling a machine that has no headroom left to give.


N cores streaming
Each ~30 GB/s demand
DRAM BW
~50-100 GB/s per socket
sum < capacity
sum > capacity
Linear scaling
2 cores: 2x, 4 cores: 4x
Bandwidth wall
More cores = same total
Per-core BW drops
Compress, cache-block, NUMA-local, fewer threads
Increase arithmetic intensity

Atomic and lock contention

Chapter 4 showed that an atomic read-modify-write (a CAS or `fetch_add`) must acquire the cache line in exclusive (Modified) state and enforce ordering. When many cores hammer the *same* atomic — a global counter, a shared queue head, a reference count on a hot object — that line becomes a single-line bottleneck: it can be Modified in exactly one cache at a time, so the cores serialize, handing the line back and forth with a coherence round trip per operation. This is true sharing (the data really is shared) rather than false sharing, but the mechanism and the symptom are the same cache-line **bouncing**, and it is a scalability killer. The same is true of a contended lock: the lock word itself is a cache line that ping-pongs among the contenders' caches, and the critical section serializes them on top of that.

The unifying theme, and the most important sentence in this chapter for concurrent code: **shared mutable state is the enemy of scaling.** Every write to a shared line is a coherence event, and coherence events serialize. The fixes are all variations on *stop sharing the written data*:

- **Per-core / sharded state.** Give each core (or thread, or shard) its own counter/accumulator on its own cache line, and combine them only when a total is actually read. A billion-increment global counter that serializes all cores becomes N independent per-core counters that never contend; you pay one cheap sum at read time. This is the Linux kernel's per-CPU-variable pattern and the design behind Java's `LongAdder` (which is exactly a striped, padded set of cells) versus a contended `AtomicLong`.
- **Reduce the frequency of shared writes. Batch** updates locally and flush to the shared structure occasionally — accumulate 1,000 events in a thread-local buffer, then do one atomic add of the batch. This turns N coherence events into N/1000.
- **Lock-free / wait-free structures where they genuinely fit** (Volume 4). These reduce the *serialization* of a lock but do *not* eliminate coherence traffic — a lock-free stack whose single head pointer is CAS'd by every thread still bounces that one line. Lock-free is not a magic scaling bullet; the scaling comes from *not concentrating writes on one line*, which is a data-layout decision, not a synchronization-primitive decision.

- **Read-mostly optimizations.** For data read far more than written, structures like RCU or seqlocks (Volume 4) let readers proceed with no writes to shared lines at all, confining coherence cost to the rare writer.

The `perf` fingerprint of atomic/lock contention is the same as false sharing — high cross-core snoop hits on modified lines (HITM), visible in `perf c2c` — plus, for locks, time spent in the futex/park paths visible in a flame graph. The distinction between *true* sharing (fix by sharding the data or reducing writes) and *false* sharing (fix by padding) is exactly what `perf c2c` is built to tell you, because it reports the offsets within the contended line that different cores touched.

Allocation and garbage-collection effects

Memory allocation is a performance anti-pattern hiding in plain sight, because `new` / `malloc` looks like a single cheap call. In a hot path it is three separate problems. First, the allocator itself has cost and, under concurrency, **contention** — a shared global heap serializes threads on its metadata locks, which is why modern allocators (jemalloc, tcmalloc, mimalloc) use per-thread caches and per-core arenas, the same "stop sharing" fix as above. Second, allocation **pollutes cache**: touching fresh memory and the allocator's bookkeeping evicts the loop's hot data, adding misses elsewhere. Third, and most importantly, allocation produces **pointer-heavy, scattered layouts** — exactly the pointer-chasing anti-pattern from earlier. A graph of small heap objects, each a separate allocation, guarantees the cache-miss-per-node behavior; the allocation *is* the layout, and the layout is the problem.

The fixes are about controlling layout and lifetime:

- **Allocate less.** Hoist allocations out of hot loops; reuse buffers; return by value into caller-provided storage. The cheapest allocation is the one you do not do.
- **Arenas / region allocators.** Allocate many objects contiguously from a bump-pointer arena and free them all at once by resetting the arena. This makes logically related objects physically adjacent (restoring locality) *and* makes allocation and deallocation nearly free (a pointer bump and a reset). Ideal for request-scoped data: arena per request, reset at the end.

- **Object pools / free lists** for expensive, frequently recycled objects, so steady-state operation does no allocation at all — the LMAX Disruptor's pre-allocated ring buffer (Chapter 1) is the archetype.
- **Value types and flat layouts.** Prefer contiguous arrays of values to arrays of pointers to objects. In managed runtimes this is the difference between a `long[]` (dense, cache-friendly) and a `Long[]` / `List<Long>` (an array of references to boxed heap objects, each a pointer chase). C# `struct`s, Java's Project Valhalla value classes, and Go's naturally value-typed structs and slices all exist to give you flat layouts on managed heaps.
- **Off-heap / manual memory for large data** (Volume 13) — large buffers, caches, and columnar stores kept outside the managed heap avoid both GC scanning cost and pointer-chasing layout.

In garbage-collected languages there is a second, latency-shaped effect: **GC pauses**. The more you allocate, the more frequently the collector runs, and even mostly-concurrent collectors have stop-the-world phases and consume CPU and memory bandwidth that steal from the application. A service that allocates aggressively in its request path pays for it not only in throughput but in **tail latency** — a request unlucky enough to be in flight during a pause absorbs the whole pause into its response time. Reducing allocation is therefore a *tail-latency* intervention as much as a throughput one: fewer, larger, longer-lived allocations mean less collector work, shorter and rarer pauses, and a tighter p99. This is a central theme of Volume 13 (managed-runtime performance) and reappears in Volume 11 (SRE, tail latency); the mechanical cause is here.

Measurement: finding these without guessing

Everything above is a hypothesis generator. The discipline that turns it into engineering is measurement, and on Linux the primary instrument is `perf`, which reads the CPU's **hardware performance counters** — dedicated registers that count microarchitectural events (cycles, instructions retired, cache misses, branch mispredictions, stalls) with negligible overhead.

Start with `perf stat`, which gives the high-level vitals for a whole program or command:

```
$ perf stat -d ./workload
        12,004.55 msec task-clock                #    1.00 CPUs
utilized
   38,204,551,010      cycles                    #    3.18 GHz
   11,204,882,140      instructions              #    0.29  insn per
cycle   <- low IPC: stalling
      612,003,884      branches
       88,441,220      branch-misses             #   14.4% of all
branches   <- bad prediction
    2,904,551,201      cache-references
    1,880,220,140      cache-misses              #   64.7% of cache
refs      <- poor locality
   26,110,884,201      stalled-cycles-backend    #   68.3% of
cycles          <- waiting on memory
```

Read it top-down. **IPC** (instructions per cycle) is the master health metric: a modern core can retire ~3–4 instructions per cycle, so an IPC of 0.3 means the core is stalled ~90% of the time and *something* is starving it. The rest of the counters tell you *what*: a high **branch-miss** rate points at the misprediction anti-pattern; a high **cache-miss** rate plus high **stalled-cycles-backend** points at poor locality or bandwidth; low IPC with neither of those, on multithreaded code, points at contention. Add `-d` (detailed) for L1/LLC breakdowns; the specific counters you will name most are `LLC-load-misses` (last-level misses that go all the way to DRAM — the expensive ones) and `cache-misses`.

Anti-pattern	Code shape	`perf` fingerprint	Fix
Pointer chasing / poor locality	Linked lists, trees, scattered heap objects	Low IPC, high `cache-misses`, high `stalled-cycles-backend`, high `LLC-load-misses`	Contiguous/array layout, arenas, indices not pointers
AoS wasting lines	Loading whole struct for one field	High `cache-misses`, low line utilization	Struct-of-arrays, hot/cold split
Cache thrashing (capacity)	Working set > cache; degrades at a size threshold	`LLC-load-misses` climb with input size, IPC falls	Blocking / tiling, shrink working set

Anti-pattern	Code shape	`perf` fingerprint	Fix
Strided access	Wrong loop order, big strides	High `cache-misses`, `dTLB-load-misses`	Sequential access, loop interchange
False sharing	Per-thread data on shared line; scales *negatively*	`perf c2c` HITM at distinct offsets in one line	`alignas(64)` padding, per-line state
Branch misprediction	Data-dependent branch in hot loop	High `branch-misses` (>~5–10%)	Branchless / `cmov`, predictable order, SIMD
Bandwidth saturation	Streaming, low arithmetic intensity	Flat throughput vs. threads, DRAM-bound `LLC-load-misses`	Reduce data movement, compress, block, compute per byte
Atomic / lock contention	Hot shared counter/lock/ queue head	`perf c2c` HITM at the *same* offset; futex in flame graph	Per-core sharding, batching, lock-free where it fits
Allocation / GC	`new` / `malloc` in hot path; GC pauses	Allocator/GC frames in flame graph; pause spikes in tails	Arenas, pools, value types, allocate less, off-heap

When `perf stat` tells you *what* class of problem you have, the next question is *where*. `perf record` samples the program and `perf report` (or `perf top` for a live view) attributes the events to functions and source lines — crucially, you can sample a *specific* event, e.g. `perf record -e cache-misses ./workload` to find exactly which lines take the misses, or `-e branch-misses` to find the mispredicting branch. Rendered as a **flame graph** (Brendan Gregg's visualization: stacks on the x-axis, sampled by width), this makes the expensive call paths obvious at a glance and is the fastest way to orient in unfamiliar code.

For the two contention anti-patterns there is a purpose-built tool: **`perf c2c`** (cache-to-cache). It records where cache lines are shared and modified across cores and reports the **HITM** events — loads that hit a line **M**odified in *another* core's cache, the exact signature of a bouncing

line. Its killer feature is that it shows the *offsets within the line* that different cores touched: if cores are hitting *different* offsets, you have **false** sharing (pad them apart); if they are hitting the *same* offset, you have **true** sharing (shard the data or reduce writes). This single distinction is otherwise very hard to make, which is why `perf c2c` is the tool for any "adding cores made it slower" mystery.

Complementary tools fill gaps. **Cachegrind** (Valgrind) *simulates* a cache and gives exact, deterministic miss counts per line — no hardware counters, no sampling noise, at the cost of a large slowdown; excellent for pinning down *which line* misses when hardware sampling is too coarse. **Intel VTune** and **AMD uProf** provide vendor-specific, deeply annotated microarchitectural analysis (top-down stall attribution, memory-access analysis, bandwidth against the roofline) that goes beyond what raw `perf` counters name. In production, **continuous profilers** (Volume 13) sample `perf`-style data fleet-wide at low overhead — the direct descendant of Google-Wide Profiling — so you find the hot loop that is wasting cycles across thousands of nodes without attaching to any single one.

The workflow is a loop, and its order matters:

1. **Measure first.** `perf stat` for the vitals; never optimize on a hunch.
2. **Identify the bottleneck class** from the counters: prediction, locality, bandwidth, or contention.
3. **Localize** with `perf record` / `report` / `top` (or `perf c2c` for contention) to the exact function and line.
4. **Apply the matching fix** from this chapter — and only that fix.
5. **Re-measure** to confirm the counter moved and the wall-clock improved. If it did not, your hypothesis was wrong; revert and return to step 2.

The cardinal sin is optimizing blind — restructuring code on intuition without a counter confirming the bottleneck. On modern hardware intuition is wrong often enough that blind optimization frequently makes things *slower* while making them uglier.

Distributed-systems lens: the same enemy at every scale

These are single-node effects, but their consequences are fleet-shaped, and the pattern of the fix is identical to the patterns you already apply to distributed systems.

Waste multiplies across the fleet. A hot loop that runs at IPC 0.4 instead of 2.0 because of a cache-miss pattern is wasting ~80% of every core it runs on. On one machine that is an annoyance; on a service deployed to 10,000 cores it is 8,000 cores of pure waste — a five-figure monthly cloud bill and a proportional carbon footprint spent stalling on memory. Mechanical-sympathy fixes are among the highest-leverage cost reductions available precisely because the per-node saving multiplies by the fleet. Chapter 1's argument — that per-node efficiency *is* a fleet-level cost concern — is cashed out here in concrete counters.

Shared mutable state limits scaling at every level, and it is the same theory each time. False sharing and atomic contention are the *single-node* face of the exact problem distributed systems spend their lives fighting: coordination on shared state does not scale. A cache line bouncing between two cores is the hardware micro-image of two nodes contending on a distributed lock or a hot database row. The cure is identical at both scales — *stop sharing the unit of coordination*: partition the data, give each core/node its own state, and combine only when necessary. Per-core counters (single-node) and sharded/partitioned datastores (distributed, Volumes 5 and 7) are the *same design* at twelve orders of magnitude apart; share-nothing architectures (Volume 7) are the fleet-scale statement of "one writer per cache line" (Chapter 4). The engineer who internalizes why the padded counter scales already understands why the partitioned service scales.

Tail latency is frequently one of these effects firing on one node. A p99 spike rarely means the median got slower; it means one request hit something the others did not. Very often that something is mechanical: a GC pause the request happened to be in flight for, a node whose memory bandwidth got saturated by a noisy neighbor, a cache-thrashing code path triggered by an unusually large input, a lock that went contended under a burst. The p99/p999 that SRE teams chase (Volume 11) is, at the bottom of the stack, often the anti-patterns in this chapter manifesting on the unlucky node — which is why reducing allocation, eliminating

contention, and fixing locality are tail-latency interventions, not just throughput ones.

The profiling workflow scales to the fleet. The measure-identify-fix-remeasure loop that finds a false-sharing bug on your laptop is the same loop, instrumented continuously and aggregated across thousands of hosts, that finds the fleet's most expensive function. Google-Wide Profiling and its descendants (Volume 13) are `perf` sampling at fleet scale; a 1% CPU regression that would be invisible on one host is a screaming, ranked line item across the fleet. Mechanical sympathy per node, multiplied by fleet scale, is the entire performance-and-cost story of a large backend — and this chapter is its practical core.

Key takeaways

- **Most performance problems that survive an algorithmic pass are hardware-effect problems.** "Same big-O, 10x different performance" is the normal case, and the cause is mechanical — cache misses, branch mispredicts, false sharing, bandwidth, contention, allocation — not operation count.
- **Poor locality is the biggest single lever.** Pointer chasing (linked lists, trees, scattered heap objects) causes a cache miss per node and defeats out-of-order execution and prefetch; AoS wastes most of every line when you read one field; strides and oversized working sets thrash the cache. Fix with contiguous layout, **struct-of-arrays** and hot/cold splitting, sequential access, and **blocking/tiling** — data-oriented design.
- **False sharing is the subtle multicore killer.** Independent variables on one 64-byte line, written by different threads, ping-pong the line via coherence and can run an order of magnitude slower while *scaling negatively*. Fix by padding/aligning hot per-thread data to its own line (`alignas(64)`, `@Contended`, `CachePadded`).
- **Unpredictable, data-dependent branches in hot loops flush the pipeline** (~15–20 cycles each). The sorted-vs-unsorted array sum is faster sorted *only* because the branch becomes predictable; the real fix is branchless code (`cmov`), predictable ordering, or SIMD — but only for branches that are both hot and genuinely unpredictable, because a well-predicted branch is nearly free.

- **Bandwidth-bound workloads do not scale with threads.** When you are limited by bytes/second, not miss latency, more cores contend for the same channel. Use the **roofline** to tell latency-bound from bandwidth-bound; fix by moving less data — locality, compression, fusion, per-byte compute, NUMA-local placement.
- **Contended atomics and locks serialize cores** by bouncing one line. Shared mutable state is the enemy of scaling: shard into per-core state, batch updates, and use lock-free/read-mostly structures where they genuinely fit — the scaling comes from *not concentrating writes on one line*.
- **Allocation is not free.** It contends, pollutes cache, and produces pointer-chasing layouts; GC turns it into tail-latency spikes. Allocate less; use arenas, pools, value types, and off-heap for big data.
- **Measure, never guess.** Each anti-pattern has a `perf`-counter fingerprint — IPC, `cache-misses`, `LLC-load-misses`, `branch-misses`, `stalled-cycles-backend`, `perf c2c` HITM. Run the loop: measure → identify the class → localize → apply the matching fix → re-measure. Optimizing blind on modern hardware usually fails.
- **Every effect multiplies across the fleet.** Per-node waste × 10,000 cores is real money and real tail latency; false sharing and atomic contention are the single-node face of distributed coordination overhead; the cure at every scale is the same — stop sharing the unit of coordination, and partition.

Further reading

- Ulrich Drepper, "What Every Programmer Should Know About Memory" (2007) — https://people.freebsd.org/~lstewart/articles/cpumemory.pdf — the definitive practitioner treatment of cache behavior, access patterns, false sharing, and NUMA, with measurements; Sections 3–6 map directly onto this chapter.
- Brendan Gregg, *Systems Performance*, 2nd ed. (Addison-Wesley, 2020), and the flame-graph and `perf` material at https://www.brendangregg.com/ — the reference for `perf`, flame graphs, and the measurement-first methodology; see especially the "CPU Flame Graphs" and `perf` one-liner pages.
- Joe Mario and Don Zickus, "C2C — False Sharing Detection in Linux Perf" — https://joemario.github.io/blog/2016/09/01/c2c-blog/ — the

definitive walkthrough of `perf c2c`, HITM events, and telling true from false sharing.

- Samuel Williams, Andrew Waterman, David Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, 2009 — the roofline model for reasoning about bandwidth-bound vs. compute-bound kernels.
- The Stack Overflow question "Why is processing a sorted array faster than processing an unsorted array?" — https://stackoverflow.com/q/11227809 — the canonical branch-prediction demonstration, with accurate explanations of the mechanism.
- Herb Sutter, "Eliminate False Sharing," Dr. Dobb's Journal, 2009 — a practitioner walkthrough of false sharing and padding fixes with numbers; pairs with the `std::hardware_*_interference_size` facilities in C++17.
- Richard Fabian, *Data-Oriented Design* — https://www.dataorienteddesign.com/dodbook/ — book-length treatment of organizing data around access patterns (SoA, hot/cold splitting) rather than around objects.
- Mike Acton, "Data-Oriented Design and C++," CppCon 2014 — https://www.youtube.com/watch?v=rX0ItVEVjHc — the influential talk making the case for laying out memory for the hardware, not the abstraction.
- Ulrich Drepper, "A Case Study in Performance: memcpy," and Agner Fog's optimization manuals at https://www.agner.org/optimize/ — microarchitectural detail on branches, `cmov`, and instruction costs across Intel/AMD parts; the authority for the branch-misprediction and branchless claims here.
- Brendan Gregg, *BPF Performance Tools* (Addison-Wesley, 2019) — eBPF-based observability that extends `perf`-style measurement into production tracing (ties to Volume 2).
- Paul E. McKenney, *Is Parallel Programming Hard, And, If So, What Can You Do About It?* — https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html — per-CPU data,

`LongAdder`-style striping, RCU, and why sharding beats contended atomics.

Chapter 9 — Number Representation and Floating Point

What this chapter covers. Every prior chapter in this volume treated numbers as a given: caches move them, SIMD lanes multiply them, coherence protocols keep them consistent. This chapter opens the number itself. A machine word is a fixed-width bit pattern, and the *meaning* of that pattern — whether `0xFFFFFFFF` is 4,294,967,295 or −1, whether `0x3FF0000000000000` is the integer 4,607,182,418,800,017,408 or the floating-point value 1.0 — is a convention imposed by the code that reads it. Most of the time the convention is invisible and correct. The rest of the time it is the source of a class of bug that is uniquely nasty for backend engineers: it is silent, it is data-dependent, it crosses service boundaries, and it corrupts *money*, *identifiers*, and *aggregates* — the three things a backend system exists to get right. This chapter is a precise account of how integers and, especially, floating-point numbers are represented, why the representation leaks, and the specific discipline that keeps the leaks out of production. The throughline is that number representation is not arithmetic trivia; it is a *correctness and consistency* problem that shows up at exactly the seams where distributed systems are already fragile — serialization boundaries, cross-language APIs, and parallel reductions across a heterogeneous fleet.

Learning goals — after this chapter you should be able to:

- Explain two's-complement integer representation and *why* it is universal (one zero, one add/subtract circuit), and predict the behavior of overflow: defined wraparound for unsigned, undefined behavior for signed C, and the real incidents both have caused.
- Read an IEEE 754 float bit-for-bit — sign, biased exponent, significand, the implicit leading 1, subnormals, and the special values `±0`, `±∞`, and `NaN` — and explain why `NaN != NaN` and why `0.1 + 0.2 != 0.3`.
- Name the four backend floating-point hazards — catastrophic cancellation, accumulation error, non-associativity, and precision loss

at large magnitude — and apply the matching fix (tolerance comparison, Kahan/pairwise summation, deterministic reduction order, integer/decimal money).

- Diagnose the classic cross-service bug where a 64-bit ID round-trips through a JSON/JavaScript `float64` and is silently corrupted above 2^53, and prescribe the fix.
- Reason about floating-point *determinism* across a fleet: why FMA, x87 80-bit intermediates, SIMD reassociation, and `-ffast-math` make the "same" computation produce different bits on different nodes, and why that breaks caching, replication, and ML reproducibility.

Integers: two's complement and its universality

An *n*-bit unsigned integer is the obvious thing: the bits are the base-2 digits, the range is `0 … 2^n − 1`, and arithmetic is modular — it wraps around 2^n with no notion of negative. The interesting design decision is how to represent *signed* integers, and here the industry converged, decades ago and essentially without exception, on **two's complement**.

In two's-complement representation the most significant bit carries *negative* weight. For an *n*-bit value with bits `b_{n-1} … b_0`:

```
value = −b_{n-1}·2^{n-1} + Σ_{i=0}^{n-2} b_i·2^i
```

So for an 8-bit byte, `0b1111_1111` is `−128 + 127 = −1`, and `0b1000_0000` is `−128`. The range is asymmetric: `−2^{n-1} … 2^{n-1} − 1`. For a 32-bit `int` that is `−2,147,483,648 … 2,147,483,647`.

Two's complement won for reasons that are directly architectural, not aesthetic:

- **There is exactly one zero.** The competing schemes — sign-magnitude (a dedicated sign bit) and ones' complement (negate by inverting all bits) — both have a distinct `+0` and `−0`. Two representations of zero mean every equality comparison and every branch-on-zero must special-case a second bit pattern. Two's complement has a single `0b0000_0000`.
- **Addition and subtraction use one circuit.** In two's complement, `a − b` is just `a + (two's-complement of b)`, and the two's complement of `b` is `~b + 1` (invert the bits, add one). The same ripple-carry adder that computes `a + b` computes `a − b` with no sign logic, no

special cases, and — critically — the *same* addition works whether you interpret the operands as signed or unsigned. The hardware does not need to know. That is why an ALU has an `ADD` and the sign interpretation is left to the flags and to the compiler. Sign-magnitude, by contrast, needs the adder to inspect signs and conditionally subtract.

- **Carry-out and wraparound are natural.** Modular wraparound at 2^n falls out of dropping the carry bit, which is exactly what a fixed-width adder does.

The one wart is the asymmetric range. There is a most-negative value (`INT_MIN`, `-2^{n-1}`) with no positive counterpart, so `-INT_MIN` overflows: negating the smallest int gives back the smallest int. `abs(INT_MIN)` is a real bug — it returns a negative number. This is not a curiosity; it has produced CVEs in image decoders and length calculations for decades.

Widths and the cost of choosing wrong

The common widths are 8, 16, 32, and 64 bits, and the choice is not free-form. A width is a promise about the maximum magnitude a value will ever take, and violating that promise is overflow. The default `int` in C, Go, Java, and most systems is 32-bit, which tops out near 2.1 billion — a number that many production quantities blow past: row counts in a large table, cumulative byte counters on a busy interface, view counts on a viral video, IDs in a high-throughput system. **The reflexive fix for anything that counts real-world events at scale is a 64-bit integer**, whose range (~9.2 quintillion signed) is large enough that overflow stops being a practical concern for counters and IDs. This is why Snowflake IDs, database bigint primary keys, and Unix nanosecond timestamps are all 64-bit. It is also, as we will see, exactly why they are dangerous when they meet JSON.

Integer overflow: wraparound, undefined behavior, and real incidents

When an arithmetic result exceeds the width, what happens depends on signedness and language, and the distinction is a genuine footgun.

Unsigned overflow is defined: it wraps modulo 2^n. `uint32` `0xFFFFFFFF + 1 == 0`. This is predictable but still dangerous — a wrapped-around length or index becomes a small or huge number that

sails past a bounds check. The classic pattern is an allocation size computed as `count * size`, which wraps to a small value; the code then allocates the small buffer but writes `count` elements into it — a heap overflow. This is a staple of memory-safety CVEs.

Signed overflow in C and C++ is *undefined behavior* (UB), and this is one of the sharpest edges in systems programming. The standard does not say signed overflow wraps; it says the program's behavior is undefined, which licenses the optimizer to assume it *never happens*. A compiler may prove that `x + 1 > x` is always true for signed `x` (because overflow "can't" occur) and delete a bounds check that depended on the wraparound. The infamous shorthand is that `if (x + 1 < x)` — a hand-rolled overflow check — can be optimized away entirely, because the compiler reasons the condition is impossible. Overflow checks must therefore be written *before* the operation (`if (x > INT_MAX - 1)`) or with compiler builtins (`__builtin_add_overflow`) or in a language that defines the behavior. Rust makes this explicit: signed and unsigned overflow *panic* in debug builds and wrap in release builds by default, with `checked_add`, `wrapping_add`, and `saturating_add` as intent-revealing choices.

The consequences are not academic. Two illustrative classes:

- **Underhanded / supply-chain attacks.** Integer-overflow bugs are a favorite of the "underhanded C" genre precisely because they are invisible in review: a size calculation that looks correct wraps on a large but attacker-controllable input, and the resulting undersized allocation becomes an exploit primitive. In a supply-chain context — a dependency deep in your build — such a bug is a plausible vector for a deliberately planted vulnerability that passes casual audit. The general lesson from the supply-chain incidents catalogued in Volume 4 applies: subtle arithmetic in untrusted code is exactly where planted vulnerabilities hide.
- **Physical-world failure.** The most cited example is the loss of the maiden flight of the Ariane 5 (Flight 501, June 1996). The reused inertial-reference software converted a 64-bit floating-point value related to horizontal velocity into a 16-bit signed integer; the Ariane 5's higher velocity produced a value outside the 16-bit range; the conversion overflowed and raised an unhandled exception; the inertial reference units shut down; and the vehicle, now flying on garbage guidance, self-destructed. The precise chain has been

studied exhaustively in the official inquiry board report — the salient point for us is that it was a *conversion-with-overflow* bug in reused code whose input assumptions no longer held.

The 2038 problem

A specific, scheduled instance of signed overflow deserves its own mention because it will actually arrive. Traditional Unix `time_t` is a **signed 32-bit** count of seconds since the epoch, 1970-01-01T00:00:00Z. A signed 32-bit second counter overflows at **2038-01-19T03:14:07Z**, after which it wraps to a large negative number and time appears to jump to December 1901. This is the integer-overflow bug with a deadline. The fix is a 64-bit `time_t`, which pushes the overflow roughly 292 billion years out; modern 64-bit platforms and updated 32-bit ABIs have moved to it, but embedded systems, on-disk formats, wire protocols, and databases that baked in 32-bit timestamps remain exposed. If your system stores or transmits a 32-bit epoch anywhere, it has a 2038 liability, and the audit is worth doing now rather than in 2037.

Endianness and the serialization boundary

A multi-byte integer must be laid out in memory as a sequence of bytes, and there are two conventions. **Big-endian** stores the most significant byte at the lowest address; **little-endian** stores the least significant byte first. The 32-bit value `0x0A0B0C0D` is `0A 0B 0C 0D` in memory big-endian and `0D 0C 0B 0A` little-endian.

Within a single machine, endianness is invisible — the CPU reads back what it wrote. It becomes real the moment bytes cross a boundary: a network socket, a memory-mapped file, a binary protocol, a device register. x86 and most ARM deployments are little-endian; many older RISC systems and some network gear are big-endian. Crucially, **network byte order is big-endian** by convention (the IP/TCP/UDP header fields are big-endian), which is why the socket API ships `htons` / `htonl` / `ntohs` / `ntohl` — host-to-network and network-to-host conversions that are no-ops on a big-endian host and byte-swaps on a little-endian one. Any binary serialization that does not pin an explicit byte order is a cross-system bug waiting for a heterogeneous fleet or a new client platform. This is the number-level face of the serialization discipline developed in Volume 3 (networking) and Volume 8 (APIs): every wire format must specify byte order, and every reader must honor it. Text formats like

JSON dodge endianness entirely by encoding numbers as decimal strings of digits — but, as the second half of this chapter shows, they trade the endianness bug for a precision bug.

Floating point: IEEE 754

Integers represent a bounded range of exact values with uniform spacing. Real computation — physics, statistics, graphics, ML, anything with fractions or a wide dynamic range — needs to represent numbers from the very small to the very large, and it accepts *approximation* as the price. The universal standard for this is **IEEE 754** (first published 1985, substantially revised as IEEE 754-2008 and 754-2019). Essentially every FPU, GPU, and language `float` / `double` implements it, which is what makes cross-platform floating point *mostly* portable — and the exceptions to "mostly" are the subject of the determinism section below.

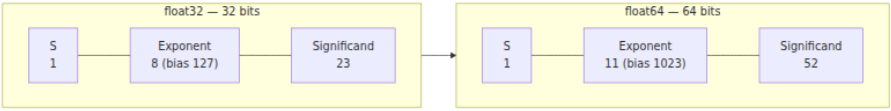
A 754 binary float encodes a number in three fields, in the spirit of scientific notation `(−1)^sign × significand × 2^exponent`:

- a **sign** bit `s`;
- a biased **exponent** field `e` of width chosen per format;
- a **significand** (also called mantissa) field `f`, the fractional bits.

For a *normalized* number, the value is:

```
(−1)^s × 1.f × 2^(e − bias)
```

The `1.` is the **implicit leading bit**: because a normalized binary significand always starts with a 1, the standard does not store it, buying one extra bit of precision for free. The exponent is stored *biased* (offset by a constant) rather than in two's complement, so that the bit patterns of floats sort in the same order as signed integers of the same width — a trick that lets hardware compare magnitudes with integer comparisons.



The two workhorse formats:

- **float32 (single precision):** 1 sign, 8 exponent (bias 127), 23 significand bits. With the implicit bit that is 24 bits of significand,

giving roughly 7 decimal significant digits and a range up to ~3.4×10^38.

- **float64 (double precision):** 1 sign, 11 exponent (bias 1023), 52 significand bits — 53 effective — for roughly 15–16 decimal digits and a range up to ~1.8×10^308. This is the default `double` in C family languages, Python's `float`, and — importantly — *every* number in JavaScript and JSON.

The ML formats: fp16, bfloat16, and FP8

Machine learning changed the economics of precision. Training and inference are bottlenecked by memory bandwidth and by the die area and energy of multiply-accumulate units, and it turns out neural networks tolerate — sometimes benefit from — dramatically reduced precision. The result is a family of narrow formats, each a different answer to the question "given very few bits, spend them on range or on precision?" (See Volume 7 on ML systems for how these interact with training stability and mixed-precision schemes.)

The key contrast is **float16 vs bfloat16**, both 16 bits:

- **float16** (IEEE half) spends 5 bits on exponent and 10 on mantissa — more precision, but a narrow dynamic range (~6×10^−5 to ~65,504) that overflows and underflows easily during training, requiring loss-scaling gymnastics.
- **bfloat16** (Google Brain's format) spends 8 bits on exponent and only 7 on mantissa. It has the *exact same exponent range as float32* — so a float32 can be truncated to bfloat16 by dropping the low 16 mantissa bits, and values rarely overflow — at the cost of much coarser precision. For deep learning this trade is usually the right one: the network cares more about dynamic range than about the low mantissa bits, and bfloat16 became the default training precision on TPUs and modern GPUs.

FP8 pushes further, to 8 bits, standardized by the Open Compute Project in two variants: **E4M3** (4 exponent, 3 mantissa — more precision, used for weights/activations) and **E5M2** (5 exponent, 2 mantissa — more range, used for gradients). At 8 bits the encodings deviate from strict IEEE 754 — E4M3 in the OCP definition, for instance, forgoes infinities to

reclaim those bit patterns for finite values — so "FP8" is a family of closely-specified formats rather than a single 754 type. The takeaway for a backend engineer serving models is that *precision is now a deployment parameter*: the same model weights exist as float32, bfloat16, and FP8 quantizations, and which one a node uses affects both its throughput and — the theme of this chapter — the exact bits it computes.

Format	Bits	S/E/M	Approx. range	Approx. significant digits	Typical use
int8	8	—	−128 … 127	exact	flags, quantized weights
int16	16	—	−32,768 … 32,767	exact	small counters, audio
int32	32	—	±2.1×10^9	exact	default int, IDs (risky)
int64	64	—	±9.2×10^18	exact	IDs, counters, nanotime
FP8 E4M3	8	1/4/3	~±448	~1	ML inference (weights)
FP8 E5M2	8	1/5/2	~±5.7×10^4	~1	ML training (gradients)
float16	16	1/5/10	~±6.5×10^4	~3	ML, graphics
bfloat16	16	1/8/7	~±3.4×10^38	~2–3	ML training/ inference
float32	32	1/8/23	~±3.4×10^38	~7	graphics, DSP, ML
float64	64	1/11/52	~±1.8×10^308	~15–16	scientific, JS/ JSON default

Subnormals and the special values

The exponent field has two reserved patterns — all-zeros and all-ones — that encode the edges of the number line.

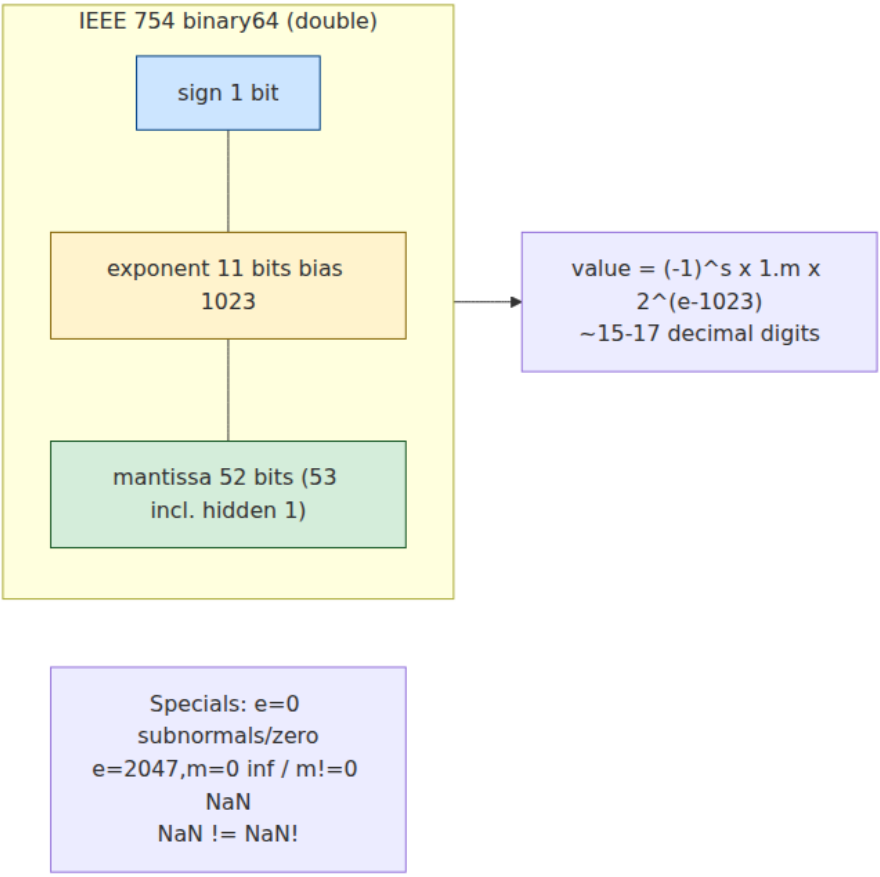
All-zero exponent signals a **subnormal** (denormal) number. Normalized numbers cannot represent values arbitrarily close to zero, because the implicit leading 1 imposes a smallest normalized magnitude. Subnormals fill the gap between that smallest normalized value and zero: the implicit leading bit becomes 0 instead of 1, so the significand is `0.f` rather than `1.f`, and the numbers get progressively less precise but degrade *gradually* to zero rather than snapping to it (this is "gradual underflow," a deliberate 754 design goal). Subnormals are correctness-preserving but can be a performance cliff: some hardware handles them via microcode or traps that run orders of magnitude slower, which is why performance-sensitive and ML code often enables **flush-to-zero / denormals-are-zero** modes — trading the tail of precision for predictable latency (a mechanical-sympathy concern in the spirit of Chapter 8).

All-one exponent signals the special values:

- **Infinity** when the significand is zero: `+∞` and `−∞`, produced by overflow or by `1.0/0.0`.
- **NaN** (Not a Number) when the significand is nonzero: the result of `0.0/0.0`, `∞ − ∞`, `sqrt(−1)`, and any operation with no meaningful real answer. The standard distinguishes **quiet NaN** (qNaN, top significand bit set) which propagates through arithmetic silently, from **signaling NaN** (sNaN) which is intended to raise an exception when used — in practice almost everything you encounter is a quiet NaN, and the distinction rarely surfaces above the FPU.
- **Signed zero:** `+0` and `−0` are distinct bit patterns. They compare *equal* (`+0 == −0` is true) but are distinguishable by operations like `1.0/x`, which yields `+∞` for `+0` and `−∞` for `−0`.

The single most bug-productive property is that **`NaN` is not equal to anything, including itself**: `NaN == NaN` is `false` and `NaN != NaN` is `true`. This is by design — NaN means "no meaningful value," so no equality holds — but it wrecks code that assumes reflexivity. A `NaN` slipped into a sort comparator produces inconsistent orderings and can corrupt or crash the sort. A `NaN` key in a hash map can never be found again. A `NaN` in a running max/min silently swallows subsequent comparisons (`NaN < x` and `NaN > x` are both false). A deduplication or set membership check on floats breaks. The canonical *detection* idiom exploits the reflexivity failure: `x != x` is true if and only if `x` is `NaN` (which is what `isnan()` does under the hood). The canonical *defense* is to reject or sanitize NaN and infinity at ingestion — a validated numeric

input at an API boundary should exclude them unless they are genuinely meaningful.



The fundamental problem: floating point is approximate

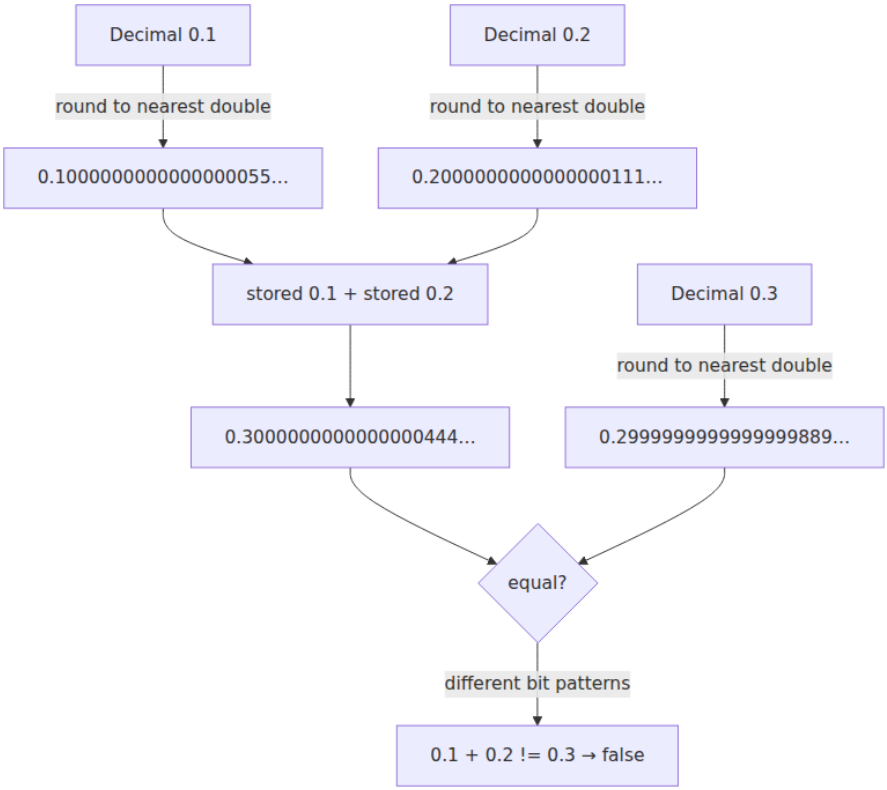
Here is the fact that every backend engineer must internalize: **most decimal fractions cannot be represented exactly in binary floating point.** A binary float can exactly represent only numbers of the form `integer × 2^k`. The decimal `0.5` is `2^−1` — exact. The decimal `0.1` is *not* a finite sum of powers of two; in binary it is the infinitely repeating fraction `0.0001100110011001100…₂`, the base-2 analog of how `1/3` is `0.333…` in decimal. Stored in a float64, `0.1` is rounded to the nearest

representable value, which is very slightly more than one tenth. The same is true of `0.2` and most "round" decimals.

This is why the most famous floating-point surprise is true in essentially every language with 754 doubles:

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Neither `0.1`, `0.2`, nor `0.3` is stored exactly. The stored `0.1` and stored `0.2` sum to a value whose nearest representable double is *not* the nearest representable double to `0.3`, and the tiny representation errors fail to cancel. The result is off by one unit in the last place — about `5.5×10^−17` — which prints as the notorious `…04` tail.



Two consequences follow, and they are the root of most floating-point bugs:

- **Rounding error is inherent and it accumulates.** Every operation whose true result is not exactly representable is rounded (by default, round-to-nearest-even), introducing an error of up to half a unit in the last place. One rounding is negligible; a billion roundings, correlated, are not.
- `==` **on floats is a bug.** Because the exact bits depend on the exact sequence of roundings, two computations that are mathematically equal routinely differ in their low bits. Testing floats for equality asks whether two approximations landed on the *identical* approximation, which is almost never what you mean.

Machine epsilon quantifies the granularity: it is the gap between 1.0 and the next representable value — `2^−52 ≈ 2.22×10^−16` for float64, `2^−23 ≈ 1.19×10^−7` for float32. It is the *relative* precision floor. The correct way to compare floats is with a tolerance that respects this scale — either an absolute epsilon for values near a known magnitude, a *relative* tolerance (`|a − b| <= eps × max(|a|, |b|)`), or a **ULP** (units-in-the-last-place) comparison that counts how many representable values lie between the two operands. Language ecosystems provide these: `math.isclose` in Python (with `rel_tol` and `abs_tol`), `numpy.isclose` / `allclose` , and equivalent helpers elsewhere. There is no universally correct tolerance — the right one depends on the magnitudes and the accumulated error of the specific computation — which is precisely why float equality cannot be papered over with a library call and must be reasoned about.

Floating-point hazards for backend systems

Approximation by itself is benign; you learn to compare with tolerance and move on. The hazards that actually break production are the ones where errors are not tiny and independent but *large* or *correlated* or *order-dependent*.

Hazard	Cause	Symptom	Fix
Catastrophic cancellation	Subtracting near-equal values cancels the leading digits, exposing rounding noise	Result has few or no correct significant digits	Reformulate the algorithm to avoid the subtraction (stable variance, stable quadratic)
Accumulation error	Summing many values; each add rounds; errors correlate and drift	Total is wrong by growing amount; order-dependent	Kahan/compensated summation, pairwise summation, or sort by magnitude
Non-associativity	`(a+b)+c ≠ a+(b+c)` because each add rounds	Parallel/sharded reductions disagree; non-reproducible	Fix a canonical reduction order or use reproducible-reduction techniques
Precision loss at magnitude	Beyond 2^53 a float64 cannot represent consecutive integers	Large IDs corrupted; large counters skip values	Keep exact integers as int64/decimal/strings, never as float
Overflow / underflow	Result exceeds format range or is flushed to zero	`±∞` or `0` silently; NaN downstream	Range checks; log-domain arithmetic; wider format

Catastrophic cancellation

When you subtract two nearly-equal floating-point numbers, the high-order digits cancel and the result is dominated by the *rounding error* in the low-order digits of the operands — you can lose most or all of your significant figures in a single subtraction. The textbook case is the naive variance formula `E[x²] − E[x]²`: for data with a large mean and small spread, `E[x²]` and `E[x]²` are two large, nearly-equal numbers whose difference is a small variance, and the subtraction annihilates precision — it can even produce a *negative* variance, which is mathematically impossible. The fix is algorithmic: use a numerically stable formulation such as Welford's online algorithm, which updates mean and variance incrementally without ever forming that difference. The same

reformulation logic applies to the quadratic formula (rationalize to avoid subtracting `b` from a nearly-equal `sqrt(b² − 4ac)`) and to computing `log(1+x)` and `exp(x)−1` for small `x`, which is why the standard library ships `log1p` and `expm1` — dedicated, cancellation-free implementations.

Accumulation error and Kahan summation

Summing a large array of floats is the most common backend numeric operation — it is what every `SUM()`, every average, every metric rollup does — and the naive loop drifts. Each addition rounds the running total to a representable value, discarding a little of the addend's low bits; when you add many numbers, especially many small numbers to a large running total, those discarded bits accumulate into a visible error. Add a million values around 1.0 and the total can be off by a noticeable amount, because once the running sum is large, each new small addend loses precision relative to it.

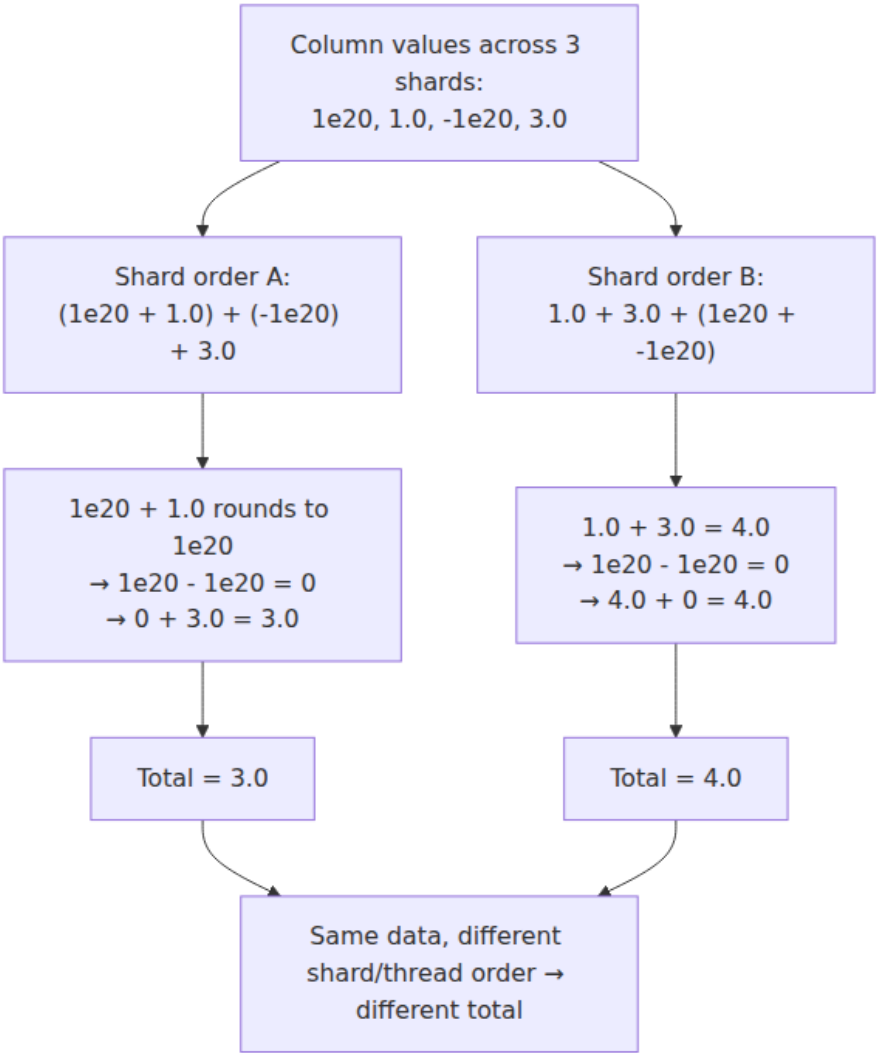
Kahan (compensated) summation fixes this by keeping a second variable that tracks the low-order bits lost on each addition and feeds them back into the next one:

```
def kahan_sum(values):
    total = 0.0
    c = 0.0          # running compensation for lost low-order bits
    for x in values:
        y = x - c           # apply previous compensation
        t = total + y       # total may be big; low bits of y are lost
here
        c = (t - total) - y # recover exactly what was lost
        total = t
    return total
```

The algebra `(t − total) − y` recovers the rounding error exactly (in the absence of overflow), so the compensation `c` carries the lost precision forward. Kahan summation makes the error roughly constant regardless of the number of terms, instead of growing with it. Alternatives with different trade-offs: **pairwise (cascade) summation** recursively sums halves and has good error behavior with better cache/vectorization properties — it is what NumPy's `sum` uses — and **sorting by magnitude** before summing (smallest first) reduces the relative error of the naive loop. The engineering point is that "sum this column" is not a solved primitive; at scale it is an algorithm choice.

Non-associativity and the distributed reduction

This hazard is the one that graduates from a single-node numerics footnote to a genuine distributed-systems correctness problem, so it gets the distributed-systems lens treatment below and a diagram here. Floating-point addition is **commutative** (`a + b == b + a`) but **not associative**: `(a + b) + c` can differ from `a + (b + c)` because the two orderings round at different intermediate values. Concretely, adding a large and a small number can round the small one away entirely, so the order in which you fold values determines whether small contributions survive.

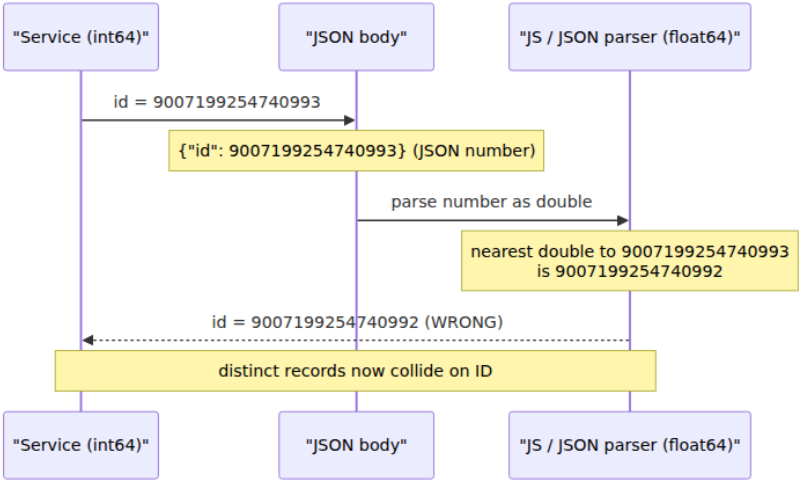


When the same aggregation runs across shards, threads, or nodes, and the runtime is free to combine partial sums in whatever order they complete, the *final total depends on scheduling* — which is non-deterministic. This is not a rounding curiosity; it means two replicas computing the "same" sum can disagree, a re-run of the same job can produce a different answer, and a cached result can mismatch a freshly computed one.

Precision loss at large magnitude, and the 64-bit-ID-through-JSON bug

Float64 has 53 bits of significand, so it can represent every integer exactly up to **2^53 = 9,007,199,254,740,992**. Above that, consecutive integers are no longer all representable — at 2^53 the spacing becomes 2, so `2^53 + 1` rounds to `2^53`. JavaScript exposes this boundary as `Number.MAX_SAFE_INTEGER = 2^53 - 1 = 9,007,199,254,740,991`, the largest integer for which `n` and `n + 1` are both exactly representable.

This collides head-on with the backend habit of using 64-bit integer IDs. A Snowflake ID, a database bigint, or any int64 identifier routinely exceeds 2^53. The danger is that **JavaScript has a single number type — float64 — and JSON, following JavaScript, defines its `number` as a double in every mainstream parser.** So the moment a 64-bit integer ID is serialized as a JSON *number* and parsed by a JavaScript client (or many other JSON libraries that decode numbers to doubles), any value above 2^53 is silently rounded to the nearest representable double — a *different integer*. The ID `9007199254740993` comes back as `9007199254740992`. Two distinct records now share an ID; a lookup misses; an idempotency key collides; a permission check matches the wrong row.



The corruption is silent — no error, no exception, just a wrong number — and it is intermittent, because IDs below 2^53 round-trip fine, so it hides until your ID space grows or a Snowflake timestamp crosses the

threshold. The fix is unambiguous and belongs to the API-design discipline of Volume 8: **serialize 64-bit identifiers as JSON strings, not numbers.** `{"id": "9007199254740993"}` is lossless everywhere, because a string of digits has no float64 rounding. Twitter's API famously learned this and began returning both `id` (number, deprecated) and `id_str` (string) for exactly this reason. The same caution applies to any int64 that flows through a JavaScript, JSON, or protobuf-to-JSON layer: protobuf's JSON mapping already encodes 64-bit integer fields as strings for this precise reason.

Money and decimals: never use float

Money is the domain where floating-point approximation stops being an academic error and becomes a financial discrepancy, an accounting mismatch, or a regulatory problem. **You must never represent monetary amounts as binary floating point.** The reason is exactly the `0.1` problem: currency is decimal, and `0.10`, `0.01`, and most prices are not exactly representable in binary. A `double` of `0.10` is slightly off; multiply it by a quantity, sum thousands of line items, apply a percentage tax, and the sub-cent errors compound into totals that do not reconcile, that round inconsistently, and that differ from what a decimal-correct system computes. `0.1 + 0.2 != 0.3` is a rounding curiosity; the same effect across a ledger is a real deviation that auditors and customers notice.

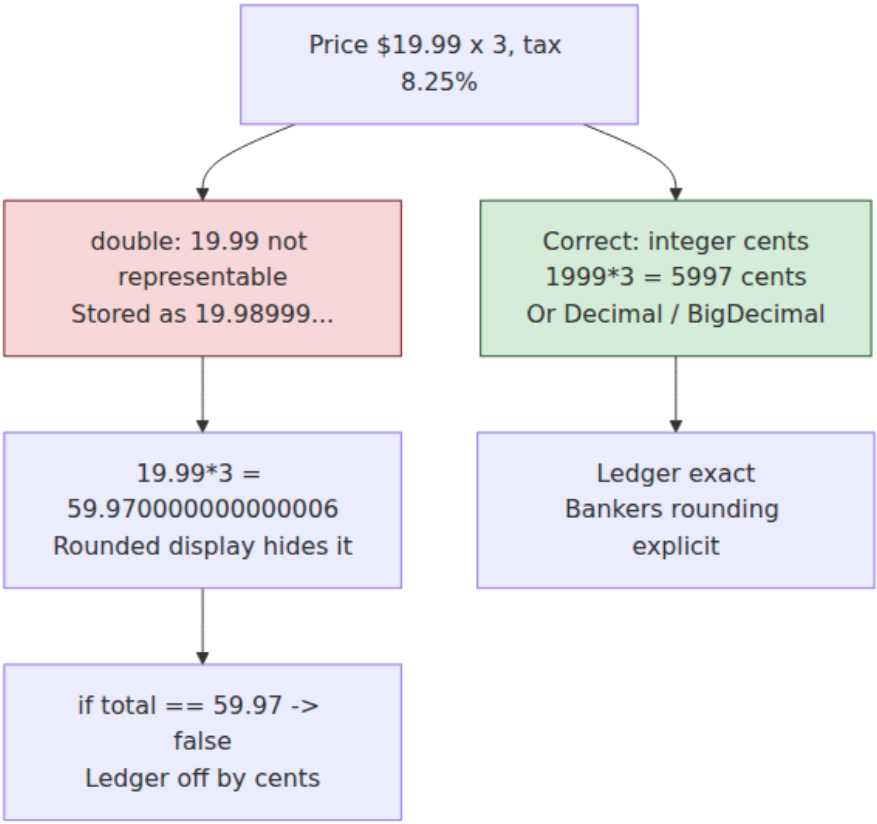
There are two correct approaches, and both eliminate binary fractions entirely:

Approach	Representation	Pros	Cons / caveats
Integer minor units	Store amounts as an integer count of the smallest unit (cents, satoshis, or a fixed number of decimal places) in int64/int128	Exact, fast, simple, database-friendly; arithmetic is plain integer math	Must track the scale/currency out of band; must handle currencies with different minor-unit exponents; risk of overflow at extreme scale (use int64/128 deliberately)

Approach	Representation	Pros	Cons / caveats
Decimal / fixed-point type	A base-10 type: SQL `DECIMAL(p,s)` / `NUMERIC`, Java `BigDecimal`, Python `decimal.Decimal`, C# `decimal`, Rust `rust_decimal`	Exact decimal arithmetic with explicit precision and rounding mode; handles fractional cents and multi-currency scales	Slower than native ints/floats; must set precision and rounding policy explicitly; not a hardware type

Integer minor units are the workhorse: store `$19.99` as the integer `1999` cents. All arithmetic is exact integer arithmetic; you divide by 100 only for display. This is simple, fast, and unambiguous, and it is why payment systems and ledgers overwhelmingly use it. Its one discipline is that *scale is metadata* — `1999` means nothing without knowing it is US cents at two decimal places — and currencies differ (JPY has zero minor-unit digits, BHD has three), so a robust money type carries currency and exponent alongside the integer.

The **decimal type** is the choice when you need fractional minor units (interest accrual, per-unit pricing, tax computed to four decimal places) or when the arithmetic is genuinely decimal. `BigDecimal`, Python `Decimal`, and SQL `NUMERIC(p,s)` all perform exact base-10 arithmetic and let you specify the rounding mode explicitly — which matters, because financial rounding rules (round-half-up, round-half-even/banker's rounding) must be *chosen*, not left to whatever the FPU does. The database column type is the last line of defense: a monetary column should be `DECIMAL` / `NUMERIC` with an explicit scale, never `FLOAT` or `DOUBLE`. IEEE 754 also defines *decimal* floating-point formats (decimal64, decimal128) used in some financial and database engines (IBM's hardware, PostgreSQL's `numeric` semantics), which give decimal exactness in a floating format — but the everyday rule stands: if a value is money, it is an integer of minor units or a decimal type, fleet-wide, at every layer.



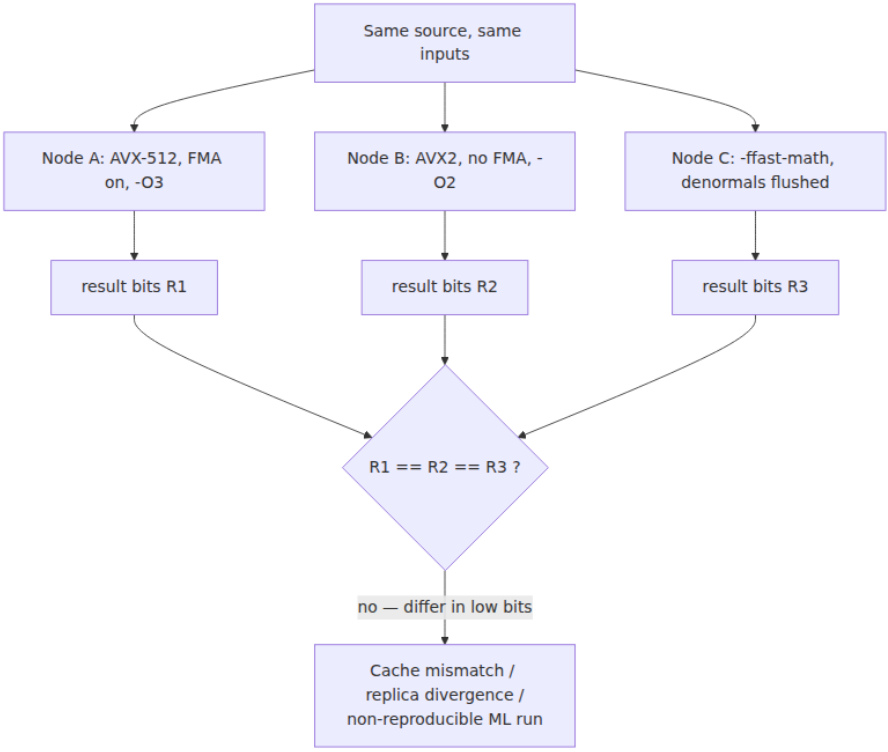
Determinism and reproducibility across a fleet

The final hazard is the most subtle and the most distributed: the *same* floating-point program, given the *same* inputs, can produce *different bits* on different machines, compilers, or optimization settings. IEEE 754 pins the result of each individual operation, but a real computation is a sequence of operations, and several things below the source level are free to change that sequence or its intermediate precision:

- **Fused multiply-add (FMA).** Modern CPUs and GPUs have an `FMA` instruction that computes `a*b + c` with a *single* rounding instead of two (one after the multiply, one after the add). This is more accurate, but it produces a *different* result than a separate multiply and add — and whether the compiler emits FMA depends on the target

architecture and flags. The same source computes different bits on a chip with FMA than on one without, or between `-O2` and a build that contracts to FMA.

- **x87 80-bit extended precision.** Legacy 32-bit x86 code using the x87 FPU computes intermediates in 80-bit extended precision and rounds to 64-bit only when a value is stored to memory. Whether a temporary stays in an 80-bit register or gets spilled to a 64-bit memory slot — a decision the *register allocator* makes — changes the result. This is the classic "it gives different answers in debug vs. release" bug. SSE/AVX (the default on 64-bit x86) largely eliminated it by computing directly in 32/64-bit, but it survives in older toolchains and targets.
- **SIMD reassociation and reduction order.** Auto-vectorizing a sum splits it across vector lanes and reassociates the additions — which, per the non-associativity result above, changes the total. A loop summed scalar, summed with AVX2 (8 lanes), and summed with AVX-512 (16 lanes) can give three different answers, all "correct," because they fold the values in different orders.
- **`-ffast-math` and friends.** This family of flags (`-ffast-math`, `/fp:fast`, `--use_fast_math`) explicitly tells the compiler to *abandon* strict IEEE semantics for speed: assume no NaNs or infinities, assume associativity (licensing free reassociation), flush denormals to zero, and use lower-precision reciprocal/sqrt approximations. It makes numeric code faster and its results non-portable and non-reproducible — and it can silently break code that relied on `NaN` propagation or on strict rounding. Enabling it is a deliberate trade, not a free optimization.



Why a backend engineer must care: the moment a computed floating-point value is *compared across nodes* or *cached and reused*, bit-level non-determinism becomes a consistency bug. Concretely:

- **Caching computed results.** If you cache a computed metric or score keyed by its inputs, and two nodes compute slightly different bits, a cache populated by node A is "wrong" from node B's perspective — or worse, cache validation that recomputes and compares by equality thrashes forever.
- **Replicated computation.** Systems that run the same computation on multiple replicas and compare results for agreement (some consensus, verification, or Byzantine-tolerance schemes) will see spurious disagreement on a heterogeneous fleet if the computation is floating point and the nodes differ in ISA, SIMD width, or build. This ties directly to the determinism and consistency material in Volume 6: *deterministic replay and state-machine replication require*

deterministic computation, and floating point is a leading source of non-determinism.

- **ML reproducibility.** Training and even inference are famously hard to reproduce bit-for-bit because of exactly these effects — GPU reduction order, mixed-precision accumulation, non-associative `all-reduce` across a distributed training job, and library kernels that reassociate. Reproducible ML (a concern for debugging, auditing, and regulatory sign-off — Volume 7) requires pinning the versions, the reduction order, the precision, and sometimes disabling fast-math and non-deterministic kernels (e.g., framework "deterministic mode" flags), at a real throughput cost.

The mitigations form a spectrum. Where you need *bit-reproducibility*, you buy it: pin the compiler and flags across the fleet, disable fast-math, fix the reduction order (a canonical serial or tree order rather than "whatever finished first"), pin library and kernel versions, and consider reproducible-summation algorithms (e.g., pre-rounding or wide-accumulator techniques that are provably order-independent). Where you only need *agreement to a tolerance*, design the comparison around a ULP or relative-error budget rather than equality, and never let a downstream decision hinge on the low bits of a distributed float reduction.

Distributed-systems lens

Number representation is a small topic on one node and a large one across a fleet, because every service, language, and CPU is free to treat a number slightly differently, and the disagreements surface exactly at the boundaries where distributed systems are already hard:

- **Non-associativity breaks determinism in parallel and sharded aggregation.** Summing a column across shards, threads, or GPU lanes in whatever order they complete yields *order-dependent totals*. For analytics this is a reproducibility and correctness problem (the same query returns different totals); for replicated state machines and consensus it is a divergence bug (replicas that must agree do not); for ML it is the reason distributed training is hard to reproduce. The fix is to make reduction order a *contract*, not an accident — see Volume 6 on determinism and consistency.
- **The 64-bit-ID-through-float64 corruption is a canonical cross-service API bug.** It lives at the JSON boundary between an int64-

native backend and a float64-native JavaScript/JSON world, it is silent, and it is intermittent. The fleet-wide fix — *large identifiers travel as strings* — is an API-contract decision (Volume 8), not a per-service patch.

- **Reproducibility across a heterogeneous fleet is a caching and consistency concern.** Different CPUs, SIMD widths, compilers, and flags compute different bits for the "same" floating-point work, so any design that caches a computed float, compares computed floats across nodes, or assumes recomputation yields the identical value is exposed. Either pin the entire numeric toolchain fleet-wide or design for tolerance, never equality.
- **Money correctness is a fleet-wide discipline, not a local one.** A single service that uses `double` for an amount contaminates every total it touches downstream. Integer-minor-unit or decimal representation has to be the invariant *across* the financial system — at the API, in the database column, in every intermediate service — because the errors compound across hops.
- **Endianness and representation must be pinned at every serialization boundary.** Byte order for binary formats, and precision for text formats, are properties of the *wire contract*. A format that leaves either unspecified is a latent cross-system bug that fires the day a new client platform, a new CPU architecture, or a larger value enters the system (Volumes 3 and 8).

The unifying observation is that a number is only unambiguous *within* one representation. The instant it crosses a boundary — a socket, a JSON body, a language runtime, a different CPU — it is re-interpreted, and every difference in signedness, width, byte order, precision, or reduction order is a place where the value can silently change. Distributed systems are, among other things, a very large number of such boundaries.

Key takeaways

- **Two's complement is universal** because it has a single zero and lets one adder do signed and unsigned add/subtract; its only wart is the asymmetric range (`INT_MIN` has no positive counterpart, so `-INT_MIN` and `abs(INT_MIN)` overflow).
- **Integer overflow is defined wraparound for unsigned but *undefined behavior* for signed C/C++** — the optimizer may delete

overflow checks that assume wraparound. Write checks before the operation or use a language (Rust) that defines the behavior. Overflow has caused real incidents (undersized allocations → memory-safety CVEs; the Ariane 5 64-bit-float-to-16-bit-int conversion overflow) and has a scheduled one: the 2038 signed-32-bit `time_t` rollover.

- **Endianness matters at every binary serialization boundary;** network byte order is big-endian (`htonl` / `ntohl`). Text formats avoid it but trade it for a precision bug.
- **IEEE 754 encodes** `(−1)^s × 1.f × 2^(e−bias)` with an implicit leading 1, subnormals for gradual underflow, and special values `±0`, `±∞`, and `NaN`. `NaN != NaN` — which breaks sorts, hash keys, and equality — and is also the standard `isnan` test (`x != x`).
- **Floating point is approximate:** `0.1`, `0.2`, `0.3` are not exactly representable, so `0.1 + 0.2 != 0.3`. Never compare floats with `==`; use a relative/ULP tolerance sized to the computation. Machine epsilon (`~2.2×10^−16` for float64) is the relative-precision floor.
- **The backend hazards are cancellation, accumulation, non-associativity, and magnitude loss.** Reformulate to avoid subtracting near-equal values (stable variance, `log1p` / `expm1`); use Kahan/pairwise summation for large sums; fix reduction order for reproducible distributed sums; and keep exact large integers out of floats.
- **The 64-bit-ID-through-JSON/JavaScript bug:** float64 is exact only up to 2^53 (`Number.MAX_SAFE_INTEGER`), and JSON numbers are doubles, so int64 IDs above 2^53 are silently corrupted at a JS/JSON boundary. **Send large identifiers as strings.**
- **Never use float for money.** Use integer minor units (cents) or a decimal/fixed-point type (`BigDecimal`, `DECIMAL(p,s)`), with an explicit rounding mode, at every layer including the database column.
- **Floating point is non-deterministic across a fleet** due to FMA, x87 80-bit intermediates, SIMD reassociation, and `-ffast-math`. For bit-reproducibility, pin the toolchain, flags, precision, and reduction order fleet-wide; otherwise design for tolerance, never equality. This underlies caching, replication, and ML reproducibility.

Further reading

- IEEE, "IEEE Standard for Floating-Point Arithmetic," IEEE 754-2019 (and the 2008 and original 1985 editions) — https://standards.ieee.org/ieee/754/6210/ — the authoritative definition of formats, rounding, special values, and operations.
- David Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, 1991 — https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html — the canonical deep treatment of rounding, cancellation, and 754 rationale.
- William Kahan, "Further remarks on reducing truncation errors," *Communications of the ACM*, 1965 — the original compensated-summation note; see also Kahan's collected lecture notes at https://people.eecs.berkeley.edu/~wkahan/ for the definitive practitioner's perspective on 754.
- Nicholas J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. (SIAM, 2002) — the rigorous reference on cancellation, summation error, and numerical stability (Chapter 4 on summation).
- ARIANE 5 Flight 501 Failure — Report by the Inquiry Board (ESA/CNES, 1996) — https://esamultimedia.esa.int/docs/esa-x-1819eng.pdf — the primary source for the 64-bit-float to 16-bit-signed-integer conversion overflow.
- Intel 64 and IA-32 Architectures Software Developer's Manuals — https://www.intel.com/sdm — for x87 80-bit extended precision, SSE/AVX floating-point behavior, and the FMA instructions.
- "The Open Compute Project OCP 8-bit Floating Point Specification (OFP8)," and Micikevicius et al., "FP8 Formats for Deep Learning," 2022 — https://arxiv.org/abs/2209.05433 — the E4M3/E5M2 definitions; and Google's bfloat16 documentation — https://cloud.google.com/tpu/docs/bfloat16.
- ECMAScript Language Specification, Number type (float64) and `Number.MAX_SAFE_INTEGER` — https://tc39.es/ecma262/ — and RFC 8259 (JSON), §6 Numbers — https://www.rfc-editor.org/rfc/rfc8259 — on why JSON numbers lose 64-bit integer precision; see the protobuf JSON mapping (https://protobuf.dev/programming-guides/json/) for the string-encoding remedy.

- "What Every Programmer Should Know About Floating-Point Arithmetic" (floating-point-gui.de) — https://floating-point-gui.de/ — an accessible practitioner's companion to Goldberg, with the comparison-with-tolerance and money guidance restated concretely.
- Martin Fowler, "Money" pattern (*Patterns of Enterprise Application Architecture*) — https://martinfowler.com/eaaCatalog/money.html — the canonical statement of the integer-minor-unit money type and why float is disqualified.
- James Demmel and Hong Diep Nguyen, "Fast Reproducible Floating-Point Summation," IEEE ARITH 2013 — the reference on order-independent, reproducible parallel summation for distributed reductions.

Chapter 10 — Hardware Support for Virtualization and Isolation

---

What this chapter covers. Every previous chapter of this volume treated the machine as *yours*: your pipeline (Chapter 2), your caches (Chapters 3–4), your NUMA nodes (Chapter 6), your cores. That was a useful fiction. In production, your code runs on a sliver of a machine that is simultaneously running other people's code — a co-tenant's batch job, another team's service, in the public cloud a total stranger's workload — and the only thing keeping their bug, their greed, and their malice out of your address space is a set of hardware mechanisms the CPU enforces on every instruction. This final chapter is about those mechanisms: privilege levels, the MMU as an isolation device, the specific silicon that makes a virtual machine possible (Intel VT-x / AMD-V, EPT/NPT, the IOMMU), the isolation *spectrum* you actually choose from in production (VMs, microVMs, gVisor, containers), the confidential-computing features that try to remove the cloud provider itself from your trust boundary (SEV-SNP, TDX, SGX), and the honest bad news: the isolation is imperfect, and speculation (Chapter 2) plus shared microarchitectural state (Chapters 2–4) leak data *across* boundaries that the architecture swears are airtight. This is the chapter where hardware architecture hands off to the operating system (Volume 2) and to the multi-tenant cloud (Volume 12): the backend engineer does not run on hardware, they run on *virtualized*,

multi-tenant hardware, and the boundaries between tenants are drawn in silicon.

Learning goals — after this chapter you should be able to:

- Explain the base isolation primitive — **privilege levels** (ring 3 vs. ring 0) — and why user code physically cannot touch hardware without trapping into the kernel, and how virtual memory (the MMU/TLB of Chapter 3) turns that into per-process **address-space isolation**.
- State the **Popek–Goldberg** criterion, explain *why classic x86 was not virtualizable* (sensitive-but-unprivileged instructions), and describe the three historical fixes: binary translation, paravirtualization, and hardware-assisted virtualization.
- Describe **VT-x/AMD-V** precisely — VMX root vs. non-root mode, the VMCS, VM exits and entries as the cost model — and **second-level address translation** (EPT/NPT) as the replacement for shadow page tables, plus the **IOMMU** (VT-d/AMD-Vi) and **SR-IOV** for device isolation and passthrough.
- Place **VMs, microVMs (Firecracker/Kata), gVisor, and containers** on the isolation-strength × overhead × density triangle and choose correctly for a given trust model.
- Reason honestly about **confidential computing** (what SEV-SNP/TDX/SGX do and do *not* guarantee) and about the **microarchitectural side-channel** class (Meltdown, Spectre, L1TF, MDS) as the permanent tax of sharing silicon in a multi-tenant fleet.

Why isolation is a hardware problem

Isolation is usually discussed as an operating-system or cloud-platform feature — namespaces, security groups, IAM. But every one of those software boundaries is ultimately *cashed out in hardware*. A namespace the kernel can be tricked into ignoring is no boundary; a process separation a `mov` instruction can read across is no separation. Isolation works at all because the CPU refuses to execute certain instructions, and refuses to resolve certain addresses, unless it is in the right state — enforced on every single instruction, at hardware speed, with no software in the fast path.

This matters to a backend engineer for three concrete reasons. First, **security**: in a multi-tenant environment your process's confidentiality and integrity depend on hardware boundaries you did not build and cannot see. Second, **performance isolation** — the "noisy neighbor": a co-tenant saturating memory bandwidth (Chapter 3), thrashing the last-level cache (Chapter 4), or hammering the same NUMA node (Chapter 6) degrades *your* tail latency to the extent the hardware fails to partition those resources. Third, **economics**: the isolation strength you need dictates whether you pack a thousand tenants onto a host (containers) or a hundred (VMs), and that density ratio is the dominant term in cloud unit cost. Isolation, overhead, and density form a triangle you cannot escape; understanding the silicon is what lets you sit on it deliberately.

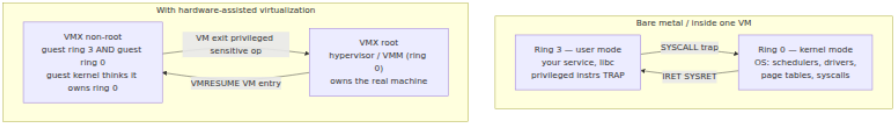
The base primitive: privilege levels

The oldest and most important isolation mechanism in a CPU is the **privilege level**, or protection ring. On x86 there are four rings, 0 through 3, but in practice operating systems use exactly two: **ring 0** (kernel/supervisor mode) and **ring 3** (user mode). Rings 1 and 2 exist, were used by a few historical systems (and, briefly, by paravirtualized Xen guests), and are otherwise dead. ARM uses the equivalent concept under the name **exception levels** (EL0 for user, EL1 for kernel, EL2 for hypervisor, EL3 for secure-monitor firmware); RISC-V calls them **privilege modes** (U, S, M). The names differ; the idea is identical.

The current ring is a piece of CPU state, and it gates two things. First, a set of **privileged instructions** will only execute in ring 0 — instructions that reconfigure the machine itself: loading the page-table base register (`mov` to `CR3`), changing control registers, executing `HLT`, `LGDT` / `LIDT` (loading the descriptor tables), `INVLPG`, reading/writing most model-specific registers via `RDMSR` / `WRMSR`, and direct port I/O (`IN` / `OUT`). Execute any of these in ring 3 and the CPU raises a **general-protection fault** instead of doing the thing. Second, the ring gates memory and I/O access via page-table permission bits and the I/O privilege level.

This is the entire foundation of the user/kernel split. Application code cannot touch a disk, reprogram the interrupt controller, or remap memory, because the instructions that would do so trap. The only way for user code to get privileged work done is to *ask the kernel*: it executes a `SYSCALL` (or `SVC` on ARM), which is a controlled, hardware-defined doorway that transitions to ring 0 at a fixed kernel entry point of the

kernel's choosing — never at an address the user controls. The kernel validates the request and does the work on the process's behalf. This is the system-call boundary, and it is the subject of Volume 2, Chapter 5 — but the *enforcement* is pure hardware: the ring bit and the trap.



The key insight for virtualization is that this two-ring model is *not enough* to run a whole guest operating system safely. A guest kernel expects to run in ring 0 and issue privileged instructions. If you demote it to ring 3, its privileged instructions trap — which is exactly what you want *if every sensitive instruction traps*. Whether they do is the crux of the classic virtualization problem.

Virtual memory as isolation

Before virtual machines, there is the humbler and more pervasive isolation mechanism you already met in Chapter 3: **virtual memory**. Each process gets its own virtual address space, and the **MMU** translates its virtual addresses through per-process **page tables** to physical frames, caching the hot translations in the **TLB**. The page-table root for the running process lives in `CR3` (x86) / `TTBR` (ARM); on a context switch the kernel loads a different root, and the same virtual address now means a different physical frame.

This is process isolation, and it is enforced by hardware on every load and store. Process A literally cannot name a byte of process B's memory: there is no virtual address in A's page tables that maps to B's private frames. It is not that the kernel checks and forbids it on each access — the kernel is not in the loop; the MMU simply has no translation, and an access to an unmapped address faults. Permission bits in the page-table entries add the finer-grained rules — writable, user-accessible, no-execute (the NX/XD bit) — enforced by the same walk. The kernel's own memory is mapped into every address space but marked supervisor-only, so ring-3 code faults if it dereferences a kernel pointer. (Hold that thought: Meltdown is precisely the story of that last guarantee failing *speculatively*, which is why KPTI later stopped mapping the kernel into user space at all.)

Virtual memory is therefore the *foundation of process isolation*, and — crucially — the technique generalizes one level up. To isolate whole virtual machines you need a *second* translation: the guest's notion of physical memory must itself be virtualized, so that guest "physical" address 0 is not host physical address 0. That second translation is what EPT/NPT provide, below. Virtual memory is the pattern; hardware virtualization applies it twice.

The classic virtualization problem: Popek–Goldberg and x86

In 1974 Gerald Popek and Robert Goldberg gave the formal criterion for whether a machine can host a classical **trap-and-emulate** virtual-machine monitor. Classify instructions into:

- **Privileged** instructions: those that trap when executed outside ring 0.
- **Sensitive** instructions: those that either read or change privileged machine state (the *control-sensitive* ones change configuration; the *behavior-sensitive* ones behave differently depending on privilege level or on the real hardware configuration).

The **Popek–Goldberg theorem** says an architecture is *efficiently virtualizable* if the set of sensitive instructions is a **subset** of the privileged instructions. The reason is elegant: run the guest kernel deprivileged in ring 3; every sensitive instruction then traps into the VMM, which emulates its effect against the guest's *virtual* machine state and returns. Non-sensitive instructions run natively at full speed. If every sensitive instruction traps, the VMM sees and controls everything that matters, and the guest cannot tell it has been deprivileged.

x86 famously **failed this test**. A 2000 analysis by Robin and Irvine catalogued seventeen sensitive-but-*unprivileged* instructions — instructions that read or depend on privileged state yet do **not** trap in ring 3; they just silently do the wrong thing. The canonical example is `POPF` / `PUSHF`: `POPF` can modify the interrupt-enable flag (`IF`), but when executed in ring 3 it does *not* fault — it simply ignores the write to `IF` silently. A deprivileged guest kernel that does `POPF` to enable interrupts gets no trap and no effect; the VMM never learns the guest wanted interrupts on, and the guest's view of the machine silently diverges from reality. Others in the list include `SGDT` / `SIDT` / `SLDT` and `SMSW` (they *read*

privileged descriptor/machine-status registers from ring 3, letting a guest observe the host's real tables), and `LAR` / `LSL` / `VERR` / `VERW` (behavior depends on privilege). Because these leak or corrupt silently, plain trap-and-emulate is impossible on classic x86. Three families of solutions emerged.

Binary translation

VMware's 1999 breakthrough was **dynamic binary translation**. The VMM does not run the guest kernel's ring-0 code directly; it scans it just ahead of execution, and translates it block by block into safe code, replacing every sensitive instruction with a call into the VMM that emulates it correctly against virtual state. User-mode guest code (the common case) runs natively; only the kernel's privileged code is translated, and translated blocks are cached so the cost amortizes. This works on stock hardware with no guest modification, which is why VMware could virtualize unmodified operating systems years before the CPU vendors caught up. The cost is the translation machinery and the fact that some operations that would be a single instruction become a VMM round trip.

Paravirtualization

Xen (2003) took the opposite tack: **change the guest**. A paravirtualized guest kernel is ported to run knowingly on top of a hypervisor. Instead of executing sensitive instructions and hoping they trap, it calls the hypervisor explicitly through **hypercalls** (the VM analog of a syscall) for privileged operations — page-table updates, interrupt control, I/O. There is no need to emulate an instruction set faithfully because the guest never issues the problematic instructions; it asks politely. This is fast — no translation, no silent-instruction problem — but it requires modifying the guest OS, so it only works for open, cooperative kernels (Linux, the BSDs) and never for an unmodified Windows. Paravirtualization survives today not as a whole-kernel strategy but as the right way to do **I/O**: `virtio` (below) is paravirtualized device access, and even hardware-virtualized guests use it.

Hardware-assisted virtualization

In 2005–2006 Intel shipped **VT-x** and AMD shipped **AMD-V (SVM)**, and the classic problem simply went away. They added a new dimension of privilege *orthogonal* to the rings.

VT-x / AMD-V: a new privilege axis

Hardware virtualization introduces two new processor operating modes: **VMX root** and **VMX non-root** (AMD's SVM has the same structure under the names *host* and *guest*). Critically, this is a *separate axis* from rings 0–3. Inside non-root mode the guest has its full complement of rings — the guest kernel runs in **guest ring 0**, believing it owns the machine — but the whole non-root mode is subordinate to root mode, where the hypervisor lives. The hardware itself now knows the difference between "the guest's kernel" and "the real supervisor."

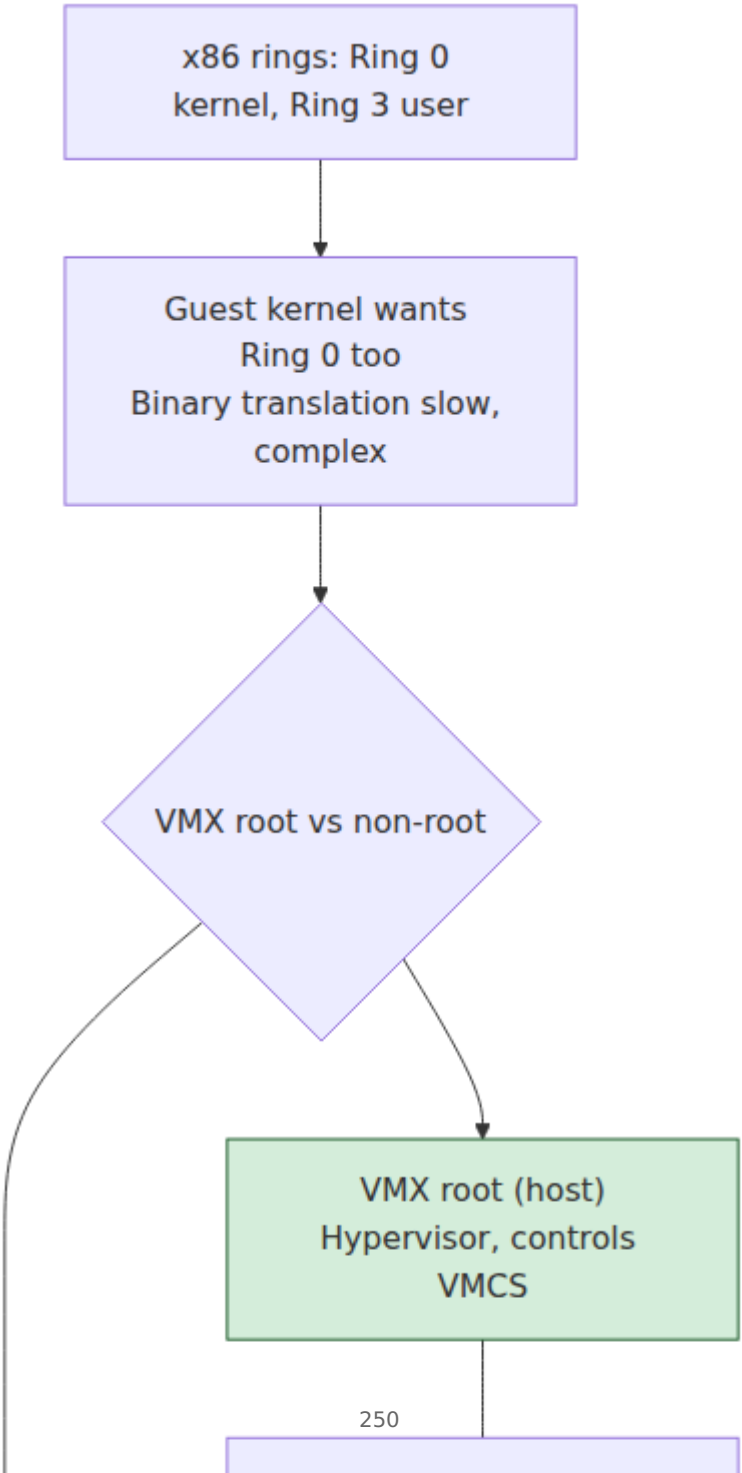
The transitions between these modes are the two operations that define hardware virtualization:

- A **VM entry** (`VMLAUNCH` the first time, `VMRESUME` thereafter) drops the CPU from root mode into the guest, loading the guest's register and control state, and runs guest code at native speed.
- A **VM exit** happens when the guest does something the hypervisor configured to be intercepted — a privileged or sensitive operation, an external interrupt, an EPT fault, an explicit `VMCALL` hypercall, etc. The CPU atomically saves guest state, restores host state, and jumps to the hypervisor's exit handler in root mode.

Both modes are backed by a control block that is the heart of the mechanism: Intel's **VMCS** (Virtual Machine Control Structure); AMD's equivalent is the **VMCB**. The VMCS is a per-vCPU in-memory structure holding the *guest state* to load on entry, the *host state* to restore on exit, and — most importantly — the **control fields** that decide *which* operations cause a VM exit. The hypervisor configures the VMCS to say, in effect: exit on `CR3` writes if I am using shadow paging (but *not* if I am using EPT); exit on I/O to these ports; exit on these MSR accesses; exit on `HLT`, on external interrupts, on exceptions of this type. Fine-grained control over exit causes is how the hypervisor keeps overhead down: the more it can let the guest do natively without exiting, the less it pays. The problematic sensitive-but-unprivileged instructions of classic x86 are now

handled directly by the hardware in non-root mode — they either behave correctly against guest state or cause a clean VM exit — so the Popek–Goldberg gap is closed in silicon.

The VM exit is the cost model of virtualization. A VM exit is a full pipeline serialization plus a state save/restore — historically hundreds to low thousands of cycles, driven down steadily across CPU generations but never free. Every intercepted operation pays it. This is why the entire art of efficient virtualization is *minimizing exits*: for memory, by using EPT so ordinary page faults and page-table edits never exit; for I/O, by using `virtio` or device passthrough so data-path operations don't trap; for interrupts, by using posted interrupts and APIC virtualization so delivering an interrupt to a guest doesn't require a round trip through the host. When you read that a workload "virtualizes with under 5% overhead," what that number really measures is how successfully the design kept the guest out of root mode.

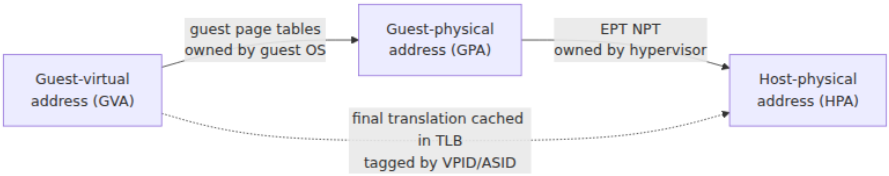

x86 rings: Ring 0
kernel, Ring 3 user
Guest kernel wants
Ring 0 too
Binary translation slow,
complex
VMX root vs non-root
VMX root (host)
Hypervisor, controls
VMCS

Memory virtualization: EPT/NPT vs. shadow page tables

A guest OS runs its own MMU logic: it builds page tables mapping **guest-virtual** addresses (GVA) to what it believes are **guest-physical** addresses (GPA). But GPA is a fiction — it must be translated again to a real **host-physical** address (HPA). There are two ways to do the second translation.

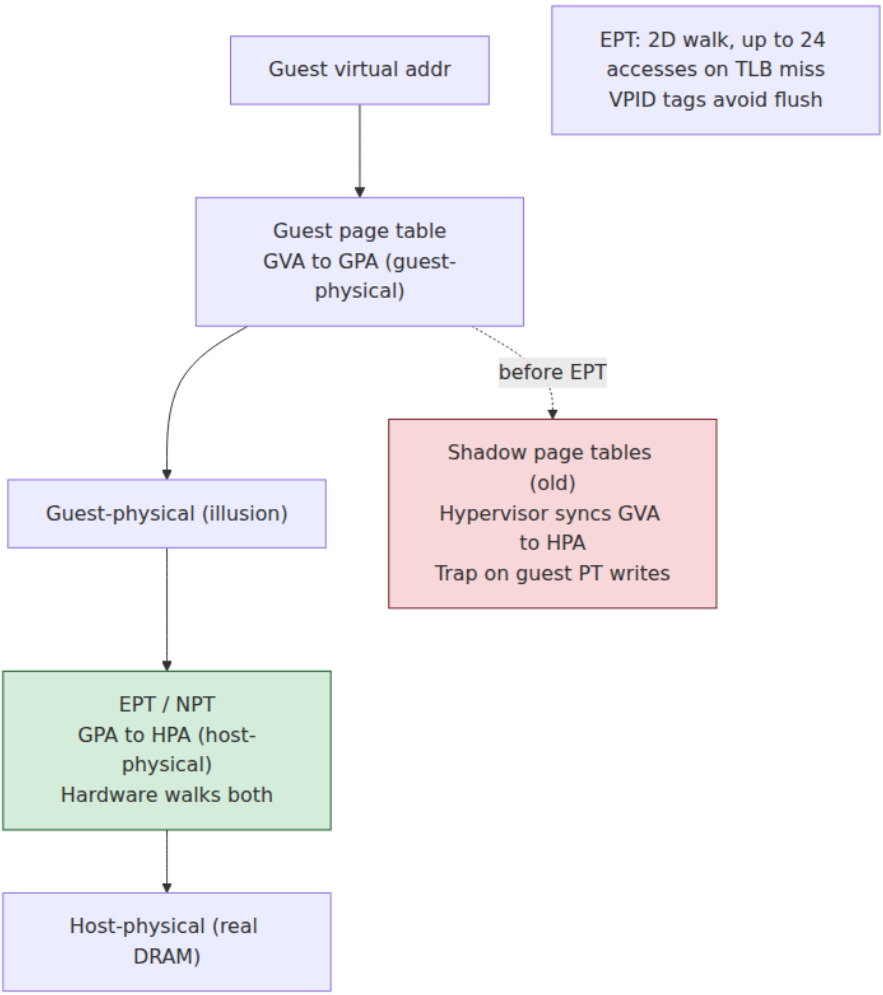
The old, software way is **shadow page tables**. The hypervisor maintains a hidden set of "shadow" page tables that map GVA directly to HPA, and points the real `CR3` at *those*. It keeps them in sync with the guest's own tables by marking the guest page tables read-only and taking a VM exit on every guest attempt to edit them, then reflecting the change into the shadows. This works but is brutal: every guest page-table write, every `CR3` reload on a guest context switch, becomes a VM exit, and the shadow structures consume memory proportional to the guest's mappings. Page-table-heavy workloads (fork-heavy servers, JITs) suffered badly.

Hardware **second-level address translation** (SLAT) fixed this: Intel **EPT** (Extended Page Tables, from Nehalem, 2008) and AMD **NPT/RVI** (Nested Page Tables, from Barcelona, 2007). The MMU now walks *two* sets of page tables in hardware. The guest's own page tables (in guest-physical space) translate GVA→GPA as normal, and a second, hypervisor-owned table (the EPT/NPT) translates GPA→HPA. The guest can edit its own page tables and reload its own `CR3` freely, with **no VM exit**, because the hypervisor no longer cares — the second-level table is where isolation is enforced, and the guest never touches it. This is exactly virtual memory applied twice, and it is what makes memory virtualization cheap enough to be the default.



The cost is a **longer page walk**. A native 4-level page walk is up to four memory references. With nested paging, *each* of those guest-level references must itself be translated through the EPT, turning a 4-level

walk into as many as 4×4 ≈ up to 24 memory accesses on a full TLB miss — the "2D page walk." Hardware hides most of this: the final GVA→HPA translation is cached in the TLB just like a native one, so a TLB *hit* costs nothing extra, and page-walk caches shortcut the intermediate levels. To avoid flushing the TLB on every VM entry/exit, the translations are tagged: Intel's **VPID** and AMD's **ASID** stamp each TLB entry with a virtual-processor identifier so guest and host (and different guests) entries coexist without cross-flushing. The residual cost is why TLB-miss-heavy workloads — those with huge, sparse working sets (Chapter 3) — see more virtualization overhead than cache-resident ones, and why **huge pages** in the guest matter even more under virtualization: they shrink both levels of the walk at once.



Device and DMA isolation: the IOMMU, VT-d, and SR-IOV

CPU and memory virtualization are only two-thirds of the machine. Devices do **DMA** — they read and write host memory directly, bypassing the CPU and therefore bypassing the MMU. A device handed to a guest that could DMA to *any* physical address would be a trivial escape: program its DMA engine to overwrite the hypervisor. The **IOMMU** closes this hole. Intel calls it **VT-d**, AMD calls it **AMD-Vi**; ARM has the SMMU. It

is an MMU *for devices*: it sits between devices and memory and translates the addresses devices use (I/O virtual addresses, IOVA) to host-physical addresses, according to per-device page tables the hypervisor controls. A device can now only touch the memory the IOMMU maps for it — DMA isolation, enforced in hardware.

The IOMMU is what makes **device passthrough** safe: you can assign a physical NIC or GPU or NVMe drive (Chapter 5) directly to a guest for native-speed I/O, and the IOMMU guarantees the device's DMA stays inside that guest's memory. On its own, one physical device serves one guest. **SR-IOV** (Single-Root I/O Virtualization) generalizes it: an SR-IOV-capable device advertises one **physical function** (PF) and many lightweight **virtual functions** (VFs), each of which looks like an independent PCIe device with its own DMA context. The hypervisor assigns one VF per guest, the IOMMU isolates each VF's DMA, and every guest gets near-native network or storage throughput without the hypervisor on the data path. This is the standard way high-performance cloud instances get their networking — the "enhanced networking" tiers are SR-IOV VFs behind the scenes.

The IOMMU is also load-bearing for the *non*-virtualized case: it is the hardware behind Linux's VFIO framework and DPDK, and it is the reason a malicious or buggy peripheral (a Thunderbolt device, say) cannot DMA over your kernel — DMA-attack protection is the IOMMU doing its day job.

Hypervisors: type 1, type 2, and KVM

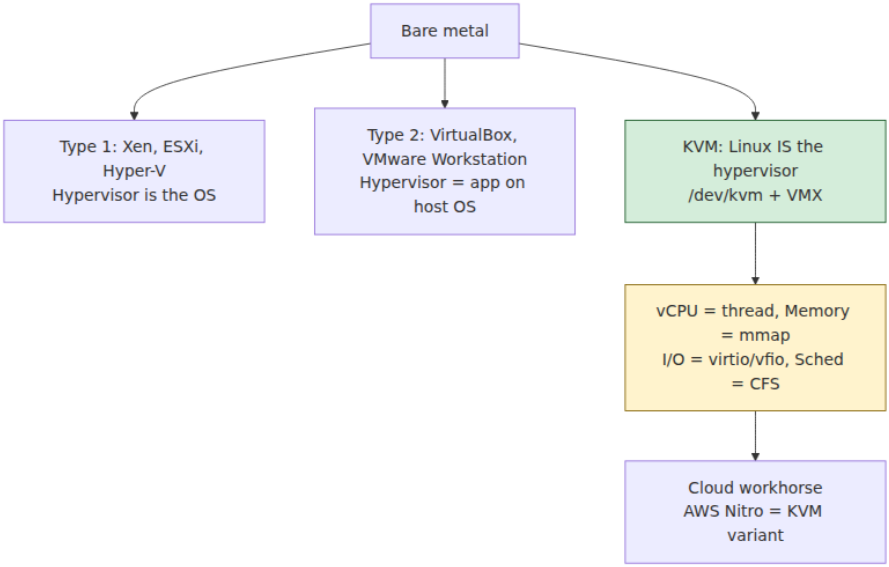
A **hypervisor** (or virtual-machine monitor, VMM) is the software that owns root mode and multiplexes the hardware among guests. The classic taxonomy:

- **Type 1 (bare-metal)**: the hypervisor runs directly on the hardware, with no host OS beneath it. VMware **ESXi**, Microsoft **Hyper-V**, and **Xen** are type 1. They are the standard for production servers because there is no general-purpose OS underneath to add overhead or attack surface.
- **Type 2 (hosted)**: the hypervisor runs as an application on a normal host OS, using the host's drivers and scheduler. VMware Workstation/Fusion and VirtualBox are type 2 — great for a laptop, not how clouds run.

KVM (Kernel-based Virtual Machine) sits awkwardly and interestingly across the line. KVM is a *Linux kernel module*: it turns the Linux kernel itself into a hypervisor by exposing VT-x/AMD-V through the `/dev/kvm` interface. Because the thing owning root mode is the Linux kernel — a full-featured OS with all its drivers and its scheduler — KVM has the *convenience* of type 2 but the *performance* of type 1 (the kernel is the bare-metal supervisor, not a guest of one), which is why the neat taxonomy breaks down and people argue about which bucket KVM belongs in. It doesn't matter; what matters is that KVM is how essentially all of the open-source cloud runs.

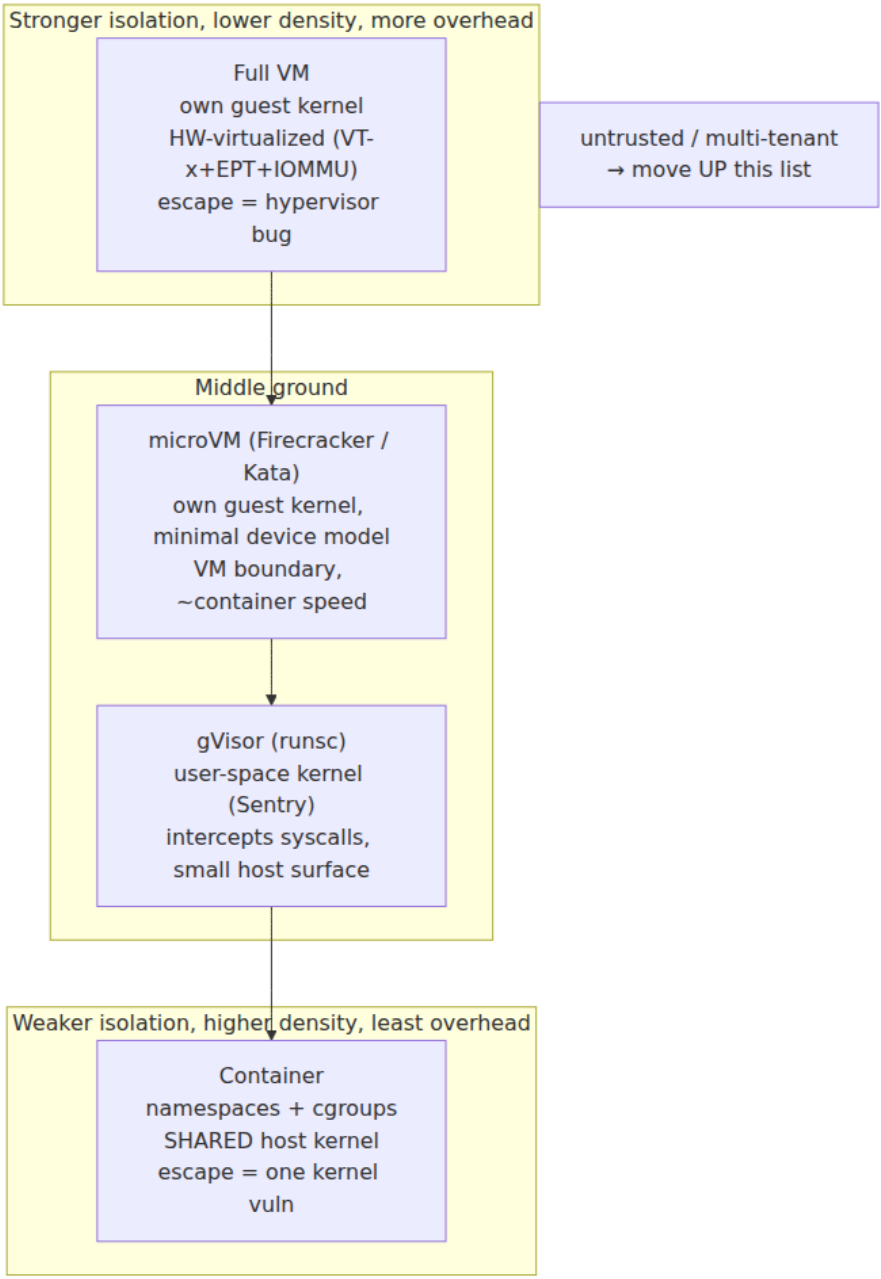
KVM alone virtualizes the CPU and memory. It does **not** emulate devices — a guest needs a disk controller, a NIC, a serial port, a BIOS. That is the job of a separate userspace process, the **VMM / device model**, and historically that is **QEMU**. The division of labor is clean: KVM (in the kernel) handles the VM-exit fast path, the vCPU threads, EPT, and interrupt injection; QEMU (in userspace) implements the virtual devices and handles the exits that need device emulation. For the data path, they use **`virtio`** — paravirtualized devices (`virtio-net`, `virtio-blk`, `virtio-scsi`) where the guest driver and the host share ring buffers, avoiding the hundreds of per-operation VM exits that emulating a *real* device register-by-register would cost. This is paravirtualization's living legacy: even a fully hardware-virtualized Linux guest uses `virtio` drivers because emulating a real e1000 NIC faithfully would be absurdly slow.

This KVM-plus-VMM structure is exactly how cloud VMs work, and it is the substrate under Volume 12. AWS's Nitro system, for instance, is a purpose-built lightweight KVM-based hypervisor that offloads networking, storage, and security to dedicated Nitro *cards* (hardware), leaving almost the entire host CPU for the guest — an architectural move to drive the virtualization tax toward zero by pushing the device model off the main CPU entirely.



The isolation spectrum: VMs, microVMs, gVisor, containers

Here is the synthesis, and the single most consequential architecture decision a platform team makes. You are running many tenants' workloads on shared hosts, and you must choose *how strongly to isolate them*. There is no free lunch: **isolation strength, runtime overhead, and density trade off against one another**, and the four dominant options sit at different points on that triangle.


Stronger isolation, lower density, more overhead
Full VM
own guest kernel
HW-virtualized (VT-x+EPT+IOMMU)
escape = hypervisor bug
untrusted / multi-tenant
→ move UP this list
Middle ground
microVM (Firecracker / Kata)
own guest kernel, minimal device model
VM boundary, ~container speed
gVisor (runsc)
user-space kernel (Sentry)
intercepts syscalls, small host surface
Weaker isolation, higher density, least overhead
Container
namespaces + cgroups
SHARED host kernel
escape = one kernel vuln

Full VMs sit at the strong-isolation corner. Each guest has its **own kernel**, and the boundary between guest and host is the hardware-virtualization boundary itself: VT-x/AMD-V, EPT, and the IOMMU. To escape, an attacker must break the *hypervisor* or the virtual device model — a small, hardened, heavily audited surface. The cost is that each VM carries a full kernel and its memory footprint, boots in seconds, and you fit fewer per host. This is the right default when tenants are mutually untrusting and the workload is long-lived.

Containers sit at the opposite corner. A container is not a virtualization construct at all — it is a normal process with a restricted view, built from two Linux kernel features: **namespaces** (which virtualize the *names* a process sees — PID, network, mount, UTS, IPC, user, cgroup namespaces) and **cgroups** (which limit and account resources — CPU, memory, I/O). These are the subject of Volume 2, Chapter 9. The decisive fact is that **all containers on a host share the single host kernel**. Isolation is process-level, drawn by the kernel in software, and the attack surface is the *entire Linux system-call interface* — hundreds of syscalls, historically a steady source of privilege-escalation CVEs. One kernel vulnerability, reachable from an unprivileged container, is a host compromise and therefore a compromise of every co-tenant. Containers win overwhelmingly on density (thousands per host), startup (milliseconds), and overhead (near zero — it *is* a native process), which is why they dominate — but their boundary is the weakest of the four, which is why you do not put mutually hostile tenants in bare containers on a shared kernel.

Between these poles are two designs that try to buy VM-grade isolation without VM-grade cost.

MicroVMs keep the real hardware-virtualization boundary — a genuine guest kernel, VT-x/EPT — but strip the *VMM* down to the bone. **Firecracker**, AWS's Rust VMM built on KVM, ships a minimal device model (a handful of `virtio` devices, no BIOS, no PCI, no legacy emulation), which shrinks both the attack surface and the boot time: a Firecracker microVM boots in ~125 ms and adds only a few megabytes of memory overhead per VM, enabling thousands per host. This is the technology under **AWS Lambda and Fargate** — every function invocation and every Fargate task gets its *own* microVM, so one tenant's code runs behind a hardware-virtualization boundary from the next tenant's, at a density and startup latency that used to require containers.

This reconciliation — hardware isolation at serverless density — is why microVMs are the quiet backbone of modern serverless (tie to Volume 12 and to the supply-chain volume, Book 6, Chapter 8, on the trust properties of serverless platforms). **Kata Containers** applies the same idea from the container direction: it is an OCI-compatible runtime that transparently runs each pod inside a lightweight VM (backed by QEMU, Firecracker, or Cloud Hypervisor), so your Kubernetes workloads get a VM boundary while still looking and deploying like containers.

gVisor takes an entirely different route: a **user-space kernel**. Google's `runsc` runtime runs the container as normal but interposes a process called the **Sentry** — a re-implementation of a large part of the Linux system-call surface, written in Go, running in user space. Guest syscalls are intercepted (via `ptrace` or a KVM-based platform) and serviced by the Sentry, which itself makes a small, tightly restricted set of real syscalls to the host kernel. The point is *defense in depth on the syscall surface*: a container exploit now has to get through the Sentry's reimplementation before it can reach the host kernel, and the host-kernel surface the Sentry exposes is a fraction of the full Linux ABI. gVisor sits in the middle of the triangle — stronger than a bare container, cheaper than a full VM — at the cost of *compatibility* (some syscalls and features are unimplemented or subtly different) and *performance* (every syscall is now an interception plus a user-space round trip, which punishes syscall-heavy and I/O-heavy workloads).

Technology	Isolation mechanism	Boundary strength	Overhead	Density	Use when
Full VM	VT-x/AMD-V + EPT + IOMMU; own kernel	Strongest — escape = hypervisor bug	Highest (full kernel, seconds to boot)	Lowest	Untrusted, long-lived tenants; strong compliance
microVM (Firecracker/ Kata)	Same HW virtualization, minimal VMM/device model	Very strong — small VMM surface	Low (~ms boot, few MB)	High	Serverless/ functions; untrusted code at scale

Technology	Isolation mechanism	Boundary strength	Overhead	Density	Use when
gVisor (runsc)	User-space kernel intercepts syscalls	Medium — shrinks host-kernel surface	Medium (syscall interception cost)	High	Untrusted code where microVM is impractical
Container	Namespaces + cgroups; shared kernel	Weakest — escape = one kernel CVE	Near zero (native process)	Highest	Trusted / first-party workloads; density-critical

The decision rule is simple to state and hard to live by: **the less you trust the workload, the higher up this list you go.** First-party services you wrote and review can share a kernel in containers; a platform running arbitrary customer code (a CI runner, a function-as-a-service, a notebook host) must not — it belongs in a microVM or, at minimum, gVisor. Where you land on the triangle is a direct input to your cloud unit economics (density) and your blast radius (isolation), and it is the daily architecture decision of any multi-tenant platform team.

Confidential computing: not trusting the hypervisor

Everything above assumes the hypervisor and the host operator are *trustworthy* — they own root mode and can read every guest's memory at will. In the public cloud that is a real assumption: you are trusting the provider's operators, firmware, and hypervisor with your plaintext data in RAM. **Confidential computing** is the hardware effort to remove that assumption — to run a workload whose memory even the hypervisor and the host OS cannot read. This directly addresses the cloud-provider-trust concern of the supply-chain volume (Book 6, Chapter 9): the provider is part of your supply chain, and confidential computing narrows what you must trust them for.

There are two generations of the idea, with importantly different scopes.

Process-level enclaves — Intel SGX. SGX (2015) lets an application carve out an **enclave**: a region of memory (the Enclave Page Cache) that

is encrypted by the memory controller and inaccessible to *everything* outside the enclave — including ring 0, the hypervisor, and DMA. Code inside the enclave runs with its data confidential and integrity-protected, and **remote attestation** lets a remote party verify (via an Intel-signed quote) that it is really talking to a genuine enclave running the expected code before provisioning secrets to it. SGX's problems are the cautionary tale of the field. The early EPC was tiny (~128 MB total, of which ~96 MB usable), so real workloads paged enclave memory in and out at brutal cost. The programming model is invasive (you must partition your app). And it has been repeatedly broken by side channels — Foreshadow/L1TF (below) read enclave memory speculatively; Plundervolt used voltage glitching; SGAxe extracted attestation keys. Intel **removed SGX from consumer Core processors** (11th/12th-gen client parts dropped it), and it survives on Xeon server parts (with a much larger EPC). Treat SGX as a specialized tool with a bloody history, not a general isolation primitive.

VM-level confidential computing — AMD SEV-SNP and Intel TDX. The newer, more practical generation encrypts an *entire VM* so an unmodified guest OS runs confidentially, with the hypervisor outside the trust boundary. AMD's line evolved in three steps: **SEV** encrypts guest memory with a per-VM key generated and held by an on-die security processor (the memory controller encrypts/decrypts with AES; the hypervisor sees only ciphertext). **SEV-ES** adds encryption of the guest's *register state* on VM exit, so the hypervisor can't read the CPU context either. **SEV-SNP** (Secure Nested Paging) adds the crucial piece: **integrity and anti-remapping protection**. A malicious hypervisor can't just *read* the memory — with SNP it also can't silently *remap*, replay, or corrupt guest pages without detection, closing a class of active attacks the earlier versions left open. Intel's **TDX** (Trust Domain Extensions, from Sapphire Rapids) provides the analogous capability — a **Trust Domain** is a confidential VM with encrypted memory and integrity protection, mediated by a small, Intel-signed **TDX module** running in a special SEAM mode that sits even below the hypervisor. Both provide remote attestation so a relying party can verify the confidential VM's identity and measurement before trusting it.

Feature	Vendor(s)	Purpose
VT-x / AMD-V	Intel / AMD	CPU virtualization: root/non-root modes, VM exits, VMCS/VMCB

Feature	Vendor(s)	Purpose
EPT / NPT	Intel / AMD	Second-level address translation (GPA→HPA) without shadow page tables
IOMMU (VT-d / AMD-Vi)	Intel / AMD	DMA remapping and device isolation; enables safe passthrough
SR-IOV	PCI-SIG	One physical device → many isolated virtual functions for guests
SGX	Intel	Process-level encrypted enclaves + attestation (deprecated on client)
SEV / SEV-ES / SEV-SNP	AMD	Confidential VMs: encrypted memory, encrypted registers, integrity
TDX	Intel	Confidential VMs (Trust Domains): encrypted, integrity-protected, attested
MPK / PKU	Intel	Fast intra-process memory-domain isolation via protection keys
CET	Intel/AMD	Control-flow integrity: shadow stack + indirect-branch tracking
Pointer Authentication / MTE / BTI	ARM	Pointer signing, memory tagging, branch-target enforcement

Be honest about what confidential computing does **not** give you. It shrinks the trusted computing base but does not eliminate it: you still trust the **CPU vendor** (silicon, microcode, and signing keys — the very keys SGX side channels have extracted), the firmware, and the attestation infrastructure. Memory encryption defeats a hypervisor *reading* RAM but does not, by itself, defeat **microarchitectural side channels** — several confidential-computing schemes have been partially broken by the same speculation-and-cache attacks discussed next. It is a genuine, valuable reduction of who you must trust — moving the cloud operator out of your data's plaintext path — not a mathematical guarantee of secrecy. Sold as the former it is a real advance; believed to be the latter it will burn you.

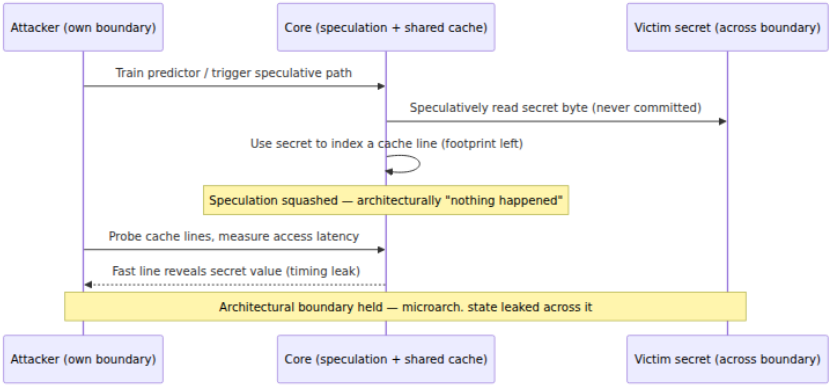
Finer-grained hardware isolation features

Below whole-VM and whole-enclave isolation, several features harden the *intra-process* boundary and matter for defense in depth. **Intel MPK / PKU** (Protection Keys for Userspace) tags each page with a 4-bit *protection key* and lets a thread change access rights for a whole key domain by writing the `PKRU` register — no syscall, no TLB shootdown — which makes it cheap to sandbox a plugin or wall off a secret region within one address space. **CET** (Control-flow Enforcement Technology) adds a hardware **shadow stack** (a protected copy of return addresses, defeating ROP) and **indirect-branch tracking** (indirect jumps must land on an `ENDBR` landing pad, defeating JOP). On ARM, **Pointer Authentication (PAC)** signs pointers with a key into their unused high bits and verifies the signature before use, making pointer corruption detectable; **MTE** (Memory Tagging) colors allocations to catch use-after-free and overflow; **BTI** is ARM's branch-target enforcement. None of these virtualize anything — they harden the boundaries the isolation stack depends on.

The leak: microarchitectural side channels

Now the honest closer. Everything above enforces the **architectural** boundary: what an instruction is *allowed* to read according to the ISA. But a modern CPU (Chapters 2–4) is full of **speculative, out-of-order, shared microarchitectural state** — branch predictors, the cache hierarchy, line-fill and store buffers, load ports — that is *not* part of the architectural contract and is *not* reset at isolation boundaries. Starting in 2018, a class of attacks showed that this state **leaks data across boundaries the architecture guarantees are sealed**. This is the deepest reason isolation is imperfect, and it is a direct consequence of the performance mechanisms this volume spent nine chapters admiring.

The common structure of every attack is: (1) get the CPU to *speculatively* access data it will never architecturally commit — the speculation is squashed, the result never appears in a register — but (2) the speculative access leaves a **microarchitectural footprint**, typically a line pulled into cache, and (3) the attacker *times* memory accesses afterward to infer which line was touched, reconstructing the secret one bit at a time. The architectural boundary held — the value was never legally read — but it leaked through timing anyway.



The landmark instances, accurately:

- **Meltdown** (CVE-2017-5754, 2018) exploited that some CPUs (chiefly Intel, some ARM) would let an out-of-order load *speculatively* read kernel memory from user mode and forward it to dependent instructions before the permission check retired — long enough to leak it through cache. It broke the user/kernel boundary directly. The mitigation, **KPTI** (Kernel Page-Table Isolation), stops mapping the kernel into the user address space at all, so there is nothing to speculatively read — at the cost of a page-table switch (and TLB pressure) on every syscall and interrupt, i.e., a real, syscall-rate-dependent performance tax.
- **Spectre** (v1 = bounds-check bypass, CVE-2017-5753; v2 = branch-target injection, CVE-2017-5715) is more general and harder to kill: it abuses the *branch predictor*. In v1 the attacker trains a bounds check to predict "in range," so the CPU speculatively reads out-of-bounds; in v2 the attacker poisons the indirect-branch predictor so speculation lands in an attacker-chosen "gadget." Spectre works *within* a privilege level and across the VM boundary. Mitigations are a grab-bag: `lfence` speculation barriers and `array_index_nospec()` for v1; **retpoline** (replacing indirect branches with a construct the predictor can't poison) and microcode controls **IBRS/IBPB/STIBP** for v2. All cost performance, and Spectre v1 in particular cannot be fully fixed in hardware — it is mitigated case by case in software, effectively forever.
- **L1TF / Foreshadow** (CVE-2018-3615/3620/3646, 2018) — the **L1 Terminal Fault** — let speculation read any data present in the **L1**

data cache past a permission fault, including *another VM's* data and *SGX enclave* data. In a multi-tenant host this is a cross-VM read. Mitigation: **flush the L1D cache on VM entry**, and — because a sibling SMT thread shares the L1 — either **disable SMT** or use core scheduling so a core is never shared across trust boundaries.

- **MDS** (Microarchitectural Data Sampling — RIDL, Fallout, ZombieLoad, 2019) leaks from even more transient buffers — **line-fill buffers, store buffers, load ports** — that are shared and not cleared at boundaries, sampling data in flight regardless of address. Mitigation: a microcode-assisted **buffer overwrite** (`VERW`) on every transition across a boundary, and again **disabling SMT** for the strongest guarantee, because the sibling thread shares those buffers.

Notice the recurring villain: **SMT** (Chapter 2). Simultaneous multithreading shares the L1 cache and these buffers between two threads on one core, so if those two threads are in different trust domains, much of the leakage is continuous rather than requiring elaborate speculation tricks. This is why the strongest response to L1TF and MDS is to **turn SMT off** — and why cloud providers offer, and security-sensitive tenants pay for, non-SMT or dedicated-core instances. It is a stark trade: SMT buys throughput (Chapter 2), and disabling it for isolation directly sacrifices that throughput — often double-digit percentages.

The multi-tenant implication is the point of this whole chapter. The "noisy neighbor" of Chapter 3 and Chapter 4 — the co-tenant thrashing your shared cache and bandwidth — has an evil twin: a co-tenant who *times* that shared state to exfiltrate your data. On shared hardware the two are the same phenomenon seen from different angles: **shared microarchitectural resources are both a performance-interference channel and an information-leakage channel.** Mitigations — KPTI, retpoline, L1D flush, buffer clears, disabling SMT — are all real performance costs, layered on top of the machine, and they are the standing tax the industry pays for having built breathtakingly fast speculative cores and then run mutually distrusting tenants on them. Hardware isolation is *good*, and steadily improving, and *not perfect*. A senior engineer running untrusted multi-tenant workloads treats side channels as a live, permanent risk to be managed — with VM/microVM boundaries, SMT policy, and up-to-date microcode — not as a solved problem.

Distributed-systems lens: the fleet runs on virtualized, multi-tenant silicon

Zoom out to the fleet. Not one of the services you operate runs on a machine it owns. It runs in a VM or a container, on a host shared with strangers, behind a hypervisor you did not write, on silicon whose isolation is enforced by the mechanisms in this chapter — and the *choice* of mechanism is a recurring, high-stakes design decision, not a detail.

The isolation spectrum **is** the multi-tenancy architecture of Volume 12. Every platform team repeatedly answers the same question — VM, microVM, gVisor, or container? — and the answer is dictated by trust: first-party services co-scheduled in containers for density; arbitrary customer code (functions, CI runners, build farms) pushed up into microVMs for a hardware boundary. **Firecracker is the concrete reconciliation** — hardware-virtualization isolation at container-like density and startup — which is precisely why serverless (Lambda, Fargate) could offer per-invocation isolation of untrusted code without the old VM tax, and why the supply-chain properties of serverless (Book 6, Chapter 8) rest on it. **Confidential computing** (SEV-SNP, TDX) is the fleet's answer to a different trust question — *must I trust the cloud operator with my plaintext?* — and it maps directly onto the provider-chain concern of Book 6, Chapter 9: it shrinks the provider from "can read all my data" to "I trust their silicon and attestation," a meaningful narrowing of the supply chain even though it is not zero-trust. And **side channels** are the fleet's worst-case multi-tenant risk: the noisy neighbor who doesn't just slow you down but reads you, which is why SMT policy and instance-isolation tiers are line items in a serious platform's threat model and its bill.

The through-line of this entire volume lands here. The privilege ring, the MMU, VT-x and EPT, the IOMMU — this stack is *what makes secure multi-tenancy possible at all*, and it is why the cloud can sell you a fraction of a machine as if it were a whole one. Its imperfections — the VM-exit tax, the 2D page walk, the isolation-versus-density triangle, the persistent side-channel leak — are not footnotes; they shape cloud pricing, instance-type menus, and security postures. The hardware you spent nine chapters learning to run *fast* is the same hardware that must, simultaneously, keep a thousand tenants from running *into each other* — and understanding both sides is the difference between using the cloud

and reasoning about it. From here, Volume 2 picks up the operating system that drives these mechanisms — ring transitions, page tables, namespaces and cgroups — and Volume 12 picks up the distributed, multi-tenant cloud they make possible.

Key takeaways

- **Isolation is enforced in hardware, on every instruction.** Privilege rings (ring 3 vs. ring 0) gate privileged instructions and force user code to trap into the kernel; the MMU/TLB give each process its own address space. These are the base primitives; everything else builds on them.
- **Classic x86 was not virtualizable** (Popek–Goldberg fails: ~17 sensitive-but-unprivileged instructions like `POPF` don't trap). The fixes were binary translation (VMware), paravirtualization (Xen/ `virtio`), and finally hardware assist (VT-x/AMD-V), which added a new privilege axis.
- **VT-x/AMD-V = VMX root vs. non-root**, controlled by the VMCS/ VMCB; **VM exits** are the cost model, and efficient virtualization is the art of avoiding them. **EPT/NPT** provide GPA→HPA translation in hardware, retiring shadow page tables at the cost of a longer (2D) page walk. The **IOMMU** (VT-d/ AMD-Vi) isolates device DMA and makes **SR-IOV** passthrough safe.
- **KVM turns Linux into the hypervisor**; QEMU (or a minimal VMM) supplies the device model; this is how cloud VMs run. Type 1 vs. type 2 is a fuzzy taxonomy that KVM straddles.
- **The isolation spectrum is the core decision**: VMs (own kernel, strongest, priciest) → microVMs (Firecracker/Kata: VM boundary at container speed — the basis of Lambda/Fargate) → gVisor (user-space kernel, shrinks host-syscall surface) → containers (namespaces+cgroups, shared kernel, weakest — one kernel CVE escapes). Less trust ⇒ move up. It's an isolation-vs-overhead-vs-density triangle.
- **Confidential computing** (SEV-SNP, TDX; SGX at process level) encrypts VM/enclave memory to remove the hypervisor/operator from the trust boundary — a real narrowing of the supply chain, but you still trust the CPU vendor and it does *not* by itself defeat side channels. Don't overstate the guarantee.

- **Side channels are the permanent tax of shared silicon.** Meltdown, Spectre, L1TF, and MDS leak across architecturally-sealed boundaries via speculation plus shared cache/buffers; SMT makes it worse. Mitigations (KPTI, retpoline, L1D flush, buffer clears, disabling SMT) all cost performance. Hardware isolation is good and improving, not perfect — treat multi-tenant side channels as a live risk.

Further reading

- Gerald J. Popek and Robert P. Goldberg, "Formal Requirements for Virtualizable Third Generation Architectures," *Communications of the ACM*, 17(7), 1974 — the founding paper; the sensitive ⊆ privileged criterion.
- John Scott Robin and Cynthia E. Irvine, "Analysis of the Intel Pentium's Ability to Support a Secure Virtual Machine Monitor," USENIX Security 2000 — the catalogue of x86's sensitive-but-unprivileged instructions.
- Keith Adams and Ole Agesen, "A Comparison of Software and Hardware Techniques for x86 Virtualization," ASPLOS 2006 — VMware's own measured comparison of binary translation vs. early VT-x; the definitive account of why early hardware assist wasn't automatically faster.
- Paul Barham et al., "Xen and the Art of Virtualization," SOSP 2003 — the paravirtualization design and hypercall model.
- Intel, *Intel 64 and IA-32 Architectures Software Developer's Manual*, Volume 3C (System Programming Guide) — the authoritative VT-x reference: VMX operation, the VMCS, VM entries/exits, EPT. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
- AMD, *AMD64 Architecture Programmer's Manual, Volume 2: System Programming* — SVM, the VMCB, and Nested Page Tables (NPT). https://www.amd.com/en/search/documentation/hub.html
- Avi Kivity et al., "kvm: the Linux Virtual Machine Monitor," Linux Symposium 2007 — how KVM exposes hardware virtualization through `/dev/kvm`.
- Alexandru Agache et al., "Firecracker: Lightweight Virtualization for Serverless Applications," USENIX NSDI 2020 — the microVM design

behind Lambda and Fargate, with its density and boot numbers. https://www.usenix.org/conference/nsdi20/presentation/agache

- The gVisor documentation, https://gvisor.dev/docs/ — architecture of the Sentry user-space kernel and the `ptrace`/KVM platforms; and the Kata Containers architecture docs, https://katacontainers.io/.
- Confidential Computing Consortium, "A Technical Analysis of Confidential Computing," https://confidentialcomputing.io/ — vendor-neutral treatment of SEV-SNP, TDX, and SGX threat models and limits. See also AMD's SEV-SNP whitepaper and Intel's TDX whitepapers on their developer sites.
- Victor Costan and Srinivas Devadas, "Intel SGX Explained," IACR ePrint 2016/086 — the thorough reference on SGX internals; pair with the Foreshadow paper (Van Bulck et al., USENIX Security 2018) for its side-channel limits.
- Paul Kocher et al., "Spectre Attacks: Exploiting Speculative Execution," and Moritz Lipp et al., "Meltdown: Reading Kernel Memory from User Space," both IEEE S&P / USENIX Security 2019 — https://spectreattack.com and https://meltdownattack.com — plus the L1TF/Foreshadow (https://foreshadowattack.eu) and MDS/RIDL/ZombieLoad (https://mdsattacks.com) sites for the cross-boundary microarchitectural attacks and their mitigations.